UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN
PROYECTO DE GRADUACIÓN II

SMILEESYNC

Gestión inteligente de citas y agenda odontológica

HEE SUNG LEE

GUATEMALA, OCTUBRE 2026

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

SMILEESYNC: GESTIÓN INTELIGENTE DE CITAS Y AGENDA ODONTOLÓGICA

PROYECTO DE GRADUACIÓN II

PRESENTADO POR:
HEE SUNG LEE

PREVIO A OPTAR AL GRADO ACADÉMICO DE:
LICENCIADO EN INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

Y EL TÍTULO PROFESIONAL DE:
INGENIERO EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN

GUATEMALA, OCTUBRE 2026

AUTORIDADES DE LA FACULTAD Y ASESOR DEL TRABAJO DE GRADUACIÓN

DECANO DE LA FACULTAD: ING. JORGE ALBERTO ARIAS TOBAR

SECRETARIO DE FACULTAD: ING. M.A. HUGO ADALBERTO HERNÁNDEZ SANTIZO

ASESOR: ING. CARLOS EDUARDO HERNÁNDEZ HERRERA

AUTORIZACIÓN PARA LA IMPRESIÓN DEL TRABAJO DE GRADUACIÓN

REGLAMENTO DEL TRABAJO DE GRADUACIÓN

Artículo 8: RESPONSABILIDAD

"Solamente el autor es responsable de los conceptos expresados en el trabajo de tesis. Su aprobación en manera alguna implica responsabilidad para la Universidad."

ÍNDICE GENERAL

Introducción..... 1
Objetivos..... 2
Objetivo general..... 2
Objetivos específicos..... 2
Hipótesis..... 3
Pregunta principal..... 3
Preguntas auxiliares..... 3
Capítulo 1. Marco Teórico..... 4
1.1 Planteamiento del problema..... 4
1.1.1 Antecedentes del problema..... 4
1.1.2 Problemática actual..... 5
1.2 Fundamentos teóricos..... 6
1.2.1 Aprendizaje automático supervisado..... 6
1.2.2 Algoritmos de regresión..... 6
1.2.3 Proceso y tipos de entrenamiento del modelo..... 7
1.2.4 Métricas de evaluación..... 8
Capítulo 2. Marco Conceptual..... 9
2.1 Antecedentes (trabajos relacionados)..... 9
2.2 Descripción de los conceptos involucrados en el desarrollo del software..... 10
2.2.1 Sistemas de información..... 10
2.2.2 Tipo de aplicación y arquitectura de software..... 11
2.2.3 Base de datos..... 11
2.2.4 Tecnologías web..... 12
2.2.5 Paradigmas de programación..... 13
2.2.6 Metodología de desarrollo del software..... 14
2.2.7 Inteligencia artificial y aprendizaje automático..... 14
2.2.8 Arquitectura y módulos del sistema..... 15
2.2.9 Computación en la nube e infraestructura de despliegue..... 17
Capítulo 3. Marco Metodológico: Análisis y Diseño..... 20
3.1 Análisis..... 20
3.1.1 Requerimientos..... 20
3.1.1.1 Cliente..... 20
3.1.1.2 Delimitación (DERCAS)..... 21
3.1.2 Análisis de la solución..... 45
3.1.2.1 Arquitectura de software..... 45
3.1.2.2 Infraestructura..... 47
3.1.2.3 Estudio de factibilidad..... 48
3.1.3 Fases de desarrollo..... 52
3.1.3.1 Descripción de las fases..... 52
3.1.3.2 Cronograma del desarrollo..... 53
3.2 Diseño del sistema..... 54
3.2.1 Diagrama de casos de uso..... 55
3.2.2 Diseño de la base de datos..... 58

3.2.3 Diagrama de clases....60
3.2.4 Diagramas de secuencia....61
3.2.5 Diagramas de actividad de los procesos....64
3.2.6 Diagrama de componentes y arquitectura de despliegue....89
3.3 Diseño de interfaces....89
3.3.1 Módulo de Seguridad y Usuarios....90
3.3.2 Módulo de Pacientes....93
3.3.3 Módulo de Catálogo Odontológico....94
3.3.4 Módulo de Agenda y Citas....97
3.3.5 Módulo de Expediente Clínico....102
3.3.6 Módulo de Plan de Tratamiento....106
3.3.7 Módulo de Cobros....107
3.3.8 Reportes del sistema....108
3.3.9 Conjunto de pruebas....109
3.4 Diseño de la investigación y validación del modelo....109
3.4.1 Población y muestra....110
3.4.2 Variables del estudio....111
3.4.3 Instrumento de recolección y depuración de los datos....111
3.4.4 Hipótesis de trabajo y criterio de validación....113
3.4.5 Métricas de evaluación y resultados esperados....113
Capítulo 4. Marco Metodológico: Desarrollo e Implementación....115
4.1 Desarrollo....115
4.1.1 Plataforma de desarrollo....115
4.1.2 Tecnologías y arquitecturas....116
4.1.2.1 Arquitectura de la aplicación....116
4.1.2.2 Bibliotecas y componentes incorporados....116
4.1.2.3 Herramientas de apoyo al desarrollo y a la documentación....119
4.1.2.4 Organización del código fuente....119
4.1.3 Seguridad....120
4.1.3.1 Autenticación....121
4.1.3.2 Autorización y control de acceso por rol....121
4.1.3.3 Protección de la sesión y del navegador....122
4.1.3.4 Protección de los datos....123
4.1.3.5 Gestión de la configuración y de los secretos....124
4.2 Codificación....125
4.2.1 Inicio de sesión....125
4.2.2 Menú principal....127
4.2.3 Funciones principales....130
4.2.3.1 Proceso 1: construcción del calendario....132
4.2.3.2 Proceso 2: estimación de la duración del procedimiento....133
4.2.3.3 Proceso 3: recomendación de espacios en la agenda....136
4.2.3.4 Proceso 4: programación de la cita y validación de la disponibilidad....138
4.2.3.5 Proceso 5: registro de la atención y realimentación del modelo....139
4.2.3.6 Funciones del módulo de Seguridad y Usuarios....141

4.2.3.7 Funciones de los módulos de Pacientes y de Catálogo....142
4.2.3.8 Funciones del módulo de Expediente Clínico....143
4.2.3.9 Funciones de los módulos de Plan de Tratamiento y de Cobros....145
4.2.3.10 Funciones del módulo de Inteligencia....146
4.2.3.11 Funciones del módulo de Reportes....152
4.2.4 Ejemplo de gráficas y reportes....153
4.3 Pruebas de implementación....157
4.3.1 Entorno y alcance....158
4.3.2 Verificación técnica previa a la entrega....158
4.3.3 Usuarios y datos de preparación del ensayo....159
4.3.4 Guion de acompañamiento: preguntas previstas y respuestas....160
4.3.5 Protocolo y registro de las sesiones....163
4.3.6 Hallazgos derivados del uso y correcciones aplicadas....164
4.3.7 Resultado y tareas pendientes....166
Referencias....167
Glosario....170
Anexos....174
Anexo A. Modelo de la base de datos....174
Anexo B. Diccionario de datos....175

ÍNDICE DE ILUSTRACIONES

Índice de figuras

Figura 1. Arquitectura general del sistema SmileeSync....16
Figura 2. Arquitectura de software de SmileeSync por capas....46
Figura 3. Cronograma de desarrollo del sistema SmileeSync....54
Figura 4. Casos de uso de los módulos administrativos....56
Figura 5. Casos de uso de la agenda, el expediente clínico, los cobros y la inteligencia....57
Figura 6. Modelo de datos de SmileeSync agrupado por módulo....59
Figura 7. Diagrama de clases del sistema SmileeSync....61
Figura 8. Diagrama de secuencia: inicio de sesión y control de acceso por rol....62
Figura 9. Diagrama de secuencia: creación de usuario con validación de datos....63
Figura 10. Diagrama de secuencia: programación de cita con predicción de duración....64
Figura 11. Diagrama de actividad: inicio de sesión y control de acceso....65
Figura 12. Diagrama de actividad: mantenimiento de usuarios....67
Figura 13. Diagrama de actividad: recuperación de contraseña....69
Figura 14. Diagrama de actividad: registro de paciente y antecedentes médicos....71
Figura 15. Diagrama de actividad: mantenimiento del catálogo odontológico....72
Figura 16. Diagrama de actividad: configuración de horarios y ausencias....73
Figura 17. Diagrama de actividad: programación de cita con estimación predictiva....74
Figura 18. Diagrama de actividad: recomendación de bloques de tiempo....75
Figura 19. Diagrama de actividad: registro de la atención y realimentación del modelo....76
Figura 20. Diagrama de actividad: reprogramación y cancelación de citas....77
Figura 21. Diagrama de actividad: registro del odontograma....78
Figura 22. Diagrama de actividad: elaboración del plan de tratamiento....80
Figura 23. Diagrama de actividad: ejecución del plan de tratamiento....82
Figura 24. Diagrama de actividad: registro de la nota clínica de evolución....84
Figura 25. Diagrama de actividad: registro de abonos y control de saldo....86
Figura 26. Diagrama de actividad: entrenamiento y versionado del modelo predictivo....88
Figura 27. Diagrama de componentes y despliegue del sistema SmileeSync....89
Figura 28. Interfaz de inicio de sesión....91
Figura 29. Interfaz del listado de usuarios....92
Figura 30. Interfaz del formulario de usuario con validaciones activas....92
Figura 31. Interfaz del listado de pacientes....93
Figura 32. Interfaz del formulario de registro de paciente....94
Figura 33. Interfaz de la ficha clínica del paciente....94
Figura 34. Interfaz del catálogo de procedimientos....94
Figura 35. Interfaz del formulario de procedimiento....95
Figura 36. Interfaz del formulario de registro de odontólogo....95
Figura 37. Interfaz del catálogo de odontólogos....96
Figura 38. Interfaz del catálogo de unidades dentales....97
Figura 39. Interfaz del módulo Principal con la agenda semanal (datos de demostración)....98
Figura 40. Interfaz de consulta de recomendación de espacios sobre el calendario (datos de demostración) 99

Figura 41. Interfaz de programación de cita con recomendación del modelo predictivo (datos de demostración).....100
Figura 42. Interfaz de consulta y seguimiento de citas.....101
Figura 43. Interfaz de detalle de la cita con registro de tiempos reales.....102
Figura 44. Interfaz del expediente clínico del paciente (datos de demostración).....103
Figura 45. Interfaz del odontograma con el registro de hallazgos por pieza.....104
Figura 46. Interfaz de la hoja de evolución clínica del paciente (datos de demostración).....105
Figura 47. Interfaz del listado de planes de tratamiento (datos de demostración).....106
Figura 48. Interfaz del detalle del plan de tratamiento con fases, renglones y sesiones sugeridas (datos de demostración).....107
Figura 49. Interfaz del control de cobros y saldos por plan (datos de demostración).....108
Figura 50. Pantalla de inicio de sesión del sistema.....125
Figura 51. Menú principal del sistema con el calendario semanal de la clínica (datos de demostración)128
Figura 52. Panel de reportes e indicadores del sistema (datos de demostración).....154
Figura 53. Panel del módulo de Inteligencia con las métricas del modelo y el histórico de versiones (datos de demostración).....157
Figura 54. Diagrama entidad-relación completo del sistema SmileeSync (vista ampliada).....174

Índice de tablas

Tabla 1. Módulos y procesos del sistema SmileeSync.....16
Tabla 2. Herramientas tecnológicas utilizadas en el desarrollo del sistema.....19
Tabla 3. Usuarios involucrados en el proyecto y sus responsabilidades.....21
Tabla 4. Descripción del problema.....23
Tabla 5. Resumen de requerimientos funcionales por módulo.....24
Tabla 6. Requerimiento REQ-SEG-01 — Inicio de sesión.....25
Tabla 7. Requerimiento REQ-SEG-02 — Mantenimiento de usuarios.....26
Tabla 8. Requerimiento REQ-SEG-03 — Control de acceso por rol.....27
Tabla 9. Requerimiento REQ-SEG-04 — Historial de auditoría.....27
Tabla 10. Requerimiento REQ-SEG-05 — Recuperación de contraseña.....28
Tabla 11. Requerimiento REQ-PAC-01 — Mantenimiento de pacientes.....28
Tabla 12. Requerimiento REQ-PAC-02 — Antecedentes médicos.....29
Tabla 13. Requerimiento REQ-CAT-01 — Administración de especialidades.....29
Tabla 14. Requerimiento REQ-CAT-02 — Administración de procedimientos.....30
Tabla 15. Requerimiento REQ-CAT-03 — Administración de odontólogos.....31
Tabla 16. Requerimiento REQ-CAT-04 — Administración de unidades dentales.....31
Tabla 17. Requerimiento REQ-AGE-01 — Horarios de disponibilidad.....32
Tabla 18. Requerimiento REQ-AGE-02 — Ausencias puntuales.....32
Tabla 19. Requerimiento REQ-AGE-03 — Programación de citas.....33
Tabla 20. Requerimiento REQ-AGE-04 — Reprogramación y cancelación.....34
Tabla 21. Requerimiento REQ-AGE-05 — Registro de tiempos reales.....34
Tabla 22. Requerimiento REQ-AGE-06 — Calendario operativo.....35
Tabla 23. Requerimiento REQ-EXP-01 — Odontograma.....36
Tabla 24. Requerimiento REQ-EXP-02 — Notas clínicas de evolución.....37
Tabla 25. Requerimiento REQ-EXP-03 — Plan de tratamiento.....37

Tabla 26. Requerimiento REQ-EXP-04 — Planificación del tratamiento en sesiones.....38
Tabla 27. Requerimiento REQ-COB-01 — Presupuesto del tratamiento.....38
Tabla 28. Requerimiento REQ-COB-02 — Registro de abonos.....39
Tabla 29. Requerimiento REQ-COB-03 — Estado de cuenta.....40
Tabla 30. Requerimiento REQ-INT-01 — Estimación de duración.....40
Tabla 31. Requerimiento REQ-INT-02 — Recomendación de bloques de tiempo.....41
Tabla 32. Requerimiento REQ-INT-03 — Administración de versiones del modelo.....42
Tabla 33. Requerimiento REQ-REP-01 — Indicadores operativos.....42
Tabla 34. Requerimiento REQ-REP-02 — Reporte de utilización.....43
Tabla 35. Requerimientos no funcionales del sistema.....44
Tabla 36. Referencias empleadas en el levantamiento de requerimientos.....44
Tabla 37. Cuadro de aceptación del proyecto.....45
Tabla 38. Comparación entre la infraestructura disponible y la sugerida.....48
Tabla 39. Requerimientos de hardware.....49
Tabla 40. Requerimientos de software.....49
Tabla 41. Recurso humano y tiempos estimados del proyecto.....50
Tabla 42. Presupuesto estimado del proyecto (en quetzales).....51
Tabla 43. Fases de desarrollo del sistema.....53
Tabla 44. Especificación del caso de uso CU-01: Iniciar sesión.....58
Tabla 45. Especificación del caso de uso CU-10: Programar cita con predicción de duración.....58
Tabla 46. Elementos del diseño de base de datos y su trazabilidad.....60
Tabla 47. Especificación de campos de la pantalla de inicio de sesión.....91
Tabla 48. Especificación de campos del formulario de usuario.....93
Tabla 49. Especificación de campos del formulario de programación de cita.....100
Tabla 50. Simbología del odontograma y su correspondencia con el hallazgo registrado.....104
Tabla 51. Reportes generados por el sistema.....109
Tabla 52. Conjunto de pruebas de los requerimientos críticos.....109
Tabla 53. Variables independientes del estudio.....111
Tabla 54. Variables dependientes del estudio.....111
Tabla 55. Criterios de inclusión y exclusión de los registros.....112
Tabla 56. Plataforma y entorno de desarrollo del sistema.....116
Tabla 57. Bibliotecas incorporadas al sistema, su función y la justificación de su selección.....118
Tabla 58. Herramientas de apoyo al desarrollo y a la documentación del proyecto.....119
Tabla 59. Organización del código fuente por aplicación del proyecto.....120
Tabla 60. Riesgos considerados y mecanismos de seguridad implementados.....124
Tabla 61. Archivos y funciones que intervienen en el inicio de sesión.....126
Tabla 62. Matriz de acceso por rol y por módulo implementada en el sistema.....130
Tabla 63. Inventario de funciones del módulo Principal y de la agenda.....132
Tabla 64. Inventario de funciones del módulo de Seguridad y Usuarios.....141
Tabla 65. Inventario de funciones de los módulos de Pacientes y de Catálogo.....143
Tabla 66. Inventario de funciones del módulo de Expediente Clínico.....144
Tabla 67. Inventario de funciones de los módulos de Plan de Tratamiento y de Cobros.....146
Tabla 68. Resultado del entrenamiento de los dos algoritmos implementados sobre 446 atenciones registradas..... 148

Tabla 69. Peso de cada variable en el modelo de bosque aleatorio y procedencia del dato.....149
Tabla 70. Inventario de funciones del módulo de Inteligencia.....152
Tabla 71. Inventario de funciones del módulo de Reportes.....152
Tabla 72. Conjuntos de verificación automatizada exigidos como condición previa a las pruebas de implementación.....159
Tabla 73. Usuarios creados para las pruebas de implementación.....160
Tabla 74. Datos de demostración cargados para las pruebas de implementación.....160
Tabla 75. Guion de acompañamiento: preguntas previstas de la recepción y sus respuestas.....161
Tabla 76. Guion de acompañamiento: preguntas previstas de los odontólogos y sus respuestas.....162
Tabla 77. Guion de acompañamiento: preguntas previstas del administrador y sus respuestas.....163
Tabla 78. Ficha de registro de las sesiones de prueba de implementación.....163
Tabla 79. Sesiones de prueba de implementación previstas, con su objetivo y los aspectos por observar 164
Tabla 80. Hallazgos derivados del uso del sistema, correcciones aplicadas y verificaciones agregadas166
Tabla 81. Diccionario de datos del sistema.....175

Introducción

La gestión eficiente de las citas constituye uno de los pilares del funcionamiento operativo de cualquier clínica de atención ambulatoria. En el ámbito de la odontología, este proceso adquiere una complejidad adicional debido a la diversidad de procedimientos que se realizan y a la variabilidad en los tiempos de atención que requiere cada paciente. En la Clínica Dental Smilee, ubicada en el departamento de Santa Rosa, Guatemala, la asignación de tiempos en la agenda se realiza actualmente con base en promedios generales, sin considerar variables que inciden directamente en la duración real de los procedimientos, como el tipo de tratamiento, el historial del paciente o el odontólogo responsable.

SmileeSync es un sistema web orientado a administrar la operación diaria de la clínica y a controlar de forma integral la agenda de citas. El sistema centraliza la información de usuarios, pacientes, procedimientos odontológicos y citas, y ofrece los procesos administrativos necesarios para registrar, consultar y actualizar dicha información. Como componente distintivo, incorpora un módulo inteligente que estima la duración probable de cada procedimiento y recomienda automáticamente los bloques de tiempo más adecuados para programar cada consulta, apoyando al personal administrativo en la planificación de la jornada.

Desde la perspectiva de la Ingeniería en Sistemas de Información, el proyecto representa una aplicación concreta del diseño de sistemas informáticos, la modelación de bases de datos y el análisis de datos aplicados a un problema real del sector salud en el contexto guatemalteco. Más allá de la automatización de tareas, el sistema busca transformar una agenda estática en una herramienta dinámica que apoye la toma de decisiones basada en información histórica.

El presente documento corresponde a la segunda entrega del proyecto y se organiza en tres capítulos. El Capítulo 1, Marco Teórico, presenta el planteamiento del problema y los fundamentos teóricos del componente predictivo. El Capítulo 2, Marco Conceptual, expone los trabajos relacionados con la investigación y describe los conceptos involucrados en el desarrollo del software. El Capítulo 3, Marco Metodológico, desarrolla el análisis y el diseño del sistema: los requerimientos y su delimitación, el análisis de la solución con sus estudios de factibilidad, las fases de desarrollo, el diseño mediante UML y el diseño de las interfaces, con el propósito de situar al lector en el contexto tecnológico y metodológico de la propuesta.

Objetivos

Objetivo general

Desarrollar SmileeSync, un sistema web para la gestión de citas y de la agenda de la Clínica Dental Smilee, construido con el lenguaje de programación Python, el framework Django y el sistema gestor de bases de datos PostgreSQL, que integre un modelo predictivo de la duración de los procedimientos y un sistema de recomendación de bloques de tiempo basados en datos históricos, con el fin de optimizar la asignación de los tiempos de atención.

Objetivos específicos

1. Analizar los procesos administrativos y los datos históricos de citas de la Clínica Dental Smilee, con el fin de identificar los requerimientos funcionales y no funcionales del sistema y determinar los factores que inciden en la duración real de los procedimientos odontológicos.
2. Diseñar la arquitectura en tres capas del sistema web y el modelo de datos relacional que soporte la gestión de usuarios, pacientes, procedimientos y agenda, así como el registro de los tiempos reales de atención, implementándolo en el sistema gestor de bases de datos PostgreSQL bajo criterios de normalización e integridad referencial.
3. Desarrollar los módulos del sistema mediante el lenguaje Python y el framework Django, aplicando el patrón Modelo-Vista-Plantilla y las operaciones CRUD con borrado lógico e historial de auditoría, de manera que se garantice la integridad y la trazabilidad de la información registrada.
4. Construir un modelo predictivo que estime la duración de los procedimientos odontológicos mediante las bibliotecas scikit-learn y XGBoost, comparando los algoritmos de regresión lineal múltiple, Random Forest y XGBoost, e integrarlo a un sistema de recomendación que sugiera los bloques de tiempo más adecuados para la agenda.
5. Evaluar el desempeño del sistema contrastando los tiempos estimados por el modelo predictivo con el método tradicional de bloques fijos utilizado por la clínica, mediante las métricas de Error Absoluto Medio (MAE), Raíz del Error Cuadrático Medio (RMSE) y Coeficiente de Determinación (R^2).

Hipótesis

Dado el carácter descriptivo-correlacional de la investigación, las hipótesis se plantean en forma de preguntas que orientan el desarrollo del proyecto y que serán respondidas en las etapas posteriores.

Pregunta principal

¿En qué medida un sistema web que incorpora un modelo predictivo basado en datos históricos mejora la precisión de la asignación de tiempos en la agenda de la Clínica Dental Smilee frente al método actual de bloques fijos?

Preguntas auxiliares

- ¿Cuáles variables —tipo de procedimiento, odontólogo responsable, día de la semana, franja horaria y número de piezas tratadas— influyen de forma significativa en la duración real de los procedimientos odontológicos?
- ¿Qué algoritmo de regresión —regresión lineal múltiple, Random Forest o XGBoost— ofrece la mayor precisión medida mediante el Error Absoluto Medio (MAE), la Raíz del Error Cuadrático Medio (RMSE) y el Coeficiente de Determinación (R^2)?
- ¿En qué grado el sistema de recomendación reduce el desajuste entre el tiempo asignado en la agenda y el tiempo real de atención?
- ¿Cómo contribuye la centralización de la información en un sistema web al control de la operación diaria y a la toma de decisiones administrativas de la clínica?

Capítulo 1. Marco Teórico

1.1 Planteamiento del problema

La Clínica Dental Smilee asigna los tiempos de atención de su agenda utilizando promedios generales y criterios empíricos derivados de la experiencia del personal administrativo. Este método no considera variables que afectan directamente la duración real de los procedimientos, como el tipo de tratamiento, el historial clínico del paciente o el odontólogo responsable. Como consecuencia, se producen retrasos acumulados, tiempos muertos en la agenda, baja utilización de los recursos disponibles y una experiencia menos satisfactoria para los pacientes.

1.1.1 Antecedentes del problema

En el sector salud, los procedimientos y servicios que se ofrecen no son lo único relevante para el correcto funcionamiento de una clínica; el área administrativa cumple un papel igualmente importante, pues gestiona inventarios, recursos humanos, atención a pacientes y servicios tercerizados. Uno de los elementos que conecta la parte administrativa con los servicios prestados y con la satisfacción del paciente es la gestión de las citas, un tema complejo debido a que cada consulta puede incluir distintos procedimientos y a que cada paciente puede requerir tiempos de atención diferentes.

El principal problema en la asignación de espacios en la agenda radica en el uso de promedios generales que no consideran la variabilidad real de los procedimientos. Esta aproximación simplificada puede traducirse en esperas acumuladas a lo largo de la jornada o, en sentido contrario, en bloques de tiempo asignados por encima de lo requerido, generando espacios improductivos que repercuten en los ingresos de la clínica. Gupta y Denton (2008) identificaron que predecir con exactitud cuánto durará cada consulta constituye uno de los mayores retos operativos en la gestión de agendas médicas, puesto que dicha incertidumbre compromete tanto la eficiencia del sistema como la experiencia del paciente.

En el contexto de las clínicas dentales, esta problemática se intensifica por la diversidad de procedimientos que se atienden. Una cita puede corresponder a una limpieza de rutina o a intervenciones de mayor complejidad, como tratamientos de conducto, extracciones quirúrgicas o restauraciones, cada una con requerimientos de tiempo sustancialmente distintos. Klassen y Rohleder (1996) documentaron que los sistemas de programación que no incorporan la

variabilidad real de los tiempos de atención tienden a acumular retrasos de forma sistemática, deteriorando el rendimiento general de la clínica.

El avance de la tecnología ha permitido incorporar herramientas de análisis de datos en la toma de decisiones de distintos sectores, incluido el de la salud. Diversas investigaciones han explorado el desarrollo de modelos analíticos orientados a estimar con mayor precisión la duración de las consultas y a respaldar la planificación de citas (Gupta y Denton, 2008). No obstante, en la mayoría de las clínicas privadas persiste el uso de métodos tradicionales que prescinden de la información histórica disponible y de las herramientas tecnológicas que podrían facilitar una asignación de tiempos más precisa.

1.1.2 Problemática actual

Actualmente, la Clínica Dental Smilee gestiona su agenda de forma manual o semi-manual, asignando a cada cita un bloque de tiempo estándar independientemente del tipo de procedimiento y del perfil del paciente. Debido a que la clínica inició operaciones recientemente, su información se encuentra dispersa y no ha sido sistematizada de manera que permita analizar el comportamiento histórico de los tiempos de atención.

Esta forma de operar ocasiona varios efectos negativos. En primer lugar, los retrasos acumulativos surgen cuando un procedimiento tarda más de lo previsto y afecta la totalidad de las citas subsiguientes. En segundo lugar, los tiempos muertos representan una pérdida de productividad directamente cuantificable, pues el especialista permanece inactivo durante períodos que podrían destinarse a atender a otros pacientes. En tercer lugar, los tiempos de espera prolongados constituyen uno de los factores de insatisfacción más frecuentes en el sector salud y pueden reducir la fidelización de los pacientes.

Si esta situación continúa sin abordarse, es probable que se mantenga una gestión de citas poco eficiente, caracterizada por retrasos frecuentes, baja utilización del tiempo disponible y menor satisfacción de los pacientes. Por ello surge la necesidad de contar con un sistema web que centralice la información de la clínica y que, mediante el análisis de los datos históricos, estime de manera más precisa la duración de los procedimientos y apoye la planificación de la agenda. El manejo de la información de los pacientes se realizará de forma anonimizada, garantizando la confidencialidad de los datos y el cumplimiento del marco ético correspondiente.

1.2 Fundamentos teóricos

Una vez planteado el problema, esta sección desarrolla los fundamentos teóricos que sustentan el componente predictivo de SmileeSync. Se describe qué es el aprendizaje automático supervisado, cómo funcionan los algoritmos de regresión que se compararán, en qué consiste el proceso de entrenamiento de un modelo y cuáles son las métricas con las que se evaluará su desempeño. Estos fundamentos son la base teórica sobre la que se construirá, en la etapa metodológica, el modelo que estima la duración de los procedimientos odontológicos.

1.2.1 Aprendizaje automático supervisado

El aprendizaje automático es una rama de la inteligencia artificial que estudia algoritmos capaces de aprender patrones a partir de datos y mejorar su desempeño con la experiencia, sin ser programados explícitamente para cada caso (Mitchell, 1997). Dentro de este campo, el aprendizaje supervisado entrena un modelo a partir de un conjunto de datos etiquetados: cada registro contiene un conjunto de variables de entrada, o predictores (X), y el valor conocido de la variable que se desea predecir, u objetivo (y). El algoritmo ajusta sus parámetros internos para aproximar la función que relaciona X con y, minimizando el error entre sus predicciones y los valores reales observados (Hastie et al., 2009).

Los problemas de aprendizaje supervisado se dividen en dos grandes tipos según la naturaleza de la variable objetivo. En la clasificación, la salida es una categoría (por ejemplo, si un paciente asistirá o no a su cita). En la regresión, la salida es un valor numérico continuo. Como el objetivo de este proyecto es estimar la duración de un procedimiento en minutos, el problema es de tipo regresión (Hastie et al., 2009). Una propiedad esencial que se busca en el modelo es la generalización: su capacidad de mantener un buen desempeño al aplicarse a datos nuevos que no vio durante el entrenamiento, y no solo sobre los datos con los que aprendió.

1.2.2 Algoritmos de regresión

La regresión lineal múltiple es el modelo de referencia (baseline) para predecir una variable continua a partir de varias variables independientes. Su forma es $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n + \varepsilon$, donde los coeficientes β se estiman por mínimos cuadrados ordinarios y cada uno cuantifica el cambio esperado en la duración ante un incremento unitario de su variable, manteniendo constantes las demás (Montgomery et al., 2012). Su principal ventaja es la interpretabilidad; su limitación es que asume relaciones lineales y puede quedarse corta cuando

existen interacciones no lineales entre variables como el tipo de procedimiento, el odontólogo y el perfil del paciente.

Un árbol de decisión es un modelo que particiona de forma recursiva el espacio de las variables mediante reglas del tipo “si… entonces…”. En regresión, la predicción de cada hoja es el promedio de los valores de la variable objetivo en esa partición (Hastie et al., 2009). Los árboles son intuitivos y no exigen supuestos sobre la distribución de los datos, pero de manera individual tienden al sobreajuste cuando su profundidad no se limita adecuadamente.

Random Forest es un método de ensamble propuesto por Breiman (2001) que construye numerosos árboles de decisión y combina sus predicciones mediante promedio. Dos mecanismos de aleatorización reducen la correlación entre árboles: el bagging (muestreo aleatorio con reemplazo de los registros) y la selección aleatoria de variables en cada división. El resultado es un modelo con menor varianza que un árbol individual, robusto ante valores atípicos y capaz de capturar relaciones no lineales, además de proporcionar una medida de la importancia de cada variable.

XGBoost (Extreme Gradient Boosting) es una implementación optimizada del gradient boosting desarrollada por Chen y Guestrin (2016). A diferencia de Random Forest, construye los árboles de forma secuencial: cada árbol nuevo se entrena para corregir los errores del conjunto de árboles anterior, minimizando una función de pérdida regularizada que penaliza la complejidad para controlar el sobreajuste (Friedman, 2001). XGBoost suele ofrecer un rendimiento sobresaliente en problemas con datos tabulares como el de esta investigación.

1.2.3 Proceso y tipos de entrenamiento del modelo

El entrenamiento de un modelo predictivo comienza con el preprocesamiento de los datos, etapa que suele concentrar la mayor parte del esfuerzo del proyecto. Incluye la limpieza de registros, el tratamiento de valores atípicos, la imputación de datos faltantes, la codificación de variables categóricas (como el tipo de procedimiento) a un formato numérico y la normalización de las variables continuas (Han et al., 2011). La calidad de esta etapa es determinante: un modelo entrenado con datos mal preparados no rinde bien por más sofisticado que sea el algoritmo.

Para poder medir la capacidad de generalización, el conjunto de datos se divide en dos partes: una de entrenamiento, con la que el modelo aprende, y una de prueba, reservada para evaluarlo con datos que no vio. Una proporción habitual es 80 % para entrenamiento y 20 % para prueba. Con el fin de obtener una estimación más robusta del rendimiento, se emplea la validación

cruzada de k particiones (k-fold): el conjunto de entrenamiento se divide en k subconjuntos y el modelo se entrena y evalúa k veces, usando cada subconjunto una vez como prueba y promediando los resultados (Kohavi, 1995).

Cada algoritmo posee hiperparámetros, que son valores de configuración fijados antes del entrenamiento y que no se aprenden de los datos —por ejemplo, el número de árboles y la profundidad máxima en Random Forest, o la tasa de aprendizaje en XGBoost—. El ajuste de estos hiperparámetros se realiza probando distintas combinaciones mediante búsqueda en cuadrícula acompañada de validación cruzada, y seleccionando la que ofrece el mejor desempeño (Géron, 2019). Durante este proceso se vigilan dos riesgos opuestos: el sobreajuste (overfitting), cuando el modelo memoriza el ruido de los datos de entrenamiento y falla ante datos nuevos, y el subajuste (underfitting), cuando el modelo es demasiado simple para capturar los patrones; el objetivo es hallar el equilibrio adecuado entre sesgo y varianza (Hastie et al., 2009).

1.2.4 Métricas de evaluación

El desempeño de un modelo de regresión se cuantifica con métricas numéricas que comparan los valores predichos con los reales. El Error Absoluto Medio (MAE) es el promedio de las diferencias absolutas entre predicción y valor real; se expresa en las mismas unidades que la variable objetivo —minutos— por lo que es directamente interpretable, y es poco sensible a valores atípicos (Willmott y Matsuura, 2005). La Raíz del Error Cuadrático Medio (RMSE) es la raíz del promedio de los errores al cuadrado; al elevar al cuadrado, penaliza con mayor severidad los errores grandes, lo que resulta útil cuando esas desviaciones son especialmente indeseables. El Coeficiente de Determinación (R^2) indica la proporción de la variabilidad de la duración que el modelo logra explicar, y tiene como valor máximo 1: los valores cercanos a 1 reflejan un mejor ajuste, mientras que un valor negativo sobre datos no vistos indica que el modelo predice peor que el simple promedio de las duraciones (Hastie et al., 2009). En conjunto, estas tres métricas permiten comparar de forma objetiva los algoritmos candidatos y seleccionar el más preciso.

Capítulo 2. Marco Conceptual

2.1 Antecedentes (trabajos relacionados)

La revisión de los trabajos relacionados permite identificar tres líneas convergentes que fundamentan el presente proyecto: la optimización de agendas clínicas mediante modelos estadísticos y de simulación; la aplicación de técnicas de aprendizaje automático supervisado para la predicción de variables clínicas; y la identificación de la odontología privada de mediana escala como un campo con alta demanda de soluciones tecnológicas y escasa investigación aplicada.

Cayirli y Veral (2003) publicaron una revisión sistemática sobre la programación ambulatoria en el sector salud que abarcó más de cinco décadas de investigación. Los autores clasificaron los sistemas de programación según su grado de adaptabilidad y concluyeron que los sistemas dinámicos, que ajustan los tiempos de cita en función de datos históricos, superan de forma consistente a los sistemas estáticos en la utilización del tiempo del profesional y en la satisfacción del paciente. La revisión identificó, además, la escasez de estudios empíricos en clínicas dentales de pequeño tamaño.

Klassen y Rohleder (1996) realizaron uno de los estudios fundacionales sobre la programación de citas con tiempos de servicio estocásticos. Mediante modelos de simulación, demostraron que la asignación de bloques de tiempo variables —ajustados al tipo de procedimiento y al historial del paciente— reduce significativamente los tiempos de espera en comparación con el sistema de bloques fijos, y establecieron que ningún sistema de programación puede considerarse óptimo si no incorpora la variabilidad inherente a los procedimientos clínicos.

Erdogan et al. (2015) revisaron la planificación y programación quirúrgica desde la investigación de operaciones e identificaron que la predicción de la duración de procedimientos es uno de los desafíos más complejos de la gestión clínica. Sus análisis revelaron que los modelos de regresión con variables categóricas de tipo de procedimiento superan a los basados en promedios simples, reduciendo los errores de predicción de forma considerable en entornos con alta variabilidad, hallazgo directamente aplicable al contexto odontológico.

Srinivas y Ravindran (2018) desarrollaron un sistema de optimización de agendas para clínicas de atención ambulatoria utilizando aprendizaje automático y programación estocástica. Sus resultados mostraron que la incorporación de variables sobre el comportamiento histórico del paciente incrementó la precisión de los tiempos estimados de servicio respecto al sistema de

bloques fijos, y que los algoritmos de ensamble, en particular Random Forest, ofrecieron el mejor equilibrio entre precisión y capacidad de generalización.

En el campo odontológico, Hung et al. (2020) revisaron las aplicaciones de inteligencia artificial y concluyeron que, a pesar de los avances en diagnóstico por imagen, la gestión automatizada de agendas en clínicas dentales permanece escasamente explorada, especialmente en América Latina. Por su parte, Tsai et al. (2016) demostraron que los sistemas automatizados de programación y recordatorio de citas, apoyados en registros históricos, permiten identificar patrones de comportamiento que anticipan la inasistencia del paciente. Finalmente, Topol (2019) argumentó que los sistemas basados en aprendizaje automático no reemplazan al profesional de la salud, sino que amplían sus capacidades y liberan tiempo de valor clínico al automatizar procesos repetitivos como la programación de citas.

2.2 Descripción de los conceptos involucrados en el desarrollo del software

Con el fin de situar al lector en el contexto tecnológico del proyecto, a continuación se describen en detalle los conceptos fundamentales que intervienen en el desarrollo de SmileeSync: los sistemas de información, el tipo de aplicación y su arquitectura de software, las bases de datos, las tecnologías web, los paradigmas de programación, la metodología de desarrollo, las técnicas de inteligencia artificial y la infraestructura de nube sobre la que se despliega. La descripción se realiza especificando, para cada concepto, la decisión tecnológica concreta adoptada en el proyecto y su justificación.

2.2.1 Sistemas de información

Un sistema de información es un conjunto integrado de componentes que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, la coordinación y el control dentro de una organización (Laudon y Laudon, 2016). Todo sistema de información se compone de cinco elementos: personas, procesos, datos, software y hardware. En SmileeSync, estos elementos corresponden, respectivamente, al personal administrativo y odontológico de la clínica; a los procesos de registro, programación y atención de citas; a los registros históricos de pacientes y procedimientos; a la aplicación web desarrollada; y a la infraestructura de cómputo sobre la cual se ejecuta. Según su nivel organizacional, SmileeSync se clasifica como un sistema de procesamiento de transacciones (TPS) que, mediante su módulo de reportes y su componente

de inteligencia artificial, incorpora funciones propias de un sistema de apoyo a la toma de decisiones (DSS).

2.2.2 Tipo de aplicación y arquitectura de software

SmileeSync se define como una aplicación web dinámica, de tipo multipágina (Multi-Page Application, MPA), con renderizado del lado del servidor y diseño responsivo. A diferencia de una aplicación web estática, cuyo contenido es fijo, una aplicación dinámica genera sus páginas en tiempo de ejecución a partir de la información almacenada en la base de datos. A diferencia de una aplicación de página única (Single-Page Application, SPA), que traslada gran parte de la lógica al navegador, el enfoque multipágina con renderizado en el servidor reduce la complejidad del cliente, mejora la mantenibilidad y disminuye los costos de infraestructura, lo que resulta apropiado para una clínica de mediana escala (Sommerville, 2011).

La aplicación adopta una arquitectura en tres capas (three-tier), que separa la presentación, la lógica de negocio y el almacenamiento de datos. La capa de presentación se ejecuta en el navegador y se encarga de la interacción con el usuario. La capa de aplicación, alojada en el servidor, contiene la lógica de negocio y coordina los módulos del sistema. La capa de datos administra la persistencia mediante el sistema gestor de bases de datos. Esta separación en capas favorece la escalabilidad, la seguridad y el mantenimiento independiente de cada nivel (Pressman y Maxim, 2015).

Internamente, la capa de aplicación se estructura según el patrón arquitectónico Modelo-Vista-Plantilla (MVT), una variante del clásico Modelo-Vista-Controlador (MVC) empleada por el framework Django. El Modelo define las entidades del dominio y su mapeo a la base de datos; la Vista contiene la lógica que procesa las solicitudes y determina la respuesta; y la Plantilla se encarga de la presentación del contenido en HTML. Adicionalmente, el sistema se organiza de forma modular, de modo que cada módulo —usuarios, pacientes, catálogo, agenda, inteligencia y reportes— constituye una aplicación independiente dentro del proyecto, con sus propios modelos, vistas y plantillas, lo que refuerza la cohesión y reduce el acoplamiento entre componentes.

2.2.3 Base de datos

Una base de datos es una colección organizada de datos relacionados que se almacena de forma estructurada para permitir su acceso, gestión y actualización eficientes. El modelo relacional, propuesto por Codd (1970) y consolidado como estándar de la industria, organiza los

datos en tablas o relaciones compuestas por filas (tuplas) y columnas (atributos), y establece vínculos entre ellas mediante claves primarias y claves foráneas (Elmasri y Navathe, 2016). El diseño de la base de datos parte de un modelo entidad-relación y aplica reglas de normalización con el fin de eliminar la redundancia y evitar anomalías de inserción, actualización y eliminación.

La administración de estas estructuras se realiza a través de un sistema gestor de bases de datos (SGBD), que garantiza la integridad, la concurrencia y la seguridad de la información, y que asegura el cumplimiento de las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad) en el procesamiento de transacciones (Date, 2004). El lenguaje estándar para definir y manipular los datos es SQL (Structured Query Language). Para SmileeSync se utilizará PostgreSQL, un SGBD relacional de código abierto reconocido por su robustez, su cumplimiento del estándar SQL y su capacidad para manejar cargas transaccionales de forma confiable sin costos de licenciamiento (PostgreSQL Global Development Group, 2024).

Sobre la base de datos se implementan las cuatro operaciones fundamentales de manipulación de datos, conocidas como CRUD: Create (insertar), Read (consultar), Update (actualizar) y Delete (eliminar). Cada proceso de los módulos del sistema implementa este conjunto de operaciones. Con el propósito de preservar la integridad y la trazabilidad de la información, el sistema evita la eliminación física de los registros y aplica dos estrategias complementarias. La primera es el borrado lógico, que consiste en cambiar el estado de un registro a “inactivo” mediante una operación de actualización, conservándolo en la tabla. La segunda son las tablas de histórico o de auditoría, en las cuales se inserta una copia del registro antes de cualquier eliminación o cambio relevante, guardando la fecha, el usuario y la operación realizada. Estas prácticas permiten auditar las modificaciones, recuperar información y sostener la base de datos histórica que alimenta al modelo predictivo.

2.2.4 Tecnologías web

Las tecnologías web son el conjunto de estándares, lenguajes y herramientas que permiten construir aplicaciones accesibles a través de un navegador. Estas aplicaciones se basan en el modelo cliente-servidor, en el que el cliente —el navegador— envía solicitudes mediante el protocolo HTTP y el servidor procesa dichas solicitudes y devuelve las respuestas correspondientes (Sommerville, 2011). El protocolo HTTP define métodos como GET, POST, PUT y DELETE, que se corresponden de manera natural con las operaciones CRUD de la base de datos.

En el lado del cliente (front-end) se emplean tres tecnologías complementarias: HTML5 para estructurar el contenido, CSS3 para definir su presentación y JavaScript para dotar de interactividad a la interfaz. Para lograr un diseño responsivo —que se adapte a computadoras, tabletas y teléfonos— se utilizará el marco de estilos Bootstrap, que provee un sistema de rejilla y componentes visuales predefinidos. En el lado del servidor (back-end), el desarrollo se apoyará en Django, un framework de alto nivel escrito en Python que promueve el desarrollo rápido y el diseño limpio (Django Software Foundation, 2024). Django incorpora de forma nativa un sistema de autenticación y de gestión de usuarios, roles y permisos; un mapeador objeto-relacional (ORM) que abstrae el acceso a la base de datos y permite trabajar con objetos en lugar de sentencias SQL; un motor de plantillas; y mecanismos de protección frente a vulnerabilidades comunes como la inyección SQL, el cross-site scripting (XSS) y la falsificación de peticiones (CSRF).

Para la comunicación con el módulo de inteligencia artificial se adoptará el estilo arquitectónico REST (Representational State Transfer), que define principios para el diseño de servicios web basados en recursos identificados por URL y manipulados mediante los métodos de HTTP (Fielding, 2000). De este modo, el componente predictivo, desarrollado en Python, expone sus estimaciones a la aplicación web a través de una interfaz de programación (API).

2.2.5 Paradigmas de programación

Un paradigma de programación es un enfoque o estilo para estructurar y organizar el código de un programa. Entre los principales se encuentran el paradigma imperativo, que describe el cómputo como una secuencia de instrucciones que modifican el estado del programa; el paradigma orientado a objetos, que organiza el software en torno a objetos que encapsulan datos y comportamiento y que se apoya en los principios de encapsulamiento, herencia y polimorfismo; y el paradigma funcional, que concibe el cómputo como la evaluación de funciones y evita el estado mutable (Sebesta, 2016). El desarrollo de SmileeSync adopta principalmente el paradigma orientado a objetos, que facilita el modelado de las entidades del dominio —usuarios, pacientes, procedimientos y citas— como clases con atributos y métodos, en correspondencia directa con las tablas de la base de datos a través del ORM.

El lenguaje seleccionado para el proyecto es Python, un lenguaje de alto nivel, interpretado y de propósito general que se ha consolidado como estándar para el desarrollo web y para la ciencia de datos, gracias a su sintaxis clara y a la riqueza de su ecosistema de bibliotecas (Van Rossum y Drake, 2009). Python es un lenguaje multiparadigma que admite los estilos imperativo, orientado

a objetos y funcional, lo que permite emplear el enfoque más adecuado para cada componente y, a la vez, integrar de manera natural el desarrollo de la aplicación web con el análisis de datos y el aprendizaje automático, evitando la fragmentación tecnológica del proyecto.

2.2.6 Metodología de desarrollo del software

El desarrollo de SmileeSync seguirá una metodología ágil de tipo incremental e iterativo, basada en el marco de trabajo Scrum (Schwaber y Sutherland, 2020). Bajo este enfoque, el sistema se construye en ciclos cortos denominados sprints, al final de los cuales se entrega un incremento funcional del producto. Cada incremento corresponde, en este proyecto, a uno o varios módulos completos —por ejemplo, el módulo de seguridad y usuarios en el primer incremento— lo que permite validar tempranamente la funcionalidad con el personal de la clínica e incorporar retroalimentación de forma continua. La elección de una metodología incremental responde a la naturaleza evolutiva del proyecto y a la necesidad de mostrar avances verificables en cada etapa del curso.

Para el componente de ciencia de datos se adoptará, de manera complementaria, la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining), que define un ciclo iterativo de seis fases: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación e implementación (Chapman et al., 2000). CRISP-DM es especialmente adecuada para proyectos en los que la calidad y disponibilidad de los datos no se conocen con exactitud hasta iniciar la recolección, pues permite regresar a fases anteriores cuando los resultados lo requieren. La combinación de Scrum para el desarrollo del software y CRISP-DM para el modelo predictivo articula, en un mismo proyecto, la construcción de la aplicación web y la del componente inteligente.

2.2.7 Inteligencia artificial y aprendizaje automático

El aprendizaje automático es una rama de la inteligencia artificial que desarrolla algoritmos capaces de aprender patrones a partir de datos y mejorar su desempeño con la experiencia, sin ser programados explícitamente para cada tarea (Mitchell, 1997). Dentro de este campo, el aprendizaje supervisado entrena un modelo con datos etiquetados para aprender la función que relaciona las variables de entrada con una variable de salida; cuando dicha salida es un valor numérico continuo, como la duración de un procedimiento, el problema se aborda mediante técnicas de regresión (Hastie et al., 2009).

El proyecto contempla la comparación de tres algoritmos: la regresión lineal múltiple, utilizada como modelo de referencia por su interpretabilidad; Random Forest, un método de ensamble que combina múltiples árboles de decisión para reducir la varianza y capturar relaciones no lineales (Breiman, 2001); y XGBoost, una implementación optimizada del gradient boosting que ha demostrado un rendimiento sobresaliente con datos tabulares (Chen y Guestrin, 2016). El desempeño de estos modelos se evaluará con métricas como el Error Absoluto Medio (MAE), la Raíz del Error Cuadrático Medio (RMSE) y el Coeficiente de Determinación (R^2). El desarrollo de este componente se apoyará en las bibliotecas de Python scikit-learn, pandas y NumPy.

A partir de las estimaciones del modelo predictivo, un sistema de recomendación sugiere elementos o decisiones relevantes con base en datos históricos y en la identificación de patrones (Ricci et al., 2011). En SmileeSync, el sistema de recomendación transforma la duración estimada de cada procedimiento en una sugerencia accionable —el bloque de tiempo más adecuado para la cita— considerando la disponibilidad de la agenda y las restricciones operativas de la clínica.

2.2.8 Arquitectura y módulos del sistema

SmileeSync se organiza en la arquitectura de tres capas descrita anteriormente —presentación, aplicación y datos— complementada por un motor de inteligencia artificial. La Figura 1 muestra la arquitectura general del sistema, en la que la capa de presentación corresponde al navegador web (HTML5, CSS3 y Bootstrap), la capa de aplicación aloja los módulos desarrollados con Django bajo el patrón MVT, y la capa de datos se gestiona mediante PostgreSQL. El motor de inteligencia artificial, desarrollado en Python, se comunica con la aplicación para proveer las predicciones de duración.

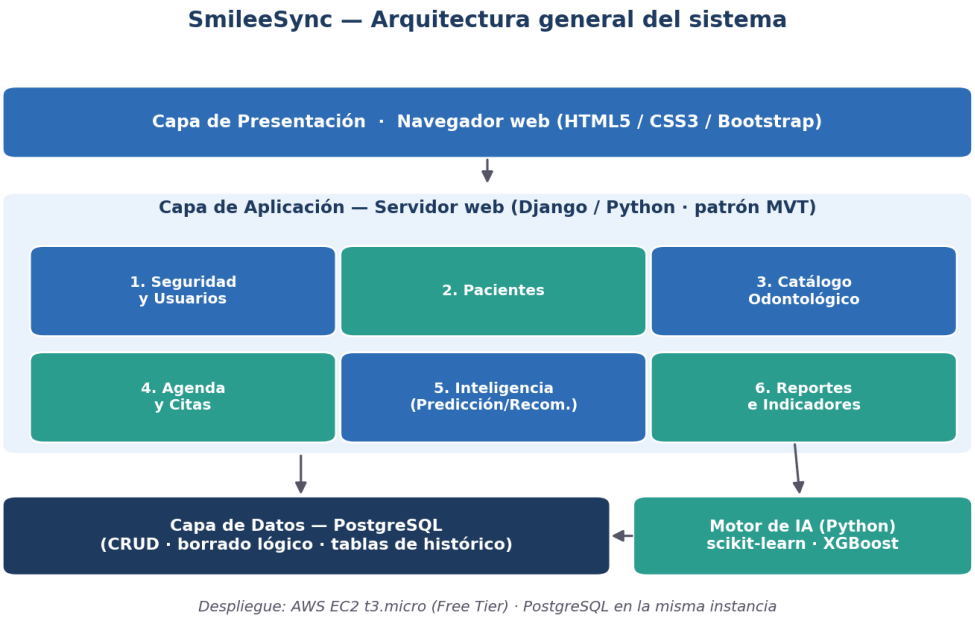


Figura 1: Arquitectura general del sistema SmileeSync **Fuente:** elaboración propia.

El sistema se compone de ocho módulos, cada uno con un mínimo de tres procesos, y cada proceso implementa las operaciones CRUD con borrado lógico e historial. Dos módulos son obligatorios —Seguridad y Usuarios, y Reportes e Indicadores— y se complementan con los de Pacientes, Catálogo Odontológico, Agenda y Citas, Expediente Clínico, Cobros e Inteligencia. La Tabla 1 resume los módulos y sus procesos.

Módulo	Procesos (cada uno con CRUD)	Estrategia de eliminación
1. Seguridad y Usuarios (obligatorio)	Gestión de usuarios; gestión de roles y permisos; control de acceso por módulo; recuperación de contraseña; consulta del historial de auditoría.	Borrado lógico (estado activo o inactivo) + tabla de histórico
2. Pacientes	Gestión de pacientes; gestión de antecedentes médicos; consulta de la ficha clínica y del historial de atenciones.	Borrado lógico + tabla de histórico
3. Catálogo Odontológico	Gestión de especialidades; gestión de procedimientos y sus tarifas; gestión de odontólogos; gestión de unidades dentales.	Borrado lógico (estado activo o inactivo)
4. Agenda y Citas (módulo principal)	Gestión de citas; gestión de horarios de disponibilidad; registro de ausencias puntuales; registro de tiempos reales de atención; calendario operativo.	Borrado lógico + tabla de histórico
5. Expediente Clínico	Registro del odontograma por pieza y cara; notas clínicas de evolución; elaboración y seguimiento del plan de tratamiento.	Registro de solo agregado (odontograma) y borrado lógico + histórico
6. Cobros	Presupuesto del tratamiento; registro de abonos; consulta del estado de cuenta por paciente.	Anulación lógica del abono + tabla de histórico
7. Inteligencia (Predicción y Recomendación)	Gestión del conjunto de entrenamiento; estimación de duración; recomendación de bloques; versionado del modelo.	Tabla de histórico (versiones del modelo)
8. Reportes e Indicadores (obligatorio)	Indicadores de la agenda; reportes de utilización; comparativo de precisión del modelo.	Solo consulta

Tabla 1: Módulos y procesos del sistema SmileeSync **Fuente:** elaboración propia.

2.2.9 Computación en la nube e infraestructura de despliegue

La computación en la nube es un modelo que permite el acceso ubicuo, cómodo y bajo demanda, a través de la red, a un conjunto compartido de recursos de cómputo configurables —como servidores, almacenamiento y aplicaciones— que pueden aprovisionarse y liberarse rápidamente con un mínimo esfuerzo de gestión (Mell y Grance, 2011). Se distinguen tres modelos de servicio: la infraestructura como servicio (IaaS), que ofrece recursos básicos como servidores virtuales y almacenamiento; la plataforma como servicio (PaaS), que provee entornos listos para desarrollar y ejecutar aplicaciones; y el software como servicio (SaaS), en el que el usuario consume una aplicación completa a través del navegador. SmileeSync se despliega bajo el modelo IaaS, aprovechando la capa gratuita de un proveedor de nube para minimizar los costos.

Un servidor es una computadora que provee servicios o recursos a otros equipos, denominados clientes, a través de una red. En la nube, estos servidores suelen ofrecerse como máquinas virtuales o instancias: entornos de cómputo aislados que emulan una computadora física y que se crean y eliminan bajo demanda. Para este proyecto se utiliza una instancia de cómputo elástico (Amazon EC2) que ejecuta el sistema operativo Ubuntu. Cada instancia se identifica en la red mediante una dirección IP; en particular se emplea una dirección IP elástica, que es una IP pública fija asociada a la instancia y que no cambia aunque el servidor se reinicie, lo que garantiza un punto de acceso estable.

Dentro del servidor, el software se organiza en dos componentes. El servidor web —Nginx— recibe las solicitudes HTTP de los usuarios, entrega los archivos estáticos y actúa como proxy inverso, redirigiendo las peticiones dinámicas hacia la aplicación. El servidor de aplicaciones —Gunicorn— ejecuta el código Python de Django a través de la interfaz WSGI (Web Server Gateway Interface), que es el estándar de comunicación entre servidores web y aplicaciones Python. Esta separación permite que Nginx atienda eficientemente las conexiones mientras Gunicorn se concentra en procesar la lógica de la aplicación.

Finalmente, el proyecto incorpora una práctica de integración y despliegue continuos (CI/CD, por sus siglas en inglés). La integración continua ejecuta de forma automática las pruebas del sistema cada vez que se registran cambios en el código, mientras que el despliegue continuo publica esos cambios en el servidor de manera automática una vez superadas las pruebas. En SmileeSync este flujo se implementa con GitHub Actions, lo que reduce los errores manuales y agiliza la actualización del sistema en producción (Sommerville, 2011).

Como síntesis de los conceptos desarrollados en esta sección, la Tabla 2 consolida las herramientas tecnológicas seleccionadas para la construcción del sistema, señalando la función concreta que cumple cada una dentro de SmileeSync. Esta correspondencia entre el concepto teórico y la decisión tecnológica adoptada constituye el punto de partida del análisis y el diseño que se desarrollan en el capítulo siguiente.

Capa o componente	Herramienta seleccionada	Función dentro de SmileeSync
Lenguaje de programación	Python 3.12	Constituye el lenguaje base tanto de la aplicación web como del componente de aprendizaje automático, lo que evita la fragmentación tecnológica del proyecto al emplear un mismo lenguaje en ambos frentes de desarrollo.
Framework de desarrollo web	Django 5.0	Implementa el patrón Modelo-Vista-Plantilla, el mapeador objeto-relacional, el sistema de autenticación y gestión de roles, y las protecciones frente a inyección SQL, secuencias de comandos en sitios cruzados (XSS) y falsificación de peticiones (CSRF).
Base de datos en producción	PostgreSQL 16	Almacena las dieciséis tablas del modelo relacional y garantiza el cumplimiento de las propiedades ACID en las transacciones de la agenda, evitando que dos citas se programen simultáneamente sobre la misma franja horaria.
Base de datos en desarrollo	SQLite	Permite ejecutar y probar el sistema en el equipo del desarrollador sin necesidad de instalar y administrar un servidor de base de datos independiente.
Interfaz de usuario	HTML5, CSS3 y Bootstrap 5.3	Definen la estructura, la presentación y el diseño responsivo de las pantallas mediante un sistema de rejilla y componentes visuales predefinidos, sin requerir un proceso de compilación ni un framework de JavaScript.
Aprendizaje automático	scikit-learn, XGBoost, pandas y NumPy	Realizan la preparación de los datos, el entrenamiento y la comparación de los algoritmos de regresión, y la exportación del modelo entrenado al formato serializado que consume la aplicación.
Servidor web	Nginx	Recibe las solicitudes HTTP de los usuarios, entrega directamente los archivos estáticos y actúa como proxy inverso hacia el servidor de aplicaciones.
Servidor de aplicaciones	Gunicorn	Ejecuta el código Python de Django a través de la interfaz WSGI, que constituye el estándar de comunicación entre servidores web y aplicaciones escritas en Python.
Infraestructura de despliegue	Amazon EC2 (Ubuntu 24.04) y AWS CDK	Provee el servidor bajo el modelo de infraestructura como servicio, definido mediante infraestructura como código para que el entorno sea reproducible y versionable.
Control de versiones e integración continua	Git, GitHub y GitHub Actions	Versionan el código fuente y automatizan la ejecución de las pruebas y el despliegue del sistema en el servidor de producción tras

Capa o componente	Herramienta seleccionada	Función dentro de SmileeSync
		cada cambio aprobado.

Tabla 2: Herramientas tecnológicas utilizadas en el desarrollo del sistema Fuente: elaboración propia.

Capítulo 3. Marco Metodológico: Análisis y Diseño

El presente capítulo desarrolla el análisis y el diseño del sistema SmileeSync. En primer término se identifica al cliente y se delimita el alcance de la solución mediante el documento de especificación de requerimientos y criterios de aceptación de software (DERCAS). Posteriormente se analiza la solución propuesta desde la perspectiva de la arquitectura de software, la infraestructura y el estudio de factibilidad, y se describen las fases de desarrollo con su respectivo cronograma. La segunda parte del capítulo presenta el diseño del sistema mediante los diagramas del Lenguaje Unificado de Modelado (UML) y el diseño de las interfaces de usuario de los módulos que componen la aplicación.

3.1 Análisis

El análisis constituye la etapa en la que se traduce la necesidad del cliente en una especificación técnica verificable. Siguiendo el enfoque propuesto por Pressman y Maxim (2015), esta etapa comprende la identificación de los interesados, la elicitación de los requerimientos, la delimitación del alcance y la evaluación de la viabilidad de la solución antes de comprometer recursos en la construcción del software.

3.1.1 Requerimientos

3.1.1.1 Cliente

El cliente del proyecto es la Clínica Dental Smilee, una clínica odontológica privada de mediana escala ubicada en el departamento de Santa Rosa, Guatemala. La clínica ofrece servicios de odontología general y de especialidades, atendiendo procedimientos que van desde profilaxis de rutina hasta tratamientos de conducto, extracciones y tratamientos de ortodoncia. Al tratarse de una organización de reciente creación, su información operativa se encuentra dispersa y no ha sido sistematizada, lo que impide analizar el comportamiento histórico de los tiempos de atención.

La necesidad expresada por el cliente se resume en tres puntos: centralizar en un solo sistema la información que actualmente se encuentra dispersa; reducir los retrasos y los tiempos muertos que genera la asignación de bloques de tiempo fijos; y contar con información objetiva sobre el desempeño de la agenda que respalde la toma de decisiones administrativas. La Tabla 3 detalla a los usuarios involucrados en el proyecto y las responsabilidades que asume cada uno.

Nombre o perfil	Rol	Responsabilidades
Hee Sung Lee	Analista, diseñador y desarrollador del sistema	Levantamiento de requerimientos, análisis, diseño, construcción, pruebas, despliegue y capacitación del sistema.
Propietario de la clínica	Patrocinador del proyecto y administrador del sistema	Provee la información del negocio, valida los requerimientos, aprueba las entregas y administra los usuarios del sistema una vez implementado.
Personal de recepción	Usuario operativo (recepcionista)	Registra pacientes, programa, reprograma y cancela citas; es el usuario de mayor interacción con la agenda.
Odontólogos	Usuario operativo (odontólogo)	Consulta la agenda diaria y la ficha clínica del paciente, y registra la duración real de cada atención.
Asistentes dentales	Usuario operativo (asistente)	Apoya el registro de pacientes y la consulta de la agenda del día.
Ing. Carlos Eduardo Hernández Herrera	Asesor del trabajo de graduación	Supervisa el cumplimiento metodológico y académico de cada etapa del proyecto.

Tabla 3: Usuarios involucrados en el proyecto y sus responsabilidades Fuente: elaboración propia.

3.1.1.2 Delimitación (DERCAS)

El documento de especificación de requerimientos y criterios de aceptación de software (DERCAS) delimita qué construye el sistema, qué queda fuera de su alcance y bajo qué condiciones se considera aceptado. La presente sección sigue la estructura del DERCAS Visión adoptada por la carrera: antecedentes del problema con su impacto y sus beneficios, descripción de la problemática, alcance, funcionalidad requerida desglosada por módulo, limitantes, requerimientos no funcionales y de documentación, y cuadro de aceptación.

Antecedentes. La Clínica Dental Smilee administra su agenda mediante un libro de citas en papel. La recepción asigna a cada tratamiento un bloque de tiempo estimado a partir de la experiencia del personal, sin distinguir el tipo de procedimiento, el número de piezas por tratar ni el ritmo de trabajo de cada profesional. La clínica no dispone de ningún sistema informático de gestión, de modo que la información de pacientes, procedimientos y atenciones se encuentra dispersa entre el libro de citas, las fichas individuales y el conocimiento no documentado del personal.

Impacto. La ausencia de una herramienta informática que apoye la planificación de la agenda produce los siguientes efectos sobre la operación de la clínica:

- Retrasos acumulados a lo largo de la jornada, originados en bloques de tiempo asignados por debajo de la duración real del procedimiento.

- Tiempos muertos entre atenciones cuando el bloque asignado excede la duración real, con la consiguiente subutilización de los sillones.
- Reprogramación reactiva de citas, que traslada el retraso a los pacientes siguientes y ocupa al personal de recepción en tareas correctivas.
- Imposibilidad de verificar la disponibilidad real de un profesional sin consultar físicamente el libro de citas.
- Redundancia e inconsistencia de los datos del paciente, registrados en más de un lugar y sin un criterio común.
- Ausencia de registro histórico, que impide evaluar el desempeño de la operación o sustentar cualquier mejora en evidencia.

Beneficio. La implantación del sistema aporta a la clínica un conjunto de beneficios directos:

- Centralización en un solo sistema de la información de usuarios, pacientes, catálogo, agenda y expediente clínico.
- Estimación de la duración de cada procedimiento a partir de las atenciones realmente registradas, y no de un promedio general.
- Recomendación automática del bloque de tiempo y de los espacios del calendario donde el procedimiento cabe completo.
- Control de la capacidad real de atención, derivada del número de unidades dentales activas y no solo de la disponibilidad del profesional.
- Trazabilidad de las operaciones mediante historiales de auditoría y borrado lógico en todas las entidades.
- Indicadores de desempeño que permiten evaluar la utilización de la agenda y la precisión del componente predictivo.
- Mejora progresiva de la estimación conforme la clínica acumula atenciones registradas.

Descripción del problema. La Tabla 4 sintetiza la problemática que da origen al proyecto, la población afectada, las consecuencias de no atenderla y las características que debe reunir una solución exitosa.

Aspecto	Descripción
¿Cuál es el problema?	La Clínica Dental Smilee asigna los tiempos de atención de su agenda utilizando promedios generales y criterios empíricos, sin considerar el tipo de tratamiento, el historial del paciente ni el odontólogo responsable. Ello ocasiona retrasos acumulados a lo largo de la jornada, tiempos muertos que reducen la productividad y una experiencia deficiente para el paciente. Adicionalmente, la información de la clínica se encuentra dispersa y no sistematizada, por lo que no existe registro histórico que permita analizar el comportamiento real de los tiempos de atención.
¿A quiénes afecta?	A los pacientes de la clínica, que experimentan esperas prolongadas; a los odontólogos, cuyo tiempo productivo se desaprovecha; al personal de recepción, que reprograma citas de forma reactiva; y al propietario, que carece de información objetiva sobre el desempeño de su operación.
¿Cuál es el impacto de no resolverlo?	Persistencia de retrasos frecuentes, baja utilización del tiempo disponible, pérdida de ingresos por espacios improductivos, insatisfacción del paciente con la consiguiente pérdida de fidelización, y ausencia de datos históricos que impide cualquier mejora sustentada en evidencia.
¿Qué beneficios aporta la solución?	Centralización de la información en un único sistema; estimación de la duración de los procedimientos a partir de datos históricos reales; recomendación automática de bloques de tiempo; trazabilidad completa de las operaciones mediante historiales de auditoría; e indicadores de desempeño que respaldan la toma de decisiones administrativas.
¿Cómo debe ser una solución exitosa?	Un sistema web centralizado, accesible desde los equipos con que ya cuenta la clínica, que administre usuarios, pacientes, catálogo, agenda y expediente clínico con control de acceso por rol, y que incorpore un modelo predictivo capaz de estimar la duración de cada procedimiento y sugerir el bloque de tiempo adecuado, mejorando su precisión conforme se acumulen datos.

Tabla 4: Descripción del problema Fuente: elaboración propia.

Alcance del sistema. SmileeSync comprende ocho módulos funcionales: Seguridad y Usuarios, Pacientes, Catálogo Odontológico, Agenda y Citas, Expediente Clínico, Cobros, Inteligencia y Reportes e Indicadores. El módulo principal del proyecto es el de Agenda y Citas, sobre el cual opera el componente predictivo que constituye el aporte central del trabajo. El sistema se despliega como una aplicación web accesible desde los equipos con que la clínica ya cuenta, sin requerir instalación local.

Exclusiones. No forman parte del alcance de este proyecto la facturación electrónica en línea, que en Guatemala exige la certificación de un proveedor autorizado por la administración tributaria; la integración con equipos de radiología o con dispositivos de diagnóstico por imagen; el control de inventario de insumos odontológicos; el portal de autoagendamiento para pacientes; y la aplicación móvil nativa, puesto que la interfaz web es responsiva y opera desde cualquier navegador.

Funcionalidad requerida. El sistema comprende 29 requerimientos funcionales, que en conjunto agrupan 158 funcionalidades. Cada requerimiento se identifica con una nomenclatura que incorpora el módulo al que pertenece, de la forma REQ-XXX-NN, donde XXX corresponde al prefijo del módulo y NN al correlativo dentro de él. La clasificación por prioridad responde a

un criterio explícito: se califica como alta aquel requerimiento sin el cual el sistema no puede operar, lo que comprende la autenticación, el control de acceso y el ciclo completo de la cita; como media, el que aporta valor operativo comprobable pero puede diferirse sin detener la atención; y como baja, el que mejora la gestión sin condicionar la operación diaria. La Tabla 5 presenta el resumen por módulo y las Tablas 6 a 34 detallan cada requerimiento con sus funcionalidades, su prioridad y su criterio de aceptación.

Módulo	ID	Nombre del requerimiento	Prioridad
Seguridad y Usuarios	REQ-SEG-01	Inicio de sesión	Alta
	REQ-SEG-02	Mantenimiento de usuarios	Alta
	REQ-SEG-03	Control de acceso por rol	Alta
	REQ-SEG-04	Historial de auditoría	Media
	REQ-SEG-05	Recuperación de contraseña	Media
Pacientes	REQ-PAC-01	Mantenimiento de pacientes	Alta
	REQ-PAC-02	Antecedentes médicos	Media
Catálogo Odontológico	REQ-CAT-01	Administración de especialidades	Media
	REQ-CAT-02	Administración de procedimientos	Alta
	REQ-CAT-03	Administración de odontólogos	Alta
	REQ-CAT-04	Administración de unidades dentales	Alta
Agenda y Citas	REQ-AGE-01	Horarios de disponibilidad	Alta
	REQ-AGE-02	Ausencias puntuales	Media
	REQ-AGE-03	Programación de citas	Alta
	REQ-AGE-04	Reprogramación y cancelación	Alta
	REQ-AGE-05	Registro de tiempos reales	Alta
	REQ-AGE-06	Calendario operativo	Alta
Expediente Clínico	REQ-EXP-01	Odontograma	Alta
	REQ-EXP-02	Notas clínicas de evolución	Alta
	REQ-EXP-03	Plan de tratamiento	Alta
	REQ-EXP-04	Planificación del tratamiento en sesiones	Media
Cobros	REQ-COB-01	Presupuesto del tratamiento	Media
	REQ-COB-02	Registro de abonos	Media
	REQ-COB-03	Estado de cuenta	Media
Inteligencia	REQ-INT-01	Estimación de duración	Alta
	REQ-INT-02	Recomendación de bloques de tiempo	Alta
	REQ-INT-03	Administración de versiones del modelo	Media
Reportes e Indicadores	REQ-REP-01	Indicadores operativos	Media
	REQ-REP-02	Reporte de utilización	Baja

Tabla 5: Resumen de requerimientos funcionales por módulo Fuente: elaboración propia.

Módulo de Seguridad y Usuarios. Resuelve la identificación de quien usa el sistema y el alcance de lo que cada perfil puede consultar y modificar. Comprende 5 requerimientos, de los cuales 3 son de prioridad alta.

Id. Requerimiento	REQ-SEG-01	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Inicio de sesión	**Módulo**	Seguridad y Usuarios
Referencia	Caso de uso UC01	**Funcionalidades**	7
No. Func.	**Funcionalidad**		
001	Permitir el acceso únicamente a los usuarios registrados y activos.		
002	Solicitar el correo electrónico como identificador de la cuenta.		
003	Solicitar la contraseña, que se almacena cifrada y nunca en texto claro.		
004	Verificar el correo, la contraseña y el estado del usuario contra la base de datos.		
005	Mostrar un mensaje genérico ante credenciales inválidas, sin revelar si el error estuvo en el correo o en la contraseña.		
006	Crear la sesión y dirigir al usuario al módulo Principal.		
007	Cerrar la sesión a solicitud del usuario e impedir que el navegador recupere las pantallas desde la caché.		

Tabla 6: Requerimiento REQ-SEG-01 — Inicio de sesión Fuente: elaboración propia.

Id. Requerimiento	REQ-SEG-02	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Mantenimiento de usuarios	**Módulo**	Seguridad y Usuarios
Referencia	Caso de uso UC03	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Registrar usuarios con nombres, apellidos, correo electrónico, teléfono y rol.		

002	Validar que el correo electrónico no esté asignado a otro usuario.
003	Exigir que la contraseña cumpla las reglas de longitud y complejidad.
004	Permitir la edición de los datos de un usuario existente.
005	Aplicar borrado lógico: el usuario dado de baja deja de acceder al sistema pero conserva su vínculo con los registros que generó.
006	Registrar la pregunta y la respuesta secretas, esta última cifrada.

Tabla 7: Requerimiento REQ-SEG-02 — Mantenimiento de usuariosFuente: elaboración propia.

Id. Requerimiento	REQ-SEG-03	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Control de acceso por rol	**Módulo**	Seguridad y Usuarios
Referencia	Caso de uso UC04	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Administrar los roles del sistema y los módulos habilitados para cada uno.		
002	Distinguir entre el acceso de consulta y el de escritura mediante un indicador de solo consulta.		
003	Habilitar por separado la escritura del expediente clínico, que corresponde al personal con competencia profesional y no a quien tiene más privilegios administrativos.		
004	Mostrar en el menú lateral únicamente los módulos habilitados para el rol.		
005	Ocultar las acciones de modificación a los roles de consulta.		
006	Denegar el acceso directo por dirección web a una vista no autorizada, con independencia de lo que muestre la interfaz.		

Tabla 8: Requerimiento REQ-SEG-03 — Control de acceso por rol Fuente: elaboración propia.

Id. Requerimiento	REQ-SEG-04	**Clasificación o prioridad**	Media
Nombre Requerimiento	Historial de auditoría	**Módulo**	Seguridad y Usuarios
Referencia	Tablas de histórico	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Registrar una copia del estado previo antes de cada actualización, desactivación o activación.		
002	Conservar en el histórico el identificador, el nombre, la acción, el responsable y la fecha.		
003	Referenciar el registro auditado por identificador entero, sin llave foránea, de modo que el histórico sobreviva al registro que describe.		
004	Permitir la consulta del historial desde la pantalla del módulo.		

Tabla 9: Requerimiento REQ-SEG-04 — Historial de auditoría Fuente: elaboración propia.

Id. Requerimiento	REQ-SEG-05	**Clasificación o prioridad**	Media
Nombre Requerimiento	Recuperación de contraseña	**Módulo**	Seguridad y Usuarios
Referencia	Caso de uso UC02	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Permitir el ingreso del correo electrónico registrado.		
002	Validar que el correo exista y corresponda a un usuario activo.		
003	Presentar la pregunta secreta asociada a la cuenta.		
004	Comparar la respuesta con el valor cifrado almacenado.		

005	Permitir el establecimiento de una contraseña nueva tras la verificación.
006	Aplicar a la contraseña nueva las mismas reglas de seguridad del registro.

Tabla 10: Requerimiento REQ-SEG-05 — Recuperación de contraseña Fuente: elaboración propia.

Módulo de Pacientes. Centraliza la información de las personas atendidas, que hasta ahora se encontraba dispersa entre la agenda en papel y las fichas individuales. Comprende 2 requerimientos, de los cuales 1 son de prioridad alta.

Id. Requerimiento	REQ-PAC-01	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Mantenimiento de pacientes	**Módulo**	Pacientes
Referencia	Caso de uso UC06	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Registrar pacientes con código único, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo y dirección.		
002	Validar que el código del paciente no esté asignado a otro registro.		
003	Permitir la búsqueda por código, nombre, apellido o teléfono.		
004	Permitir la edición de los datos del paciente.		
005	Aplicar borrado lógico, conservando el historial de citas del paciente.		
006	Registrar cada operación en el historial de auditoría del módulo.		

Tabla 11: Requerimiento REQ-PAC-01 — Mantenimiento de pacientes Fuente: elaboración propia.

Id. Requerimiento	REQ-PAC-02	**Clasificación o prioridad**	Media
Nombre Requerimiento	Antecedentes médicos	**Módulo**	Pacientes

Referencia	Caso de uso UC08	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Registrar los antecedentes médicos relevantes del paciente, indicando tipo y descripción.		
002	Presentar los antecedentes vigentes en la ficha clínica del paciente.		
003	Permitir retirar un antecedente mediante borrado lógico.		
004	Mostrar los antecedentes al consultar el expediente, de modo que estén a la vista antes de cualquier procedimiento.		

Tabla 12: Requerimiento REQ-PAC-02 — Antecedentes médicos Fuente: elaboración propia.

Módulo de Catálogo Odontológico. Administra los elementos que la clínica declara una vez y reutiliza en cada cita: especialidades, procedimientos, profesionales y sillones. Comprende 4 requerimientos, de los cuales 3 son de prioridad alta.

Id. Requerimiento	REQ-CAT-01	**Clasificación o prioridad**	Media
Nombre Requerimiento	Administración de especialidades	**Módulo**	Catálogo Odontológico
Referencia	Caso de uso UC10	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Registrar las especialidades odontológicas con nombre y descripción.		
002	Validar que el nombre de la especialidad no se repita.		
003	Permitir la edición y el borrado lógico de cada especialidad.		
004	Impedir la eliminación física de una especialidad vinculada a procedimientos o profesionales.		

Tabla 13: Requerimiento REQ-CAT-01 — Administración de especialidades Fuente: elaboración propia.

Id. Requerimiento	REQ-CAT-02	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Administración de procedimientos	**Módulo**	Catálogo Odontológico
Referencia	Caso de uso UC09	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Registrar procedimientos con código único, nombre, especialidad, nivel de complejidad y duración promedio de referencia.		
002	Registrar el precio de lista con que se presupuestan los planes de tratamiento.		
003	Validar que la duración declarada esté dentro de un rango admisible.		
004	Permitir la edición y el borrado lógico del procedimiento.		
005	Ofrecer únicamente los procedimientos activos al programar una cita.		

Tabla 14: Requerimiento REQ-CAT-02 — Administración de procedimientos Fuente: elaboración propia.

Id. Requerimiento	REQ-CAT-03	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Administración de odontólogos	**Módulo**	Catálogo Odontológico
Referencia	Caso de uso UC11	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Registrar odontólogos con nombres, apellidos, número de colegiado, especialidad y teléfono.		
002	Validar que el número de colegiado no esté asignado a otro profesional.		
003	Permitir vincular al odontólogo con una cuenta de usuario del sistema.		
004	Advertir en el listado cuando un profesional no tiene horarios configurados, pues en ese estado el sistema rechaza todas sus citas.		

005	Permitir la edición y el borrado lógico del profesional.

Tabla 15: Requerimiento REQ-CAT-03 — Administración de odontólogos Fuente: elaboración propia.

Id. Requerimiento	REQ-CAT-04	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Administración de unidades dentales	**Módulo**	Catálogo Odontológico
Referencia	Caso de uso UC22	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Registrar las unidades dentales o sillones con código único, nombre y ubicación.		
002	Administrar las unidades como registros del catálogo, de modo que incorporar un sillón adicional amplíe la capacidad de la clínica sin modificar el sistema.		
003	Permitir dar de baja una unidad mediante borrado lógico, conservando su vínculo con las citas históricas.		
004	Derivar de las unidades activas la capacidad máxima de atención simultánea.		

Tabla 16: Requerimiento REQ-CAT-04 — Administración de unidades dentales Fuente: elaboración propia.

Módulo de Agenda y Citas. Es el módulo principal del sistema. Concentra la programación, la consulta y el cierre de las citas, así como la disponibilidad real de cada profesional. Comprende 6 requerimientos, de los cuales 5 son de prioridad alta.

Id. Requerimiento	REQ-AGE-01	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Horarios de disponibilidad	**Módulo**	Agenda y Citas
Referencia	Caso de uso UC16	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Definir las franjas de atención de cada odontólogo por día de la semana.		
002	Permitir replicar una misma franja de lunes a viernes en una sola operación.		

003	Validar que la hora de finalización sea posterior a la de inicio.
004	Impedir el registro de franjas que se traslapen entre sí.
005	Permitir retirar una franja mediante borrado lógico, advirtiendo cuántas citas futuras quedan dentro de ella.
006	Totalizar las horas semanales declaradas por cada profesional.

Tabla 17: Requerimiento REQ-AGE-01 — Horarios de disponibilidad Fuente: elaboración propia.

Id. Requerimiento	REQ-AGE-02	**Clasificación o prioridad**	Media
Nombre Requerimiento	Ausencias puntuales	**Módulo**	Agenda y Citas
Referencia	Caso de uso UC23	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Registrar la ausencia de un odontólogo en una fecha concreta, por toda la jornada o por un rango de horas.		
002	Permitir que el propio odontólogo registre sus ausencias, aunque su rol sea de consulta para el resto de la agenda.		
003	Impedir que un profesional registre o retire ausencias de otro.		
004	Calcular la disponibilidad efectiva como el patrón semanal menos las ausencias de esa fecha.		
005	Advertir cuántas citas ya programadas caen dentro de la ausencia, sin cancelarlas automáticamente.		

Tabla 18: Requerimiento REQ-AGE-02 — Ausencias puntuales Fuente: elaboración propia.

Id. Requerimiento	REQ-AGE-03	**Clasificación o prioridad**	Alta
Nombre	Programación de citas	**Módulo**	Agenda y Citas

Requerimiento			
Referencia	Caso de uso UC12	**Funcionalidades**	7
No. Func.	**Funcionalidad**		
001	Registrar citas asociando paciente, odontólogo, procedimiento, fecha, hora de inicio y duración asignada.		
002	Impedir el registro de citas en fechas anteriores al día en curso.		
003	Validar que la cita completa quepa dentro de una franja de disponibilidad efectiva del odontólogo.		
004	Impedir el traslape con otra cita vigente del mismo profesional.		
005	Verificar que exista una unidad dental libre en el intervalo solicitado y asignar la primera disponible de forma automática.		
006	Calcular la hora de finalización a partir de la duración asignada.		
007	Aplicar las mismas validaciones con independencia de la pantalla desde la cual se origine la cita.		

Tabla 19: Requerimiento REQ-AGE-03 — Programación de citas Fuente: elaboración propia.

Id. Requerimiento	REQ-AGE-04	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Reprogramación y cancelación	**Módulo**	Agenda y Citas
Referencia	Caso de uso UC14	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Permitir modificar la fecha, la hora o la duración de una cita vigente.		
002	Aplicar a la reprogramación las mismas validaciones del registro inicial.		
003	Marcar la cita reprogramada con un estado que la distinga de la original.		

004	Permitir la cancelación registrando el motivo.
005	Conservar la cita cancelada mediante borrado lógico, de modo que siga disponible para el análisis posterior.
006	Liberar la unidad dental y el espacio en la agenda al cancelar.

Tabla 20: Requerimiento REQ-AGE-04 — Reprogramación y cancelación Fuente: elaboración propia.

Id. Requerimiento	REQ-AGE-05	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Registro de tiempos reales	**Módulo**	Agenda y Citas
Referencia	Caso de uso UC15	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Registrar la hora de inicio real y la hora de finalización real de la atención.		
002	Validar que la hora de finalización sea posterior a la de inicio.		
003	Calcular la duración real en minutos y marcar la cita como atendida.		
004	Presentar la desviación entre la duración asignada y la real.		
005	Incorporar la atención al conjunto de entrenamiento del modelo predictivo.		
006	Cerrar el renglón del plan de tratamiento que la cita atiende, si procede.		

Tabla 21: Requerimiento REQ-AGE-05 — Registro de tiempos reales Fuente: elaboración propia.

Id. Requerimiento	REQ-AGE-06	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Calendario operativo	**Módulo**	Agenda y Citas

Referencia	Caso de uso UC13	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Presentar las citas en una rejilla por semana o por día, con navegación entre períodos y retorno al día en curso.		
002	Distinguir por color el estado de cada cita.		
003	Preseleccionar la agenda propia cuando el usuario está vinculado a una ficha de odontólogo, permitiéndole consultar la de toda la clínica.		
004	Permitir abrir el detalle de una cita desde la rejilla.		
005	Permitir programar una cita pulsando una casilla libre, con la fecha y la hora precargadas.		
006	Presentar los indicadores de citas del día, utilización de la agenda, pacientes activos y error medio del modelo.		

Tabla 22: Requerimiento REQ-AGE-06 — Calendario operativo Fuente: elaboración propia.

Módulo de Expediente Clínico. Registra qué se le hace al paciente y cómo evoluciona, y traduce el diagnóstico en un plan de tratamiento presupuestado en tiempo y en dinero. Comprende 4 requerimientos, de los cuales 3 son de prioridad alta.

Id. Requerimiento	REQ-EXP-01	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Odontograma	**Módulo**	Expediente Clínico
Referencia	Caso de uso UC25	**Funcionalidades**	7
No. Func.	**Funcionalidad**		
001	Presentar la arcada completa de treinta y dos piezas permanentes en nomenclatura FDI, agrupadas por cuadrante.		
002	Registrar hallazgos por pieza y por cara, indicando tipo, fecha y observación.		

003	Agregar cada hallazgo sin modificar los anteriores, de modo que la pieza conserve su historia y el odontograma muestre el registro vigente.
004	Validar que la pieza pertenezca a la nomenclatura y que la fecha no sea futura.
005	Impedir que un hallazgo que describe la pieza completa se registre sobre una sola cara.
006	Permitir retirar un hallazgo mal capturado, con lo que vuelve a regir el registro previo.
007	Resumir el número de piezas registradas, tratadas, ausentes y con hallazgos por atender.

Tabla 23: Requerimiento REQ-EXP-01 — Odontograma Fuente: elaboración propia.

Id. Requerimiento	REQ-EXP-02	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Notas clínicas de evolución	**Módulo**	Expediente Clínico
Referencia	Caso de uso UC26	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Registrar por visita el motivo, los hallazgos clínicos, el procedimiento realizado, los materiales y las indicaciones al paciente.		
002	Vincular la nota a la cita atendida que la origina, admitiendo una sola nota por cita.		
003	Registrar el número de piezas tratadas, el uso de anestesia y la aparición de complicaciones, que son las variables que explican la variabilidad de la duración.		
004	Validar que las piezas enumeradas coincidan con el número declarado.		
005	Exigir la descripción de la complicación cuando se marca que la hubo.		
006	Presentar la evolución del paciente en orden cronológico.		

Tabla 24: Requerimiento REQ-EXP-02 — Notas clínicas de evolución Fuente: elaboración propia.

Id. Requerimiento	REQ-EXP-03	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Plan de tratamiento	**Módulo**	Expediente Clínico
Referencia	Caso de uso UC27	**Funcionalidades**	7
No. Func.	**Funcionalidad**		
001	Registrar el plan con su diagnóstico, el odontólogo responsable y la fecha de elaboración.		
002	Desglosar el plan en renglones con fase, procedimiento, pieza, número de piezas y precio unitario.		
003	Proponer el precio de lista del catálogo y permitir ajustarlo, conservando en el renglón la tarifa con que se presupuestó.		
004	Obtener del componente predictivo la duración estimada de cada renglón y registrar el origen de esa estimación.		
005	Administrar el estado del plan: borrador, aprobado, en curso, finalizado o rechazado.		
006	Exigir al menos un renglón antes de aprobar el plan.		
007	Actualizar el avance del plan conforme se completan sus renglones.		

Tabla 25: Requerimiento REQ-EXP-03 — Plan de tratamiento Fuente: elaboración propia.

Id. Requerimiento	REQ-EXP-04	**Clasificación o prioridad**	Media
Nombre Requerimiento	Planificación del tratamiento en sesiones	**Módulo**	Expediente Clínico
Referencia	Caso de uso UC29	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Repartir los renglones pendientes en sesiones de sillón, respetando el orden de las fases.		

002	Limitar cada sesión a un máximo de ciento veinte minutos de sillón.
003	Totalizar los minutos estimados del plan y los que quedan pendientes.
004	Permitir agendar un renglón enviando al formulario de cita el paciente, el odontólogo, el procedimiento y la duración estimada al presupuestar.
005	Vincular la cita creada con el renglón y marcarlo como agendado.
006	Impedir el agendado de renglones de un plan que no esté vigente.

Tabla 26: Requerimiento REQ-EXP-04 — Planificación del tratamiento en sesiones Fuente: elaboración propia.

Módulo de Cobros. Controla lo acordado con el paciente y lo efectivamente abonado, de modo que el saldo de cada tratamiento sea consultable sin revisar expedientes uno por uno. Comprende 3 requerimientos.

Id. Requerimiento	REQ-COB-01	**Clasificación o prioridad**	Media
Nombre Requerimiento	Presupuesto del tratamiento	**Módulo**	Cobros
Referencia	Caso de uso UC30	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Calcular el subtotal del plan como la suma de los importes de sus renglones.		
002	Aplicar el descuento acordado y obtener el total.		
003	Excluir del cálculo los renglones anulados.		
004	Presentar el presupuesto junto con las sesiones estimadas, de modo que el paciente conozca el costo y el número de visitas antes de aceptar.		

Tabla 27: Requerimiento REQ-COB-01 — Presupuesto del tratamiento Fuente: elaboración propia.

Id. Requerimiento	REQ-COB-02	**Clasificación o prioridad**	Media
Nombre Requerimiento	Registro de abonos	**Módulo**	Cobros
Referencia	Caso de uso UC31	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Registrar abonos con fecha, monto, forma de pago, referencia y observaciones.		
002	Validar que el monto sea mayor que cero y no exceda el saldo pendiente.		
003	Impedir el registro de abonos con fecha posterior al día en curso.		
004	Recalcular el saldo del plan con cada abono.		
005	Permitir la anulación lógica de un abono mal registrado, conservando su traza en el estado de cuenta.		
006	Restringir el registro y la anulación de abonos al personal con competencia de cobro.		

Tabla 28: Requerimiento REQ-COB-02 — Registro de abonos Fuente: elaboración propia.

Id. Requerimiento	REQ-COB-03	**Clasificación o prioridad**	Media
Nombre Requerimiento	Estado de cuenta	**Módulo**	Cobros
Referencia	Caso de uso UC30	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Listar los planes vigentes ordenados por saldo pendiente.		
002	Presentar para cada plan el total presupuestado, lo abonado y el saldo.		
003	Totalizar el monto por cobrar y el cobrado en el mes en curso.		

004	Permitir filtrar entre los planes con saldo y la totalidad de los vigentes.
005	Advertir cuando un plan se cierra conservando saldo por cobrar.

Tabla 29: Requerimiento REQ-COB-03 — Estado de cuenta Fuente: elaboración propia.

Módulo de Inteligencia. Contiene el componente predictivo: estima la duración de cada procedimiento y recomienda el bloque de tiempo que conviene reservar. Comprende 3 requerimientos, de los cuales 2 son de prioridad alta.

Id. Requerimiento	REQ-INT-01	**Clasificación o prioridad**	Alta
Nombre Requerimiento	Estimación de duración	**Módulo**	Inteligencia
Referencia	Caso de uso UC17	**Funcionalidades**	6
No. Func.	**Funcionalidad**		
001	Estimar la duración de un procedimiento a partir del procedimiento, el odontólogo, el día, la hora y el número de piezas.		
002	Aplicar una cadena de respaldo de tres niveles: modelo entrenado, promedio histórico de atenciones reales y, en última instancia, duración de referencia del catálogo.		
003	Exigir un mínimo de tres atenciones registradas para considerar fiable el promedio histórico.		
004	Preferir el promedio del propio odontólogo cuando dispone de registros suficientes, pues el ritmo de trabajo varía entre profesionales.		
005	Declarar en la interfaz el origen de cada estimación.		
006	Continuar operando si el modelo no está disponible, sin impedir el agendado.		

Tabla 30: Requerimiento REQ-INT-01 — Estimación de duración Fuente: elaboración propia.

Id. Requerimiento	REQ-INT-02	**Clasificación o prioridad**	Alta
Nombre	Recomendación de bloques de tiempo	**Módulo**	Inteligencia

Requerimiento			
Referencia	Caso de uso UC18	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Calcular un bloque mínimo y un bloque máximo recomendados alrededor de la duración estimada.		
002	Identificar sobre el calendario los espacios donde el procedimiento consultado cabe completo.		
003	Descartar los espacios que ya pasaron, los que se traslapan con una cita vigente y aquellos en que no queda una unidad dental libre.		
004	Informar cuántos espacios disponibles existen en el período mostrado.		
005	Permitir programar la cita directamente desde un espacio sugerido.		

Tabla 31: Requerimiento REQ-INT-02 — Recomendación de bloques de tiempo Fuente: elaboración propia.

Id. Requerimiento	REQ-INT-03	**Clasificación o prioridad**	Media
Nombre Requerimiento	Administración de versiones del modelo	**Módulo**	Inteligencia
Referencia	Caso de uso UC19	**Funcionalidades**	5
No. Func.	**Funcionalidad**		
001	Registrar cada modelo entrenado con su algoritmo, su fecha de entrenamiento y la ruta del archivo serializado.		
002	Almacenar las métricas de error absoluto medio, raíz del error cuadrático medio y coeficiente de determinación.		
003	Mantener una sola versión activa en producción.		
004	Conservar las versiones anteriores para efectos comparativos.		
005	Registrar cada predicción generada junto con la cita a la que corresponde, de modo que pueda		

	contrastarse después con la duración real.

Tabla 32: Requerimiento REQ-INT-03 — Administración de versiones del modelo Fuente: elaboración propia.

Módulo de Reportes e Indicadores. Expone las cifras con que la clínica puede evaluar su operación y la precisión del modelo. Comprende 2 requerimientos.

Id. Requerimiento	REQ-REP-01	**Clasificación o prioridad**	Media
Nombre Requerimiento	Indicadores operativos	**Módulo**	Reportes e Indicadores
Referencia	Caso de uso UC20	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Calcular el número de citas atendidas por período.		
002	Calcular el porcentaje de utilización de la agenda, tomando como capacidad el menor valor entre los odontólogos disponibles y las unidades dentales activas.		
003	Calcular la desviación media entre la duración asignada y la real.		
004	Presentar los indicadores en la pantalla principal del sistema.		

Tabla 33: Requerimiento REQ-REP-01 — Indicadores operativos Fuente: elaboración propia.

Id. Requerimiento	REQ-REP-02	**Clasificación o prioridad**	Baja
Nombre Requerimiento	Reporte de utilización	**Módulo**	Reportes e Indicadores
Referencia	Caso de uso UC21	**Funcionalidades**	4
No. Func.	**Funcionalidad**		
001	Generar el detalle de utilización por odontólogo y por período.		
002	Presentar los procedimientos más frecuentes de la clínica.		
003	Comparar la precisión del modelo contra la duración de referencia del catálogo.		

004	Permitir la exportación del reporte para su consulta fuera del sistema.

Tabla 34: Requerimiento REQ-REP-02 — Reporte de utilización Fuente: elaboración propia.

Limitantes y restricciones. El proyecto se encuentra condicionado por cinco factores. El primero, y de mayor peso sobre el componente predictivo, es que la clínica no dispone de un registro digital previo de duraciones, por lo que el modelo parte de la duración de referencia del catálogo y mejora conforme se acumulan atenciones reales. El segundo es la capacidad física instalada: la clínica cuenta con dos unidades dentales compartidas por los profesionales, lo que limita la atención simultánea con independencia del número de odontólogos disponibles. El tercero es el presupuesto de infraestructura, que restringe el despliegue a una instancia de cómputo de capacidad reducida. El cuarto es el calendario académico, cuyas fechas de entrega no admiten prórroga. El quinto es la disponibilidad del personal de la clínica para las actividades de validación y capacitación, que debe acomodarse a su jornada de atención.

Requerimientos no funcionales. La Tabla 35 presenta las cualidades que debe exhibir el sistema, agrupadas por categoría.

Categoría	ID	Descripción
Seguridad	REQNF-01	Las contraseñas y las respuestas secretas se almacenan cifradas mediante una función de derivación de claves, nunca en texto plano. El sistema implementa protección frente a inyección SQL, secuencias de comandos en sitios cruzados (XSS) y falsificación de peticiones (CSRF). Todas las vistas exigen autenticación previa.
Integridad	REQNF-02	Ningún registro se elimina físicamente de la base de datos: toda baja se ejecuta mediante borrado lógico. Las operaciones sobre la agenda se realizan dentro de transacciones que garantizan las propiedades ACID. Toda modificación relevante queda asentada en una tabla de auditoría.
Rendimiento	REQNF-03	Las operaciones de consulta responden en un máximo de tres segundos bajo condiciones normales de operación. La estimación de duración generada por el modelo predictivo se obtiene en línea, sin interrumpir el flujo de programación de la cita. El sistema soporta la operación concurrente del personal de la clínica.
Usabilidad	REQNF-04	La interfaz es responsiva y operable desde computadora, tableta o teléfono, sin requerir la instalación de software adicional. Todas las pantallas comparten la misma estructura de navegación. Los mensajes de error son específicos, se muestran junto al campo que los origina e indican la acción correctiva.
Disponibilidad	REQNF-05	El sistema se encuentra disponible durante el horario de atención de la clínica. El punto de acceso es estable mediante una dirección IP fija. Los respaldos de la base de datos se ejecutan de forma periódica y programada.
Mantenibilidad	REQNF-06	El código se organiza de forma modular, con una aplicación independiente por dominio funcional. El código fuente permanece versionado en un repositorio con integración continua, y el despliegue se ejecuta de forma automatizada tras superar las pruebas.
Confidencialidad	REQNF-07	El acceso a la información clínica se restringe mediante control por roles, y la

		escritura del expediente se reserva al personal con competencia profesional declarada. Los datos utilizados para el entrenamiento del modelo se tratan de forma anonimizada, empleando atributos del procedimiento y no la identidad del paciente.
Portabilidad	REQNF-08	El sistema opera indistintamente sobre PostgreSQL en producción y sobre SQLite en el entorno de desarrollo, sin modificar el código de la aplicación. El despliegue no depende del sistema operativo del equipo cliente.

Tabla 35: Requerimientos no funcionales del sistema Fuente: elaboración propia.

Requerimientos de documentación. El proyecto contempla la elaboración del presente documento de graduación conforme a las normas APA en su séptima edición, el DERCAS de visión y el DERCAS técnico, el diccionario de datos del modelo relacional, el manual de usuario por módulo y el manual de despliegue de la infraestructura. La documentación técnica del código se mantiene dentro del propio repositorio.

Otros requisitos. Por tratarse de un sistema nuevo, no existen requerimientos de migración de datos desde una solución anterior ni de interoperabilidad con sistemas preexistentes de la clínica. La carga inicial del catálogo y del padrón de pacientes se realiza mediante las pantallas de mantenimiento del propio sistema. No se requiere la compra de licencias de software, puesto que la totalidad de las tecnologías empleadas son de código abierto.

Referencias. La Tabla 36 identifica las fuentes de información empleadas para la elaboración de los requerimientos.

Tipo de referencia	Referencia	Descripción	Fecha
Documento físico	Libro de citas de la clínica	Agenda en papel donde la recepción registra las citas, el procedimiento y la hora de atención. Constituye la única fuente histórica de duraciones con que cuenta la clínica.	2024–2026
Entrevista	Propietario de la clínica	Levantamiento de la problemática, del flujo de trabajo vigente y de las expectativas sobre el sistema.	Marzo de 2026
Entrevista	Personal de recepción y odontólogos	Identificación de los criterios con que se asignan los bloques de tiempo y de las causas habituales de retraso.	Marzo de 2026
Observación directa	Jornada de atención	Registro del tiempo real empleado en los procedimientos más frecuentes y de la ocupación de las unidades dentales.	Abril de 2026
Documento normativo	Plantilla DERCAS Visión	Estructura de especificación de requerimientos adoptada por la carrera de Ingeniería en Sistemas de Información.	2024

Tabla 36: Referencias empleadas en el levantamiento de requerimientos Fuente: elaboración propia.

Cuadro de aceptación. La Tabla 37 identifica a los responsables de la validación formal de cada entrega del proyecto y el ámbito sobre el cual recae su aprobación.

Responsable	Puesto	Ámbito de aprobación
Hee Sung Lee	Analista y desarrollador del sistema	Elaboración y entrega de los productos de análisis, diseño, desarrollo y documentación.
Propietario de la Clínica	Patrocinador del proyecto	Validación de los requerimientos, de la funcionalidad

Dental Smilee		entregada y de la puesta en producción del sistema.
Ing. Carlos Eduardo Hernández Herrera	Asesor del trabajo de graduación	Aprobación metodológica y académica de cada entrega del proyecto.

Tabla 37: Cuadro de aceptación del proyecto Fuente: elaboración propia.

3.1.2 Análisis de la solución

A partir de los requerimientos delimitados, esta sección justifica las decisiones estructurales de la solución: la arquitectura de software adoptada, el gestor de base de datos seleccionado, el tipo de aplicación a desarrollar, la infraestructura sobre la cual se ejecuta el sistema y la viabilidad del proyecto en sus cinco dimensiones.

3.1.2.1 Arquitectura de software

SmileeSync adopta una arquitectura en tres capas (three-tier) que separa la presentación, la lógica de negocio y el almacenamiento de datos. La capa de presentación se ejecuta en el navegador del usuario; la capa de aplicación, alojada en el servidor, contiene la lógica de negocio y coordina los módulos; y la capa de datos administra la persistencia mediante el sistema gestor de bases de datos. Esta separación favorece la escalabilidad, la seguridad y el mantenimiento independiente de cada nivel (Pressman y Maxim, 2015).

Internamente, la capa de aplicación se estructura según el patrón Modelo-Vista-Plantilla (MVT) del framework Django, variante del clásico Modelo-Vista-Controlador. Cada uno de los ocho módulos constituye una aplicación independiente dentro del proyecto, con sus propios modelos, vistas y plantillas, lo que refuerza la cohesión interna y reduce el acoplamiento entre componentes. El motor predictivo se implementa como un componente independiente que expone sus estimaciones a la capa de aplicación, de modo que el modelo de aprendizaje automático pueda reentrenarse o sustituirse sin modificar el resto del sistema.

La Figura 2 presenta la arquitectura de software del sistema. Cada capa depende únicamente de la inmediata inferior, de modo que una modificación en la presentación no alcanza a las reglas de negocio ni al almacenamiento. La capa de dominio concentra el criterio de disponibilidad y el servicio de estimación, que son consumidos por la validación de la cita, por la recomendación de bloques y por el indicador de utilización; centralizarlos evita que cada consumidor resuelva por su cuenta la misma pregunta con criterios distintos. El componente predictivo se representa fuera de la pila porque se consulta como un servicio y porque su ausencia

no interrumpe la operación: la estimación recae entonces en el promedio histórico o en la duración del catálogo.

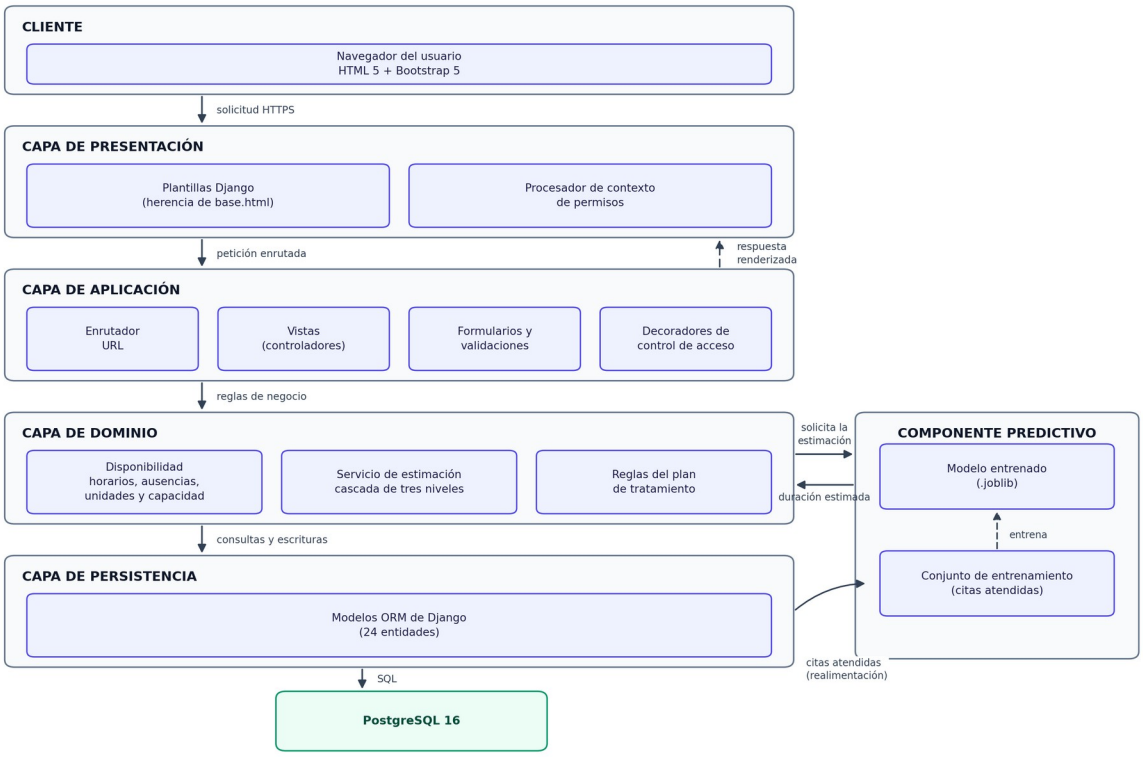


Figura 2: Arquitectura de software de SmileeSync por capas Fuente: elaboración propia.

a) Base de datos. El sistema emplea el modelo relacional, propuesto por Codd (1970) y consolidado como estándar de la industria, que organiza los datos en tablas vinculadas mediante claves primarias y foráneas (Elmasri y Navathe, 2016). El gestor seleccionado es PostgreSQL 16, por tres razones concretas: garantiza el cumplimiento de las propiedades ACID en el procesamiento de transacciones, lo que resulta indispensable cuando dos usuarios intentan programar simultáneamente una cita sobre la misma franja horaria (Date, 2004); es software de código abierto, por lo que no genera costos de licenciamiento que graven el presupuesto de la clínica; y ofrece tipos de datos y funciones de fecha y hora que facilitan los cálculos de duración y disponibilidad sobre los que opera la agenda (PostgreSQL Global Development Group, 2024). Durante el desarrollo se emplea SQLite, que permite ejecutar el sistema en el equipo del desarrollador sin administrar un servidor de base de datos independiente.

El modelo de datos se compone de veinticuatro tablas distribuidas en los ocho módulos del sistema. Su diseño aplica dos decisiones transversales. La primera es el borrado lógico: ninguna

tabla principal admite la eliminación física de registros; las bajas se representan cambiando el valor del campo de estado, lo que preserva la integridad referencial de las citas históricas. La segunda son las tablas de auditoría, que conservan una copia del registro previo a cada cambio junto con la fecha, el usuario responsable y la acción ejecutada. Ambas decisiones sostienen la base histórica que alimenta al modelo predictivo, cuyo desempeño depende directamente de la cantidad y la calidad de los datos acumulados.

b) Tipo de aplicación a desarrollar. Se evaluaron las tres alternativas habituales. Una aplicación de escritorio exigiría instalar y actualizar el software en cada equipo de la clínica y dificultaría la sincronización de la agenda entre la recepción y los consultorios. Una aplicación móvil nativa implicaría desarrollar y mantener versiones separadas para distintas plataformas, con un costo desproporcionado frente al beneficio, considerando que el personal administrativo opera principalmente desde computadoras de escritorio. La aplicación web, en cambio, se ejecuta desde cualquier navegador sin instalación previa, centraliza la información en un único servidor —de modo que todos los usuarios observan siempre el mismo estado de la agenda—, permite actualizar el sistema en un solo punto y, mediante un diseño responsivo, se adapta a computadoras, tabletas y teléfonos (Sommerville, 2011).

Por lo anterior, SmileeSync se desarrolla como una aplicación web dinámica de tipo multipágina (Multi-Page Application) con renderizado del lado del servidor. Este enfoque, frente al de una aplicación de página única, reduce la complejidad del cliente, disminuye los requerimientos de infraestructura y simplifica el mantenimiento, características apropiadas para el contexto y el presupuesto de la clínica.

3.1.2.2 Infraestructura

a) Infraestructura disponible. La clínica cuenta actualmente con equipos de cómputo de escritorio en el área de recepción y en los consultorios, conexión a internet de banda ancha comercial y una red local inalámbrica. No dispone de servidores propios, de personal de tecnologías de la información ni de licencias de software especializado para la gestión clínica. Esta situación descarta el alojamiento del sistema en un servidor físico dentro de la clínica, pues implicaría costos de adquisición, mantenimiento, respaldo eléctrico y administración que la organización no está en capacidad de asumir.

b) Infraestructura sugerida. Se propone desplegar el sistema bajo el modelo de infraestructura como servicio (IaaS), que ofrece recursos de cómputo aprovisionados bajo demanda sin inversión inicial en hardware (Mell y Grance, 2011). La Tabla 38 contrasta ambos escenarios y detalla la configuración propuesta.

Elemento	Infraestructura disponible	Infraestructura sugerida
Servidor de aplicaciones	No se dispone de servidor propio.	Instancia de cómputo en la nube (Amazon EC2, tipo t3.small) con Ubuntu Server 24.04 LTS.
Servidor web	No aplica.	Nginx como proxy inverso y servidor de archivos estáticos; Gunicorn como servidor de aplicaciones WSGI.
Base de datos	Información dispersa en archivos y registros manuales.	PostgreSQL 16 instalado en la misma instancia, con respaldos programados.
Punto de acceso	No aplica.	Dirección IP elástica que garantiza un punto de acceso estable aunque el servidor se reinicie.
Equipos de usuario	Computadoras de escritorio en recepción y consultorios.	Se conservan los equipos existentes; únicamente se requiere un navegador web actualizado.
Conectividad	Internet de banda ancha comercial y red local inalámbrica.	Se conserva la conexión actual; se recomienda un plan de respaldo de conectividad móvil.
Despliegue y versionado	No aplica.	Repositorio en GitHub con integración y despliegue continuos mediante GitHub Actions.
Administración	Sin personal de tecnologías de la información.	Administración remota del servidor mediante acceso seguro (SSH), sin requerir personal permanente en sitio.

Tabla 38: Comparación entre la infraestructura disponible y la sugerida Fuente: elaboración propia.

La configuración propuesta concentra todos los componentes en una única instancia de cómputo. Esta decisión responde al criterio de contención de costos y resulta adecuada para el volumen de operación de la clínica; no obstante, implica que la base de datos reside en el disco de la instancia, por lo que la ejecución periódica de respaldos constituye una tarea crítica de operación y no una recomendación opcional.

3.1.2.3 Estudio de factibilidad

El estudio de factibilidad determina si el proyecto es realizable y conveniente antes de comprometer recursos en su ejecución. Permite anticipar los riesgos económicos, dimensionar las necesidades técnicas y humanas, y establecer si el proyecto debe continuar o detenerse. Se desarrolla en cinco dimensiones complementarias, pues un proyecto técnicamente viable puede resultar inviable si la organización no está en capacidad de operarlo o si su costo excede el beneficio esperado.

a) Factibilidad técnica. Contempla los recursos técnicos necesarios para la operación óptima del sistema dentro de la organización. La Tabla 39 detalla los requerimientos de hardware y la Tabla 40 los de software. La particularidad de este proyecto es que la mayor parte de los recursos ya se encuentra disponible o se sustituye por servicios en la nube, lo que reduce drásticamente la inversión en equipamiento.

Cantidad	Descripción	Detalle y situación
3	Computadoras de escritorio	Procesador Intel Core i3 o superior, 8 GB de RAM, 256 GB de almacenamiento, monitor, teclado y ratón. Ubicadas en recepción y consultorios. Ya disponibles en la clínica; no requieren inversión.
1	Servidor de aplicaciones y base de datos	Instancia Amazon EC2 t3.small: 2 vCPU, 2 GB de RAM y 30 GB de almacenamiento SSD, con Ubuntu Server 24.04 LTS. Sustituye al servidor físico; se contrata bajo demanda.
1	Router inalámbrico	Router de banda dual para la red local de la clínica. Ya disponible; no requiere inversión.
1	Servicio de internet	Conexión de banda ancha comercial con ancho de banda suficiente para el acceso simultáneo del personal. Ya contratado por la clínica.
1	Unidad de respaldo de energía (UPS)	UPS de 650 VA para proteger el equipo de recepción ante cortes de energía. Recomendado; constituye la única adquisición de hardware sugerida.
1	Equipo de desarrollo	Computadora portátil del desarrollador, utilizada para el análisis, la construcción y las pruebas del sistema. Ya disponible.

Tabla 39: Requerimientos de hardware Fuente: elaboración propia.

Cantidad	Descripción	Detalle y tipo de licencia
1	Sistema operativo del servidor	Ubuntu Server 24.04 LTS. Licencia libre, sin costo.
1	Lenguaje de programación	Python 3.12. Licencia de código abierto, sin costo.
1	Framework de desarrollo web	Django 5.0. Licencia BSD de código abierto, sin costo.
1	Sistema gestor de base de datos	PostgreSQL 16 en producción y SQLite en desarrollo. Licencias de código abierto, sin costo.
1	Servidor web y de aplicaciones	Nginx y Gunicorn. Licencias de código abierto, sin costo.
1	Bibliotecas de aprendizaje automático	scikit-learn, XGBoost, pandas y NumPy. Licencias de código abierto, sin costo.
1	Marco de estilos de interfaz	Bootstrap 5.3 y Bootstrap Icons. Licencia MIT, sin costo.
1	Entorno de desarrollo y control de versiones	Visual Studio Code, Git y GitHub en su nivel gratuito. Sin costo.
3	Navegador web	Google Chrome, Microsoft Edge o Mozilla Firefox actualizado, en los equipos de la clínica. Sin costo.

Tabla 40: Requerimientos de software Fuente: elaboración propia.

El componente de mayor incertidumbre técnica es el modelo predictivo, cuya precisión depende del volumen de datos históricos disponibles. Este riesgo se mitiga estableciendo la regresión lineal múltiple como modelo de referencia, de modo que el sistema opere con estimaciones razonables desde etapas tempranas, y adoptando algoritmos de ensamble conforme

la base histórica crezca (Hastie et al., 2009). Con base en lo expuesto, el proyecto se considera técnicamente factible.

b) Factibilidad operativa. Determina las necesidades de recurso humano a lo largo del ciclo de vida del proyecto, tanto para su desarrollo como para la administración del sistema una vez implementado. La Tabla 41 detalla el personal involucrado y los tiempos estimados.

Cantidad	Perfil o actividad	Dedicación estimada
1	Analista, diseñador y desarrollador del sistema	Asumido por el estudiante como trabajo de graduación, a lo largo de los cuatro meses que comprende el proyecto.
1	Administrador del sistema (usuario)	Perfil asumido por el propietario de la clínica tras la implementación, sin dedicación exclusiva.
2	Usuarios operativos de recepción	Personal existente de la clínica; no se requiere contratación adicional.
2	Odontólogos usuarios del sistema	Personal existente de la clínica; registran los tiempos reales de atención como parte de su flujo habitual de trabajo.
1	Capacitación en el uso del sistema	Dos sesiones de cuatro horas cada una, distribuidas en cinco días hábiles, dirigidas al personal administrativo y clínico.
1	Acompañamiento posterior a la implementación	Quince días hábiles de soporte y resolución de dudas tras la puesta en producción.

Tabla 41: Recurso humano y tiempos estimados del proyecto Fuente: elaboración propia.

El sistema se inserta en procesos que el personal ya ejecuta —registrar un paciente, programar una cita, atenderla— sin exigir un cambio en la forma de trabajar de la clínica, sino en el medio en que se registra la información. La interfaz sigue convenciones de uso extendido, lo que reduce la curva de aprendizaje, y el control de acceso por rol garantiza que cada usuario observe únicamente lo que necesita. Se identifica una barrera operativa principal: el compromiso de registrar la hora real de finalización de cada atención, dato del que depende el aprendizaje del modelo y que supone un hábito nuevo para el equipo clínico; se mitiga integrando dicho registro como una sola acción dentro del flujo de cierre de la cita. No se requiere contratar personal adicional, por lo que el proyecto se considera operativamente factible.

c) Factibilidad económica. Determina la disponibilidad monetaria necesaria para la realización del proyecto, a partir de los recursos identificados en las factibilidades técnica y operativa. La Tabla 42 presenta el presupuesto estimado en quetzales.

Concepto	Detalle	Costo estimado
Computadoras de escritorio	Se utilizan los tres equipos con que ya cuenta la clínica.	Q 0.00
Unidad de respaldo de energía (UPS)	UPS de 650 VA para el equipo de recepción.	Q 650.00
Servidor en la nube (primer año)	Instancia EC2 t3.small e IP elástica, cubiertas parcialmente por la capa gratuita del proveedor.	Q 1,600.00
Servicio de internet	Se utiliza el servicio de banda ancha ya contratado por la	Q 0.00

Concepto	Detalle	Costo estimado
	clínica.	
Subtotal de hardware e infraestructura	Suma de los conceptos anteriores.	Q 2,250.00
Licencias de sistema operativo y servidor	Ubuntu Server, Nginx y Gunicorn: software de código abierto.	Q 0.00
Licencias de desarrollo	Python, Django, PostgreSQL, Bootstrap, scikit-learn y XGBoost: software de código abierto.	Q 0.00
Herramientas de desarrollo y versionado	Visual Studio Code, Git y GitHub en su nivel gratuito.	Q 0.00
Subtotal de software	Suma de los conceptos anteriores.	Q 0.00
Desarrollo del sistema	Análisis, diseño, programación y pruebas, asumidos por el estudiante como trabajo de graduación.	Q 0.00
Capacitación del personal	Dos sesiones de capacitación impartidas por el desarrollador.	Q 0.00
Subtotal de recurso humano	Suma de los conceptos anteriores.	Q 0.00
Costo total de implementación	Inversión inicial requerida por la clínica.	Q 2,250.00
Costo anual de operación posterior	Instancia y almacenamiento en la nube una vez concluida la capa gratuita.	Q 1,600.00 a Q 2,400.00

Tabla 42: Presupuesto estimado del proyecto (en quetzales) Fuente: elaboración propia.

El presupuesto evidencia que la inversión inicial requerida por la clínica asciende a Q 2,250.00, cifra considerablemente menor a la de un proyecto equivalente con servidor físico y licenciamiento comercial, gracias a la adopción de software de código abierto en la totalidad de la pila tecnológica y al modelo de infraestructura en la nube. Frente a este costo, el beneficio económico proviene de la recuperación de tiempo productivo: si la asignación más precisa de los bloques de tiempo permitiera recuperar únicamente una hora de agenda efectiva por semana y por odontólogo, el valor de las atenciones adicionales que esa hora habilita superaría con amplitud el costo anual de la infraestructura. A ello se suman beneficios no cuantificados directamente, como la reducción del tiempo de espera del paciente y la disponibilidad de información histórica para la toma de decisiones. El proyecto se considera, por tanto, económicamente factible.

d) Factibilidad legal. Todas las tecnologías empleadas se distribuyen bajo licencias de código abierto que permiten su uso, modificación y despliegue sin contraprestación económica, por lo que no existe riesgo de infracción a la Ley de Derecho de Autor y Derechos Conexos de Guatemala, ni se incurre en el uso de software sin licencia. El único contrato vigente que requiere el proyecto es el servicio de internet con que ya cuenta la clínica, al que se suma la contratación del servicio de cómputo en la nube. En cuanto al tratamiento de la información, el sistema administra datos personales y clínicos de los pacientes, amparados por el deber de confidencialidad que rige el ejercicio profesional en el ámbito de la salud y por las disposiciones sobre datos

personales sensibles contenidas en la legislación guatemalteca en materia de acceso a la información pública. En consecuencia, el sistema adopta tres medidas: el acceso a la información clínica se restringe mediante control por roles; las contraseñas se almacenan cifradas; y los datos empleados para entrenar el modelo predictivo se procesan de forma anonimizada, utilizando únicamente atributos del procedimiento y no la identidad del paciente. La implementación se realiza con el consentimiento expreso del propietario de la clínica, quien autoriza el uso de la información con fines académicos bajo condiciones de anonimización. El proyecto se considera legalmente factible.

e) Factibilidad ambiental. El proyecto presenta un impacto ambiental favorable. La digitalización de la agenda y de los expedientes reduce de manera directa el consumo de papel, tóner y espacio físico de archivo que demanda el registro manual. El despliegue en infraestructura de nube compartida resulta energéticamente más eficiente que la operación de un servidor físico dedicado dentro de la clínica, pues los centros de datos operan con índices de aprovechamiento energético superiores a los de un equipo individual en funcionamiento permanente. Al no requerir la adquisición de equipos de cómputo nuevos, el proyecto tampoco genera residuos de aparatos eléctricos y electrónicos. En cuanto al ambiente de trabajo, la incorporación del sistema no introduce equipamiento adicional en los espacios de atención, por lo que no modifica las condiciones físicas ni ergonómicas del personal; únicamente se recomienda verificar la adecuada iluminación y la distancia visual de los equipos existentes para salvaguardar la salud visual de los usuarios. El único consumo energético atribuible corresponde al de la instancia de cómputo, cuyo dimensionamiento se ajustó deliberadamente al mínimo necesario para la operación. El proyecto se considera ambientalmente factible.

3.1.3 Fases de desarrollo

3.1.3.1 Descripción de las fases

El desarrollo del sistema sigue una metodología ágil de tipo incremental e iterativo basada en el marco de trabajo Scrum (Schwaber y Sutherland, 2020), complementada con la metodología CRISP-DM para el componente de ciencia de datos (Chapman et al., 2000). Bajo este enfoque, el sistema se construye en ciclos cortos denominados sprints, al final de los cuales se entrega un incremento funcional verificable. La combinación de ambas metodologías responde a que el

desarrollo del software y la construcción del modelo predictivo avanzan en paralelo pero con ritmos distintos: el primero progresa por módulos completos, mientras que el segundo depende de la acumulación progresiva de datos históricos. La Tabla 43 describe las fases del proyecto.

Fase	Descripción	Productos
1. Levantamiento de requerimientos	Identificación de los interesados, elicitación de necesidades mediante entrevistas con el personal de la clínica y observación de los procesos de recepción y atención.	Requerimientos funcionales y no funcionales; delimitación del alcance (DERCAS).
2. Análisis de la solución	Evaluación de alternativas de arquitectura, selección de tecnologías y elaboración del estudio de factibilidad.	Arquitectura definida; estudio de factibilidad; propuesta de infraestructura.
3. Diseño del sistema	Modelado de la base de datos, elaboración de los diagramas UML y diseño de las interfaces de usuario.	Diagrama entidad-relación; diccionario de datos; diagramas de clases, casos de uso, secuencia y componentes; diseño de pantallas.
4. Desarrollo por incrementos	Construcción iterativa de los módulos mediante sprints, iniciando por Seguridad y Usuarios, seguido del módulo Principal y Agenda, Pacientes y Catálogo, y finalmente Reportes. Una vez en marcha el módulo de Agenda y Citas se imparten las dos sesiones de capacitación previstas en la Tabla 41, de modo que el registro de los tiempos reales de atención inicie desde ese momento.	Módulos funcionales desplegados en el servidor de producción.
5. Preparación de datos (CRISP-DM)	Digitalización de las anotaciones de la agenda manual mediante la ficha de registro y consolidación con los registros generados por el sistema; limpieza, tratamiento de valores atípicos, codificación de variables categóricas y construcción del conjunto de entrenamiento.	Conjunto de datos depurado y documentado.
6. Modelado y evaluación	Entrenamiento y comparación de la regresión lineal múltiple, Random Forest y XGBoost mediante validación cruzada, ajuste de hiperparámetros y selección del modelo con mejor desempeño.	Modelo entrenado y exportado; métricas MAE, RMSE y R^2 documentadas.
7. Integración del componente inteligente	Incorporación del modelo seleccionado al flujo de programación de citas y construcción del sistema de recomendación de bloques de tiempo.	Módulo de Inteligencia operativo dentro del sistema.
8. Pruebas y evaluación del sistema	Ejecución de pruebas funcionales, de integración y de aceptación; comparación de los tiempos estimados por el modelo frente al método tradicional de bloques fijos.	Informe de pruebas; evaluación del impacto del sistema.
9. Despliegue y documentación final	Puesta en producción de la versión definitiva y elaboración de la documentación técnica y de usuario.	Sistema en producción; manuales; documento final de graduación.

Tabla 43: Fases de desarrollo del sistema Fuente: elaboración propia.

3.1.3.2 Cronograma del desarrollo

El cronograma que presenta la Figura 3 distribuye las fases descritas a lo largo del período de ejecución del proyecto. Las actividades se agrupan en tres frentes de trabajo —análisis y diseño,

desarrollo de módulos, y modelo predictivo y cierre— que avanzan de forma parcialmente simultánea. Los hitos señalados corresponden a las entregas formales establecidas en el curso de Proyecto de Graduación II.

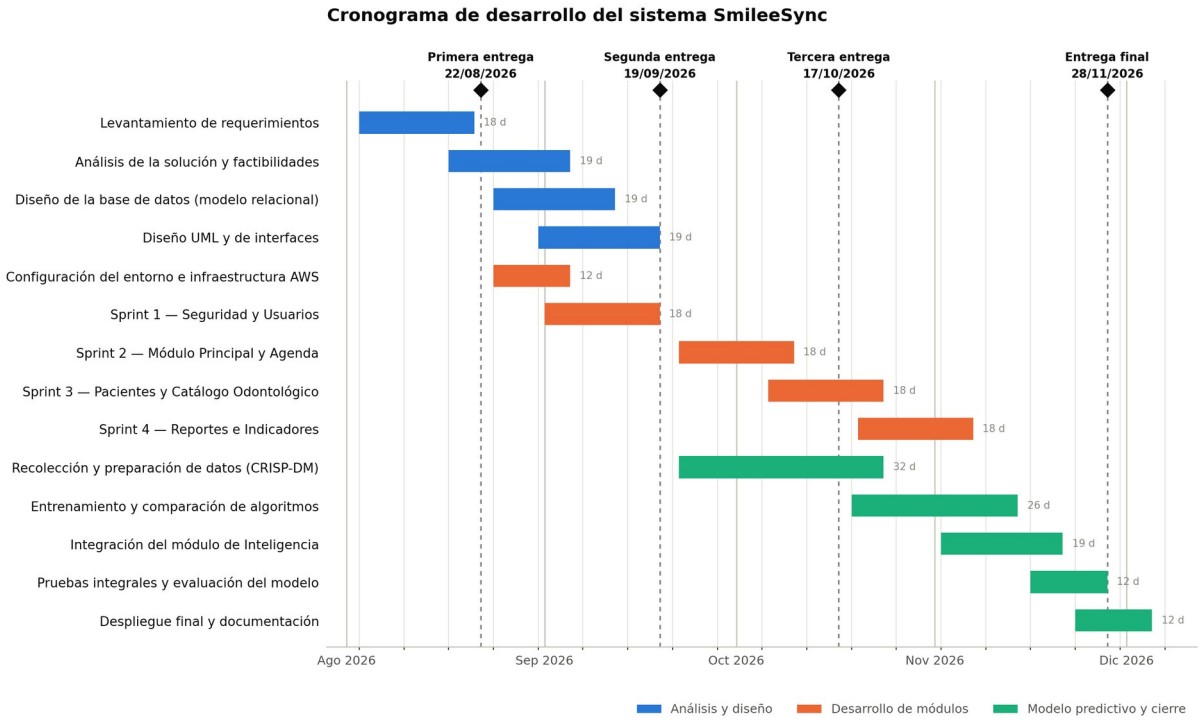


Figura 3: Cronograma de desarrollo del sistema SmileeSync Fuente: elaboración propia.

El traslape deliberado entre actividades responde a una dependencia propia de este proyecto: el conjunto de entrenamiento se conforma en dos tiempos. La digitalización de las anotaciones de la agenda manual puede iniciar tan pronto se define la estructura del conjunto de datos, mientras que los registros de mayor precisión solo se producen cuando el módulo de Agenda y Citas entra en operación y el personal registra los tiempos reales de cada atención. Por esa razón el entrenamiento del modelo se ubica en la última parte del cronograma y no al inicio, pese a constituir el componente distintivo del sistema.

3.2 Diseño del sistema

El diseño del sistema se documenta mediante los diagramas del Lenguaje Unificado de Modelado (UML), que permiten representar la estructura y el comportamiento del software antes de su construcción. Se presentan cinco vistas complementarias: la funcionalidad desde la perspectiva del usuario mediante el diagrama de casos de uso, el modelo de datos mediante el

diagrama entidad-relación, la estructura estática mediante el diagrama de clases, el comportamiento dinámico de los procesos críticos mediante diagramas de secuencia, y la organización física del sistema mediante el diagrama de componentes.

3.2.1 Diagrama de casos de uso

El diagrama de casos de uso presenta la funcionalidad del sistema desde la perspectiva de quienes lo utilizan. Se identifican cuatro actores humanos —administrador, recepcionista, odontólogo y asistente— y un actor de sistema, el motor predictivo, que participa en la estimación de duraciones y en el entrenamiento del modelo. Los casos de uso se agrupan en paquetes que corresponden a los módulos del sistema.

Las relaciones de inclusión expresan dependencias obligatorias: programar una cita incluye necesariamente la estimación de la duración del procedimiento, y esta a su vez incluye la recomendación del bloque de tiempo. Las relaciones de extensión representan comportamiento opcional, como el reentrenamiento del modelo a partir del registro de los tiempos reales de atención. La asignación de casos de uso a cada actor se corresponde directamente con los permisos configurados en el rol, de modo que el diagrama documenta también la matriz de control de acceso del sistema.

El sistema comprende treinta y dos casos de uso distribuidos en ocho módulos. Para conservar su legibilidad, el diagrama se presenta en dos vistas complementarias: la Figura 4 reúne los casos de uso administrativos y la Figura 5 los de la operación clínica, donde se concentra el componente predictivo.

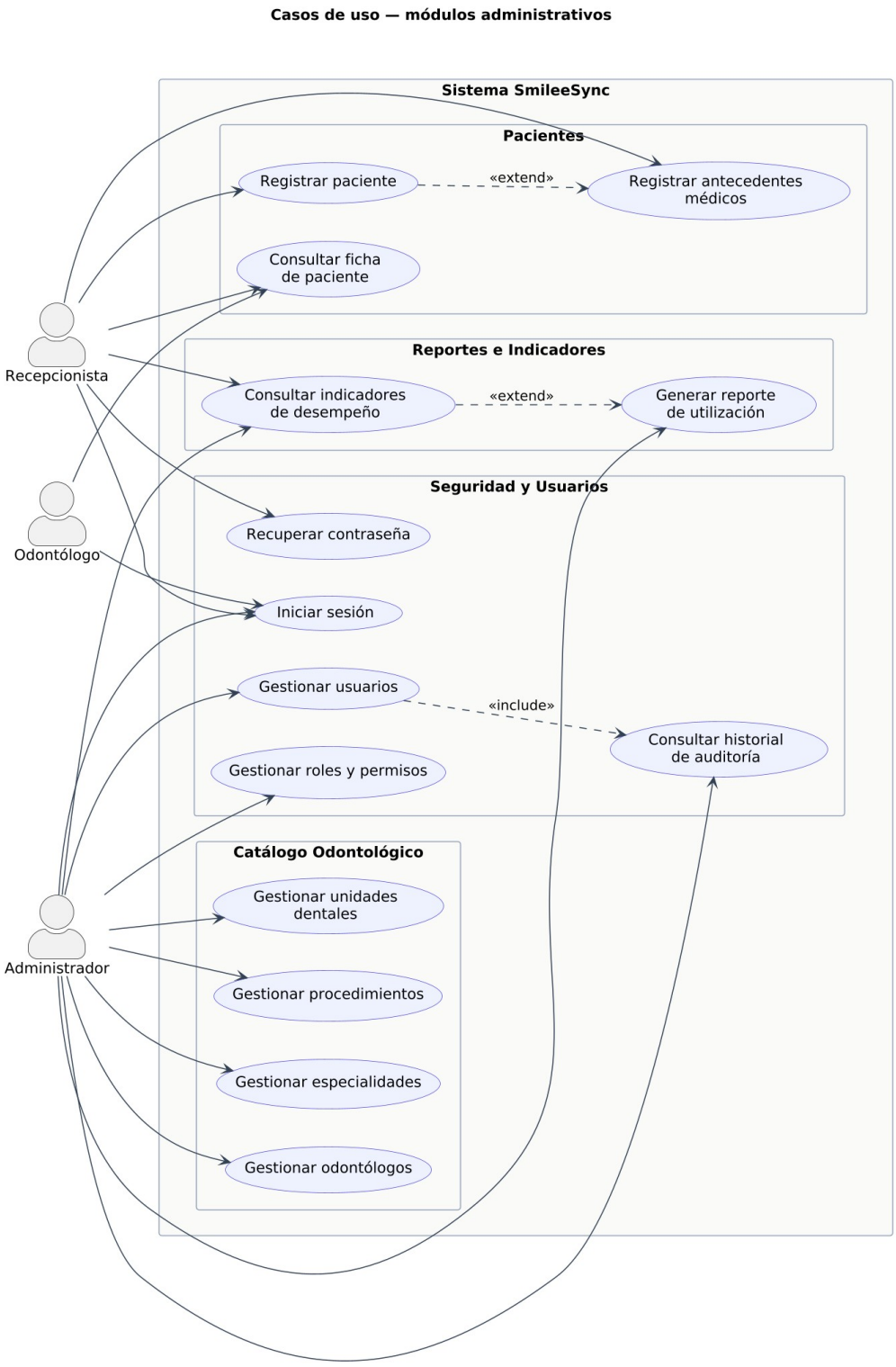


Figura 4: Casos de uso de los módulos administrativos Fuente: elaboración propia.

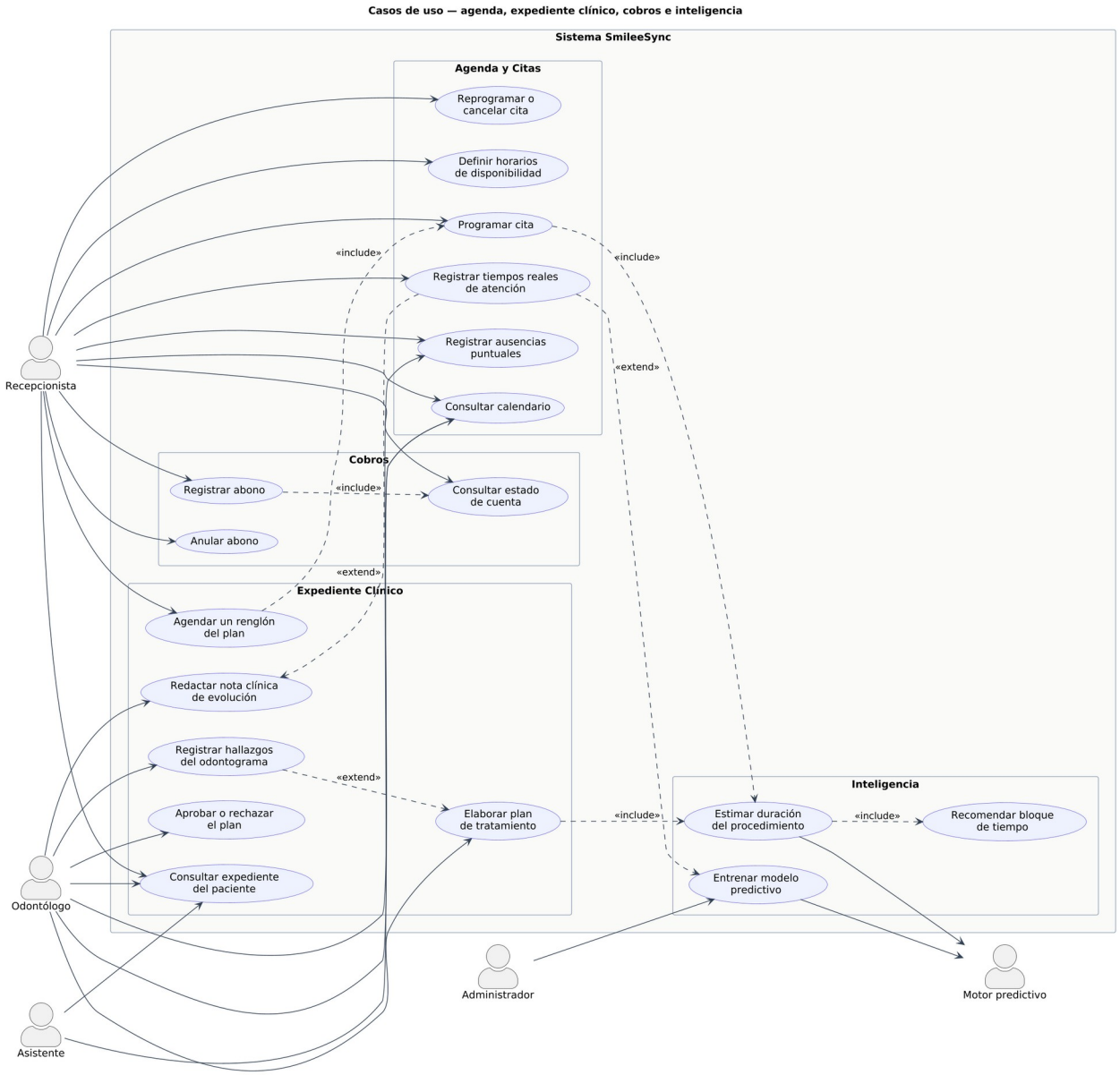


Figura 5: Casos de uso de la agenda, el expediente clínico, los cobros y la inteligencia Fuente: elaboración propia.

A continuación se especifican en detalle los dos casos de uso de mayor relevancia para el proyecto. La Tabla 44 documenta el inicio de sesión, punto de entrada al sistema y base del control de acceso.

Elemento	Descripción
ID del requerimiento	REQ-01
ID del caso de uso	CU-01
Actores	Administrador, recepcionista, odontólogo y asistente.
Descripción	Proceso inicial encargado de verificar las credenciales de acceso al sistema y de determinar los módulos habilitados para el usuario según su rol.
Precondiciones	El usuario debe encontrarse registrado en el sistema y su estado debe ser activo.

Elemento	Descripción
Secuencia normal	1. El sistema presenta el formulario de acceso. 2. El usuario ingresa su correo electrónico. 3. El usuario ingresa su contraseña. 4. El usuario confirma el envío de los datos. 5. El sistema valida el formato de los datos ingresados. 6. El sistema verifica la existencia del correo en la base de datos. 7. El sistema compara la contraseña con el valor cifrado almacenado. 8. El sistema verifica que el estado del usuario sea activo. 9. El sistema crea la sesión y evalúa los permisos del rol. 10. El sistema presenta el módulo Principal con el menú correspondiente a su rol.
Excepciones	5. Formato inválido: el sistema muestra el mensaje de validación junto al campo correspondiente y conserva el formulario. 6. y 7. Credenciales incorrectas: el sistema muestra el mensaje "Correo o contraseña incorrectos" sin revelar cuál de los dos datos es erróneo, evitando la enumeración de usuarios. 8. Usuario inactivo: el sistema deniega el acceso e informa que la cuenta se encuentra desactivada. 9. Error de base de datos: el sistema presenta un mensaje de error general y el incidente queda registrado para su revisión.
Postcondiciones	El usuario queda autenticado, con una sesión activa y acceso restringido a los módulos autorizados para su rol.

Tabla 44: Especificación del caso de uso CU-01: Iniciar sesión Fuente: elaboración propia.

La Tabla 45 especifica la programación de una cita con estimación predictiva, caso de uso que concentra la propuesta de valor del sistema.

Elemento	Descripción
ID del requerimiento	REQ-10, REQ-13 y REQ-14
ID del caso de uso	CU-10
Actores	Recepcionista (actor principal) y motor predictivo (actor de sistema).
Descripción	Proceso mediante el cual se programa una cita odontológica, estimando la duración del procedimiento a partir del modelo predictivo y recomendando el bloque de tiempo más adecuado.
Precondiciones	El paciente, el procedimiento y el odontólogo deben encontrarse registrados y activos; el odontólogo debe tener definidos sus horarios de disponibilidad.
Secuencia normal	1. El usuario selecciona el paciente. 2. El usuario selecciona el procedimiento a realizar. 3. El usuario selecciona el odontólogo y la fecha. 4. El sistema consulta la disponibilidad del odontólogo para esa fecha. 5. El sistema invoca al motor predictivo con el procedimiento, el odontólogo, el día de la semana, la franja horaria y el número de piezas. 6. El motor predictivo carga la versión activa del modelo y calcula la duración estimada. 7. El sistema calcula el bloque de tiempo recomendado y almacena la predicción generada. 8. El sistema presenta la duración estimada, el bloque sugerido y los factores considerados. 9. El usuario confirma o ajusta la duración propuesta. 10. El sistema verifica que no exista traslape con otra cita. 11. El sistema registra la cita y la presenta en el calendario.
Excepciones	4. Sin disponibilidad: el sistema informa que el odontólogo no posee franjas libres en la fecha seleccionada y sugiere la siguiente fecha disponible. 6. Modelo no disponible o sin datos suficientes: el sistema emplea la duración promedio del catálogo como valor de referencia e informa al usuario que la estimación no proviene del modelo. 10. Traslape de citas: el sistema impide el registro, señala la cita en conflicto y solicita seleccionar otro horario.
Postcondiciones	La cita queda registrada con su duración asignada, y la predicción generada queda almacenada para su posterior contraste con la duración real.

Tabla 45: Especificación del caso de uso CU-10: Programar cita con predicción de duración Fuente: elaboración propia.

3.2.2 Diseño de la base de datos

El diagrama entidad-relación que presenta la Figura 6 describe el modelo de datos completo del sistema: veinticuatro tablas distribuidas en los ocho módulos, con sus atributos, tipos de dato, claves primarias, claves foráneas y restricciones de unicidad. Las relaciones continuas representan

claves foráneas declaradas en la base de datos, mientras que las relaciones punteadas corresponden a las referencias lógicas de las tablas de auditoría.

Dos aspectos del modelo merecen destacarse. En primer lugar, las tablas de historial no declaran una clave foránea hacia la entidad que auditan, sino que conservan su identificador como un valor entero simple. Esta decisión es deliberada: garantiza que el registro de auditoría sobreviva íntegro aunque el registro original cambie, y evita que la integridad referencial impida conservar la evidencia de un cambio. En segundo lugar, la tabla dato_entrenamiento almacena una copia desnormalizada de los atributos que alimentan al modelo predictivo, de modo que el conjunto de entrenamiento permanezca estable aunque el catálogo de procedimientos se modifique con posterioridad.

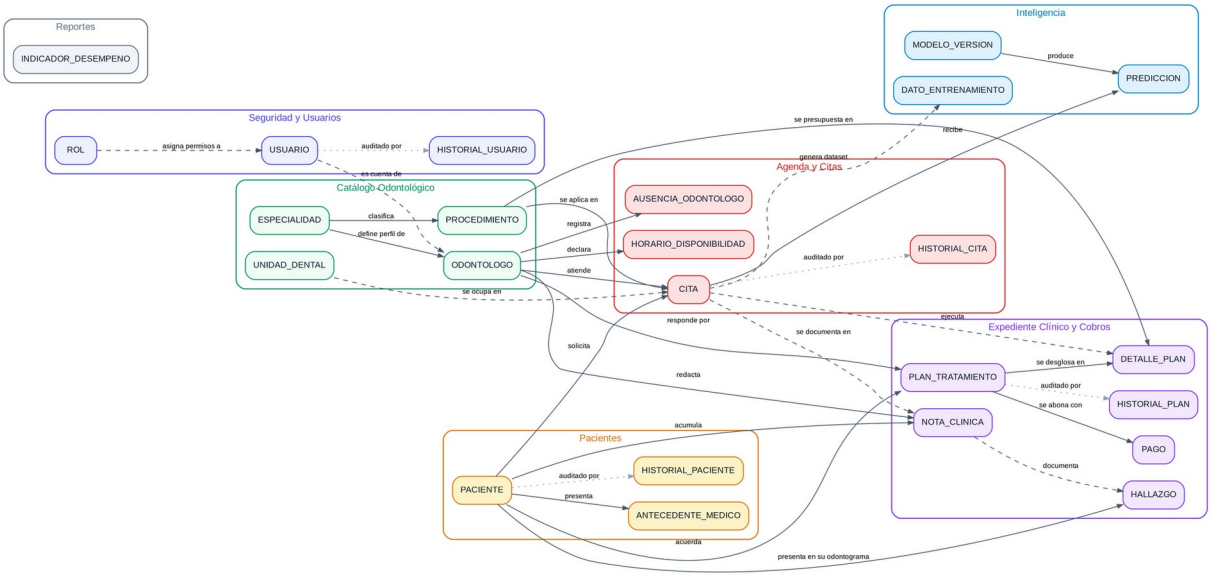


Figura 6: Modelo de datos de SmileeSync agrupado por módulo Fuente: elaboración propia.

La Tabla 46 relaciona cada elemento del modelo de datos con el requerimiento que lo origina, lo que permite verificar la trazabilidad entre el análisis y el diseño.

Requerimientos	Elemento del modelo	Descripción
REQ-SEG-03	rol	Registra los roles del sistema, los módulos habilitados para cada uno y los indicadores de solo consulta y de escritura clínica.
REQ-SEG-01, REQ-SEG-02, REQ-SEG-05	usuario	Registra a los usuarios, sus credenciales cifradas, su rol y la pregunta secreta de recuperación.
REQ-SEG-04	historial_usuario	Conserva el estado previo de cada usuario antes de su modificación o cambio de estado.
REQ-PAC-01	paciente	Registra los datos de identificación y contacto de las personas atendidas por la clínica.
REQ-PAC-02	antecedente_medico	Registra las condiciones médicas relevantes del paciente que

		condicionan la atención.
REQ-PAC-01	historial_paciente	Conserva el estado previo de cada ficha de paciente antes de su modificación.
REQ-CAT-01	especialidad	Clasifica los procedimientos y define el perfil de cada profesional.
REQ-CAT-02	procedimiento	Registra el catálogo de tratamientos con su complejidad, su duración de referencia y su tarifa.
REQ-CAT-03	odontologo	Registra a los profesionales y su vínculo con una cuenta de usuario del sistema.
REQ-CAT-04	unidad_dental	Registra los sillones de la clínica, de los que se deriva la capacidad de atención simultánea.
REQ-AGE-01	horario_disponibilidad	Define el patrón semanal de atención de cada odontólogo.
REQ-AGE-02	ausencia_odontologo	Registra las excepciones puntuales al patrón semanal, por jornada completa o por rango de horas.
REQ-AGE-03, REQ-AGE-04, REQ-AGE-05	cita	Registra la cita con sus tiempos programados y reales, el procedimiento, el profesional y la unidad asignada.
REQ-SEG-04, REQ-AGE-04	historial_cita	Conserva el estado previo de cada cita antes de su reprogramación, cancelación o cierre.
REQ-EXP-01	hallazgo	Registra el estado de cada pieza dental por cara. Es una tabla de solo agregado: el odontograma vigente es el último registro de cada combinación.
REQ-EXP-02	nota_clinica	Documenta la visita y aporta al modelo las variables de piezas tratadas, anestesia y complicación.
REQ-EXP-03	plan_tratamiento	Registra el tratamiento acordado con su diagnóstico, su estado y su descuento.
REQ-EXP-03, REQ-EXP-04	detalle_plan	Desglosa el plan en renglones con su precio y la duración que el modelo estimó al presupuestarlos.
REQ-COB-02	pago	Registra los abonos aplicados al plan y admite su anulación lógica.
REQ-COB-01, REQ-COB-03	historial_plan	Conserva la traza de los cambios de estado del plan y de los cobros aplicados.
REQ-INT-03	modelo_version	Registra cada versión entrenada del modelo con su algoritmo y sus métricas de error.
REQ-INT-01	dato_entrenamiento	Acumula las atenciones cerradas que constituyen el conjunto de entrenamiento.
REQ-INT-01, REQ-INT-02	prediccion	Registra cada estimación generada para contrastarla después con la duración real.
REQ-REP-01, REQ-REP-02	indicador_desempeno	Almacena las cifras consolidadas de la operación y de la precisión del modelo.

Tabla 46: Elementos del diseño de base de datos y su trazabilidad Fuente: elaboración propia.

3.2.3 Diagrama de clases

El diagrama de clases de la Figura 7 representa la estructura estática del sistema desde la perspectiva de la programación orientada a objetos. Las clases se agrupan en paquetes que corresponden a los ocho módulos de la aplicación, y cada una expone sus atributos y, cuando aplica, sus métodos de negocio. La correspondencia entre las clases y las tablas de la base de datos se resuelve mediante el mapeador objeto-relacional del framework, lo que permite operar sobre objetos en lugar de sentencias SQL.

La clase Usuario concentra la lógica de control de acceso mediante el método puede(modulo), que evalúa los permisos definidos en la clase Rol y determina tanto las opciones

visibles del menú como el acceso efectivo a cada vista. La clase HistorialUsuario expone el método de clase registrar(), invocado antes de cada modificación para preservar el estado previo. Las relaciones de composición —representadas con rombo relleno— indican dependencia existencial: los antecedentes médicos no existen sin su paciente, ni las predicciones sin la cita que las origina.

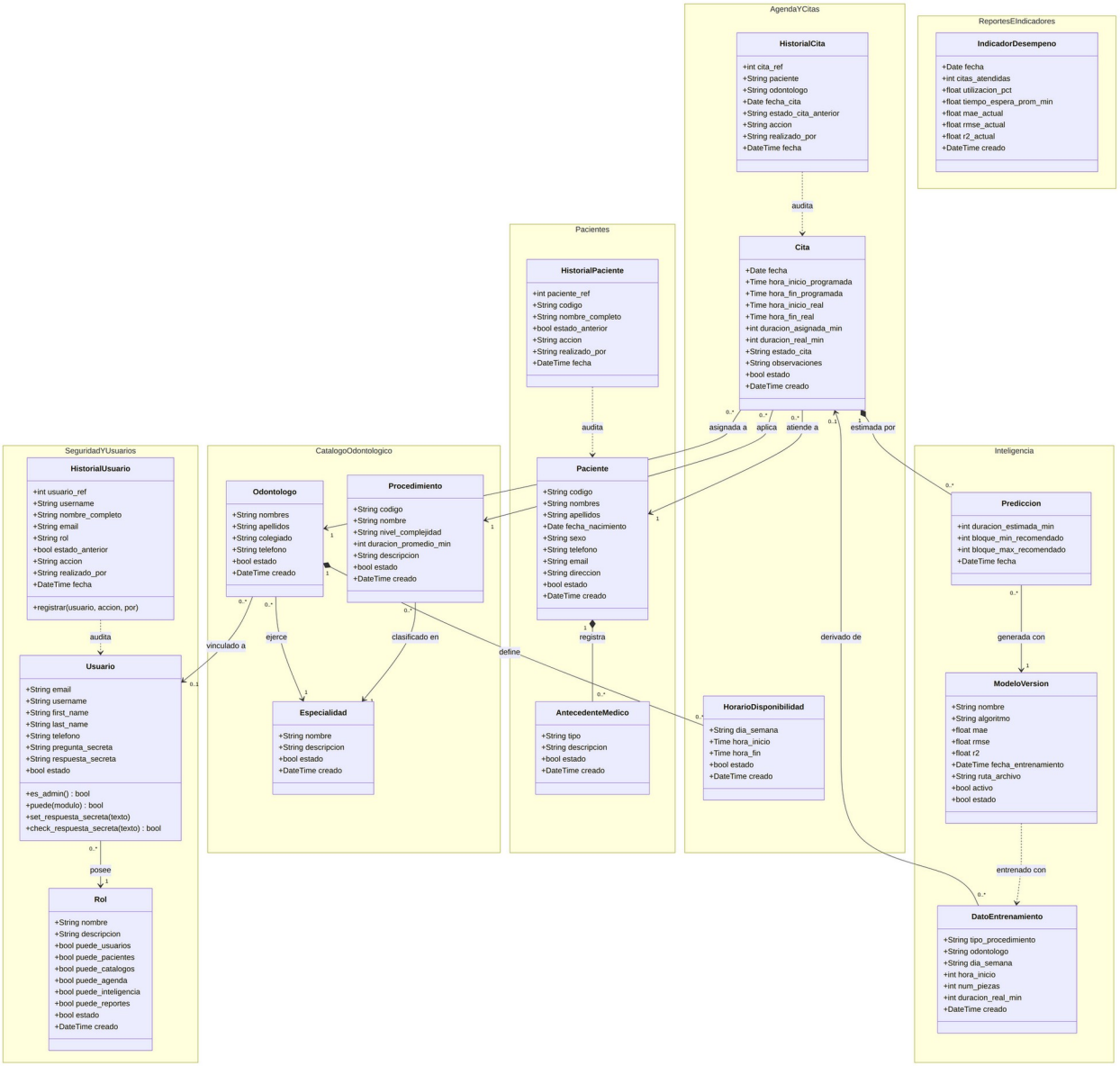


Figura 7: Diagrama de clases del sistema SmileeSync Fuente: elaboración propia.

3.2.4 Diagramas de secuencia

Los diagramas de secuencia describen el comportamiento dinámico del sistema mostrando el intercambio de mensajes entre los objetos participantes a lo largo del tiempo. Se documentan los tres procesos de mayor relevancia para el proyecto: la autenticación con control de acceso por

rol, la creación de usuarios con sus validaciones, y la programación de una cita con estimación predictiva de la duración.

La Figura 8 presenta el proceso de inicio de sesión. El diagrama evidencia que el control de acceso no se limita a ocultar opciones del menú, sino que se verifica en cada vista antes de ejecutar la operación solicitada, lo que impide el acceso mediante la manipulación directa de la dirección web.

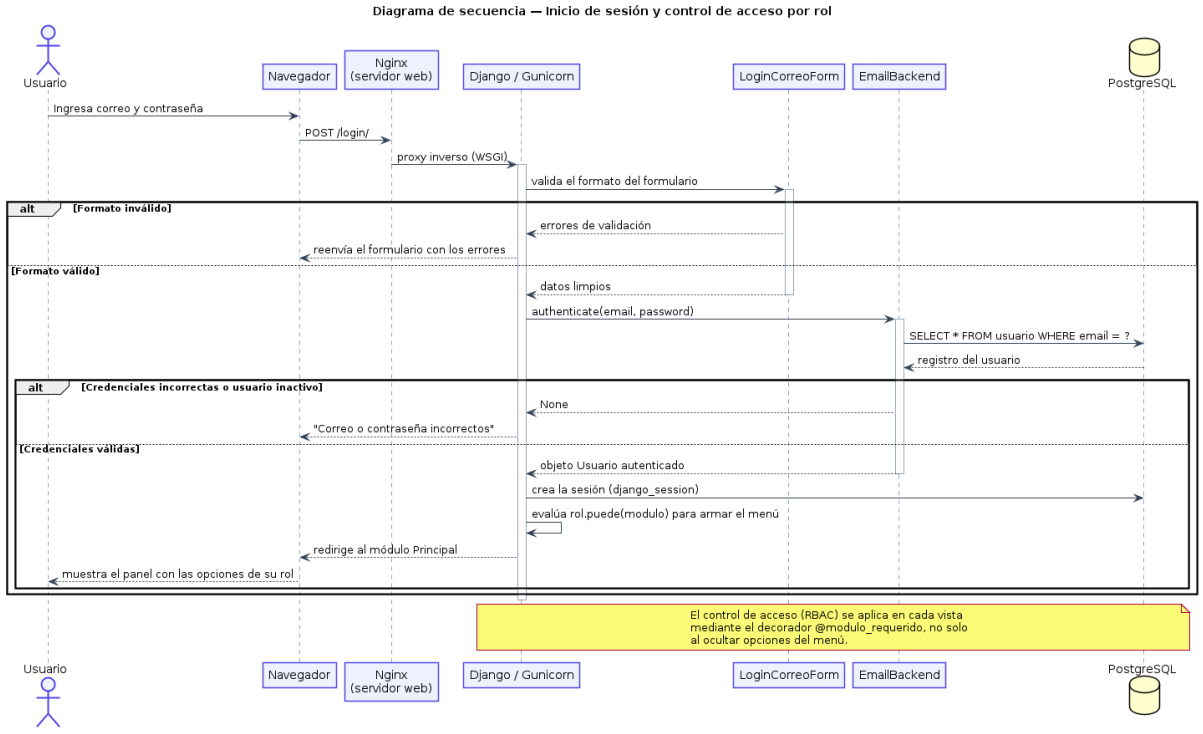


Figura 8: Diagrama de secuencia: inicio de sesión y control de acceso por rol Fuente: elaboración propia.

La Figura 9 detalla la creación de un usuario. El diagrama muestra la secuencia de validaciones que se ejecutan antes de persistir el registro: la verificación de que el nombre y el apellido contengan únicamente caracteres alfabéticos, la validación del formato del correo electrónico y la comprobación de que no se encuentre duplicado, y la coincidencia entre ambas contraseñas. Únicamente cuando todas las validaciones se satisfacen se ejecuta el guardado, el cifrado de la contraseña y el registro correspondiente en la tabla de auditoría.

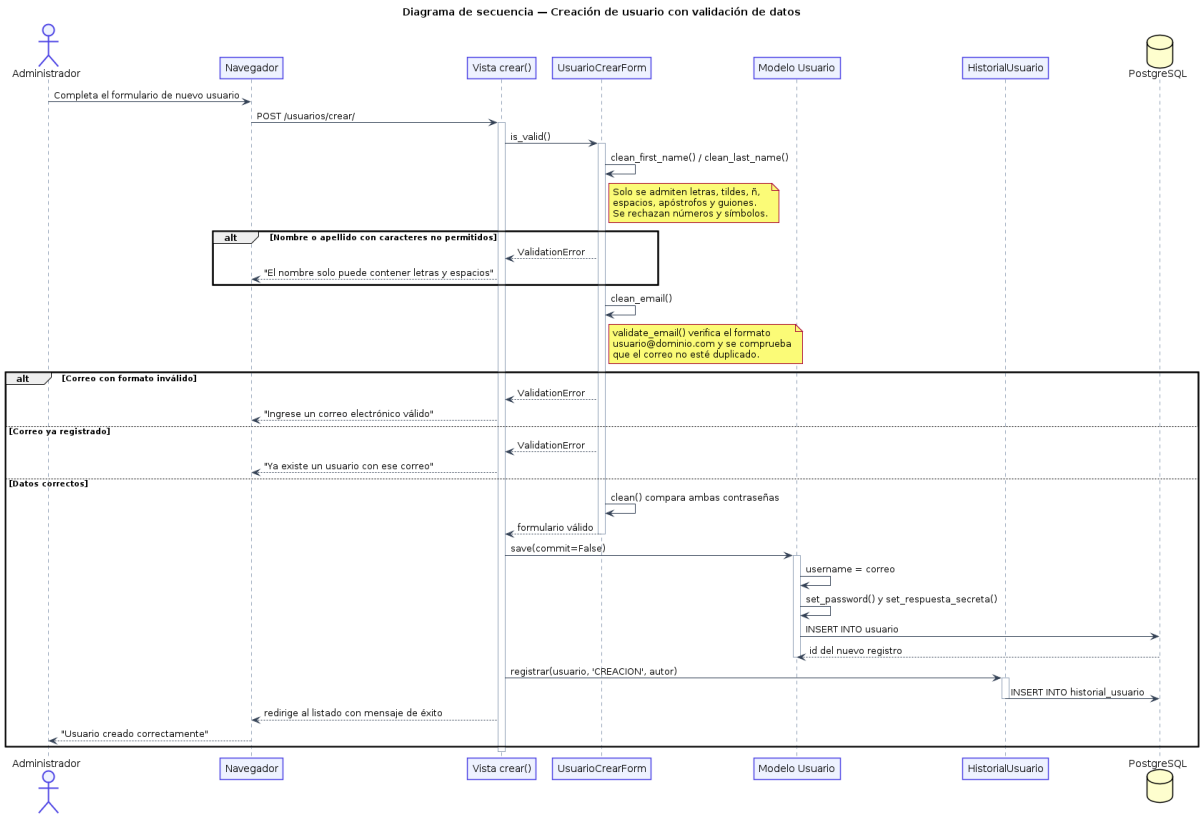


Figura 9: Diagrama de secuencia: creación de usuario con validación de datos Fuente: elaboración propia.

La Figura 10 corresponde al proceso distintivo del sistema. Al seleccionar el paciente, el procedimiento y el odontólogo, la aplicación consulta la disponibilidad del profesional e invoca al motor predictivo, el cual carga la versión activa del modelo entrenado, calcula la duración estimada y determina el bloque de tiempo recomendado. La estimación se almacena antes de presentarse al usuario, de modo que quede registro de cada predicción y pueda contrastarse posteriormente con la duración real. El ciclo se cierra cuando, finalizada la atención, el tiempo efectivamente empleado se incorpora al conjunto de entrenamiento y permite reentrenar el modelo, mejorando progresivamente su precisión.

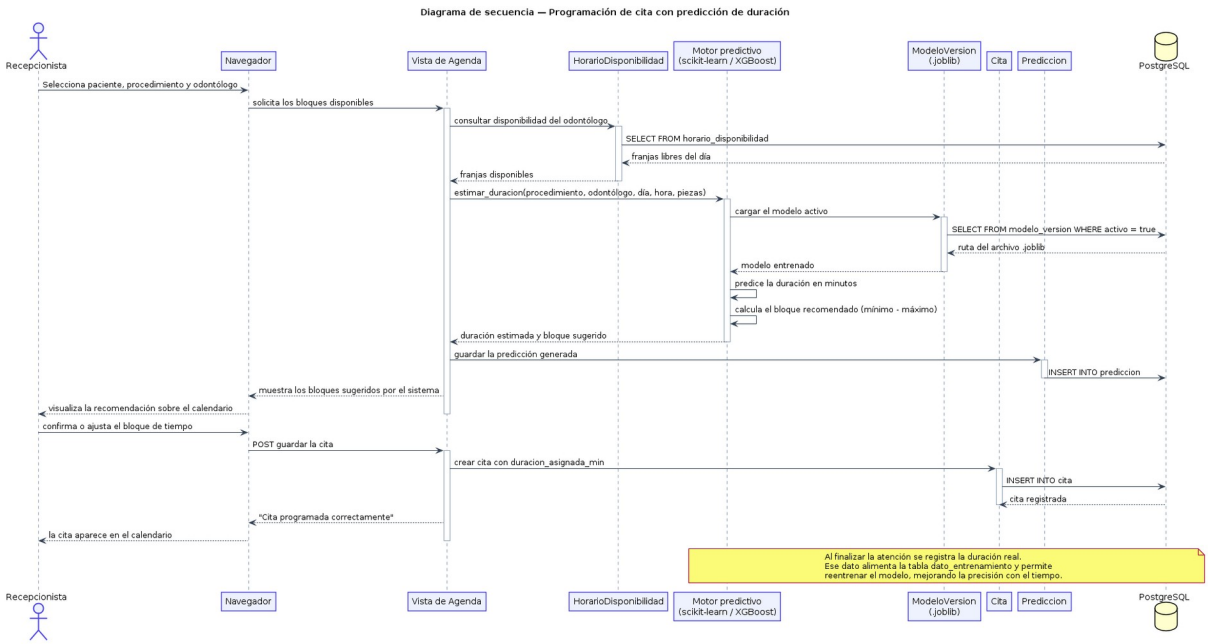


Figura 10: Diagrama de secuencia: programación de cita con predicción de duración Fuente: elaboración propia.

3.2.5 Diagramas de actividad de los procesos

Los diagramas de secuencia de la sección anterior describen el intercambio de mensajes entre los componentes del sistema. Esta sección los complementa con el flujo de trabajo de cada proceso, expresado en diagramas de actividad con calles por responsable. La distinción importa porque permite verificar dos cosas que la secuencia no muestra: el orden en que se aplican las validaciones y quién ejecuta cada actividad. Se documentan los 16 procesos del sistema, uno por cada funcionalidad principal de los ocho módulos.

Inicio de sesión y control de acceso (Figura 11). El usuario se identifica con su correo electrónico; el sistema verifica la contraseña cifrada y el estado de la cuenta, y construye el menú a partir de los módulos habilitados para su rol.

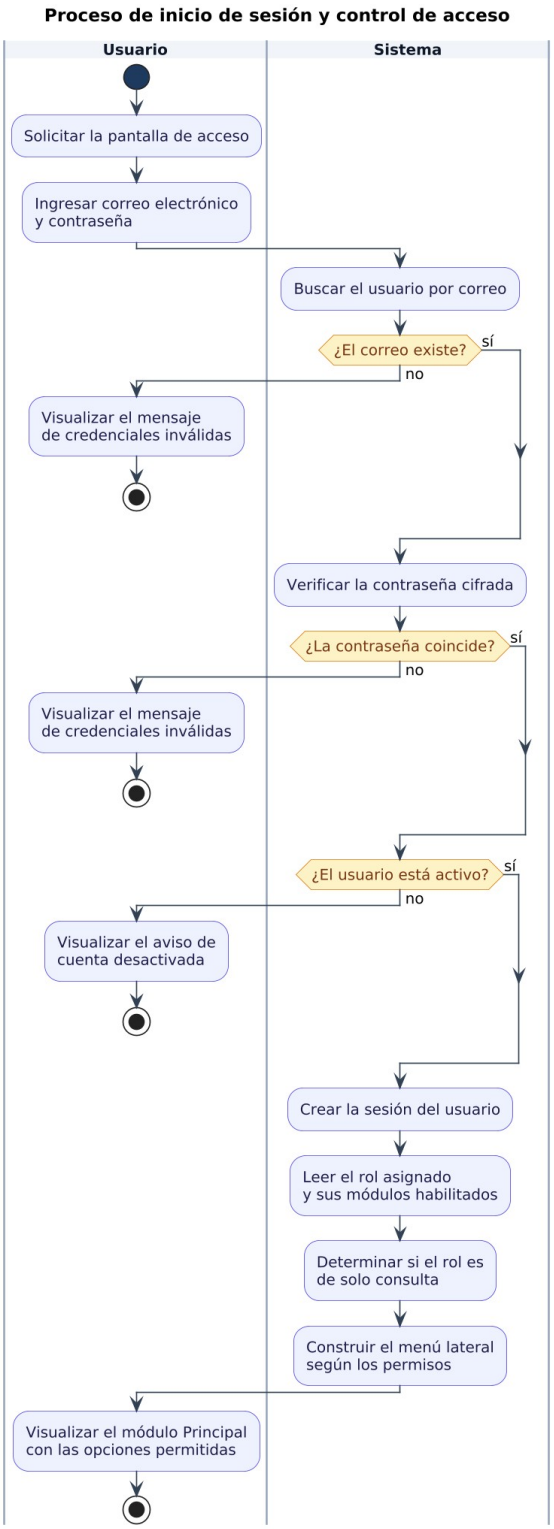


Figura 11: Diagrama de actividad: inicio de sesión y control de acceso Fuente: elaboración propia.

Mantenimiento de usuarios (Figura 12). El administrador captura los datos del usuario; el sistema valida que el correo no esté asignado, exige una contraseña conforme a las reglas de seguridad y deja constancia de la creación en el historial de auditoría.

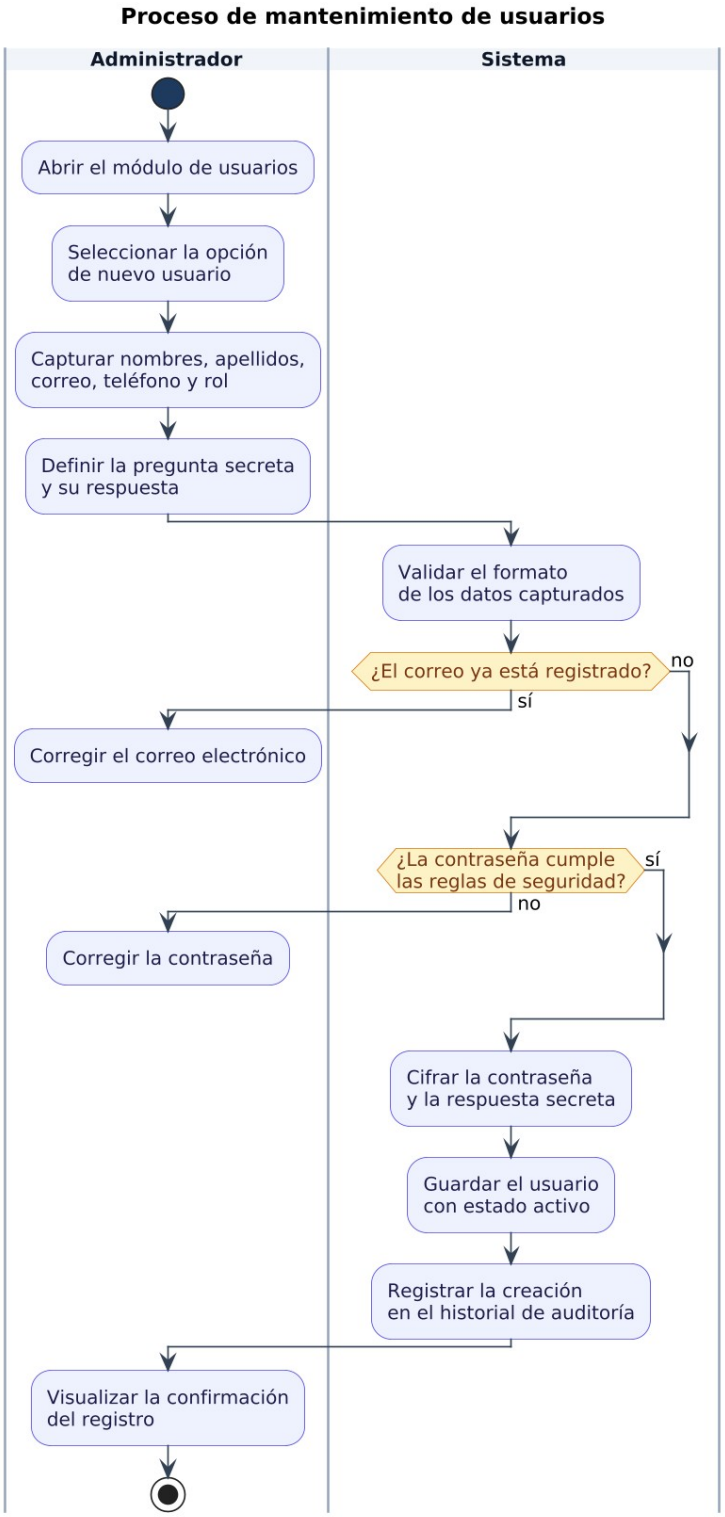


Figura 12: Diagrama de actividad: mantenimiento de usuarios Fuente: elaboración propia.

Recuperación de contraseña (Figura 13). La recuperación se resuelve con la pregunta secreta registrada al dar de alta la cuenta, cuya respuesta se compara contra el valor cifrado almacenado.

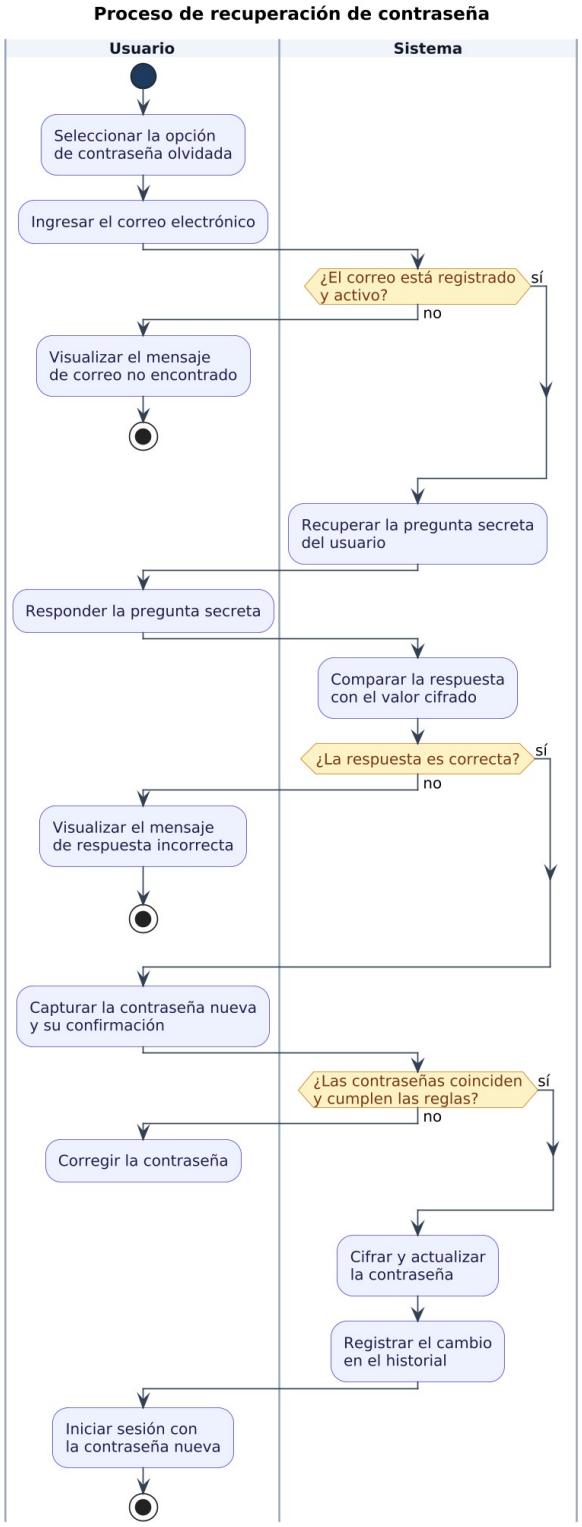


Figura 13: Diagrama de actividad: recuperación de contraseña Fuente: elaboración propia.

Registro de paciente y antecedentes médicos (Figura 14). El proceso comienza por la búsqueda, para evitar el registro duplicado, y concluye con la captura iterativa de los antecedentes médicos relevantes.

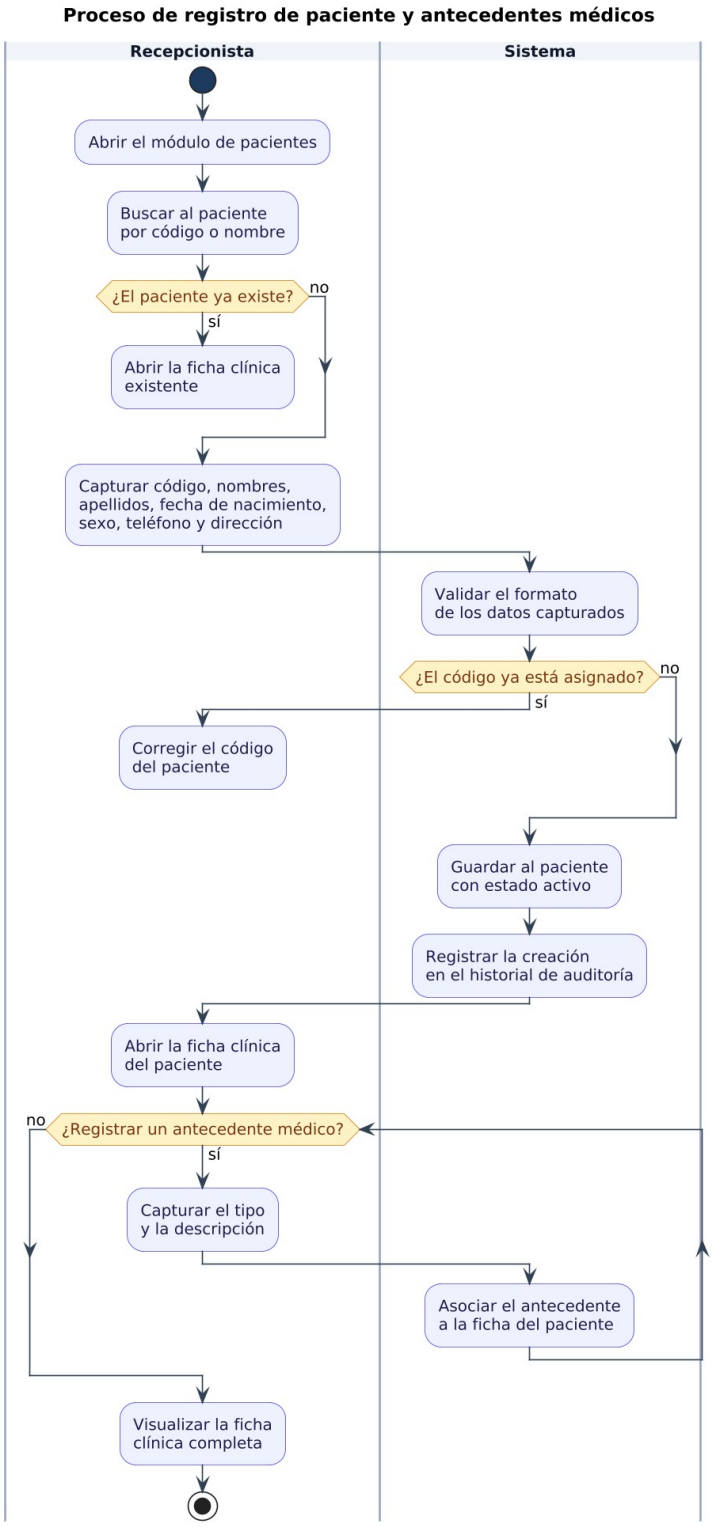


Figura 14: Diagrama de actividad: registro de paciente y antecedentes médicos Fuente: elaboración propia.

Mantenimiento del catálogo odontológico (Figura 15). Las cuatro entidades del catálogo comparten el mismo comportamiento, de modo que el proceso se representa una sola vez. La baja nunca elimina el registro: lo marca como inactivo para conservar su vínculo con las citas históricas.

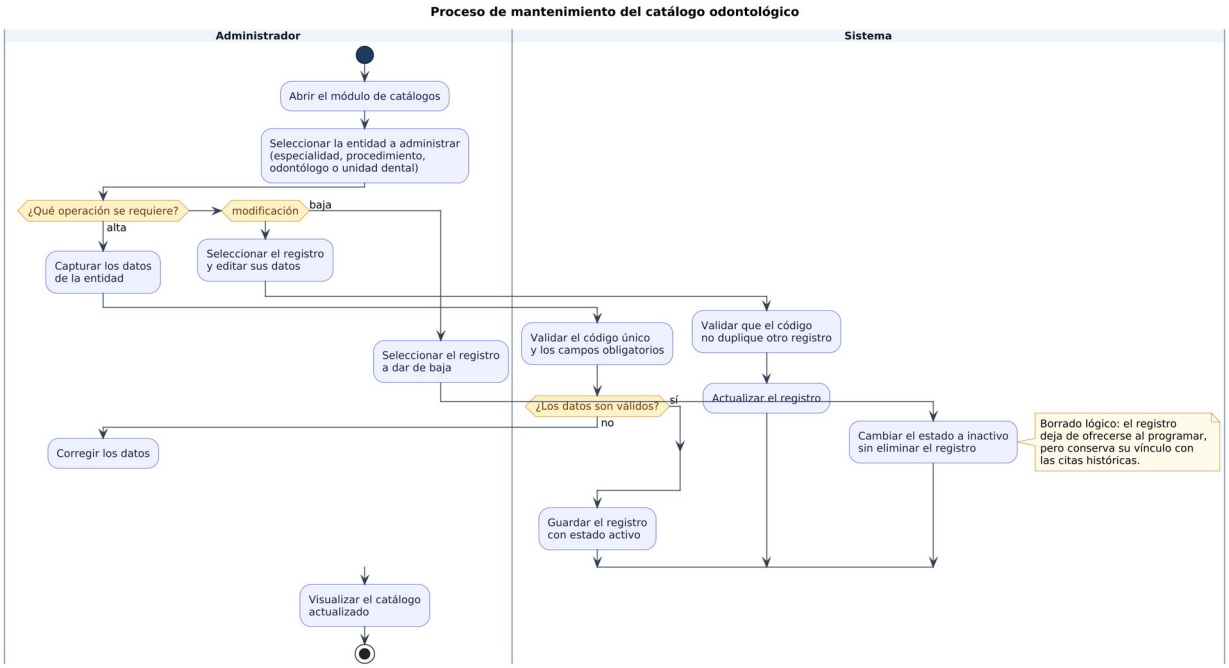


Figura 15: Diagrama de actividad: mantenimiento del catálogo odontológico Fuente: elaboración propia.

Configuración de horarios y ausencias (Figura 16). El patrón semanal define la disponibilidad habitual y la ausencia la descuenta en una fecha concreta. La disponibilidad efectiva es la diferencia entre ambos.

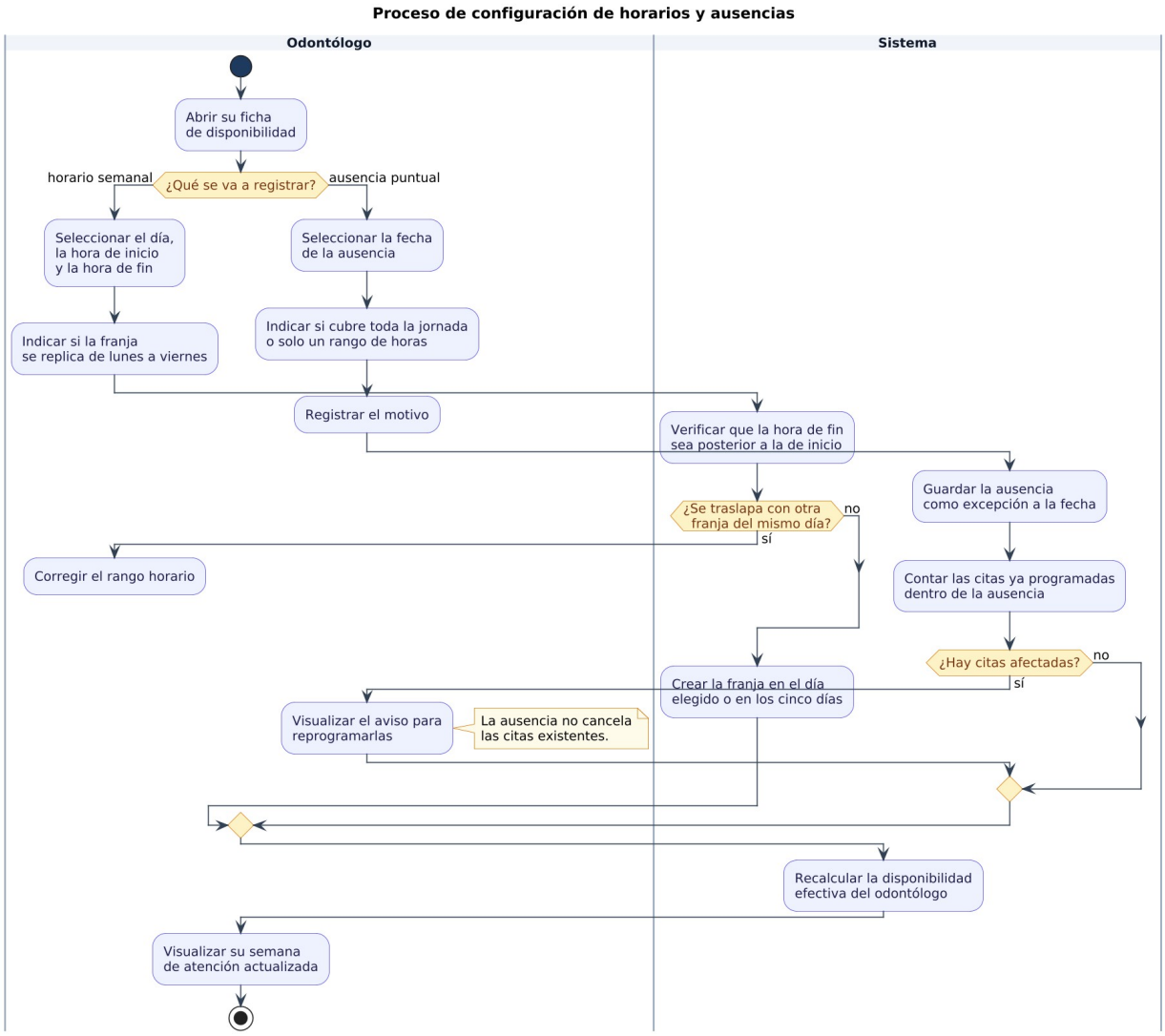


Figura 16: Diagrama de actividad: configuración de horarios y ausencias Fuente: elaboración propia.

Programación de cita con estimación predictiva (Figura 17). Es el proceso central del módulo principal. Antes de guardar, el sistema verifica cuatro condiciones: que la fecha no sea pasada, que la cita quepa en una franja efectiva, que no se traslape con otra cita del profesional y que quede una unidad dental libre.

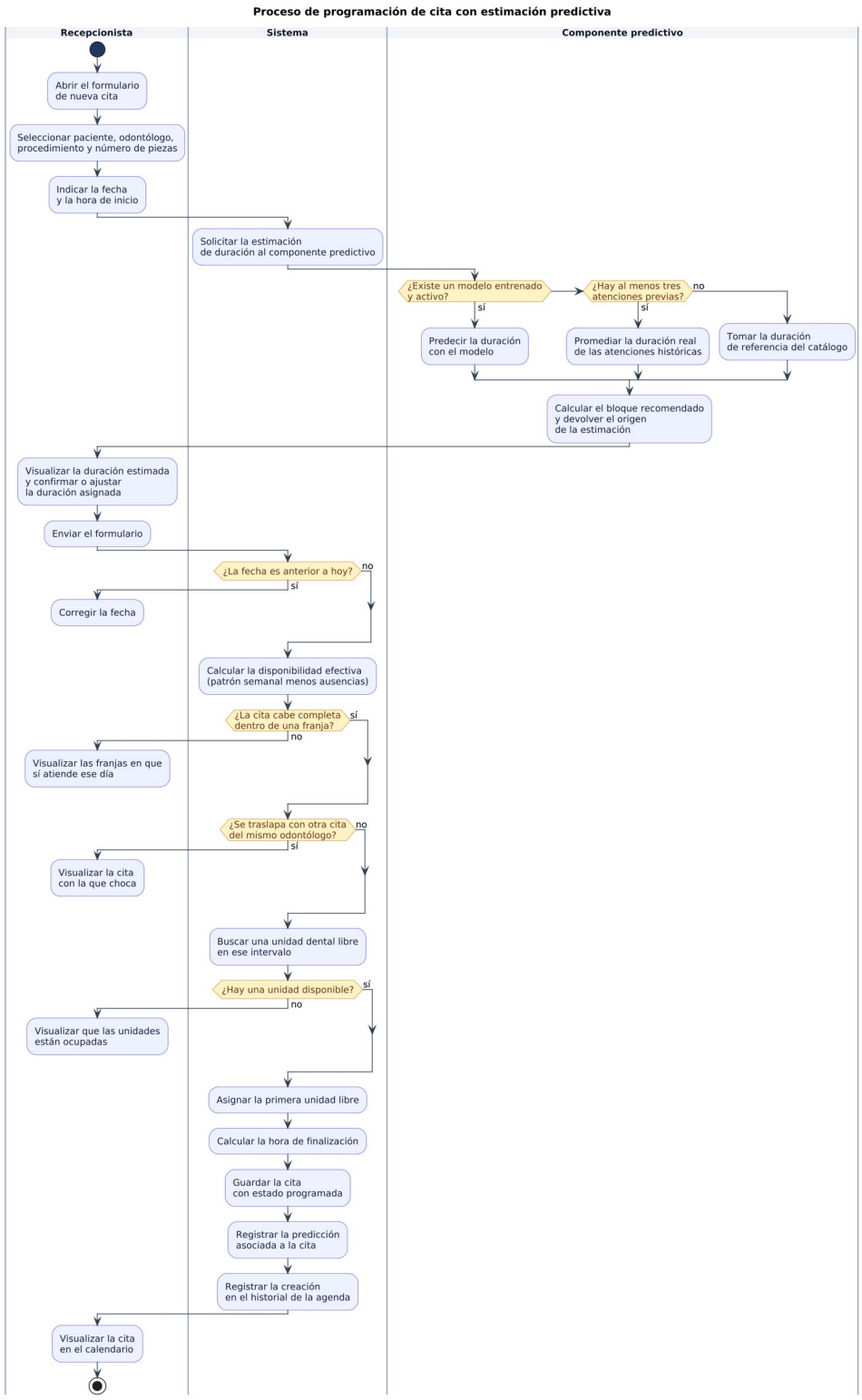


Figura 17: Diagrama de actividad: programación de cita con estimación predictiva Fuente: elaboración propia.

Recomendación de bloques de tiempo (Figura 18). El sistema recorre las franjas disponibles en pasos de quince minutos y descarta los inicios que ya pasaron, los que chocan con una cita vigente y aquellos en que no queda sillón libre.

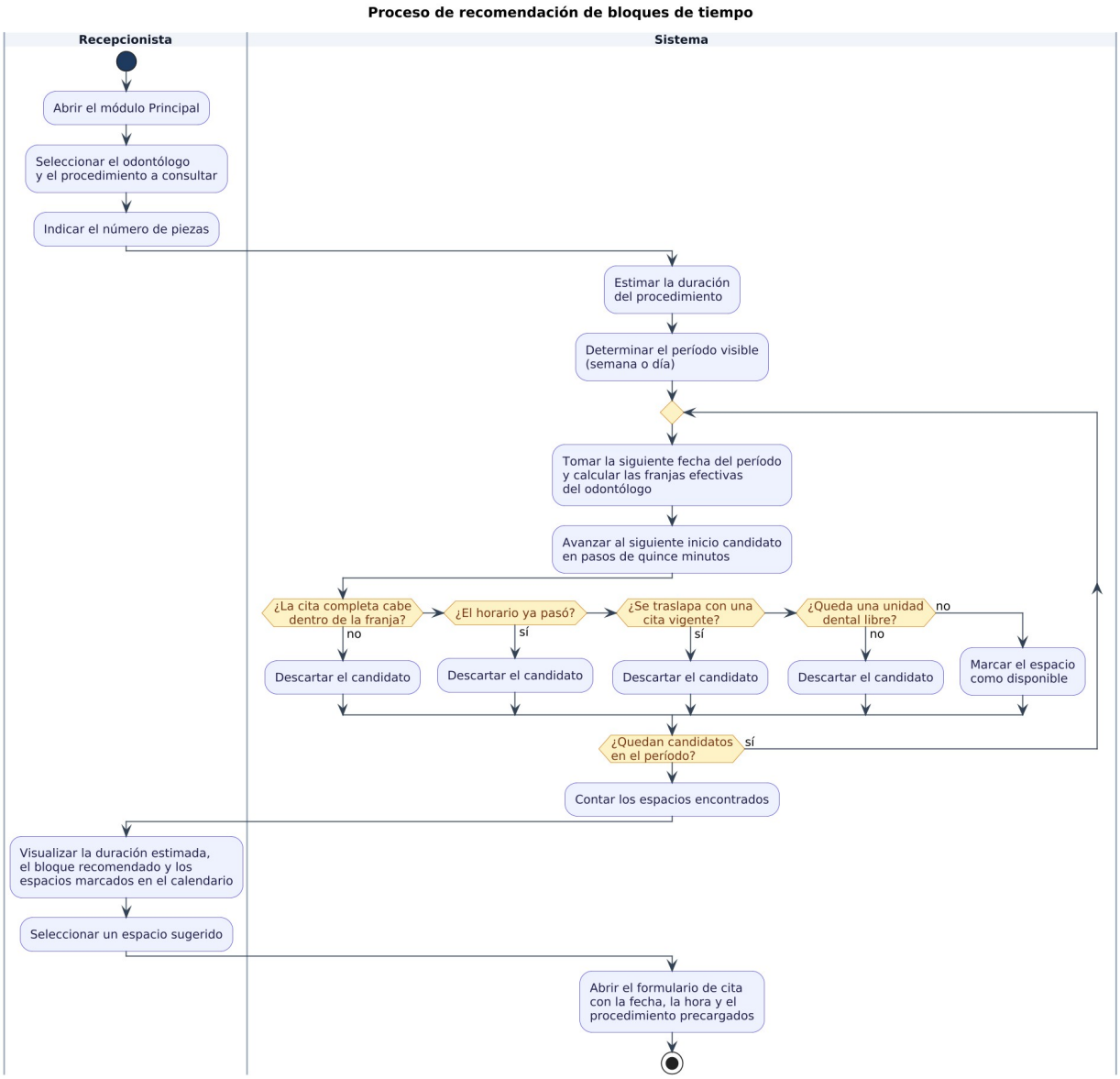


Figura 18: Diagrama de actividad: recomendación de bloques de tiempo Fuente: elaboración propia.

Registro de la atención y realimentación del modelo (Figura 19). Al cerrar la cita, la duración real se incorpora al conjunto de entrenamiento. Si la cita atendía un renglón de un plan de tratamiento, el renglón se cierra y el avance del plan se recalcula sin intervención del usuario.

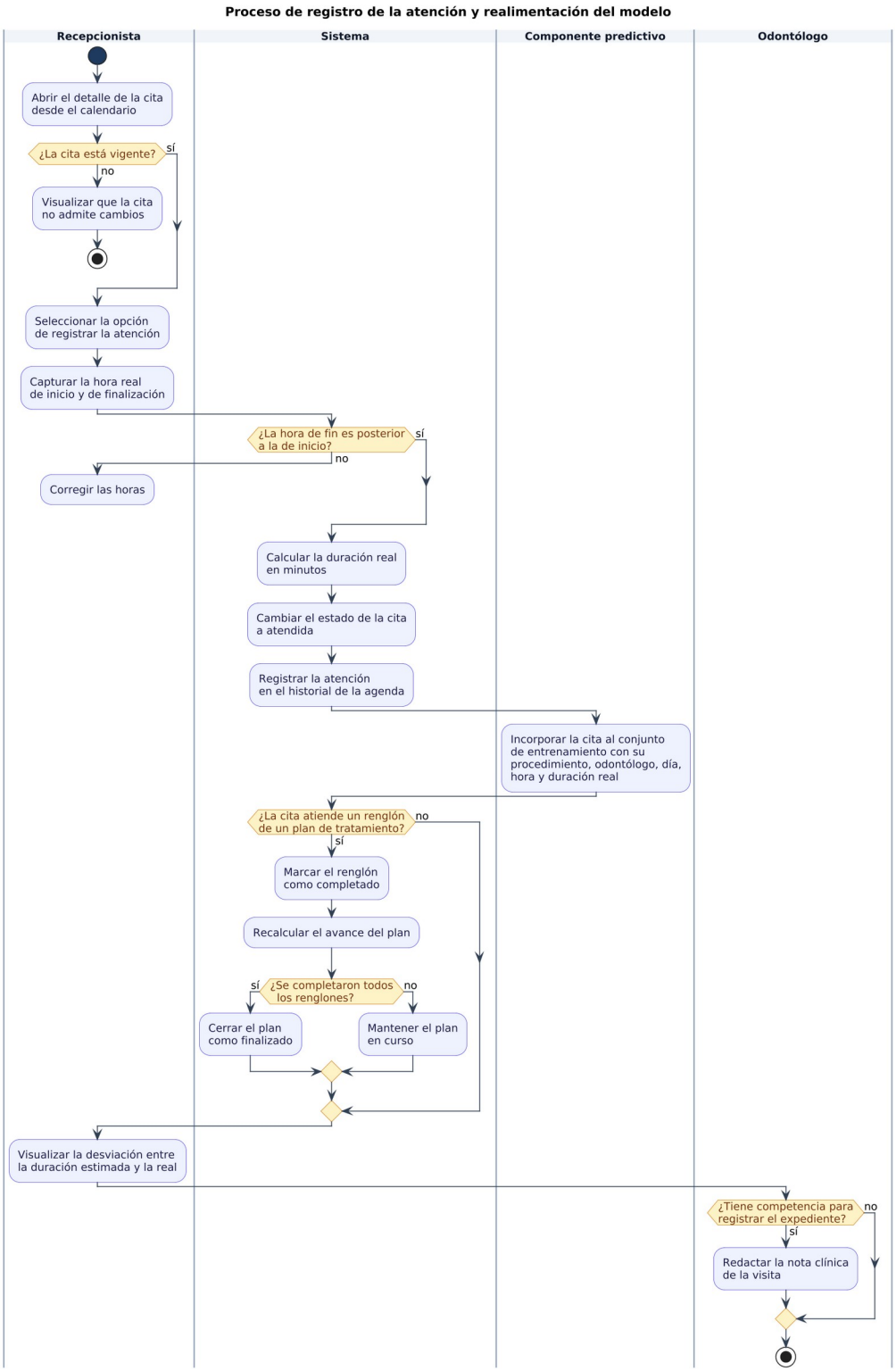


Figura 19: Diagrama de actividad: registro de la atención y realimentación del modelo Fuente: elaboración propia.

Reprogramación y cancelación de citas (Figura 20). La reprogramación aplica las mismas validaciones del registro inicial. La cancelación conserva la cita mediante borrado lógico y libera la unidad dental.

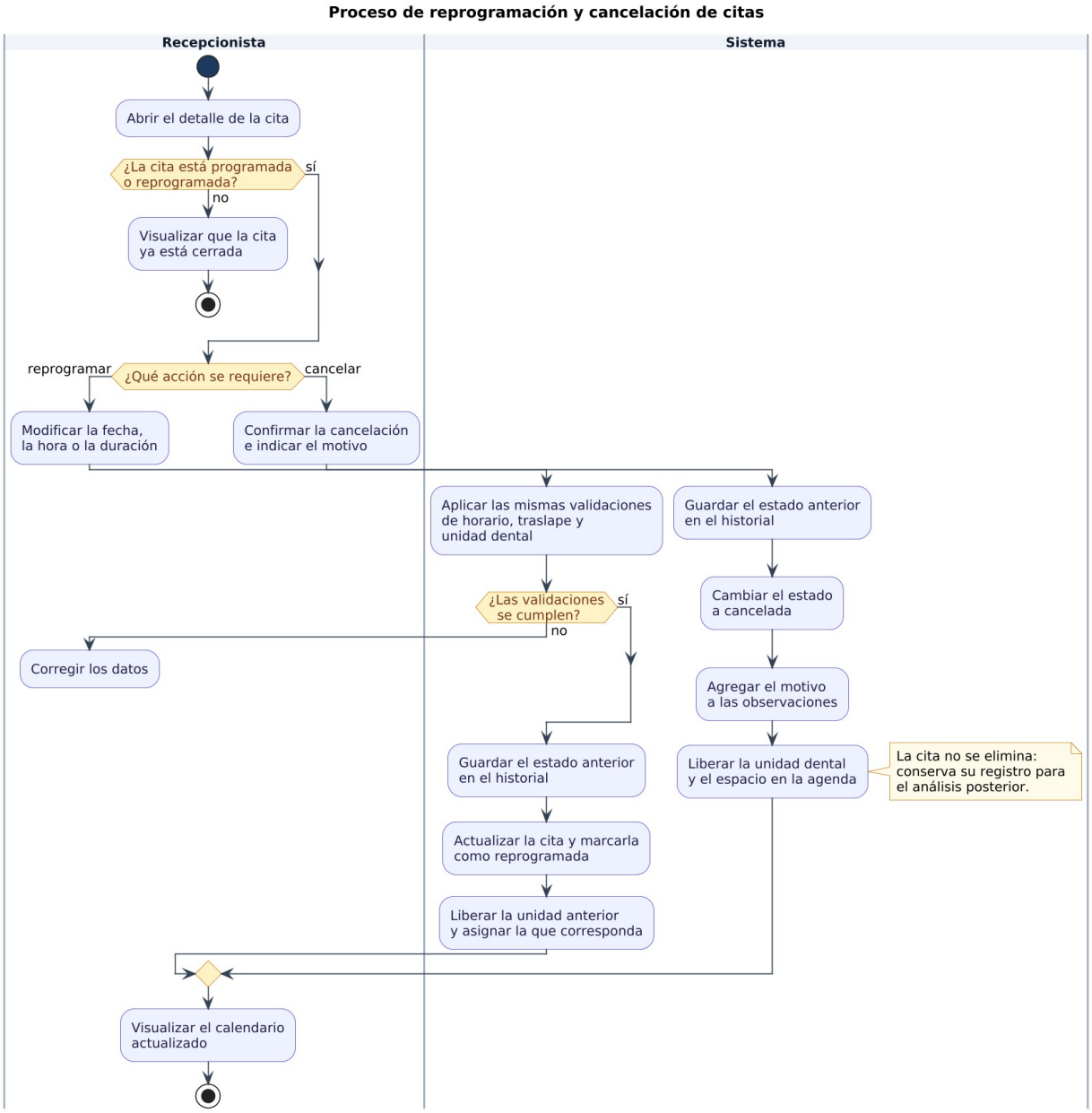


Figura 20: Diagrama de actividad: reprogramación y cancelación de citas Fuente: elaboración propia.

Registro del odontograma (Figura 21). Cada hallazgo se agrega sin modificar los anteriores, de modo que la pieza conserva su historia clínica y el odontograma presenta el registro vigente.

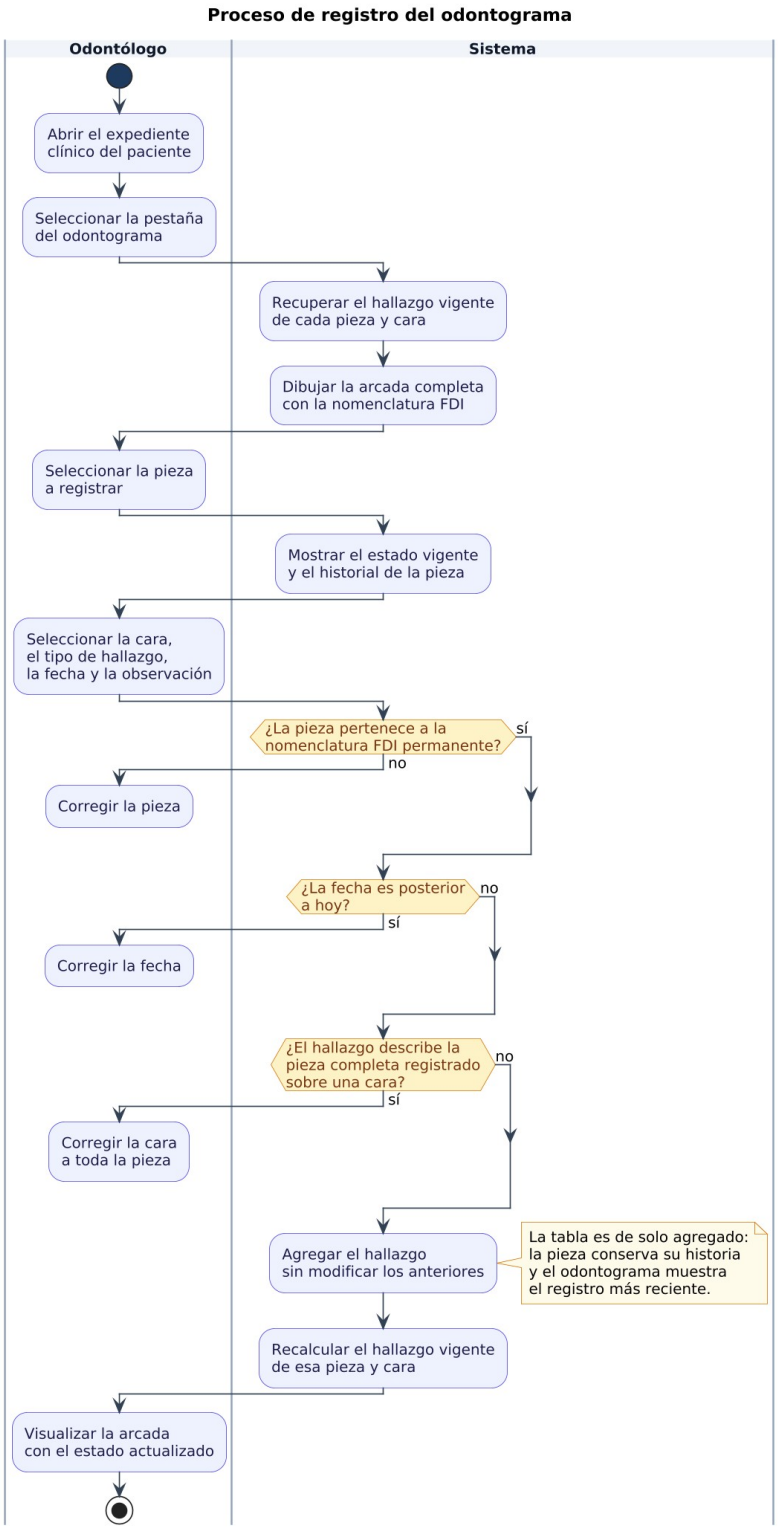


Figura 21: Diagrama de actividad: registro del odontograma Fuente: elaboración propia.

Elaboración del plan de tratamiento (Figura 22). Cada renglón recibe del componente predictivo su duración estimada, con lo que el plan queda presupuestado en tiempo además de en dinero, y se reparte en sesiones antes de agendar la primera cita.

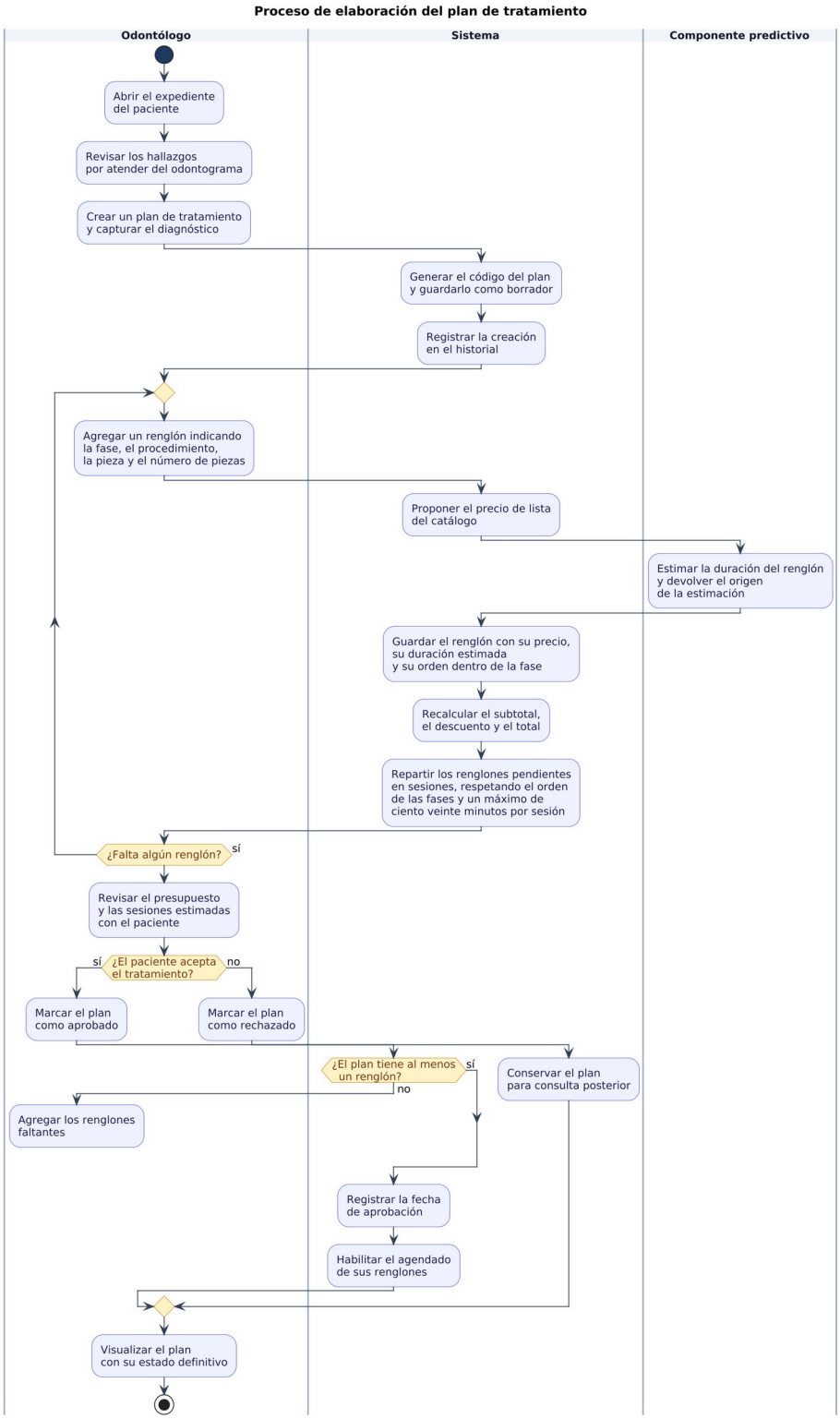


Figura 22: Diagrama de actividad: elaboración del plan de tratamiento Fuente: elaboración propia.

Ejecución del plan de tratamiento (Figura 23). El renglón aprobado se agenda con sus datos precargados, se atiende y se cierra automáticamente, de modo que el avance del plan no dependa de una marca manual.

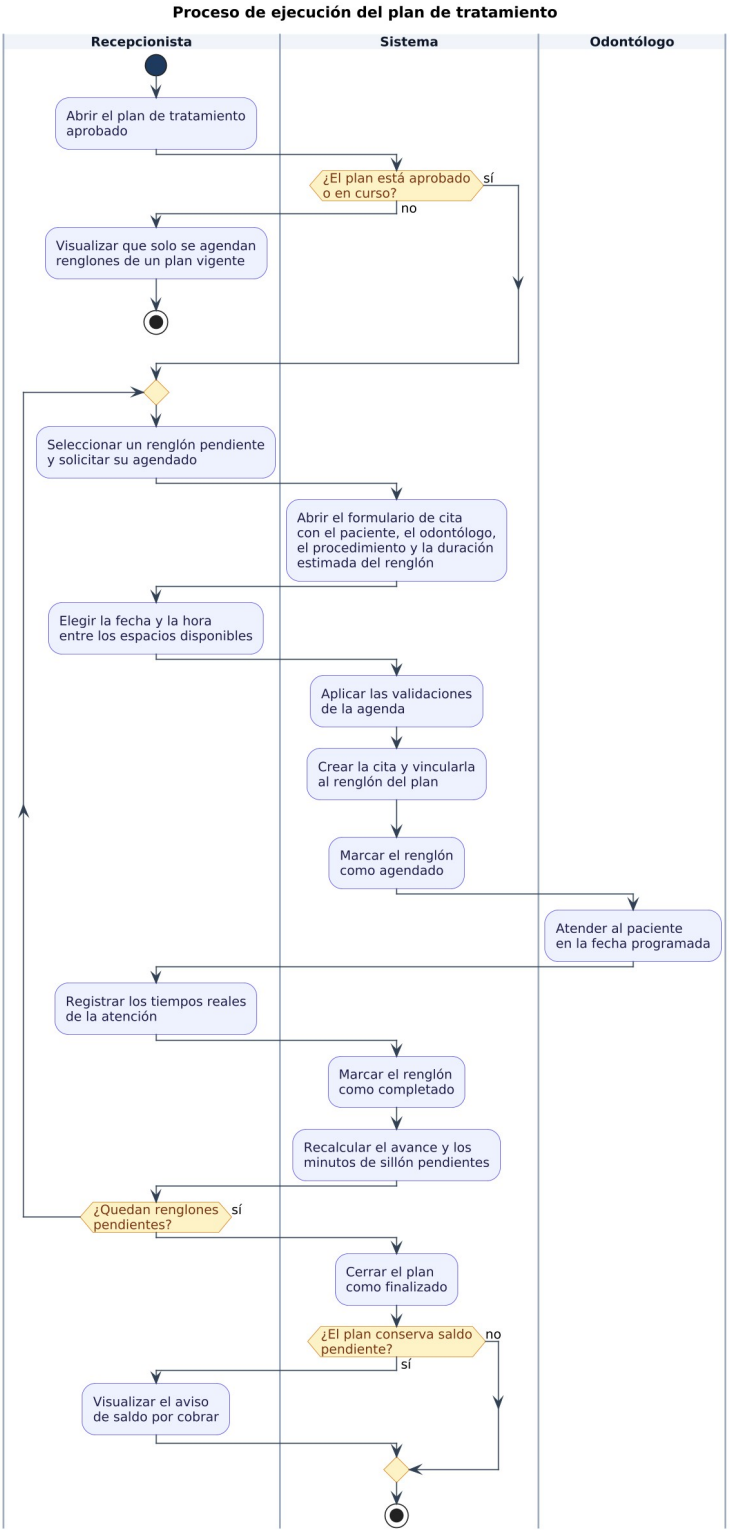


Figura 23: Diagrama de actividad: ejecución del plan de tratamiento Fuente: elaboración propia.

Registro de la nota clínica de evolución (Figura 24). La nota documenta la visita y aporta al modelo las tres variables que el registro de tiempos no puede conocer: piezas tratadas, uso de anestesia y aparición de complicaciones.

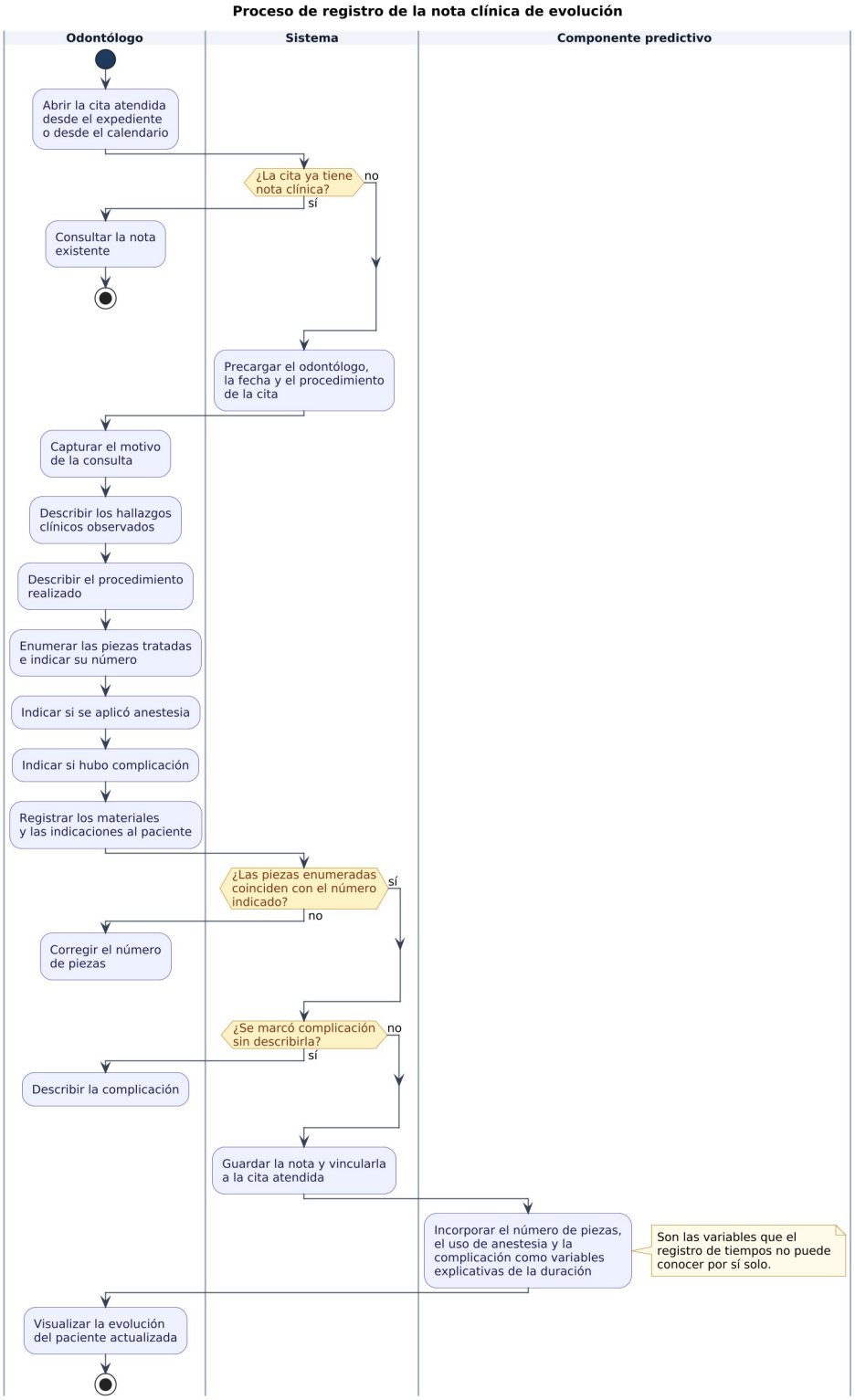


Figura 24: Diagrama de actividad: registro de la nota clínica de evolución Fuente: elaboración propia.

Registro de abonos y control de saldo (Figura 25). El abono no puede exceder el saldo vigente ni fecharse en el futuro. La corrección de un abono se resuelve por anulación lógica, conservando la traza en el estado de cuenta.

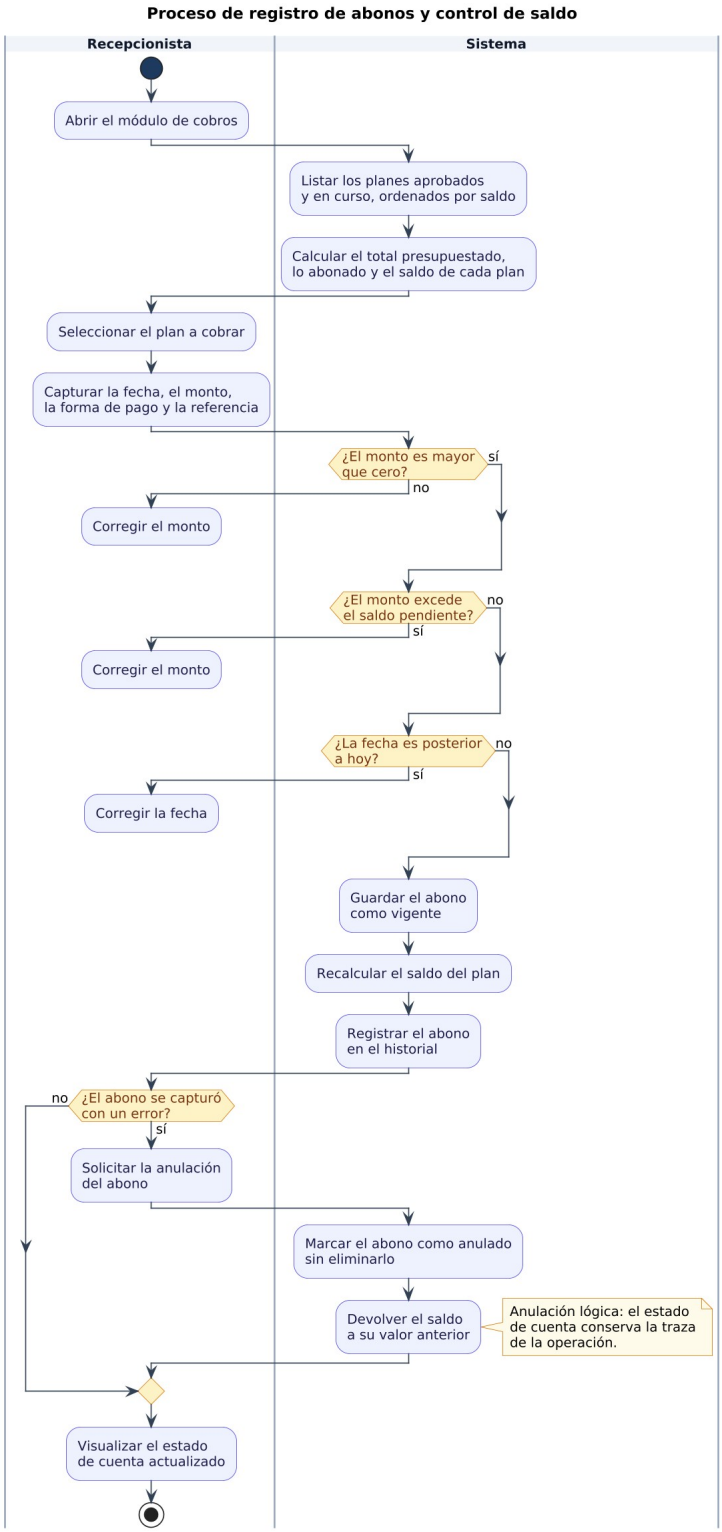


Figura 25: Diagrama de actividad: registro de abonos y control de saldo Fuente: elaboración propia.

Entrenamiento y versionado del modelo predictivo (Figura 26). La versión nueva se activa únicamente si reduce el error frente a la que está en producción; en caso contrario se conserva como histórico para efectos comparativos.

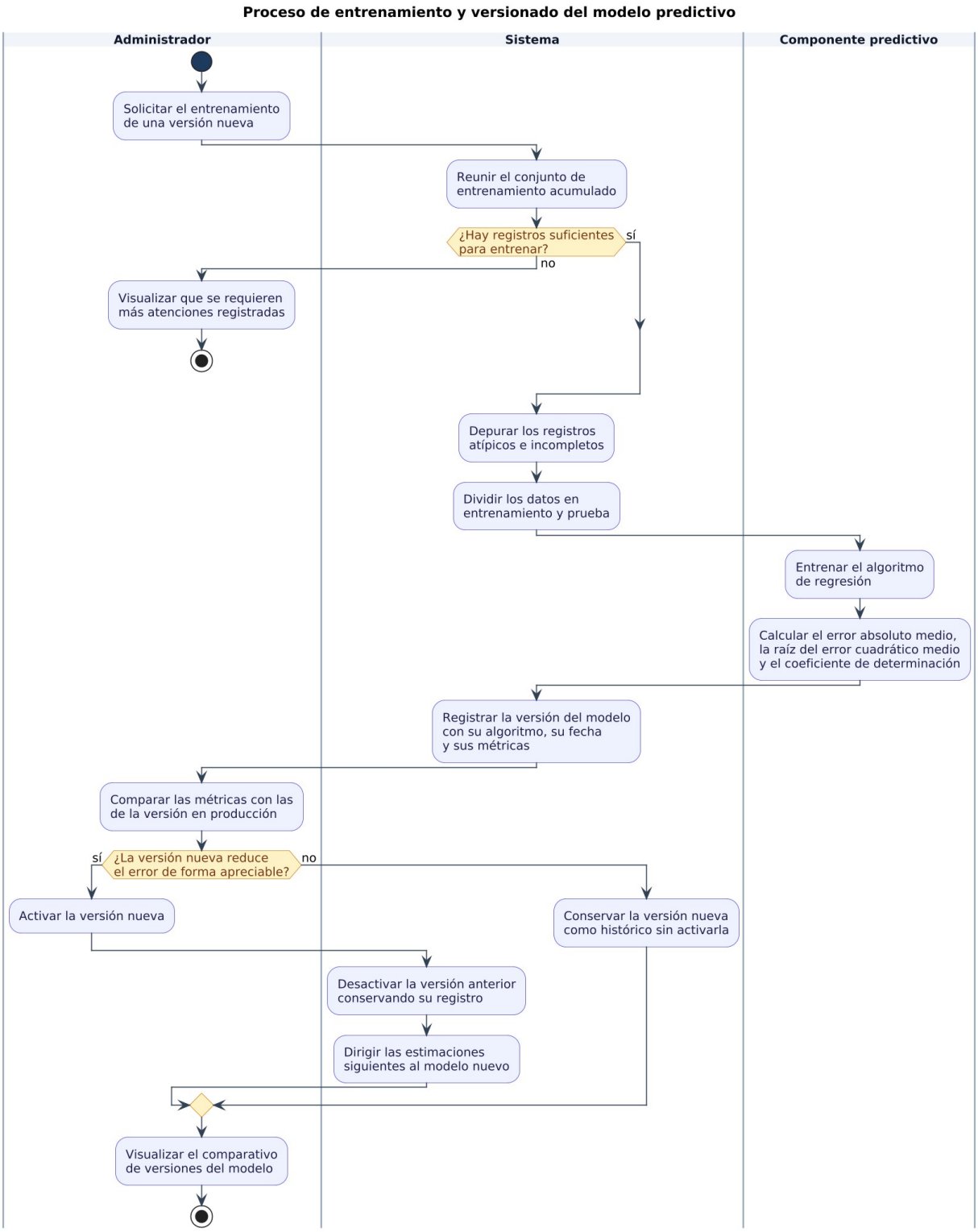


Figura 26: Diagrama de actividad: entrenamiento y versionado del modelo predictivo Fuente: elaboración propia.

3.2.6 Diagrama de componentes y arquitectura de despliegue

El diagrama de componentes de la Figura 27 representa la organización física del sistema y la distribución de sus elementos sobre la infraestructura de despliegue. Las solicitudes del navegador ingresan por Nginx, que atiende directamente los archivos estáticos y delega las peticiones dinámicas a Gunicorn, el cual ejecuta la aplicación Django. Los ocho módulos acceden a la base de datos exclusivamente a través del mapeador objeto-relacional, lo que centraliza el acceso a los datos y evita la dispersión de sentencias SQL en el código. El motor predictivo se representa como un componente independiente que carga el modelo entrenado desde un artefacto serializado, y el flujo de integración y despliegue continuos aparece como un elemento externo que actúa sobre el servidor tras cada cambio aprobado en el repositorio.

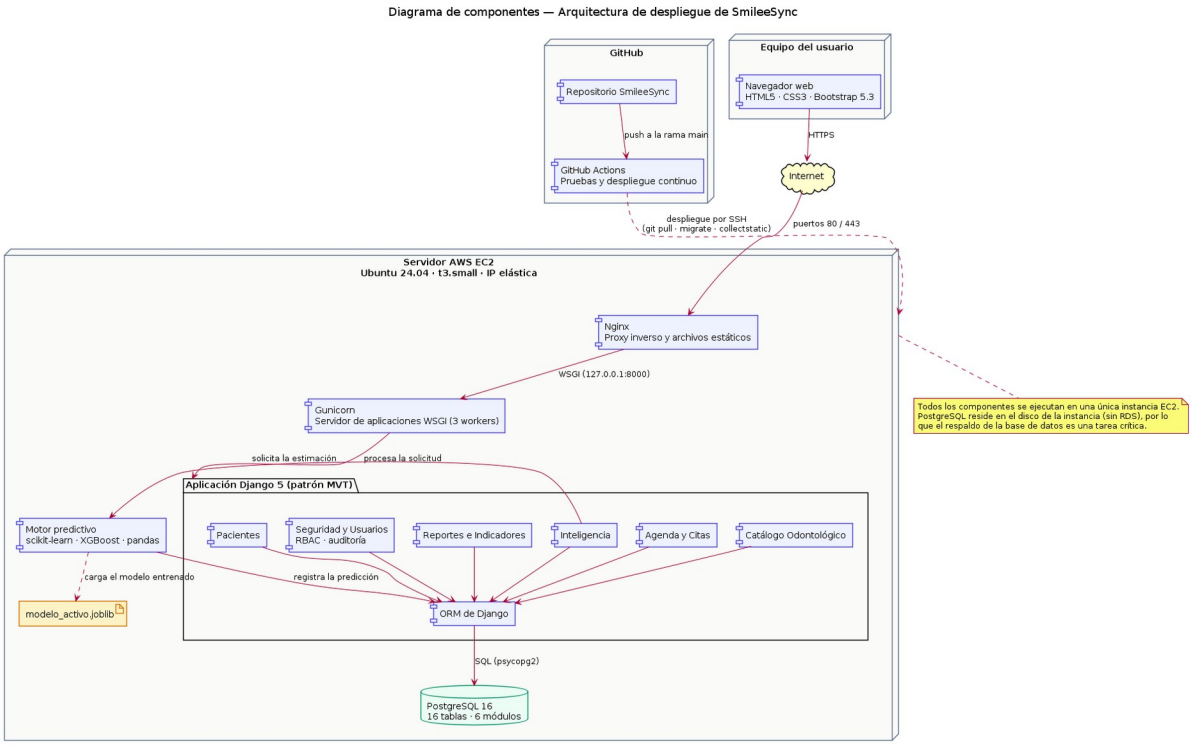


Figura 27: Diagrama de componentes y despliegue del sistema SmileeSync Fuente: elaboración propia.

3.3 Diseño de interfaces

El diseño de las interfaces de usuario se rige por tres criterios. El primero es la consistencia estructural: todas las pantallas comparten la misma disposición —menú lateral con los módulos habilitados según el rol, barra superior con la identificación del usuario y área central de contenido —, de modo que el usuario reconozca de inmediato dónde se encuentra. El segundo es la economía visual: se privilegian listados con búsqueda, formularios agrupados por bloques temáticos y una

jerarquía tipográfica clara, evitando elementos decorativos que no aporten información. El tercero es la retroalimentación inmediata: toda validación se comunica junto al campo que la origina y con un mensaje que indica la acción correctiva.

Las interfaces se implementan con el marco de estilos Bootstrap, que provee un sistema de rejilla responsiva y componentes visuales estandarizados, lo que garantiza que las pantallas se adapten a computadoras, tabletas y teléfonos sin desarrollar versiones separadas. A continuación se presentan las pantallas diseñadas, agrupadas por módulo. Las del módulo de Reportes y las del módulo de Inteligencia se documentan en el capítulo 4, junto con el código que genera sus gráficas. En la especificación de campos, la letra R indica que el campo es requerido y la letra E que corresponde a un campo de entrada.

3.3.1 Módulo de Seguridad y Usuarios

La pantalla de inicio de sesión (Figura 28) constituye el punto de entrada al sistema. Solicita el correo electrónico y la contraseña, y ofrece el acceso al procedimiento de recuperación mediante pregunta secreta. El acceso se realiza por correo electrónico y no por nombre de usuario, decisión que reduce los errores de autenticación al emplear un dato que el usuario ya conoce y no necesita memorizar. La Tabla 47 especifica sus campos.

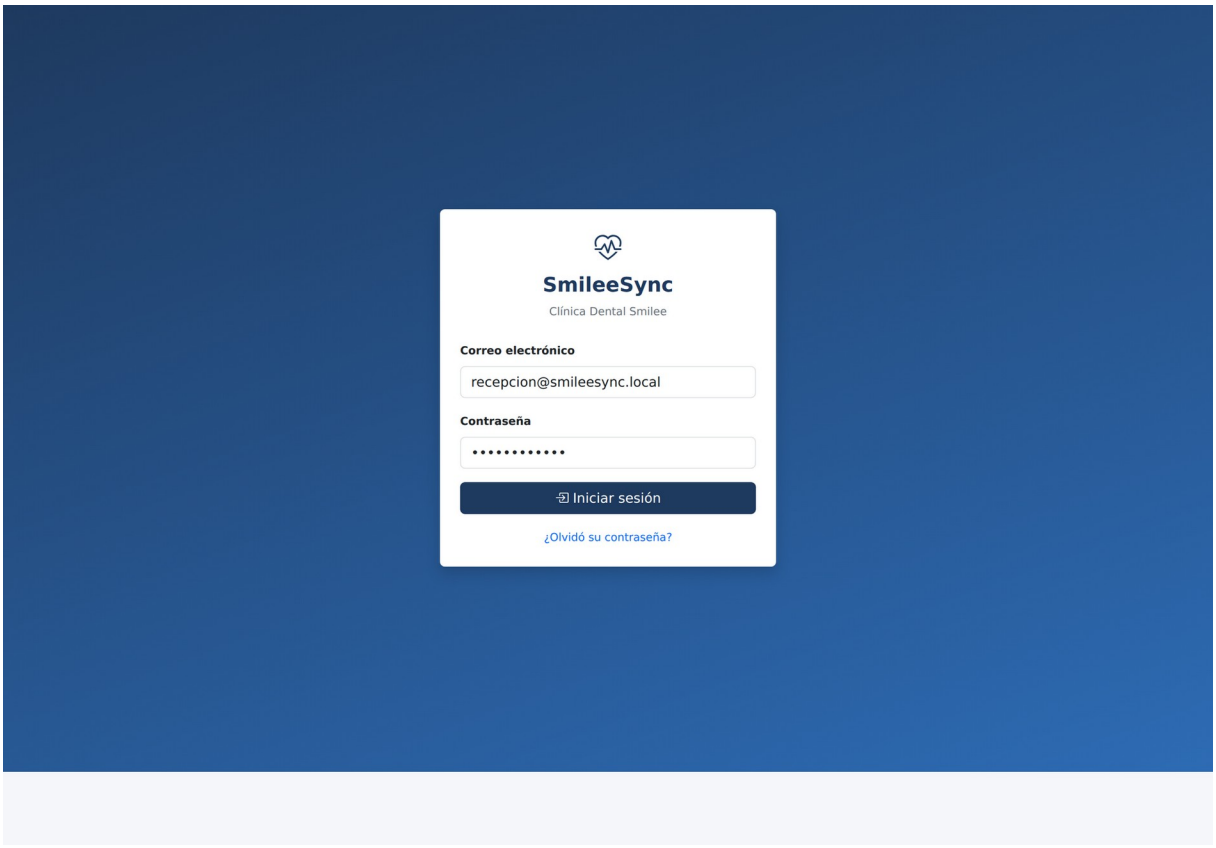


Figura 28: Interfaz de inicio de sesión Fuente: elaboración propia.

Campo	Tipo de campo	Observación
Correo electrónico	R, E	Campo de entrada que registra el correo del usuario; constituye el identificador de acceso y se valida contra el formato usuario@dominio.com.
Contraseña	R, E	Campo de entrada de texto no visible; se compara contra el valor cifrado almacenado en la base de datos.
¿Olvidó su contraseña?	Enlace	Dirige al procedimiento de recuperación mediante la pregunta secreta configurada por el usuario.

Tabla 47: Especificación de campos de la pantalla de inicio de sesión Fuente: elaboración propia.

El listado de usuarios (Figura 29) presenta al personal registrado con su rol, su estado y las acciones disponibles. La activación y desactivación aplican borrado lógico, por lo que un usuario inactivo permanece visible en el listado, diferenciado visualmente, y conserva su vínculo con los registros históricos que generó.

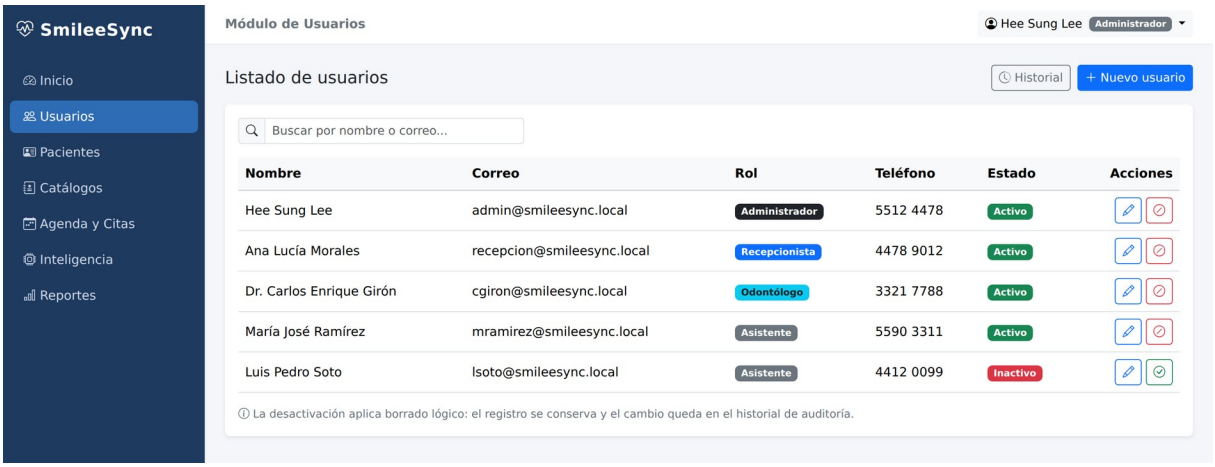


Figura 29: Interfaz del listado de usuarios Fuente: elaboración propia.

El formulario de usuario (Figura 30) se presenta deliberadamente en estado de error para documentar el comportamiento de las validaciones. El campo de nombre rechaza el valor por contener caracteres numéricos, y el campo de correo lo rechaza por no corresponder al formato de una dirección electrónica válida. La Tabla 48 especifica el comportamiento de cada campo.

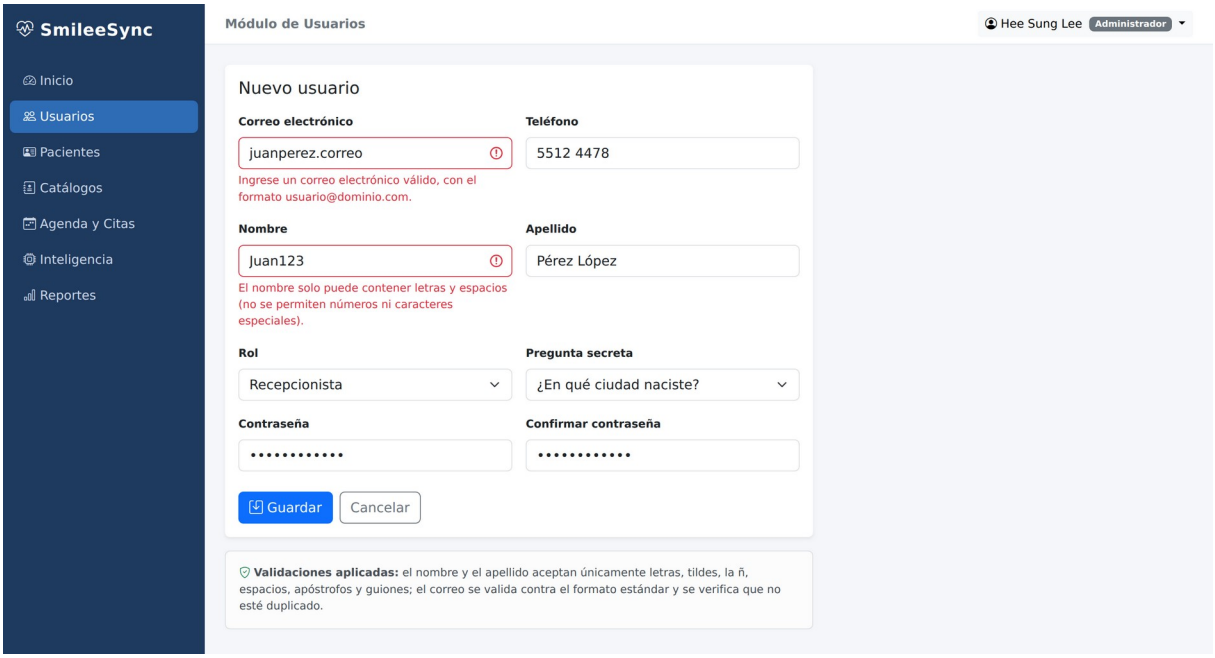


Figura 30: Interfaz del formulario de usuario con validaciones activas Fuente: elaboración propia.

Campo	Tipo de campo	Observación
Correo electrónico	R, E	Se valida el formato usuario@dominio.com y se verifica que no exista otro usuario registrado con el mismo correo.
Nombre	R, E	Admite únicamente letras, tildes, la ñ, espacios, apóstrofos y guiones; rechaza números y caracteres especiales.
Apellido	R, E	Aplica la misma validación que el campo de nombre.
Teléfono	E	Campo de entrada opcional para el número de contacto del usuario.

Campo	Tipo de campo	Observación
Rol	R, Selección	Lista desplegable que presenta únicamente los roles activos; determina los módulos a los que tendrá acceso el usuario.
Pregunta secreta	R, Selección	Lista desplegable de preguntas predefinidas, utilizada en el proceso de recuperación de contraseña.
Respuesta secreta	R, E	Se almacena cifrada y normalizada a minúsculas para la comparación posterior.
Contraseña	R, E	Campo de texto no visible; se almacena cifrado mediante una función de derivación de claves.
Confirmar contraseña	R, E	Debe coincidir exactamente con el campo anterior; de lo contrario el formulario es rechazado.

Tabla 48: Especificación de campos del formulario de usuario Fuente: elaboración propia.

3.3.2 Módulo de Pacientes

El listado de pacientes (Figura 31) permite localizar a un paciente por código, nombre o teléfono, y muestra la fecha de su última cita como referencia rápida para el personal de recepción.

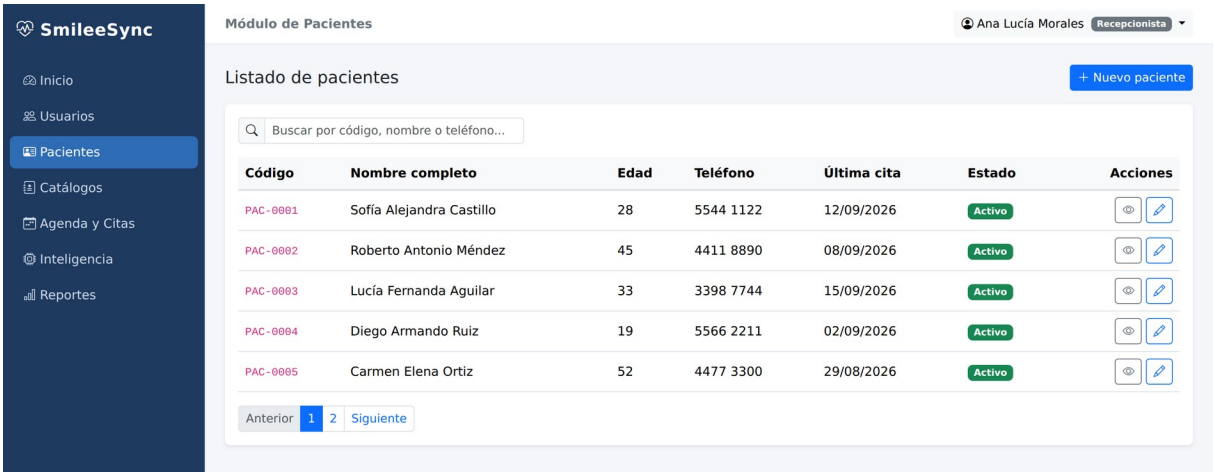


Figura 31: Interfaz del listado de pacientes Fuente: elaboración propia.

El formulario de registro de paciente (Figura 32) organiza la información en datos de identificación y datos de contacto. El código del paciente se genera automáticamente para evitar duplicidades y errores de digitación.

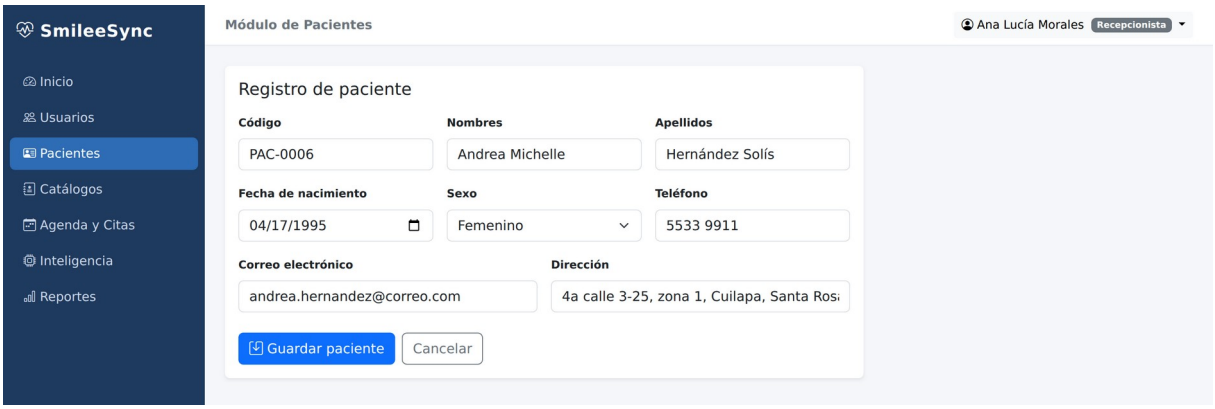


Figura 32: Interfaz del formulario de registro de paciente Fuente: elaboración propia.

La ficha del paciente (Figura 33) consolida en una sola vista la información personal, los antecedentes médicos y el historial de citas atendidas con su duración real. Esta pantalla está dirigida principalmente al odontólogo, quien requiere consultar los antecedentes —una alergia documentada, por ejemplo— inmediatamente antes de la atención.



Figura 33: Interfaz de la ficha clínica del paciente Fuente: elaboración propia.

3.3.3 Módulo de Catálogo Odontológico

El catálogo de procedimientos (Figura 34) organiza los servicios de la clínica por especialidad e indica su nivel de complejidad y su duración promedio de referencia. Este valor constituye la estimación inicial que emplea el sistema mientras el modelo predictivo no dispone de datos históricos suficientes.



Figura 34: Interfaz del catálogo de procedimientos Fuente: elaboración propia.

El formulario de procedimiento (Figura 35) permite registrar un nuevo servicio indicando su código, su especialidad, su nivel de complejidad y su duración promedio.



Figura 35: Interfaz del formulario de procedimiento Fuente: elaboración propia.

El formulario de odontólogo (Figura 36) registra al profesional y permite vincularlo con su usuario del sistema, lo que habilita que consulte su propia agenda y registre los tiempos reales de atención.



Figura 36: Interfaz del formulario de registro de odontólogo Fuente: elaboración propia.

El listado de odontólogos (Figura 37) presenta a los profesionales registrados con su número de colegiado, su especialidad y el usuario del sistema con que cada uno ingresa. La columna del usuario vinculado es la que permite que el profesional vea su propia agenda al iniciar sesión y que registre sus ausencias sin intervención de la recepción.

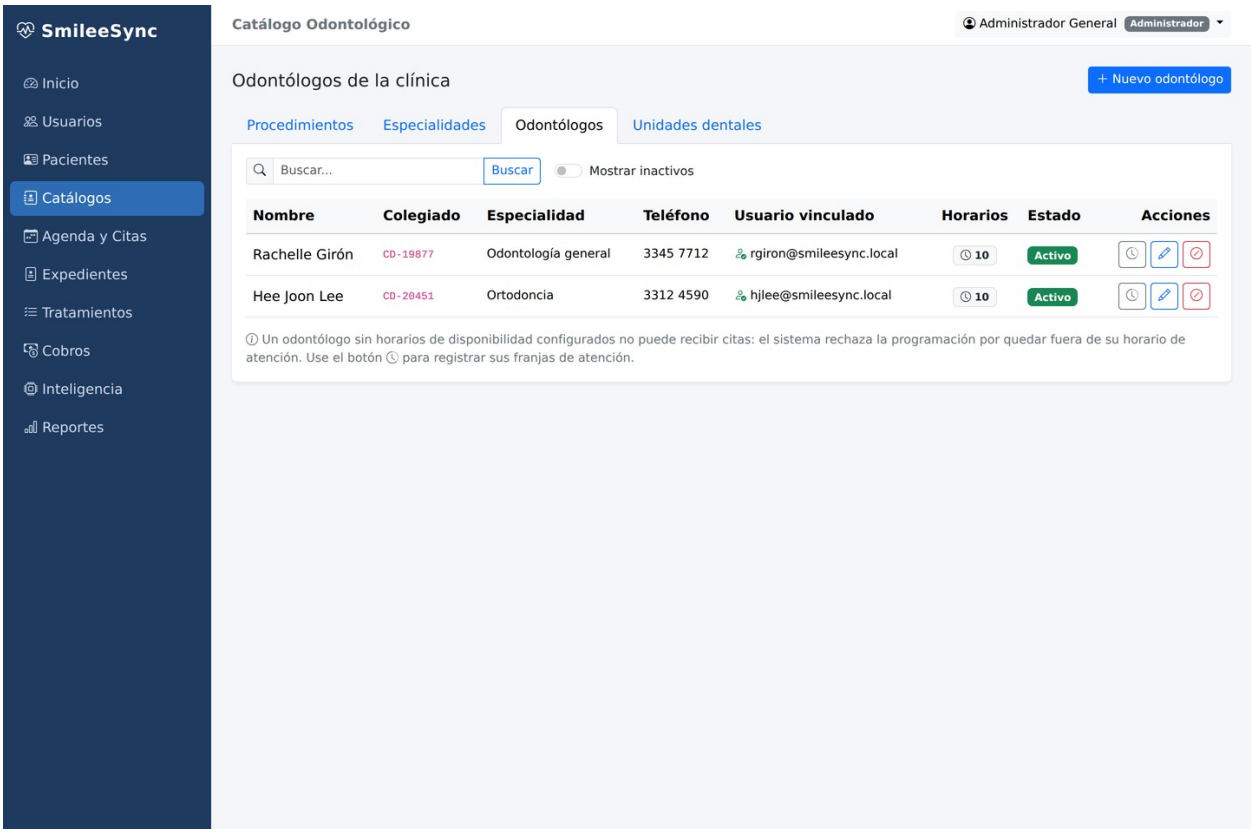


Figura 37: Interfaz del catálogo de odontólogos Fuente: elaboración propia.

El catálogo de unidades dentales (Figura 38) registra los sillones con que cuenta la clínica. Es el catálogo más breve del sistema y, al mismo tiempo, el que impone la restricción física de la operación: la capacidad de atención simultánea no la determina el número de profesionales, sino el menor entre ese número y el de unidades activas. Dar de alta un sillón nuevo es un registro en esta pantalla y el sistema ajusta en consecuencia la validación de las citas, la recomendación de bloques y el indicador de utilización, sin modificar el código.

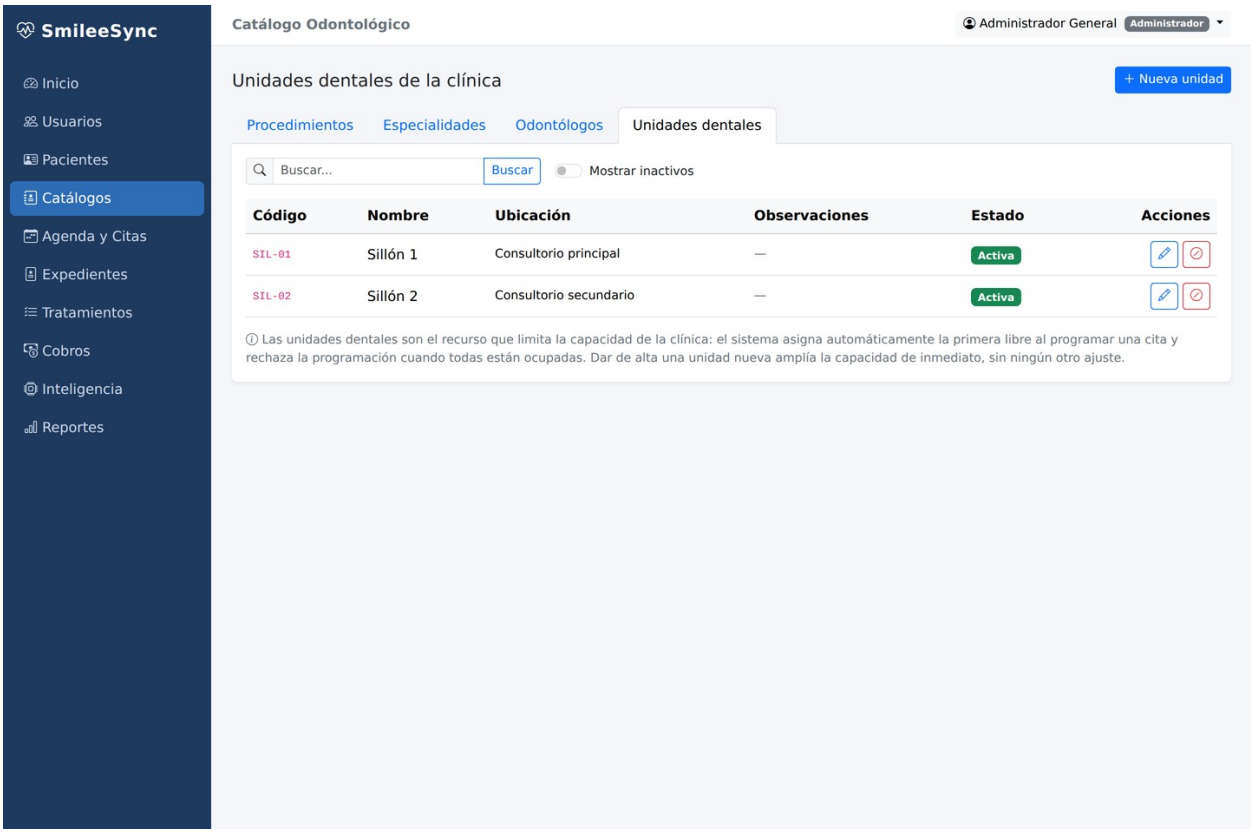


Figura 38: Interfaz del catálogo de unidades dentales Fuente: elaboración propia.

3.3.4 Módulo de Agenda y Citas

La vista de calendario (Figura 39) constituye la pantalla principal del sistema y el espacio de trabajo habitual del personal de recepción. Presenta la agenda semanal con las citas diferenciadas por color según su estado, acompañada de los indicadores operativos del día. Esta disposición permite identificar de un vistazo los espacios disponibles y la carga de trabajo de la jornada. Las citas, los indicadores y las métricas que aparecen en la figura corresponden a datos de demostración cargados para ilustrar el funcionamiento de la pantalla.

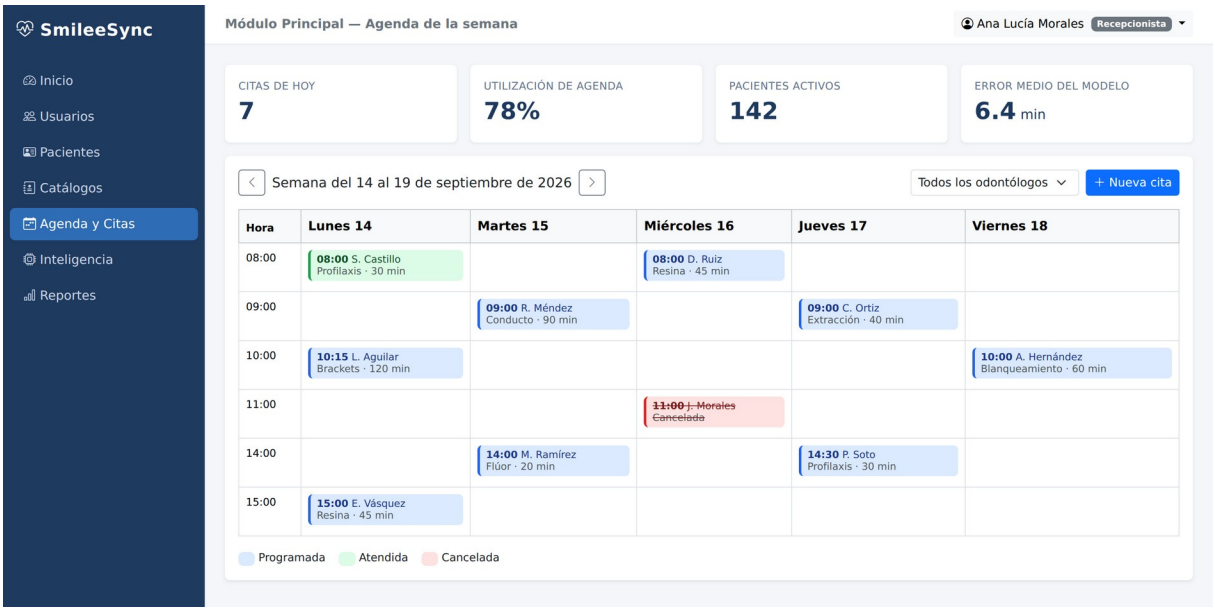


Figura 39: Interfaz del módulo Principal con la agenda semanal (datos de demostración) Fuente: elaboración propia.

Sobre esa misma pantalla opera la consulta de recomendación (Figura 40). El usuario elige un procedimiento, el profesional que lo atenderá y el número de piezas; el sistema estima la duración, declara de qué fuente proviene la estimación y resalta en la rejilla los espacios de la semana donde ese procedimiento cabe completo. La consulta responde antes de que exista una cita, de modo que la recepción puede ofrecer al paciente una hora concreta durante la misma llamada telefónica.

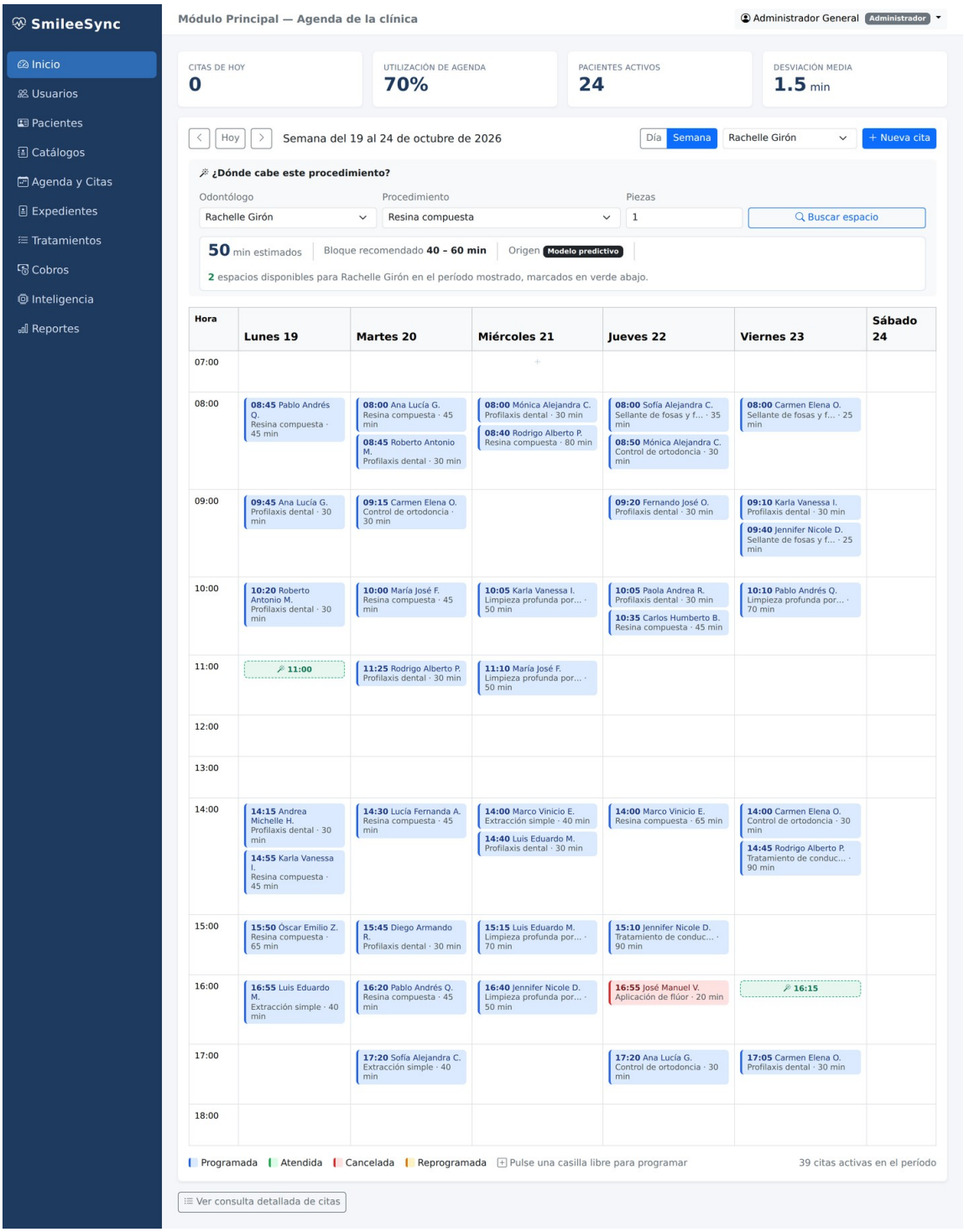


Figura 40: Interfaz de consulta de recomendación de espacios sobre el calendario (datos de demostración) Fuente: elaboración propia.

El formulario de programación de cita (Figura 41) materializa la propuesta de valor del sistema. Al seleccionar el paciente, el procedimiento y el odontólogo, el panel lateral muestra la duración estimada por el modelo predictivo, el bloque de tiempo recomendado, la versión del

modelo empleada y sus métricas de precisión, junto con los factores considerados en la estimación. El campo de duración se precarga con el valor sugerido, pero permanece editable: la decisión final corresponde al usuario, y el sistema actúa como apoyo y no como sustituto del criterio del personal. La Tabla 49 especifica sus campos. La versión del modelo y las métricas de precisión que muestra el panel son valores de demostración: el entrenamiento del modelo con los datos reales de la clínica está previsto en la fase correspondiente del cronograma (sección 3.1.3.2).

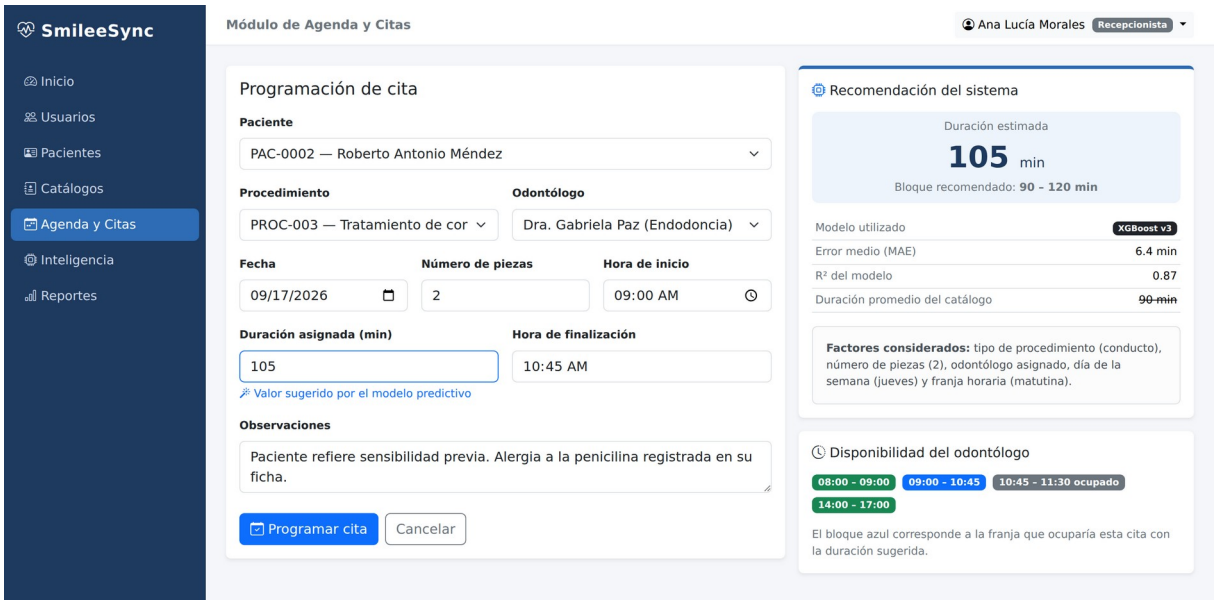


Figura 41: Interfaz de programación de cita con recomendación del modelo predictivo (datos de demostración) Fuente: elaboración propia.

Campo	Tipo de campo	Observación
Paciente	R, Selección	Lista de pacientes activos, identificados por su código y su nombre completo.
Procedimiento	R, Selección	Lista de procedimientos activos del catálogo; su selección activa el cálculo de la estimación.
Odontólogo	R, Selección	Lista de odontólogos activos; determina las franjas de disponibilidad aplicables.
Fecha	R, E	Campo de tipo fecha; no admite fechas anteriores al día actual.
Número de piezas	R, E	Campo numérico que constituye una de las variables de entrada del modelo predictivo.
Hora de inicio	R, E	Campo de tipo hora; se valida contra la disponibilidad del odontólogo y contra las citas ya programadas.
Duración asignada	R, E	Se precarga con la duración estimada por el modelo, pero admite modificación por parte del usuario.
Hora de finalización	Calculado	Se calcula automáticamente a partir de la hora de inicio y la duración asignada; no es editable.
Observaciones	E	Campo de texto libre para anotaciones relevantes sobre la cita.

Tabla 49: Especificación de campos del formulario de programación de cita Fuente: elaboración propia.

La consulta de citas (Figura 42) presenta el registro de las citas dentro de un rango de fechas, contrastando el tiempo asignado con el tiempo real de atención. Esta comparación cumple

una doble función: permite al administrador evaluar la precisión de la programación y constituye la fuente que alimenta el conjunto de entrenamiento del modelo.



Figura 42: Interfaz de consulta y seguimiento de citas Fuente: elaboración propia.

La pantalla de detalle de la cita (Figura 43) es donde el ciclo se cierra. Al concluir la atención se registran la hora real de inicio y la de finalización, y el sistema calcula la duración efectiva y la contrasta con la que había asignado. Ese registro cumple dos funciones: documenta la atención y constituye el dato con que el modelo predictivo se vuelve a entrenar, de modo que la estimación de la clínica mejora con su propia operación.

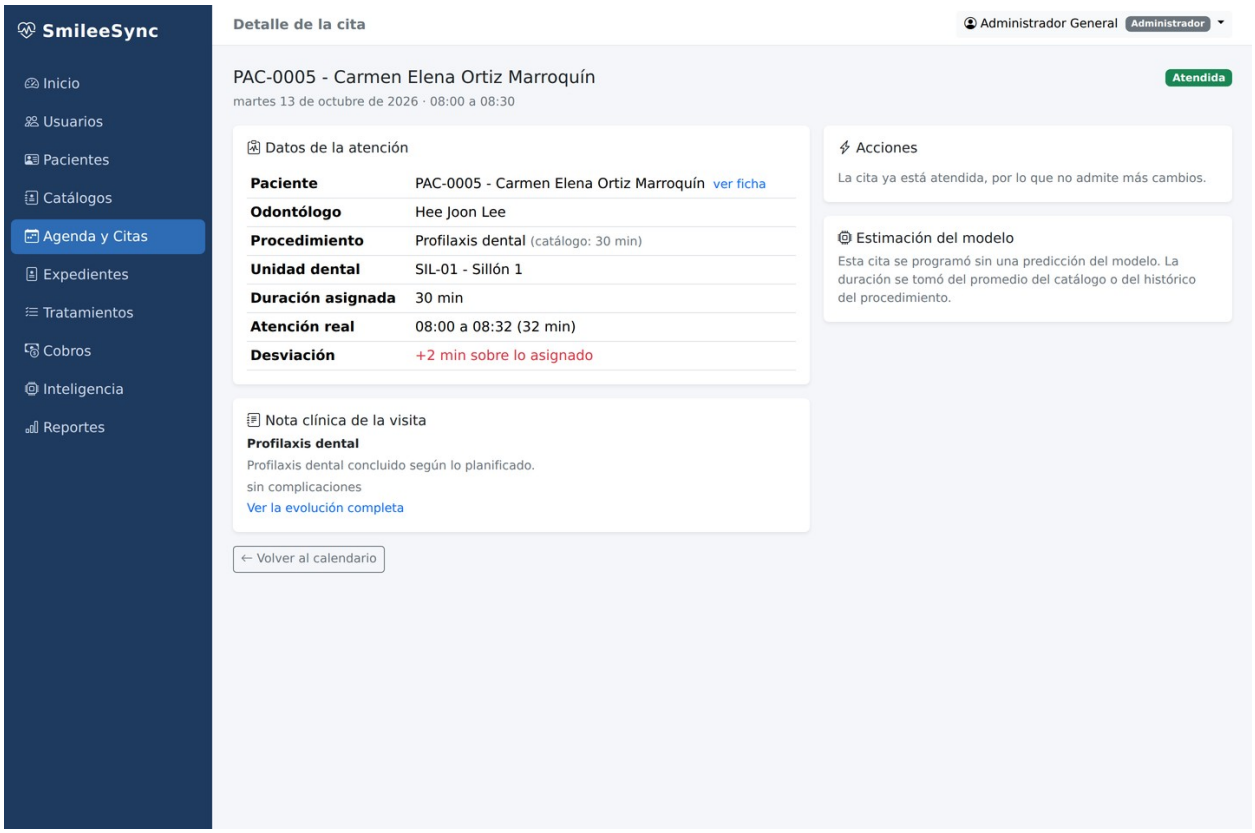


Figura 43: Interfaz de detalle de la cita con registro de tiempos realesFuente: elaboración propia.

3.3.5 Módulo de Expediente Clínico

El expediente clínico responde a una pregunta distinta de la que resuelve la agenda: no cuándo se atiende al paciente, sino qué se le ha hecho y en qué estado se encuentra. La pantalla del expediente (Figura 44) reúne en una sola vista el odontograma vigente, el resumen de hallazgos por atender y la última nota de evolución, de modo que el profesional reconstruya la situación del paciente antes de comenzar la consulta.

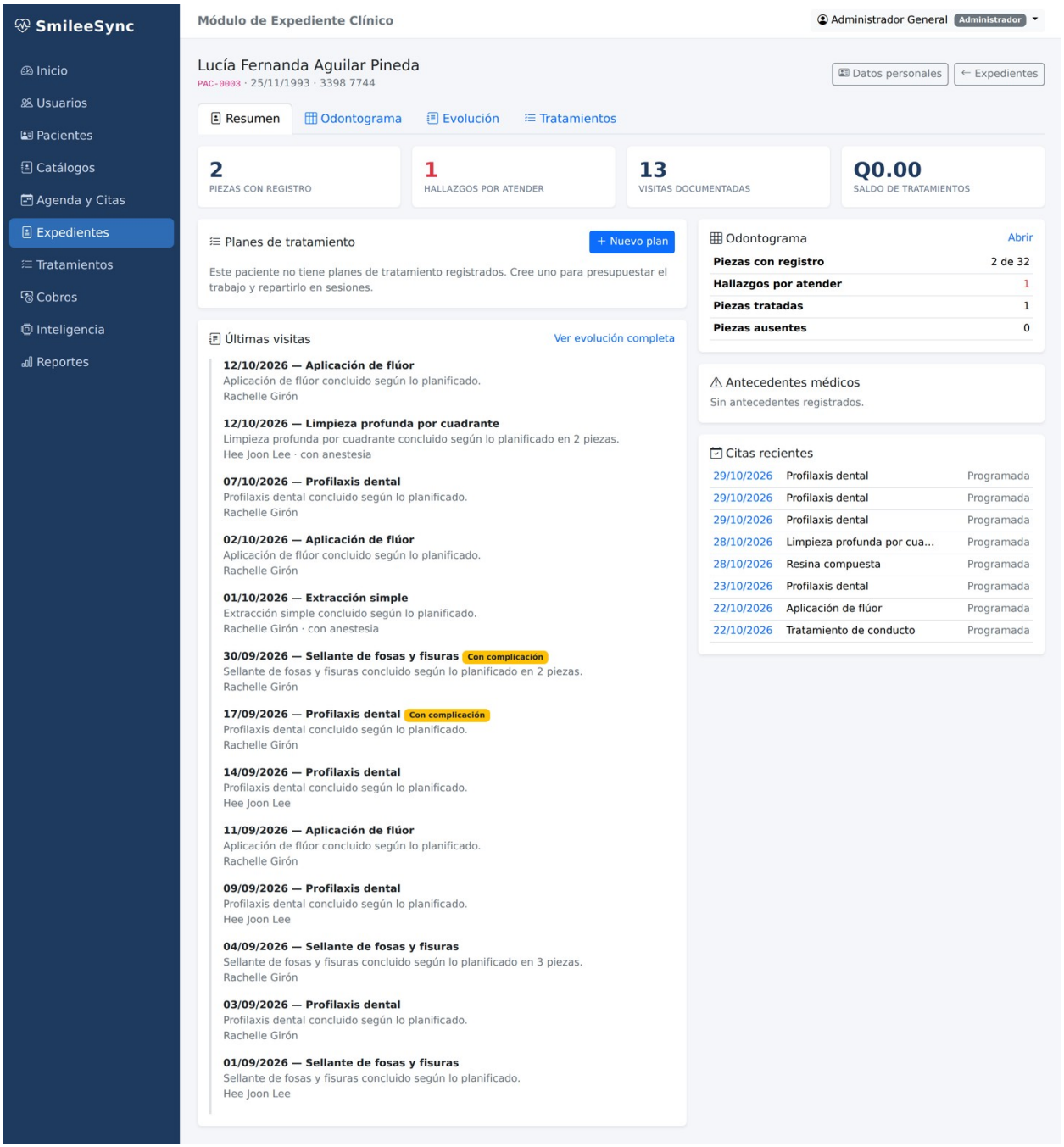


Figura 44: Interfaz del expediente clínico del paciente (datos de demostración) Fuente: elaboración propia.

El odontograma (Figura 45) representa las treinta y dos piezas de la dentición permanente conforme a la notación de dos dígitos de la Federación Dental Internacional, en la que el primer dígito identifica el cuadrante y el segundo la posición de la pieza desde la línea media. La rejilla se dispone en cuatro cuadrantes con la orientación habitual del expediente odontológico, y el color de cada pieza resume el hallazgo vigente, con una letra que distingue entre hallazgos del mismo color. La Tabla 50 recoge esa simbología.

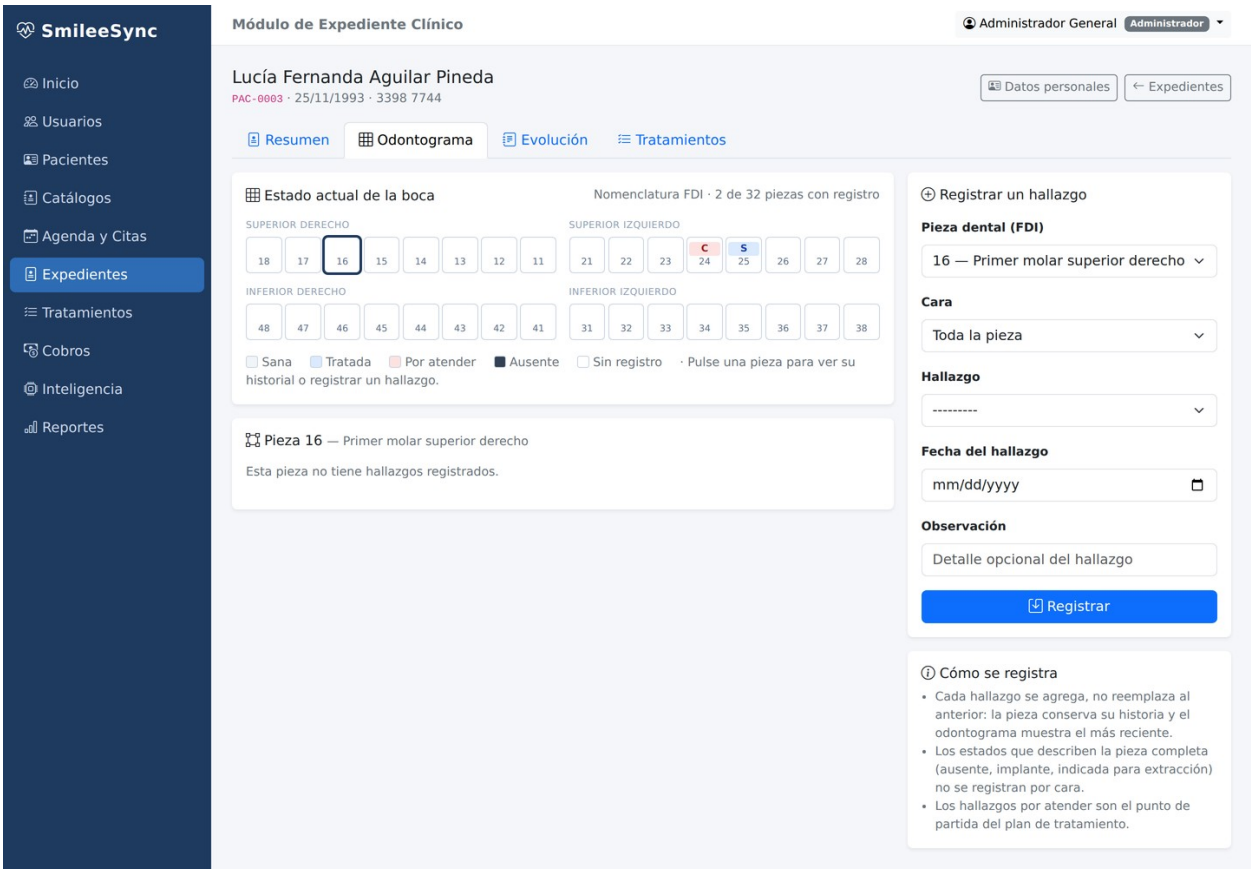


Figura 45: Interfaz del odontograma con el registro de hallazgos por pieza Fuente: elaboración propia.

Hallazgo	Símbolo	Color de la pieza	Significado clínico
Sana	·	Gris	Pieza sin hallazgos registrados.
Caries	C	Rojo	Lesión activa pendiente de tratamiento.
Fractura	F	Rojo	Pérdida de estructura pendiente de atención.
Indicada para extracción	X	Rojo	Pieza cuya exodoncia está indicada.
Obturada	O	Azul	Restauración de resina o amalgama colocada.
Sellante	S	Azul	Sellante de fosas y fisuras aplicado.
Endodoncia	E	Azul	Tratamiento de conducto concluido.
Corona	K	Azul	Corona protésica colocada.
Prótesis	P	Azul	Pieza sustituida por prótesis.
Implante	I	Azul	Implante colocado.
Ausente	—	Gris oscuro	Pieza no presente en la boca.

Tabla 50: Simbología del odontograma y su correspondencia con el hallazgo registrado Fuente: elaboración propia.

La tabla de hallazgos es de solo agregado: cada observación se registra con su fecha y no se modifica después. El odontograma vigente es el último hallazgo activo de cada combinación de pieza y cara, de modo que el expediente conserva la evolución completa de cada diente sin requerir una tabla de auditoría aparte y sin que una corrección borre lo que se observó antes.

La hoja de evolución (Figura 46) presenta las notas clínicas en orden cronológico inverso. Cada nota corresponde a una visita atendida y registra el motivo, el diagnóstico, el procedimiento realizado, las piezas tratadas, el uso de anestesia y la aparición de complicaciones. Las tres últimas variables cumplen una doble función: documentan el acto clínico y alimentan al modelo predictivo, pues son las que explican por qué dos atenciones del mismo procedimiento no duran lo mismo.

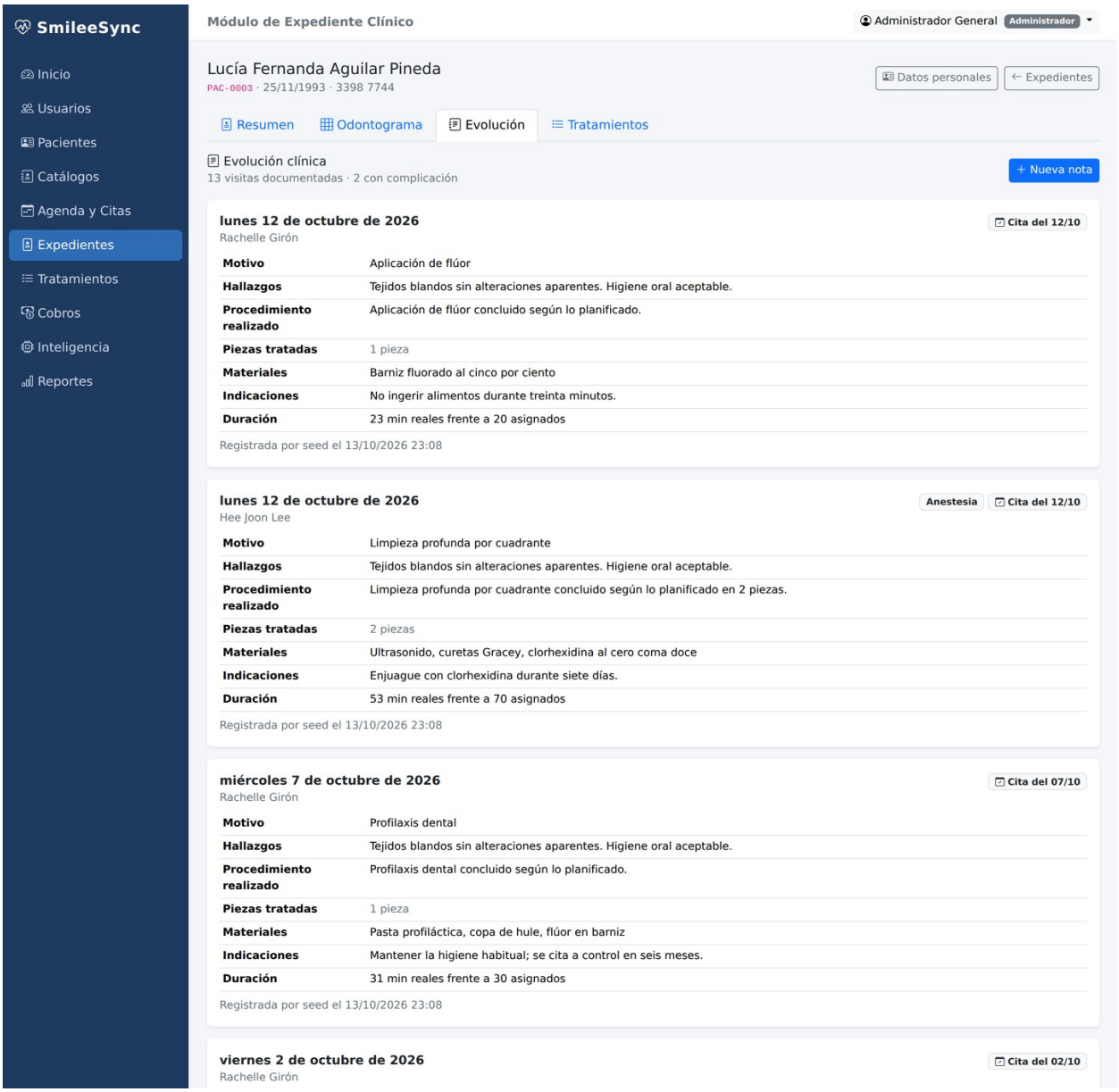


Figura 46: Interfaz de la hoja de evolución clínica del paciente (datos de demostración) Fuente: elaboración propia.

3.3.6 Módulo de Plan de Tratamiento

El plan de tratamiento es el documento con que la clínica acuerda con el paciente el trabajo por realizar y su costo. El listado de planes (Figura 47) presenta los planes abiertos con su estado, su avance y su saldo pendiente.

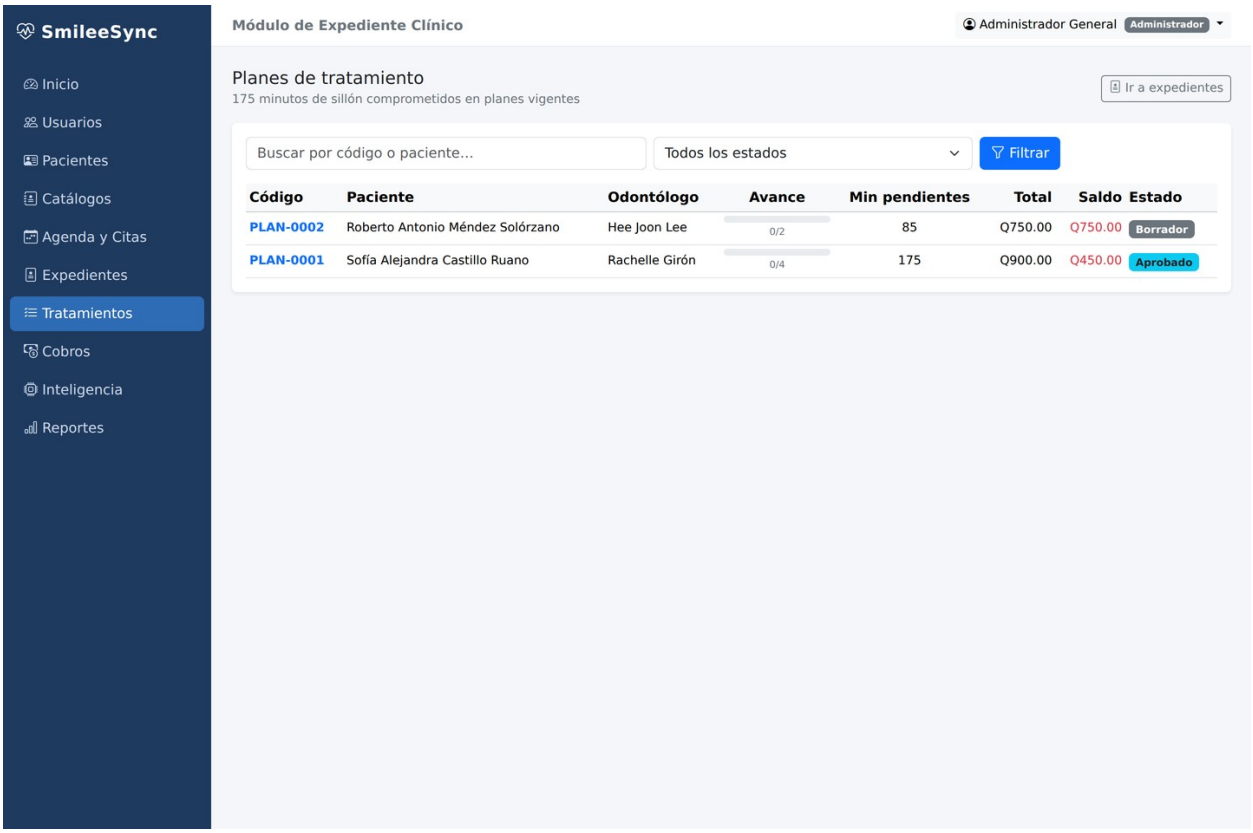


Figura 47: Interfaz del listado de planes de tratamiento (datos de demostración) Fuente: elaboración propia.

El detalle del plan (Figura 48) organiza el tratamiento en renglones agrupados por fases. Cada renglón corresponde a un procedimiento sobre una pieza determinada y conserva la duración que el modelo estimó al presupuestarlo, junto con la fuente de esa estimación. Es el punto donde el componente predictivo deja de resolver una cita aislada y pasa a planificar un tratamiento completo: con la duración de cada renglón, el sistema reparte el trabajo pendiente en sesiones que no exceden las dos horas de sillón, respetando el orden de las fases, y anticipa cuántas visitas requerirá el plan antes de agendar la primera.

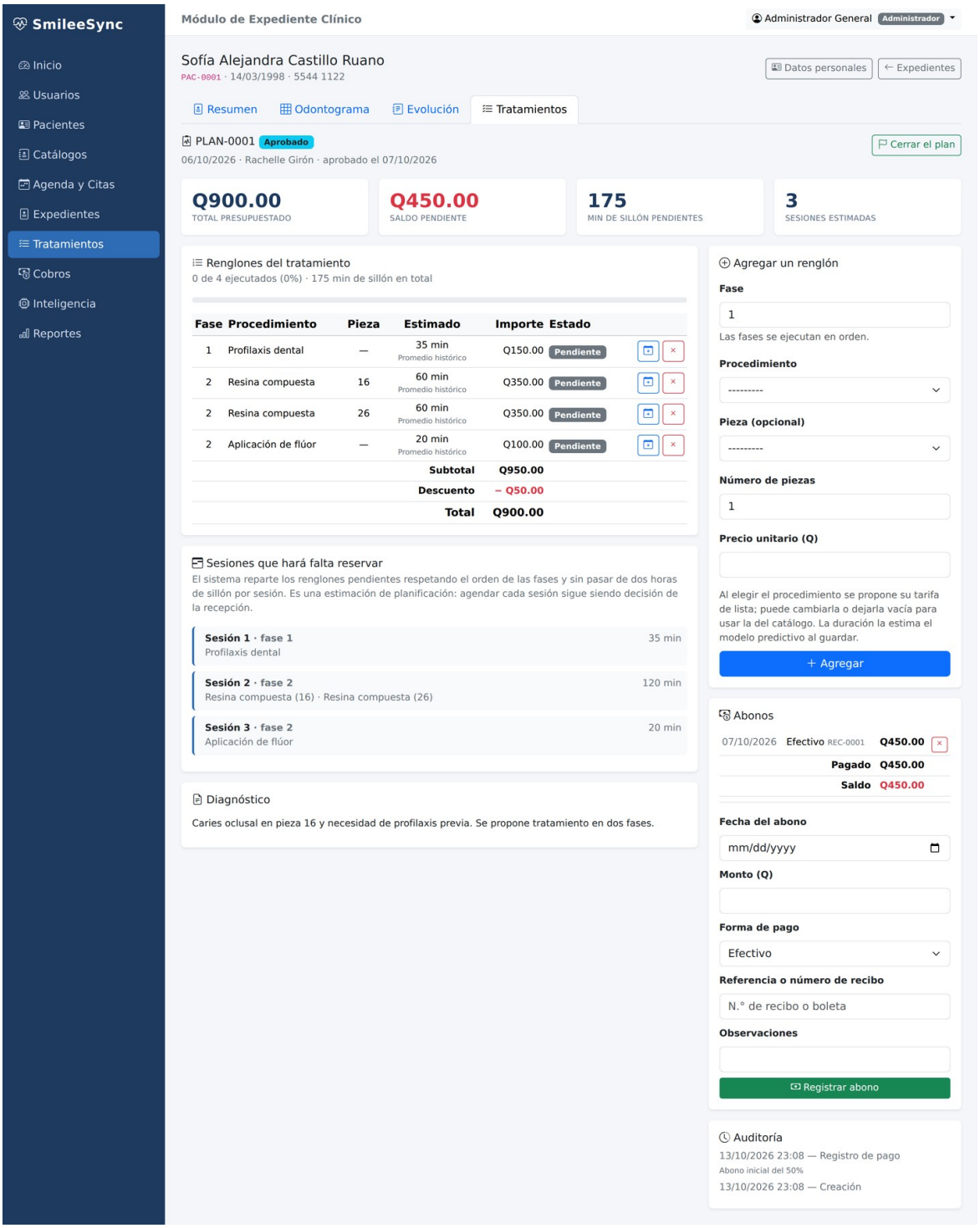


Figura 48: Interfaz del detalle del plan de tratamiento con fases, renglones y sesiones sugeridas (datos de demostración)
Fuente: elaboración propia.

3.3.7 Módulo de Cobros

El módulo de cobros (Figura 49) presenta el estado financiero de los planes aprobados: el total presupuestado, lo abonado y el saldo pendiente de cada paciente. Los pagos se registran como

abonos aplicados al plan, de modo que el saldo se deriva siempre de la suma de los abonos vigentes y no de un campo que deba mantenerse al día. La anulación de un pago no borra el registro: lo marca como inactivo, con lo que la trazabilidad del cobro se conserva.

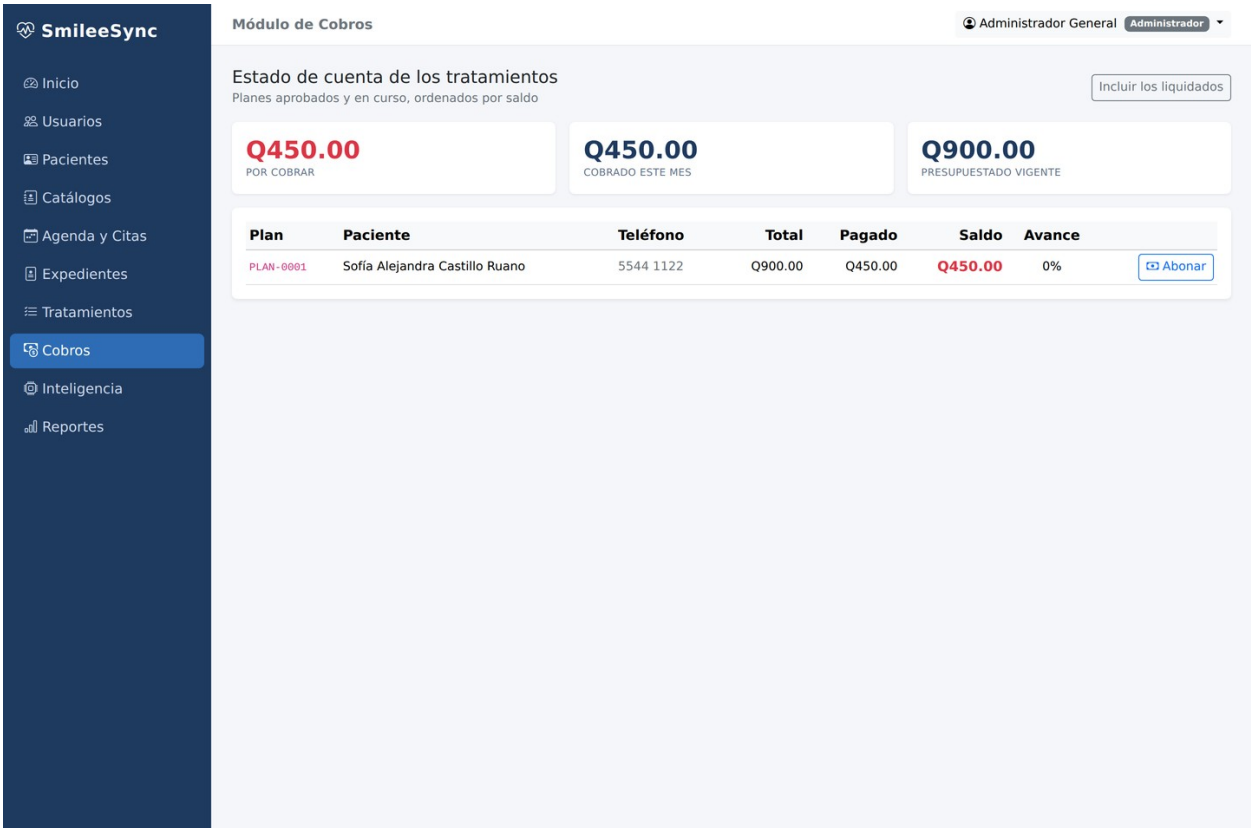


Figura 49: Interfaz del control de cobros y saldos por plan (datos de demostración) Fuente: elaboración propia.

3.3.8 Reportes del sistema

La Tabla 51 relaciona los reportes que genera el sistema con el requerimiento que los origina. Estos reportes constituyen la salida del módulo de Reportes e Indicadores y son la base sobre la cual el administrador evalúa el desempeño de la clínica.

ID del requerimiento	Reporte	Descripción
REQ-16	Reporte de citas atendidas	Extracto de las citas efectivamente atendidas durante un rango de fechas, agrupadas por odontólogo y por procedimiento.
REQ-16	Reporte de utilización de la agenda	Porcentaje del tiempo disponible que fue efectivamente ocupado, con el detalle de los espacios improductivos identificados.
REQ-16	Reporte de tiempos de espera	Tiempo de espera promedio de los pacientes, comparado entre períodos.
REQ-15, REQ-16	Reporte de precisión del modelo	Métricas MAE, RMSE y R^2 de la versión activa del modelo, con la comparación entre la duración estimada y la duración real de las citas atendidas.
REQ-04	Reporte de auditoría	Extracto del historial de operaciones efectuadas sobre usuarios, pacientes

ID del requerimiento	Reporte	Descripción
		y citas, indicando fecha, responsable y acción.

Tabla 51: Reportes generados por el sistema Fuente: elaboración propia.

3.3.9 Conjunto de pruebas

La Tabla 52 presenta el conjunto de pruebas definido para validar los requerimientos críticos del sistema. Cada prueba verifica el comportamiento esperado ante datos válidos e inválidos, lo que permite comprobar de forma objetiva el cumplimiento de los criterios de aceptación establecidos en el DERCAS.

ID del requerimiento	Campo o proceso evaluado	Validación esperada
REQ-01	Correo electrónico en el inicio de sesión	Campo requerido, de tipo texto con formato de correo. Un correo no registrado o una contraseña incorrecta producen un mensaje genérico sin revelar cuál dato es erróneo.
REQ-01	Estado del usuario	Un usuario con estado inactivo no puede iniciar sesión, aun cuando sus credenciales sean correctas.
REQ-02	Nombre y apellido del usuario	El ingreso de números o caracteres especiales es rechazado con un mensaje específico. Valores como "María José" o "López-García" son aceptados.
REQ-02	Correo electrónico del usuario	Un valor sin arroba o sin dominio es rechazado. Un correo ya registrado produce el mensaje de duplicidad.
REQ-02	Confirmación de contraseña	Si ambos campos no coinciden, el formulario es rechazado y no se crea el usuario.
REQ-03	Acceso a un módulo no autorizado	El ingreso directo a la dirección de un módulo no habilitado para el rol es denegado, aunque la opción no aparezca en el menú.
REQ-04	Registro de auditoría	Toda actualización o cambio de estado genera un registro en la tabla de historial con la fecha, el responsable y la acción ejecutada.
REQ-10	Traslape de citas	El sistema impide registrar dos citas que se superpongan para el mismo odontólogo y señala la cita en conflicto.
REQ-13	Estimación de duración	La estimación se obtiene en línea y queda almacenada con la versión del modelo utilizada. Si el modelo no está disponible, el sistema emplea la duración promedio del catálogo e informa de ello.
REQ-12	Registro de duración real	Al registrar la hora de finalización, el sistema calcula la duración real e incorpora el registro al conjunto de entrenamiento.

Tabla 52: Conjunto de pruebas de los requerimientos críticos Fuente: elaboración propia.

3.4 Diseño de la investigación y validación del modelo

Las secciones anteriores describen el análisis y el diseño del sistema. La presente sección delimita el marco con el que se evalúa su componente predictivo: la población de registros sobre la que se entrena el modelo, las variables consideradas, el instrumento con que se recolectan los datos, los criterios con que se depuran y el umbral cuantitativo a partir del cual se considera que el modelo cumple su propósito.

3.4.1 Población y muestra

La población de la investigación está constituida por la totalidad de las citas atendidas en la Clínica Dental Smilee desde el inicio de sus operaciones, el 24 de marzo de 2025, junto con las que el sistema registre a partir de su puesta en producción. Esa población proviene de dos fuentes de distinta naturaleza. La primera es la agenda manual que la clínica lleva desde su apertura, en la que el personal anota, además de la cita programada, el tiempo que tomó cada atención; tal como se expone en la sección 1.1.2, esa información se encuentra dispersa y no sistematizada, por lo que no admite análisis en su forma actual, aunque sí constituye un registro recuperable. La segunda es el propio sistema: cada atención que se cierra en el módulo de Agenda y Citas incorpora la hora real de inicio, la hora real de finalización y la duración efectiva calculada automáticamente. Considerando ambas fuentes, se estima que la población asciende a alrededor de 1,000 registros. Las dos difieren en precisión: la anotación manual es una estimación del personal, mientras que el registro del sistema procede de marcas de tiempo exactas. Esa diferencia se atiende en la fase de preparación de datos y sustenta los criterios de depuración descritos más adelante.

Sobre esa población, finita y de tamaño conocido, se calcula el número de registros necesario para que la muestra resulte representativa, mediante la fórmula de población finita establecida en la guía metodológica de la Universidad Mariano Gálvez de Guatemala:

$$n = (Z^2 \times N \times p \times q) / (e^2 \times (N - 1) + Z^2 \times p \times q)$$

Donde Z = 1.96 corresponde a un nivel de confianza del 95 %; N = 1,000 es el número estimado de registros que componen la población; p = 0.5 y q = 0.5 son las probabilidades de éxito y de fracaso; y e = 0.05 representa un margen de error del 5 %. Al sustituir los valores se obtiene:

$$n = (3.8416 \times 250) / (2.4975 + 0.9604) = 960.4 / 3.4579 \approx 278 \text{ registros}$$

El resultado, 278 registros, corresponde al tamaño mínimo con el que la muestra representa a la población con un 95 % de confianza y un 5 % de margen de error, y es el criterio que establece la guía metodológica de la Universidad. Conviene precisar su alcance: esa fórmula está construida para estimar una proporción y, por sí sola, no demuestra que ese número de observaciones baste para entrenar un modelo de regresión. La suficiencia del conjunto de datos se verifica de forma empírica mediante curvas de aprendizaje, que muestran cómo evoluciona el error conforme se incorporan más registros: si la curva continúa descendiendo de manera apreciable al alcanzar el

total disponible, el conjunto resulta insuficiente y el modelo se reentrena conforme la clínica acumule nuevas atenciones. El entrenamiento, por lo demás, no se limita a esos 278 registros: se emplea la totalidad de los disponibles, ya que en aprendizaje automático el rendimiento mejora con el volumen. Para la evaluación, el conjunto se divide mediante muestreo estratificado por tipo de procedimiento, de manera que todos queden representados —incluidos los de baja frecuencia — y el rendimiento reportado no quede sesgado hacia los más comunes.

3.4.2 Variables del estudio

El modelo predictivo se construye a partir de las variables que, según la revisión de la literatura y la estructura de la base de datos descrita en la sección 3.2.2, inciden en la duración real de un procedimiento odontológico. La Tabla 53 detalla las variables independientes que el modelo recibe como entrada y la Tabla 54 las variables dependientes que se busca estimar.

Variable	Tipo	Descripción
Tipo de procedimiento	Categórica	Clasificación del tratamiento odontológico según el catálogo de la clínica.
Número de piezas dentales	Numérica discreta	Cantidad de piezas tratadas durante la consulta. Se toma del registro cuando lo consigna; en caso contrario se imputa según el criterio de la sección 3.4.3.
Odontólogo responsable	Categórica	Profesional que atiende la cita; recoge las diferencias de ritmo y técnica.
Hora de inicio de la cita	Numérica continua	Franja horaria en que se realiza la consulta.
Día de la semana	Categórica	Día hábil en que se programa la cita.

Tabla 53: Variables independientes del estudio Fuente: elaboración propia.

Variable	Tipo	Descripción
Duración real del procedimiento	Numérica continua	Minutos transcurridos entre la hora de inicio real y la hora de finalización registradas.
Desajuste en la asignación de tiempo	Numérica continua	Diferencia entre la duración asignada en la agenda y la duración real.

Tabla 54: Variables dependientes del estudio Fuente: elaboración propia.

3.4.3 Instrumento de recolección y depuración de los datos

El instrumento de recolección es doble, en correspondencia con las dos fuentes descritas. Para las atenciones anteriores a la puesta en producción se emplea una ficha de registro estandarizada con la que se digitalizan las anotaciones de la agenda manual; para las posteriores, la fuente es el registro que genera el propio sistema. La ficha captura, por cada cita, el identificador secuencial anonimizado, el código del odontólogo responsable, el tipo de procedimiento, la fecha de la cita, la hora de inicio programada, la hora de inicio real y la hora de finalización cuando

existen, la duración asignada, la duración real y las observaciones relevantes. El identificador del paciente no se incorpora a la ficha, con el fin de preservar la confidencialidad de la información clínica. Los registros digitalizados se incorporan al conjunto de datos en una carga independiente, previa a la puesta en producción, por lo que su procedencia queda determinada sin necesidad de un campo adicional en la base de datos. El uso de esta información con fines académicos cuenta con la autorización expresa de la dirección de la clínica.

La ficha responde a los principios de parsimonia y de completitud: se incluyen únicamente las variables que aportan capacidad predictiva y, al mismo tiempo, todos los campos necesarios para entrenar y evaluar el modelo, de modo que no sea preciso volver a la fuente para recuperar información omitida (Han et al., 2011).

No todos los registros resultan aptos para el entrenamiento. La Tabla 55 resume los criterios con los que se decide la incorporación o el descarte de cada registro.

Criterio	Condición
Inclusión — registros del sistema	El registro consigna la hora de inicio real, la hora de finalización, el tipo de procedimiento y el código del odontólogo responsable.
Inclusión — registros digitalizados	El registro consigna la duración anotada por el personal, el tipo de procedimiento y el código del odontólogo responsable. Al carecer de hora real de inicio, se emplea la hora programada en su lugar.
Exclusión	Citas canceladas antes de haberse iniciado.
Exclusión	Registros con duración inferior a cinco minutos o superior a cuatro horas, por corresponder a errores de captura o a casos atípicos extremos.
Exclusión	Registros a los que falta alguno de los campos exigidos por su fuente, según los criterios de inclusión anteriores.
Exclusión	Citas de emergencia no programada, cuya duración no responde a los patrones de la agenda regular.

Tabla 55: Criterios de inclusión y exclusión de los registros Fuente: elaboración propia.

Los registros que presentan valores faltantes en variables secundarias —señaladamente el número de piezas tratadas, que la anotación manual no siempre consigna—, pero que conservan completos los campos exigidos por su fuente, se conservan mediante imputación por la mediana del tipo de procedimiento correspondiente. Esta estrategia resulta robusta ante distribuciones asimétricas y valores atípicos, condición habitual en los datos de duración de procedimientos clínicos (Han et al., 2011). La proporción de registros imputados, desagregada por fuente, se reporta junto con las estadísticas descriptivas del conjunto de datos final.

3.4.4 Hipótesis de trabajo y criterio de validación

Las hipótesis del proyecto se enuncian en forma de preguntas al inicio del documento, conforme al carácter descriptivo-correlacional de la investigación. Para efectos de la validación del componente predictivo, esas preguntas se traducen en un par de hipótesis contrastables:

H1: la implementación de un modelo predictivo basado en datos históricos —que considere el tipo de procedimiento, el odontólogo responsable, el número de piezas tratadas, el día de la semana y la franja horaria— permite estimar con mayor precisión la duración de los procedimientos odontológicos y, en consecuencia, optimizar la asignación de tiempos en la agenda de la Clínica Dental Smilee, reduciendo el desajuste entre el tiempo asignado y el tiempo real de atención.

H0: la implementación de un modelo predictivo basado en datos históricos no produce una diferencia significativa en la precisión de la estimación de la duración de los procedimientos odontológicos frente al método de asignación de tiempos que la clínica utiliza actualmente.

La hipótesis nula se contrasta con una sola regla de decisión, enunciada en los mismos términos en que está redactada: sobre los mismos casos se calcula el error del modelo y el que habría producido el método de bloques fijos que la clínica utiliza hoy, y H0 se rechaza cuando se cumplen dos condiciones de forma conjunta: que la reducción del error absoluto medio sea de al menos un 25 % y que la diferencia por pares resulte estadísticamente significativa. Para ese contraste se emplea la prueba de rangos con signo de Wilcoxon con un nivel de significancia $\alpha = 0.05$; se opta por una prueba no paramétrica porque los errores de predicción de duraciones clínicas no siguen una distribución normal. Las predicciones se obtienen mediante validación cruzada repetida, pero se promedian por registro antes del contraste, de manera que cada registro aporte un único par y no se vulnere el supuesto de independencia entre observaciones. Como evidencia complementaria se examina la importancia de las variables por permutación, que mide cuánto se degrada la predicción al alterar cada variable y permite verificar que el modelo se apoya en los factores previstos y no en asociaciones espurias (Hastie et al., 2009).

3.4.5 Métricas de evaluación y resultados esperados

El rendimiento de los algoritmos candidatos —regresión lineal múltiple, Random Forest y XGBoost— se evalúa sobre el subconjunto de prueba mediante tres métricas. El error absoluto

medio (MAE) expresa el error promedio en minutos, por lo que resulta directamente interpretable en términos del impacto sobre la agenda. La raíz del error cuadrático medio (RMSE) es más sensible a los errores de gran magnitud. El coeficiente de determinación (R^2) indica la proporción de varianza explicada por el modelo (Willmott y Matsuura, 2005).

La selección del algoritmo se realiza mediante validación cruzada de diez pliegues sobre el subconjunto de entrenamiento, tomando como criterio el menor MAE promedio. Se privilegia el MAE sobre el RMSE porque, en el contexto de la agenda clínica, interesa reducir el error promedio de estimación antes que penalizar de forma desproporcionada los casos extremos.

La meta de desempeño del proyecto es la que acaba de enunciarse como criterio de decisión: que el modelo reduzca en al menos un 25 % el error absoluto medio del método de bloques fijos sobre los mismos casos. Se expresa en términos relativos porque el catálogo de la clínica abarca procedimientos de entre 20 y 120 minutos, de modo que un mismo error absoluto pesa de manera muy distinta según el procedimiento: diez minutos son la mitad de una aplicación de flúor y menos de la décima parte de una colocación de brackets. Junto a ese criterio se reportan, sin carácter decisorio, el error absoluto medio, la raíz del error cuadrático medio y el coeficiente de determinación del modelo seleccionado, así como el error medio desagregado por tipo de procedimiento, que es la lectura que permite juzgar la utilidad práctica de la estimación en cada caso. Como valores de referencia deseables se toman un MAE de diez minutos, un RMSE de quince minutos y un R^2 de 0.6; son metas definidas por el propio proyecto y no resultados tomados de otros estudios.

Capítulo 4. Marco Metodológico: Desarrollo e Implementación

Los capítulos precedentes establecieron qué debía construirse y con qué estructura. El presente capítulo documenta la construcción. La primera sección describe la plataforma de desarrollo, las tecnologías y arquitecturas empleadas —con la justificación de cada biblioteca incorporada— y los mecanismos de seguridad implementados. La segunda presenta el código fuente del sistema: el inicio de sesión, el menú principal, las funciones principales de cada módulo y un ejemplo de la generación de gráficas y reportes. La tercera documenta las pruebas de implementación realizadas sobre el sistema en funcionamiento.

4.1 Desarrollo

4.1.1 Plataforma de desarrollo

El sistema se desarrolló en un equipo personal con sistema operativo Windows 11, utilizando Visual Studio Code como entorno de edición y Git como sistema de control de versiones, con el repositorio alojado en GitHub. Las dependencias de Python se instalaron en un entorno virtual propio del proyecto, de modo que las versiones empleadas quedaran aisladas de las del sistema operativo y pudieran reproducirse en el servidor a partir del archivo de requerimientos.

El entorno de desarrollo y el de producción se distinguen en un solo punto: el motor de base de datos. En desarrollo se utiliza SQLite, que no requiere servidor y permite reconstruir la base completa borrando un archivo; en producción se utiliza PostgreSQL. El resto del código es idéntico, porque el acceso a los datos se realiza siempre a través del mapeador objeto-relacional del framework y no mediante sentencias escritas para un motor concreto. La selección del motor se resuelve por variable de entorno, de manera que el mismo código fuente opera en ambos entornos sin modificación.

El despliegue se realiza sobre una instancia EC2 de Amazon Web Services del tipo t3.small, con dos núcleos virtuales y dos gigabytes de memoria, que ejecuta en un solo servidor la aplicación, el servidor web y la base de datos. Esa restricción de memoria no es un dato accesorio: condiciona decisiones de diseño que se documentan más adelante, señaladamente la de mantener el entrenamiento del modelo predictivo fuera del proceso que atiende las peticiones. La Tabla 56 resume la plataforma.

Componente	Versión	Papel en el proyecto

Python	3.12	Lenguaje de programación del sistema y del componente predictivo.
Django	5.0	Framework de aplicación web; estructura todo el sistema.
SQLite	3	Base de datos del entorno de desarrollo; no requiere servidor.
PostgreSQL	16	Base de datos del entorno de producción.
Gunicorn	22	Servidor de aplicación WSGI que ejecuta Django en producción con tres procesos de trabajo.
Nginx	1.24	Servidor web de borde: atiende el puerto 80, entrega los archivos estáticos y reenvía el resto a Gunicorn.
Ubuntu Server	22.04 LTS	Sistema operativo de la instancia de producción.
Amazon EC2	t3.small	Servidor de producción: 2 vCPU y 2 GB de memoria.
Git / GitHub	—	Control de versiones y repositorio remoto del proyecto.
GitHub Actions	—	Integración y despliegue continuos hacia el servidor.
AWS CDK (Python)	2	Definición de la infraestructura como código.
Visual Studio Code	—	Entorno de edición del código fuente.

Tabla 56: Plataforma y entorno de desarrollo del sistema Fuente: elaboración propia.

4.1.2 Tecnologías y arquitecturas

4.1.2.1 Arquitectura de la aplicación

El sistema se construyó sobre el patrón modelo-vista-plantilla que propone Django, que es la variante del modelo-vista-controlador en la que el framework asume el papel del controlador y el desarrollador escribe el modelo, la vista y la plantilla. Sobre ese patrón se dispuso la separación en capas descrita en la sección 3.1.2.1: la capa de presentación comprende las plantillas y los formularios; la capa de dominio concentra las reglas de negocio en módulos independientes de la interfaz; la capa de acceso a datos corresponde a los modelos y al mapeador objeto-relacional.

La decisión de diseño con mayor consecuencia práctica fue extraer de las vistas dos criterios de negocio y concentrarlos en módulos propios. El primero es la disponibilidad efectiva de la clínica, que reside en el módulo agenda/disponibilidad.py; el segundo es la estimación de duraciones, que reside en inteligencia/servicio.py. Tres partes del sistema necesitan saber si un espacio está libre —la validación al programar una cita, la recomendación de bloques sobre el calendario y el indicador de utilización de la agenda— y mientras cada una resolvía esa pregunta por su cuenta, bastaba modificar una para que las tres dejaran de coincidir. Al concentrar el criterio en un módulo, las tres lo consultan y el sistema responde con un solo criterio.

4.1.2.2 Bibliotecas y componentes incorporados

La Tabla 57 relaciona cada biblioteca incorporada al sistema con la función que cumple y con la razón por la que fue seleccionada frente a las alternativas consideradas. El criterio general de selección fue la parsimonia: cada dependencia añadida es un componente que debe instalarse,

actualizarse y mantenerse compatible en un servidor de dos gigabytes de memoria, de modo que solo se incorporó aquello que resolvía un problema que el framework no resolvía por sí mismo.

Biblioteca	Versión	Para qué se utilizó	Por qué se seleccionó
Django	5.0	Estructura completa de la aplicación: enrutamiento, mapeador objeto-relacional, migraciones de esquema, sistema de autenticación, sesiones, validación de formularios, motor de plantillas y sitio de administración.	Frente a un microframework como Flask, Django aporta ya construidos el sistema de autenticación, el modelo de usuario extensible, las migraciones y las protecciones contra los ataques web comunes. Reimplementar esos componentes habría consumido el tiempo del proyecto sin aportar al objeto de la investigación, que es el modelo predictivo.
Gunicorn	22.0	Servidor de aplicación que ejecuta el proyecto en producción con tres procesos de trabajo.	El servidor de desarrollo de Django es de un solo hilo y su propia documentación advierte que no debe usarse en producción. Gunicorn es el servidor WSGI de uso más extendido en el ecosistema de Python y se administra como un servicio del sistema operativo.
WhiteNoise	6.6	Entrega de los archivos estáticos —hojas de estilo, scripts e imágenes— desde el propio proceso de la aplicación, con compresión y nombres versionados por contenido.	La alternativa habitual es un servicio de almacenamiento con red de distribución de contenido, que para una clínica con cinco usuarios concurrentes añadiría costo mensual y una dependencia externa sin beneficio perceptible. WhiteNoise resuelve el mismo problema sin infraestructura adicional.
psycopg2-binary	2.9	Controlador de conexión entre Django y PostgreSQL en producción.	Es el controlador que la documentación de Django recomienda para PostgreSQL. Se eligió la distribución binaria porque no requiere compilar código nativo en el servidor, lo que en una instancia de dos gigabytes evita instalar un compilador completo.
python-dotenv	1.0	Carga de las variables de entorno desde el archivo local de configuración.	El servicio del sistema operativo recibe sus variables de un archivo de entorno, pero los comandos administrativos ejecutados a mano no las recibirían. Con esta biblioteca el servidor y los comandos leen la misma configuración, de modo que una migración ejecutada manualmente no se aplique por error sobre otra base de datos.
scikit-learn	1.4	Construcción, entrenamiento y evaluación del modelo predictivo: codificación de las variables categóricas, regresión lineal múltiple, bosque aleatorio, partición del conjunto en entrenamiento y prueba, y cálculo de las métricas de error.	Es la biblioteca de referencia para aprendizaje automático clásico en Python y la que implementa los algoritmos que el marco teórico de este trabajo describe. Su aporte decisivo es la tubería de procesamiento, que serializa el preprocesamiento junto con el estimador: así el proceso web no puede aplicar una codificación distinta de la del entrenamiento, que es una de las fuentes de error más frecuentes al poner un modelo en producción.
pandas	2.2	Construcción de la tabla de datos con que se entrena el modelo y del registro individual con que se solicita cada predicción.	El transformador de columnas de scikit-learn selecciona las variables por nombre, lo que exige una estructura tabular con encabezados. Resolverlo con arreglos de NumPy obligaría a mantener a mano el mismo orden de columnas en el entrenamiento y en la predicción, con el riesgo de que una variable se desplace y el modelo prediga sobre datos equivocados sin informar error alguno.

joblib	1.3	Serialización del modelo entrenado a un archivo en disco y lectura de ese archivo por el proceso web.	Es el mecanismo que la documentación de scikit-learn recomienda para persistir modelos, porque almacena los arreglos numéricos de forma más eficiente que el módulo estándar de serialización de Python. Se emplea con compresión de nivel tres, que reduce el archivo del bosque aleatorio a poco más de medio megabyte.
Bootstrap	5.3.3	Sistema de rejilla responsiva y componentes de interfaz: formularios, tablas, tarjetas, alertas, menús desplegables y distintivos de estado.	Permite que las pantallas se adapten a distintos tamaños de ventana sin escribir un sistema de estilos propio. Frente a alternativas como Tailwind CSS, no requiere un paso de compilación de los estilos, lo que mantiene el despliegue reducido a actualizar el código de Python y las plantillas.
Bootstrap Icons	1.11.3	Iconografía de la navegación, de los botones y de los estados.	Es el conjunto de iconos del mismo proyecto que el sistema de estilos, de modo que la apariencia resulta consistente y no se añade un segundo origen de recursos externos.
Chart.js	4.4.1	Dibujo de las gráficas del módulo de reportes y del panel del modelo predictivo sobre un lienzo del navegador.	Las gráficas se dibujan en el navegador del usuario y no en el servidor. Generarlas en el servidor con una biblioteca como Matplotlib consumiría memoria de la instancia en cada consulta y produciría imágenes fijas, sin la lectura de valores al pasar el cursor que la interpretación de los reportes requiere.

Tabla 57: Bibliotecas incorporadas al sistema, su función y la justificación de su selección Fuente: elaboración propia.

Las tres bibliotecas del componente predictivo —scikit-learn, pandas y joblib— se declaran en el archivo de requerimientos, pero se importan únicamente dentro de las funciones que las necesitan y no al inicio de los módulos. La razón es la memoria del servidor: importar esas tres bibliotecas eleva el consumo del proceso que lo haga de 44 a 182 megabytes, según la medición realizada sobre el conjunto de datos del sistema. Con tres procesos de trabajo de Gunicorn y una base de datos PostgreSQL en la misma instancia de dos gigabytes, cargarlas al arrancar cada proceso consumiría memoria incluso en las peticiones que no estiman nada. La importación diferida mantiene ese costo acotado al momento en que efectivamente se predice y, una vez leído, el modelo se conserva en memoria para las peticiones siguientes.

Se consideró además la biblioteca XGBoost, mencionada en las preguntas auxiliares de la investigación como algoritmo candidato. Se decidió no incorporarla al sistema por dos razones. La primera es que introduce una dependencia con código nativo compilado, más pesada de instalar y de mantener en la instancia de producción. La segunda es que, sobre el conjunto de datos disponible, el bosque aleatorio alcanzó un coeficiente de determinación de 0.939 y redujo en 41.8 % el error del método que la clínica emplea hoy, de modo que el margen de mejora que podría aportar un algoritmo de potenciación del gradiente no justificaba el costo de la dependencia. La

comparación entre los dos algoritmos efectivamente implementados se presenta en la sección 4.2.3.10.

4.1.2.3 Herramientas de apoyo al desarrollo y a la documentación

Un segundo grupo de herramientas no forma parte del sistema desplegado, pero intervino en su construcción y en la elaboración de este documento. Se declaran por transparencia metodológica: los diagramas y las capturas que ilustran el trabajo no se dibujaron a mano ni se recortaron de la pantalla, sino que se generaron a partir del código fuente, de modo que reflejen el sistema tal como está construido y puedan regenerarse cuando el sistema cambie. La Tabla 58 las relaciona.

Herramienta	Para qué se utilizó	Por qué se seleccionó
PlantUML	Generación de los diagramas de actividad de los dieciséis procesos, de los diagramas de casos de uso y del diagrama de clases.	Los diagramas se escriben como texto y se versionan junto con el código fuente, de modo que un cambio en un proceso se refleje regenerando el diagrama y no redibujándolo.
Graphviz	Generación del diagrama entidad-relación agrupado por módulo.	Dispone automáticamente veinticuatro entidades y veintinueve relaciones con agrupación por subgrafos, lo que resultó legible donde una disposición automática sin agrupar no lo era.
Matplotlib	Generación del diagrama de arquitectura de software por capas.	Permite construir el diagrama por programa, con las mismas proporciones y la misma paleta del resto de las ilustraciones.
Playwright y Chromium	Captura automatizada de las veintiséis pantallas del sistema que ilustran el capítulo 3 y el presente capítulo.	Las capturas se toman todas con el mismo ancho de ventana y el mismo factor de escala, sobre el mismo conjunto de datos de demostración, de modo que las figuras del documento resulten comparables entre sí.
python-docx	Composición de este documento: inserción de secciones, tablas, figuras y pies con el formato establecido.	Permite que el diccionario de datos, los requerimientos y las especificaciones se generen desde su origen —los modelos del sistema— y no se transcriban a mano, con lo que el documento no se desactualiza respecto del código.

Tabla 58: Herramientas de apoyo al desarrollo y a la documentación del proyecto Fuente: elaboración propia.

4.1.2.4 Organización del código fuente

El repositorio del proyecto se organiza en cuatro directorios de primer nivel: app contiene el proyecto Django; infra, la definición de la infraestructura como código; ml, los artefactos del entrenamiento del modelo; y docs, la documentación y las entregas. Dentro de app, cada módulo funcional del sistema corresponde a una aplicación de Django con sus propios modelos, vistas, formularios, direcciones y plantillas. Esa correspondencia entre módulo y aplicación es deliberada: permite que el código de un módulo se lea, se pruebe y se modifique sin abrir los demás. La Tabla

59 relaciona cada aplicación con su responsabilidad y con su tamaño aproximado en líneas de código.

Aplicación	Responsabilidad	Archivos principales	Líneas
config	Configuración del proyecto, enrutamiento raíz, validadores compartidos y formatos regionales.	settings.py, urls.py, validadores.py, formats/es/formats.py	≈ 290
usuarios	Autenticación, roles y control de acceso; mantenimiento de usuarios, recuperación de contraseña y auditoría.	models.py, views.py, forms.py, backends.py, decorators.py, middleware.py, context_processors.py	≈ 660
pacientes	Registro de pacientes, antecedentes médicos e historial de cambios.	models.py, views.py, forms.py	≈ 300
catalogo	Especialidades, procedimientos, odontólogos y unidades dentales.	models.py, views.py, forms.py	≈ 400
agenda	Módulo Principal: calendario, programación de citas, horarios, ausencias y registro de tiempos reales. Incluye el módulo de disponibilidad efectiva.	views.py, forms.py, models.py, disponibilidad.py	≈ 1,335
clinico	Expediente clínico con odontograma, notas de evolución, planes de tratamiento por fases y cobros.	models.py, views.py, forms.py, piezas.py	≈ 1,345
inteligencia	Servicio de estimación de duraciones, entrenamiento y versionado del modelo predictivo.	servicio.py, entrenamiento.py, models.py, views.py, management/commands/entrenar.py	≈ 710
reportes	Indicadores operativos y reportes de precisión de la estimación.	servicio.py, views.py, models.py	≈ 230

Tabla 59: Organización del código fuente por aplicación del proyecto Fuente: elaboración propia.

El código fuente se escribió siguiendo cuatro convenciones constantes. La primera es que los identificadores están en español —nombres de modelos, de campos, de funciones y de variables —, salvo los que el framework impone, con lo que el vocabulario del código coincide con el de la clínica y con el de este documento. La segunda es el borrado lógico: ninguna entidad se elimina de la base de datos, sino que se marca con el campo estado en falso, de modo que un procedimiento dado de baja conserve su vínculo con las citas históricas que lo utilizaron. La tercera es que toda modificación de un registro sensible deja constancia en una tabla de historial, con el estado anterior y el usuario que la realizó. La cuarta es que cada módulo y cada función no evidente llevan una cadena de documentación que explica no lo que hace el código, que se lee en el código, sino por qué está escrito así, con lo que las decisiones de diseño quedan registradas donde se aplican.

4.1.3 Seguridad

El sistema administra información de salud de pacientes identificados, de modo que la seguridad no se limita a controlar quién entra. Se implementó en cinco planos: la autenticación, que establece quién es el usuario; la autorización, que determina qué puede hacer; la protección de

la sesión; la protección de los datos frente a los ataques web habituales; y la gestión de los secretos de configuración.

4.1.3.1 Autenticación

La identificación del usuario se realiza con su correo electrónico, que el modelo declara como campo único, en lugar del nombre de usuario que Django emplea por omisión. Para ello se implementó un respaldo de autenticación propio, que localiza al usuario por correo sin distinguir mayúsculas y delega en el framework la verificación de la contraseña y del estado de la cuenta.

Las contraseñas nunca se almacenan: la base de datos guarda el resumen criptográfico que produce el algoritmo PBKDF2 con función de resumen SHA-256 y 720,000 iteraciones, que es el mecanismo predeterminado de Django 5. El número de iteraciones es lo que encarece deliberadamente cada intento de adivinación por fuerza bruta. Además, toda contraseña nueva debe superar los cuatro validadores del framework: longitud mínima, disimilitud respecto de los datos del propio usuario, ausencia de la lista de contraseñas comunes y prohibición de contraseñas enteramente numéricas.

El respaldo de autenticación incorpora una precaución que no es evidente en su lectura. Cuando el correo no existe, en lugar de responder de inmediato, calcula el resumen de la contraseña recibida y lo descarta. Sin esa operación, un correo inexistente se rechazaría de inmediato y uno existente con contraseña equivocada tardaría el tiempo del cálculo del resumen, con lo que la diferencia de tiempos de respuesta permitiría averiguar qué correos están registrados en el sistema.

La recuperación de contraseña se resuelve con una pregunta secreta elegida de una lista cerrada al dar de alta la cuenta. La respuesta tampoco se almacena en claro: se normaliza a minúsculas y se guarda con el mismo algoritmo de resumen que las contraseñas. El proceso se divide en tres pasos y cada uno verifica en la sesión que el anterior se haya completado, de modo que no sea posible llegar a la pantalla de definición de la nueva contraseña sin haber respondido correctamente.

4.1.3.2 Autorización y control de acceso por rol

El control de acceso se implementó como un modelo basado en roles. Cada rol declara, con un campo booleano por módulo, a cuáles de los ocho módulos del sistema tiene acceso, y dos campos adicionales matizan ese acceso. El campo de solo consulta distingue entre ver y modificar:

un rol puede tener acceso a un módulo y, sin embargo, no poder crear ni alterar registros en él. El campo de escritura clínica se declara aparte porque el expediente responde a una lógica distinta de la administrativa: quien lo escribe es el profesional con competencia para hacerlo, aunque su rol sea de solo consulta para la agenda de la clínica, y no lo escribe la recepción, aunque sí modifique otros módulos. Derivar ese permiso del campo de solo consulta habría dado el resultado contrario en los dos casos.

La restricción se aplica en la vista, no en la plantilla. Cada vista lleva un decorador que exige acceso al módulo y, cuando modifica información, que exige además permiso de escritura; si no se cumplen, la petición se rechaza antes de ejecutar la vista. Ocultar el botón en la pantalla es una comodidad de la interfaz, no una medida de seguridad: un usuario que escriba la dirección a mano llegaría igualmente a la vista. Para que la interfaz y la seguridad no se contradigan, un procesador de contexto pone a disposición de todas las plantillas el mismo cálculo de permisos que aplica el decorador, de modo que el menú y los botones se construyan a partir de la misma fuente que autoriza.

4.1.3.3 Protección de la sesión y del navegador

La sesión se mantiene con la cookie de sesión de Django, marcada como inaccesible al código JavaScript de la página, de modo que una inyección de script no pueda leer el identificador de sesión y suplantar al usuario. Todos los formularios que modifican información incorporan el testigo contra falsificación de petición entre sitios que el framework verifica en el servidor, con lo que una página externa no puede provocar una acción en nombre de un usuario con sesión abierta. El encabezado que prohíbe mostrar el sistema dentro de un marco está fijado en denegar, lo que impide el ataque de superposición de interfaces.

Se agregó además un componente intermedio propio que impide al navegador almacenar en caché las páginas dinámicas. Sin él, un usuario que cierra su sesión en la computadora compartida de la recepción y pulsa el botón de retroceso del navegador vería de nuevo la página con los datos del paciente, servida desde la caché local aunque la sesión ya no exista en el servidor. El componente excluye deliberadamente los archivos estáticos, que sí deben poder almacenarse.

4.1.3.4 Protección de los datos

Todo acceso a la base de datos se realiza a través del mapeador objeto-relacional, que construye las consultas con parámetros enlazados; el sistema no concatena en ningún punto datos del usuario dentro de una sentencia SQL, que es el mecanismo del que depende la inyección de código SQL. En la salida, el motor de plantillas de Django escapa por omisión todo valor que se interpola, de modo que un dato guardado con etiquetas HTML se muestre como texto y no se ejecute en el navegador de quien lo consulta.

La entrada se valida en el servidor y no solo en el navegador. Los validadores de nombres, correos, códigos y teléfonos están centralizados en un módulo compartido, de manera que el criterio con que se valida el nombre de un usuario sea el mismo con que se valida el de un paciente. A esa validación se añade la de las reglas de negocio en los formularios: que una cita no se programe en el pasado, que quepa en la disponibilidad efectiva del profesional, que no se traslape con otra y que quede una unidad dental libre. Esas reglas no son validaciones de formato, pero su lugar es el mismo, porque un dato que las incumple corrompe la operación igual que un dato mal escrito.

La integridad histórica se protege por dos vías complementarias. La primera es el borrado lógico ya descrito. La segunda son las tablas de historial de usuarios, pacientes, citas y planes, que conservan el estado anterior, la acción realizada, el usuario responsable y la fecha. Esas tablas no referencian con llave foránea al registro que describen, sino que guardan su identificador y una copia de sus datos visibles, de modo que el historial sobreviva a la desaparición del registro original y la auditoría no quede incompleta. La Tabla 60 resume la correspondencia entre cada riesgo considerado y el mecanismo que lo atiende, con indicación del lugar del código donde está implementado.

Riesgo	Mecanismo implementado	Dónde está implementado
Robo de contraseñas desde la base de datos	Resumen criptográfico PBKDF2-SHA256 con 720,000 iteraciones; la contraseña nunca se almacena.	Mecanismo predeterminado de Django; modelo Usuario
Contraseñas débiles	Cuatro validadores obligatorios: longitud, disimilitud, lista de contraseñas comunes y prohibición de valores numéricos.	config/settings.py (AUTH_PASSWORD_VALIDATORS)
Enumeración de cuentas por tiempo de respuesta	El respaldo de autenticación calcula un resumen incluso cuando el correo no existe, para igualar el tiempo de respuesta.	usuarios/backends.py
Acceso a un módulo sin autorización	Decorador que exige el permiso del rol antes de ejecutar la vista.	usuarios/decorators.py (modulo_requerido)

Modificación de datos por un rol de solo consulta	Decorador de escritura que verifica el permiso en la vista y no en la plantilla.	usuarios/decorators.py (escritura_requerida)
Escritura del expediente clínico por personal no facultado	Permiso de escritura clínica declarado por separado del permiso administrativo.	usuarios/models.py (Rol.escritura_clinica)
Inyección de código SQL	Consultas construidas por el mapeador objeto-relacional con parámetros enlazados; sin concatenación de entradas.	Todo el acceso a datos del sistema
Ejecución de scripts inyectados (XSS)	Escape automático de todo valor interpolado en las plantillas.	Motor de plantillas de Django
Falsificación de petición entre sitios (CSRF)	Testigo por formulario, verificado en el servidor en toda petición que modifica información.	CsrfViewMiddleware y plantillas
Robo de la sesión desde el navegador	Cookie de sesión inaccesible al código JavaScript de la página.	SESSION_COOKIE_HTTPONLY
Superposición de interfaces (clickjacking)	Encabezado que prohíbe mostrar el sistema dentro de un marco.	XFrameOptionsMiddleware (X_FRAME_OPTIONS = DENY)
Lectura de datos de un paciente tras cerrar la sesión	Encabezados que impiden al navegador almacenar en caché las páginas dinámicas.	usuarios/middleware.py (NoCacheMiddleware)
Pérdida de la trazabilidad de la información	Borrado lógico en todas las entidades y tablas de historial con el estado anterior y el usuario responsable.	Campo estado y tablas HistorialUsuario, HistorialPaciente, HistorialCita e HistorialPlan
Exposición de credenciales en el repositorio	Toda credencial se lee de variables de entorno; el archivo de entorno y las llaves están excluidos del control de versiones.	config/settings.py y .gitignore

Tabla 60: Riesgos considerados y mecanismos de seguridad implementados Fuente: elaboración propia.

4.1.3.5 Gestión de la configuración y de los secretos

Ninguna credencial figura en el código fuente. La clave secreta de Django, la contraseña de la base de datos, los nombres de dominio autorizados y el indicador de depuración se leen de variables de entorno, con valores predeterminados aptos solo para el entorno de desarrollo. En el servidor, esas variables se entregan al servicio desde un archivo de entorno que no forma parte del repositorio, y el archivo de exclusiones del control de versiones descarta explícitamente los archivos de entorno, las llaves privadas y los certificados. Las credenciales que el flujo de despliegue continuo necesita para entrar al servidor —la dirección, el usuario y la llave privada de acceso remoto— se almacenan como secretos del repositorio en la plataforma de integración continua y se inyectan durante la ejecución, de modo que no aparecen en el archivo del flujo.

Queda pendiente una medida de endurecimiento que se documenta por honestidad metodológica: el servidor atiende actualmente por el protocolo HTTP sin cifrar. Mientras el sistema opera con datos de demostración la exposición es acotada, pero el manejo de información clínica real exige habilitar HTTPS con un certificado y activar en consecuencia las marcas de cookie segura y de redirección obligatoria. Es la primera tarea del plan de puesta en producción definitiva.

4.2 Codificación

La presente sección documenta el código fuente del sistema. Comienza por el inicio de sesión y la construcción del menú, que son los dos puntos por los que todo usuario pasa; continúa con el inventario de las funciones principales de cada módulo, con indicación de lo que hace cada una y con el fragmento de código de las más significativas; y concluye con un ejemplo completo de la generación de gráficas y reportes. Los fragmentos se reproducen tal como están en el repositorio, con la indicación del archivo al que pertenecen; donde el fragmento se abrevia, la omisión se señala con puntos suspensivos.

4.2.1 Inicio de sesión

El sistema no expone ninguna pantalla sin sesión abierta: toda vista lleva el decorador que exige autenticación y, al no haberla, redirige al formulario de ingreso (Figura 50). El formulario solicita el correo electrónico y la contraseña, y ante un fallo responde con un mensaje único —«Correo o contraseña incorrectos»— que deliberadamente no distingue cuál de los dos datos falló, para no confirmar a un tercero que un correo está registrado en el sistema.

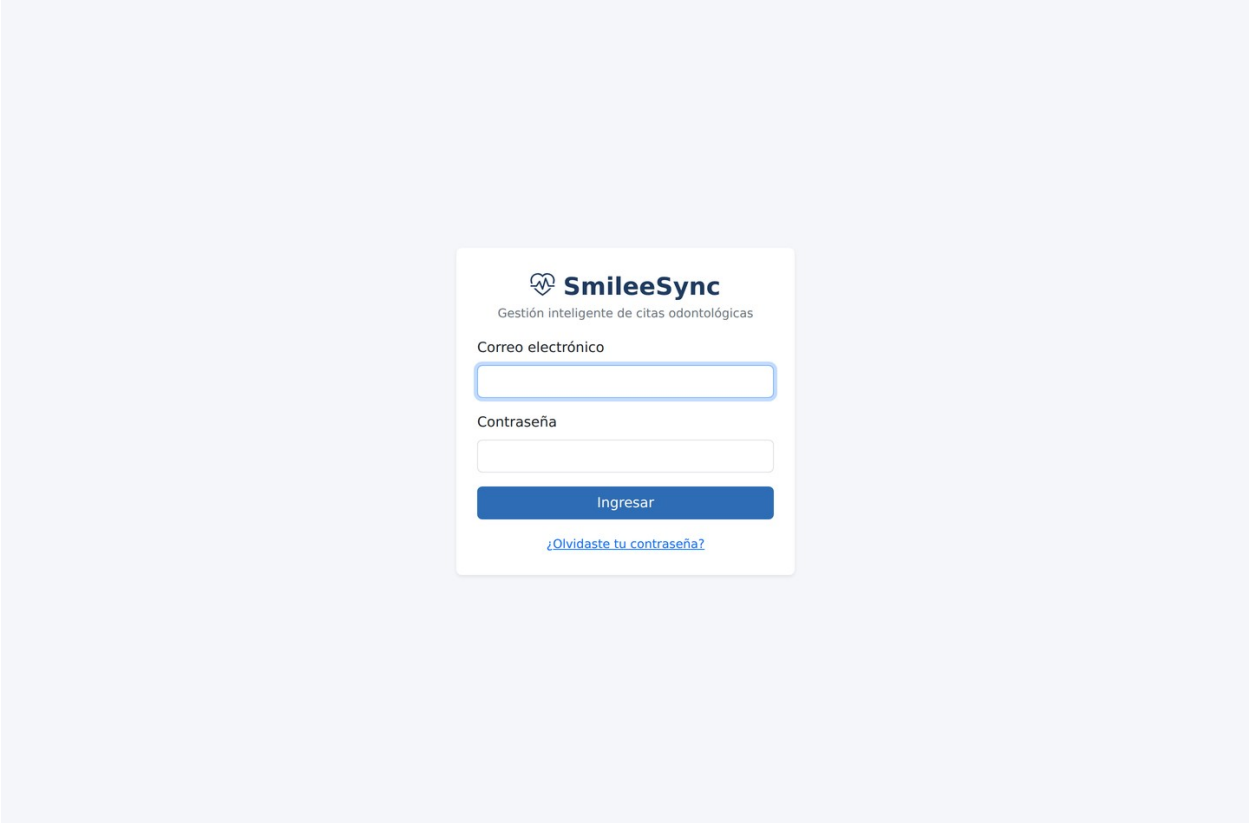


Figura 50: Pantalla de inicio de sesión del sistema Fuente: elaboración propia.

La Tabla 61 relaciona los archivos que intervienen en el proceso con la función que cumple cada uno.

Archivo	Función o clase	Qué hace
config/urls.py	ruta login/	Publica la vista de autenticación de Django con la plantilla y el formulario propios del sistema.
usuarios/forms.py	LoginCorreoForm	Sustituye la etiqueta del campo de usuario por «Correo electrónico» y aplica los estilos del sistema.
usuarios/backends.py	EmailBackend.authenticate	Localiza al usuario por su correo sin distinguir mayúsculas, verifica la contraseña contra el resumen almacenado y comprueba que la cuenta esté activa.
usuarios/models.py	Usuario	Declara el correo como campo único e incorpora el rol, el estado y la pregunta secreta.
usuarios/middleware.py	NoCacheMiddleware	Impide que el navegador conserve en caché las páginas con sesión, de modo que el botón de retroceso no las muestre tras cerrar sesión.
usuarios/views.py	recuperar_correo, recuperar_pregunta, recuperar_nueva	Resuelven en tres pasos la recuperación de la contraseña mediante la pregunta secreta.

Tabla 61: Archivos y funciones que intervienen en el inicio de sesiónFuente: elaboración propia.

El enrutamiento reutiliza la vista de autenticación del framework y le entrega la plantilla y el formulario del sistema, con lo que el manejo de la sesión, del testigo contra falsificación de peticiones y de la redirección posterior queda a cargo de código ya probado.

app/config/urls.py

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('login/', auth_views.LoginView.as_view(
        template_name='registration/login.html',
        authentication_form=LoginCorreoForm), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    # El módulo Principal (calendario de la agenda) es la raíz del sitio.
    path('', include('agenda.urls')),
    path('usuarios/', include('usuarios.urls')),
    ...
]
```

La verificación de las credenciales se implementó en un respaldo de autenticación propio. La línea que calcula y descarta un resumen cuando el correo no existe es la que iguala el tiempo de respuesta entre un correo inexistente y uno existente con contraseña equivocada, conforme se explicó en la sección 4.1.3.1.

app/usuarios/backends.py

```
class EmailBackend(ModelBackend):
    """Permite iniciar sesión con el correo electrónico en lugar del username."""
    def authenticate(self, request, username=None, password=None, **kwargs):
        UserModel = get_user_model()
        email = kwargs.get('email') or username
        if not email or not password:
            return None
        try:
```

```
            user = UserModel.objects.get(email__iexact=email.strip())
        except UserModel.DoesNotExist:
            UserModel().set_password(password)  # iguala el tiempo de respuesta
            return None
        except UserModel.MultipleObjectsReturned:
            return None
        if user.check_password(password) and self.user_can_authenticate(user):
            return user
        return None
```

El componente intermedio que acompaña al inicio de sesión no interviene en la autenticación, sino en lo que ocurre al terminarla. Añade a cada respuesta dinámica los encabezados que impiden al navegador conservarla en caché, y excluye los archivos estáticos, que sí deben almacenarse para no volver a descargarse en cada pantalla.

app/usuarios/middleware.py

```
class NoCacheMiddleware:
    def __call__(self, request):
        response = self.get_response(request)
        # No aplicar a archivos estáticos (deben poder cachearse).
        if not request.path.startswith(settings.STATIC_URL):
            response['Cache-Control'] = 'no-store, no-cache, must-revalidate, max-age=0'
            response['Pragma'] = 'no-cache'
            response['Expires'] = '0'
        return response
```

4.2.2 Menú principal

Tras autenticarse, el usuario llega al módulo Principal (Figura 51), que ocupa la raíz del sitio. La pantalla se compone de tres partes: el menú lateral con los módulos a los que el usuario tiene acceso, la franja superior con su nombre, su rol y, cuando corresponde, el distintivo de solo consulta, y el área de trabajo con los indicadores del período y el calendario de la clínica.

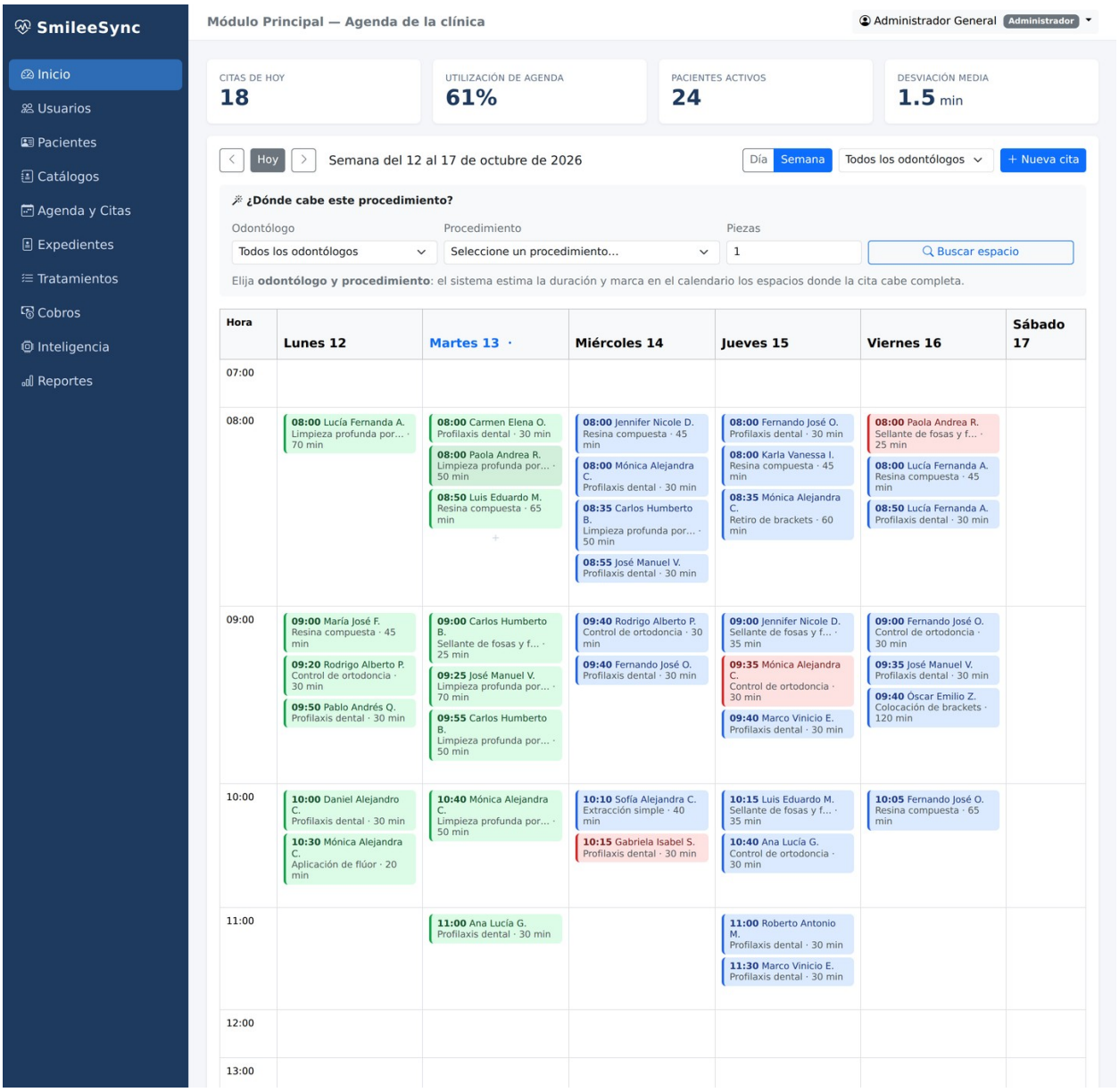


Figura 51: Menú principal del sistema con el calendario semanal de la clínica (datos de demostración) Fuente: elaboración propia.

El menú no está escrito fijo en la plantilla: se construye a partir de los permisos del rol del usuario que tiene la sesión abierta. Cada entrada se condiciona al permiso del módulo correspondiente, de modo que la recepción no ve la entrada del módulo de inteligencia y el odontólogo no ve la de usuarios. El mismo archivo de plantilla sirve, así, a los cuatro roles del sistema.

app/templates/base.html

```
<ul class="nav flex-column py-2">
  <li class="nav-item"><a class="nav-link {% if request.resolver_match.url_name == 'principal' %}
active{% endif %}"
```

```
      href="{% url 'agenda:principal' %}"><i class="bi bi-speedometer2"></i> Inicio</a></li>

  {% if user.rol.puede_usuarios or user.es_admin %}
  <li class="nav-item"><a class="nav-link" href="{% url 'usuarios:lista' %}">
     <i class="bi bi-people"></i> Usuarios</a></li>
  {% endif %}

  {% if user.rol.puede_clinico or user.es_admin %}
  <li class="nav-item"><a class="nav-link" href="{% url 'clinico:lista' %}">
     <i class="bi bi-file-medical"></i> Expedientes</a></li>
  {% endif %}

  {% if user.rol.puede_inteligencia or user.es_admin %}
  <li class="nav-item"><a class="nav-link" href="{% url 'inteligencia:panel' %}">
     <i class="bi bi-cpu"></i> Inteligencia</a></li>
  {% endif %}
  ...
</ul>
```

La decisión de qué puede hacer el usuario, y no solo qué puede ver, se concentra en dos métodos del modelo de usuario. El primero resuelve el acceso al módulo; el segundo, el permiso de escritura, con la excepción del expediente clínico, que se rige por la competencia profesional declarada en el rol y no por los privilegios administrativos.

app/usuarios/models.py — clase Usuario

```
def puede(self, modulo):
    """Devuelve True si el usuario tiene acceso al módulo indicado."""
    if self.es_admin:
        return True
    return bool(self.rol and getattr(self.rol, f'puede_{modulo}', False))

def puede_escribir(self, modulo):
    """Acceso de escritura: exige el módulo y que el rol no sea de consulta.

    El módulo clínico es la excepción y se rige por `escritura_clinica`: el
    expediente lo escribe quien tiene competencia profesional declarada,
    con independencia de sus privilegios administrativos.
    """
    if self.es_admin:
        return True
    if not self.puede(modulo):
        return False
    if modulo == 'clinico':
        return bool(self.rol and self.rol.escritura_clinica)
    return not self.es_solo_consulta
```

Para que la interfaz y la autorización no puedan contradecirse, un procesador de contexto expone a todas las plantillas el resultado de esos mismos métodos. Así, la condición con que la plantilla oculta un botón proviene del mismo cálculo con que el decorador rechaza la petición, y no de una regla escrita dos veces.

app/usuarios/context_processors.py

```
MODULOS = ['usuarios', 'pacientes', 'catalogos', 'agenda', 'clinico',
           'finanzas', 'inteligencia', 'reportes']
```

```
def permisos(request):
    usuario = getattr(request, 'user', None)
    if usuario is None or not usuario.is_authenticated:
        return {'escritura': {}, 'solo_consulta': False}
    return {
        'escritura': {m: usuario.puede_escribir(m) for m in MODULOS},
        'solo_consulta': usuario.es_solo_consulta,
    }
```

La Tabla 62 presenta la correspondencia entre los cuatro roles definidos y los módulos que cada uno alcanza, con indicación de si el acceso incluye escritura. Es la matriz que el sistema aplica y la que determina, en consecuencia, el menú que cada usuario encuentra al ingresar.

Módulo	Administrador	Recepcionista	Odontólogo	Asistente
Usuarios	Escritura	Sin acceso	Sin acceso	Sin acceso
Pacientes	Escritura	Escritura	Consulta	Sin acceso
Catálogos	Escritura	Sin acceso	Sin acceso	Sin acceso
Agenda y citas	Escritura	Escritura	Consulta	Consulta
Expediente clínico	Escritura	Consulta	Escritura	Consulta
Cobros	Escritura	Escritura	Sin acceso	Sin acceso
Inteligencia	Escritura	Sin acceso	Sin acceso	Sin acceso
Reportes	Consulta	Consulta	Sin acceso	Sin acceso

Tabla 62: Matriz de acceso por rol y por módulo implementada en el sistema Fuente: elaboración propia.

El rol de odontólogo merece una precisión. Su acceso a la agenda es de consulta, porque la programación de citas es una función de la recepción; sin embargo, el profesional sí registra sus propias ausencias, que son información suya. Es el único caso del sistema en que un rol de consulta escribe, y está acotado a los registros del odontólogo vinculado a su propio usuario mediante una comprobación explícita en la vista.

4.2.3 Funciones principales

El módulo Principal es el que concentra el aporte del sistema y el que reúne, con sus dependencias, algo más de la mitad del código escrito. La presente sección documenta primero el inventario completo de sus funciones y a continuación sus cinco procesos, cada uno con el fragmento de código que lo resuelve; después inventaría las funciones de los demás módulos con el fragmento de la más representativa de cada uno. La Tabla 63 relaciona las funciones del módulo Principal con lo que hace cada una.

Archivo	Función	Qué hace
views.py	principal	Construye la pantalla de inicio: resuelve el período solicitado, filtra las citas, calcula los indicadores, pide la estimación cuando hay una consulta en curso y arma la rejilla del calendario.
views.py	_rejilla	Organiza las citas en una matriz de horas por días y determina

		el rango de horas visible, incluyendo las de los espacios recomendados.
views.py	_huecos_sugeridos	Calcula el primer inicio viable de cada hora para una cita de la duración estimada, recorriendo la jornada en pasos de quince minutos.
views.py	_semana_solicitada, _fecha_solicitada, _lunes_de	Interpretan el período pedido en la dirección y, ante un valor inválido, recaen en el día o la semana actuales.
views.py	estimar	Atiende la consulta de estimación en formato JSON que el formulario de cita realiza cada vez que cambia uno de sus campos.
views.py	_estimacion_para	Obtiene la estimación con los datos que el formulario ya tiene, para mostrarla junto a los errores de validación sin perder lo capturado.
views.py	crear	Programa una cita nueva; registra la predicción, deja constancia en el historial y, cuando la cita proviene de un plan, liga el renglón correspondiente.
views.py	editar, cancelar	Reprograman y cancelan una cita conservando en el historial el estado anterior; la cancelación nunca elimina el registro.
views.py	registrar_tiempos	Registra los tiempos reales de la atención, incorpora el dato al conjunto de entrenamiento y cierra los renglones del plan que la cita atendió.
views.py	_cerrar_renglones	Marca como completados los renglones del plan ejecutados por la cita y recalcula el avance del plan.
views.py	lista, detalle	Consultan las citas por rango de fechas, odontólogo y estado, y muestran el detalle de una cita con su desviación de tiempo.
views.py	horarios, quitar_horario	Administran el patrón semanal de disponibilidad de cada profesional.
views.py	ausencias, quitar_ausencia	Registran y retiran las ausencias de una fecha concreta.
disponibilidad.py	franjas_efectivas	Calcula los intervalos en que el profesional atiende una fecha: su patrón semanal menos las ausencias registradas para ese día.
disponibilidad.py	_restar	Resta un intervalo de una lista de intervalos; un corte en medio parte el intervalo en dos y uno que lo cubre lo elimina.
disponibilidad.py	hay_traslape	Determina si dos intervalos se traslapan: cada uno empieza antes de que el otro termine.
disponibilidad.py	cabe_en_franja	Verifica que la cita completa quepa dentro de una sola franja efectiva.
disponibilidad.py	unidades_libres, asignar_unidad	Determinan qué unidades dentales están desocupadas en un intervalo y toman la primera disponible.
disponibilidad.py	unidades_activas	Lista las unidades dentales dadas de alta en el catálogo.
disponibilidad.py	capacidad_minutos	Calcula los minutos de atención posibles en un período como el menor entre profesionales disponibles y unidades activas en cada intervalo de quince minutos.
forms.py	CitaForm.clean	Valida las cuatro condiciones de una cita: duración positiva, cabida en la disponibilidad efectiva, ausencia de traslape y unidad dental libre.
forms.py	CitaForm.clean_fecha	Impide programar una cita nueva en una fecha pasada, sin bloquear la reprogramación de una cita antigua.
forms.py	CitaForm.save	Calcula la hora de fin y asigna automáticamente la unidad dental, que no se pide al usuario porque las unidades son intercambiables.
forms.py	RegistrarTiemposForm	Valida los tiempos reales y calcula la duración efectiva de la atención.
forms.py	HorarioForm, AusenciaForm	Validan que las franjas no se traslapen entre sí y que la hora de fin sea posterior a la de inicio.
forms.py	sumar_minutos	Suma minutos a una hora sin desbordar al día siguiente.

models.py	Cita	Entidad de la cita: paciente, profesional, procedimiento, fecha, horas programadas y reales, duración asignada y real, estado y unidad dental.
models.py	HorarioDisponibilidad, AusenciaOdontologo	Patrón semanal de atención y ausencias por fecha de cada profesional.
models.py	HistorialCita	Historial de auditoría de la cita: acción, estado anterior, responsable y fecha.

Tabla 63: Inventario de funciones del módulo Principal y de la agenda Fuente: elaboración propia.

4.2.3.1 Proceso 1: construcción del calendario

El calendario es la pantalla de trabajo habitual de la recepción y la que todo usuario encuentra al ingresar. La vista resuelve primero el período —semana o día, según el parámetro recibido—, consulta las citas de ese rango con sus relaciones precargadas en una sola consulta para no incurrir en una consulta por cita, y entrega a la plantilla una matriz de filas por hora y columnas por día.

La función que arma esa matriz incorpora una decisión que no es evidente: el rango de horas visible no se calcula solo con las citas existentes, sino también con los espacios recomendados. Sin esa consideración, una recomendación situada a una hora en que no hay ninguna cita quedaría fuera de la rejilla y el usuario no la vería, que es precisamente el caso en que la recomendación resulta más útil.

app/agenda/views.py — función _rejilla

```
def _rejilla(citas, inicio, dias_visibles, sugeridos=None):
    """Organiza las citas en una rejilla de horas por días para la plantilla."""
    rejilla = {}
    for cita in citas:
        indice_dia = (cita.fecha - inicio).days
        if 0 <= indice_dia < dias_visibles:
            hora = cita.hora_inicio_programada.hour
            rejilla.setdefault(hora, {}).setdefault(indice_dia, []).append(cita)

    # El rango de horas visible abarca tanto las citas existentes como los
    # espacios sugeridos: de lo contrario una recomendación podría quedar
    # fuera de la rejilla y el usuario no la vería.
    horas = sorted(rejilla.keys()) + [h for _, h in (sugeridos or {})]
    if horas:
        desde = max(HORA_APERTURA, min(horas) - 1)
        hasta = min(HORA_CIERRE, max(horas) + 2)
    else:
        desde, hasta = 8, 18

    filas = []
    for hora in range(desde, hasta):
        celdas = []
        for d in range(dias_visibles):
            fecha = inicio + timedelta(days=d)
            celdas.append({
                'indice': d, 'citas': rejilla.get(hora, {}).get(d, []),
                'fecha': fecha.isoformat(), 'hora': f'{hora:02d}:00',
```

```
            'sugerida': (sugeridos or {}).get((d, hora)),
        })
    filas.append({'hora': f'{hora:02d}:00', 'celdas': celdas})
return filas
```

Cada casilla lleva su fecha y su hora, de modo que la plantilla pueda enlazar directamente a la programación de una cita en ese espacio. Esa es la razón por la que la casilla libre del calendario es un enlace y no un simple espacio en blanco: al pulsarla, el formulario de cita llega con la fecha, la hora y el profesional ya definidos, con lo que el usuario solo captura el paciente y el procedimiento.

La misma vista calcula los cuatro indicadores de la franja superior. El más relevante de ellos es la utilización de la agenda, que no se mide contra las horas declaradas por los profesionales sino contra la capacidad real de la clínica, calculada como el menor entre profesionales disponibles y unidades dentales activas en cada intervalo de quince minutos. Medirla contra las horas declaradas sobrestimaría la capacidad y reportaría una utilización inferior a la real, que es justamente el dato que la clínica necesita para decidir si le hace falta otro sillón.

4.2.3.2 Proceso 2: estimación de la duración del procedimiento

La estimación de duraciones es el objeto de la investigación y, en el código, un servicio con una sola puerta de entrada. Todo el sistema —el calendario, el formulario de cita, el plan de tratamiento— solicita la estimación a la misma función, que aplica una cadena de respaldo de tres niveles: el modelo entrenado, si existe una versión en producción; el promedio histórico real de la clínica, si hay al menos tres atenciones del mismo procedimiento; y la duración de referencia del catálogo, como último respaldo.

La cadena no es una redundancia defensiva sino un requisito de la implantación. Una clínica que estrena el sistema no tiene historial, de modo que un sistema que solo supiera estimar con el modelo entrenado sería inservible durante sus primeros meses, que son precisamente aquellos en que la clínica debe adoptarlo. Cada estimación declara, además, de qué nivel proviene, de manera que quien agenda sepa qué tan fundado es el número que está viendo.

app/inteligencia/servicio.py — función estimar_duracion

```
def estimar_duracion(procedimiento, odontologo, fecha, hora, num_piezas=1):
    """Estima la duración de un procedimiento y el bloque de tiempo sugerido."""
    # --- Nivel 1: modelo predictivo entrenado ---
    modelo = _modelo_activo()
    if modelo:
        prediccion = _estimar_con_modelo(modelo, procedimiento, odontologo,
                                         fecha, hora, num_piezas)
        if prediccion:
```

```
            duracion = _redondear(prediccion)
            minimo, maximo = _bloque_recomendado(duracion)
            est = Estimacion(duracion, minimo, maximo, 'modelo',
                             f'Estimado por {modelo.nombre} '
                             f'({modelo.get_algoritmo_display()}).')
            est.modelo = modelo
            return est

    # --- Nivel 2: promedio histórico real de la clínica ---
    promedio, total, es_del_odontologo = _estimar_con_historico(
        procedimiento, odontologo)
    if promedio:
        duracion = _redondear(promedio)
        minimo, maximo = _bloque_recomendado(duracion)
        alcance = ('de este odontólogo' if es_del_odontologo
                   else 'de todos los odontólogos')
        return Estimacion(duracion, minimo, maximo, 'historico',
                          f'Promedio de {total} atenciones previas {alcance}. '
                          f'La precisión aumentará al acumularse más registros.')

    # --- Nivel 3: duración de referencia del catálogo ---
    duracion = _redondear(procedimiento.duracion_promedio_min)
    minimo, maximo = _bloque_recomendado(duracion)
    return Estimacion(duracion, minimo, maximo, 'catalogo',
                      'Aún no hay suficientes atenciones registradas de este '
                      'procedimiento; se usa la duración de referencia del catálogo.')
```

El servicio no devuelve un número suelto sino un objeto con la duración estimada, el bloque recomendado —un rango del 85 % al 120 % de la estimación, para no quedarse corto ni bloquear agenda de más—, el origen de la estimación y el texto con que la pantalla lo explica. Devolver también la explicación es deliberado: es lo que permite que la interfaz declare la procedencia del número sin que cada pantalla tenga que redactarla por su cuenta.

La lectura del modelo desde el disco requirió una corrección durante el desarrollo. En la primera versión, la función que predice invocaba la carga del archivo serializado en cada estimación, lo que significaba deserializar medio megabyte por cada cita que se consultaba. Se agregó una caché en memoria del módulo, de modo que el archivo se lea una sola vez por versión y se descarte únicamente cuando se activa otra. La verificación automatizada mide esa diferencia: la primera estimación tras limpiar la caché resulta sensiblemente más costosa que las siguientes, y estas se mantienen por debajo de cien milisegundos.

app/inteligencia/servicio.py — caché del modelo

```
# El archivo del modelo puede pesar varios megabytes. Deserializarlo en cada
# estimación significaría releerlo en cada cita que se agenda, de modo que se
# conserva en memoria y solo se recarga cuando cambia la versión activa.
_CACHE = {'version': None, 'estimador': None, 'anestesia': {}}



def _cargar(modelo):
    """Devuelve el estimador de la versión indicada, cargándolo una sola vez."""
    if _CACHE['version'] == modelo.pk and _CACHE['estimador'] is not None:
```

```
        return _CACHE['estimador']
    try:
        import joblib
    except ImportError:
        return None
    if not modelo.ruta_archivo or not os.path.exists(modelo.ruta_archivo):
        return None
    try:
        estimador = joblib.load(modelo.ruta_archivo)
    except Exception:
        # Un archivo dañado o entrenado con otra versión de la biblioteca no
        # debe impedir que se agende: la cadena de respaldo continúa.
        return None
    _CACHE['version'] = modelo.pk
    _CACHE['estimador'] = estimador
    return estimador
```

Al predecir, dos de las siete variables del modelo plantean un problema que el diseño tuvo que resolver: en el momento de agendar no se sabe todavía si se aplicará anestesia ni si surgirá una complicación, porque ambas se conocen al atender. La anestesia se resuelve con el comportamiento histórico del procedimiento —si más de la mitad de sus atenciones la usaron, se asume que la usará—; la complicación se asume ausente, porque por definición es imprevista y el sistema estima el caso nominal. Omitir ambas variables del entrenamiento habría sido la alternativa, pero son dos de las tres que más explican la duración, de modo que la precisión habría caído.

app/inteligencia/servicio.py — función _estimar_con_modelo

```
estimador = _cargar(modelo)
if estimador is None:
    return None
try:
    import pandas as pd
except ImportError:
    return None


try:
    # Los nombres de las columnas deben coincidir con los del entrenamiento.
    entrada = pd.DataFrame([{
        'procedimiento': procedimiento.nombre,
        'odontologo': f'{odontologo.nombres} {odontologo.apellidos}',
        'dia_semana': DIAS[fecha.weekday()],
        'hora_inicio': hora.hour,
        'num_piezas': num_piezas,
        'anestesia': _anestesia_habitual(procedimiento),
        'complicacion': 0,
    }])
    return float(estimador.predict(entrada)[0])
except Exception:
    # Un fallo del modelo no debe impedir agendar una cita.
    return None
```

El manejo de excepciones de ese fragmento responde a un criterio explícito: ninguna falla del componente predictivo puede impedir que la clínica agende una cita. Si las bibliotecas no están instaladas, si el archivo del modelo desapareció o si fue entrenado con una versión incompatible

de la biblioteca, la función devuelve un valor nulo y la cadena de respaldo continúa con el promedio histórico. Una de las verificaciones automatizadas comprueba exactamente ese escenario: apunta la versión activa a una ruta inexistente y confirma que la estimación sigue estando disponible, con el origen declarado en otro nivel de la cadena.

La estimación se consulta desde el navegador sin recargar la página. Cada vez que el usuario cambia el procedimiento, el profesional, la fecha, la hora o el número de piezas, el formulario solicita la estimación a una vista que responde en formato JSON y actualiza el panel de recomendación.

app/agenda/views.py — función estimar

```
@login_required
@modulo_requerido('agenda')
def estimar(request):
    """Devuelve la estimación de duración en formato JSON."""
    id_proc = (request.GET.get('procedimiento') or '').strip()
    id_odo = (request.GET.get('odontologo') or '').strip()
    if not (id_proc.isdigit() and id_odo.isdigit()):
        return JsonResponse({'disponible': False})

    procedimiento = Proc.objects.filter(pk=int(id_proc), estado=True).first()
    odontologo = Odo.objects.filter(pk=int(id_odo), estado=True).first()
    if not (procedimiento and odontologo):
        return JsonResponse({'disponible': False})
    ...
    est = estimar_duracion(procedimiento, odontologo, fecha, hora, piezas)
    return JsonResponse({
        'disponible': True, 'duracion': est.duracion_min,
        'bloque_min': est.bloque_min, 'bloque_max': est.bloque_max,
        'origen': est.origen, 'etiqueta_origen': est.etiqueta_origen,
        'detalle': est.detalle,
    })
```

4.2.3.3 Proceso 3: recomendación de espacios en la agenda

La estimación por sí sola no resuelve el problema de la recepción. Saber que un tratamiento de conducto tomará noventa minutos no dice dónde caben noventa minutos en la semana. El sistema recorre, entonces, la disponibilidad de la semana en pasos de quince minutos y devuelve el primer inicio viable de cada hora, que la plantilla resalta sobre la rejilla del calendario.

Un inicio es viable cuando cumple cuatro condiciones a la vez: la cita completa cabe dentro de una franja efectiva del profesional, no se traslapa con ninguna de sus citas vigentes, no corresponde a un momento ya pasado y queda al menos una unidad dental libre. La última condición es la que distingue la recomendación de un simple cálculo de huecos en la agenda del

profesional: la clínica tiene menos sillones que odontólogos, de modo que dos profesionales libres a la misma hora no significan dos citas simultáneas posibles.

app/agenda/views.py — función _huecos_sugeridos

```
PASO_SUGERENCIA_MIN = 15



def _huecos_sugeridos(odontologo, fechas, duracion, hoy, ahora):
    """Primer inicio viable de cada hora para una cita de `duracion` minutos."""
    if not (odontologo and duracion):
        return {}

    ocupadas = {}
    for c in (Cita.objects
              .filter(odontologo=odontologo, fecha__in=fechas, estado=True)
              .exclude(estado_cita='CANCELADA')):
        ocupadas.setdefault(c.fecha, []).append((c.hora_inicio_programada,
                                         c.hora_fin_programada))

    sugeridos = {}
    for indice, fecha in enumerate(fechas):
        # Disponibilidad efectiva: el patrón semanal menos las ausencias.
        for f_ini, f_fin in franjas_efectivas(odontologo, fecha):
            candidato = datetime.combine(fecha, f_ini)
            limite = datetime.combine(fecha, f_fin)
            while candidato + timedelta(minutes=duracion) <= limite:
                inicio_h = candidato.time()
                fin_h = (candidato + timedelta(minutes=duracion)).time()
                clave = (indice, inicio_h.hour)
                ya_paso = fecha < hoy or (fecha == hoy and inicio_h < ahora)
                choca = any(hay_traslape(inicio_h, fin_h, oi, of)
                            for oi, of in ocupadas.get(fecha, []))
                # Un espacio solo sirve si además queda un sillón libre.
                if not ya_paso and not choca and clave not in sugeridos:
                    if unidades_libres(fecha, inicio_h, fin_h):
                        sugeridos[clave] = inicio_h.strftime('%H:%M')
                candidato += timedelta(minutes=PASO_SUGERENCIA_MIN)
    return sugeridos
```

La disponibilidad del profesional no se lee directamente de su patrón semanal, porque ese patrón describe cómo atiende habitualmente y no si estará ese día concreto. La disponibilidad efectiva es el patrón menos las ausencias registradas para la fecha, y se calcula restando intervalos: una ausencia a media jornada parte la franja en dos y una que la cubre por completo la elimina. Esa resta es la única operación del módulo que no resulta evidente, y es también la que permite que una ausencia de dos horas no cancele el resto del día.

app/agenda/disponibilidad.py

```
def _restar(intervalos, quitar):
    """Resta un intervalo de una lista de intervalos, en minutos."""
    inicio_q, fin_q = quitar
    resultado = []
    for inicio, fin in intervalos:
        if fin_q <= inicio or inicio_q >= fin:      # no se tocan
            resultado.append((inicio, fin))
            continue
```

```
        if inicio_q > inicio:                       # queda un tramo antes
            resultado.append((inicio, inicio_q))
        if fin_q < fin:                             # queda un tramo después
            resultado.append((fin_q, fin))
    return resultado



def franjas_efectivas(odontologo, fecha):
    """Intervalos (hora_inicio, hora_fin) en que el odontólogo atiende."""
    intervalos = [
        (_a_minutos(f.hora_inicio), _a_minutos(f.hora_fin))
        for f in HorarioDisponibilidad.objects.filter(
            odontologo=odontologo, dia_semana=DIAS[fecha.weekday()], estado=True)
        if f.hora_fin > f.hora_inicio
    ]
    if not intervalos:
        return []

    for ausencia in AusenciaOdontologo.objects.filter(
            odontologo=odontologo, fecha=fecha, estado=True):
        intervalos = _restar(intervalos, (_a_minutos(ausencia.hora_inicio),
                                          _a_minutos(ausencia.hora_fin)))
        if not intervalos:
            break

    return [(_a_hora(i), _a_hora(f)) for i, f in sorted(intervalos)]
```

4.2.3.4 Proceso 4: programación de la cita y validación de la disponibilidad

La recomendación orienta, pero no obliga: el usuario puede programar una cita en cualquier espacio. Por eso la validación no se delega a la pantalla sino que reside en el formulario, que verifica cuatro condiciones antes de permitir el guardado y, cuando alguna falla, devuelve un mensaje que indica qué hacer en lugar de limitarse a rechazar.

app/agenda/forms.py — método CitaForm.clean

```
fin = sumar_minutos(inicio, duracion)
if fin <= inicio:
    self.add_error('duracion_asignada_min',
                   'La duración debe hacer que la cita termine después de '
                   'haber empezado.')
    return limpio
limpio['hora_fin_programada'] = fin


# --- la cita debe caber en la disponibilidad efectiva del odontólogo ---
dia = DIAS[fecha.weekday()]
tiene_patron = HorarioDisponibilidad.objects.filter(
    odontologo=odontologo, dia_semana=dia, estado=True).exists()
franjas = franjas_efectivas(odontologo, fecha)


if not tiene_patron:
    self.add_error('fecha', f'{odontologo} no tiene horario de atención '
                            f'configurado para ese día de la semana.')
elif not franjas:
    self.add_error('fecha', f'{odontologo} tiene registrada una ausencia que '
                            f'cubre toda su jornada del '
                            f'{fecha.strftime("%d/%m/%Y")}.')
elif not any(f_ini <= inicio and fin <= f_fin for f_ini, f_fin in franjas):
```

```
    disponibles = ', '.join(f'{i.strftime("%H:%M")} a {t.strftime("%H:%M")}'
                            for i, t in franjas)
    self.add_error('hora_inicio_programada',
                   f'La cita queda fuera del horario de atención. '
                   f'{odontologo} atiende ese día de {disponibles}.')
```

Las dos validaciones restantes verifican el traslape con otras citas del mismo profesional y la disponibilidad de una unidad dental. La segunda incorpora un detalle de redacción que surgió de las pruebas de implementación: el mensaje distingue el caso de una sola unidad activa del de varias, porque con un único sillón dado de alta el mensaje genérico producía la frase «Las 1 unidades dentales están ocupadas».

app/agenda/forms.py — método CitaForm.clean (continuación)

```
# --- no puede traslaparse con otra cita del mismo odontólogo ---
otras = Cita.objects.filter(odontologo=odontologo, fecha=fecha, estado=True
                            ).exclude(estado_cita='CANCELADA')
if self.instance.pk:
    otras = otras.exclude(pk=self.instance.pk)
for otra in otras:
    if hay_traslape(inicio, fin, otra.hora_inicio_programada,
                    otra.hora_fin_programada):
        self.add_error('hora_inicio_programada',
                       f'Se traslapa con la cita de {otra.paciente} '
                       f'({otra.hora_inicio_programada.strftime("%H:%M")} a '
                       f'{otra.hora_fin_programada.strftime("%H:%M")}).')
        break

# --- debe quedar una unidad dental libre ---
unidades = unidades_activas()
if not unidades:
    self.add_error(None, 'No hay unidades dentales activas en el catálogo, '
                         'por lo que no es posible programar citas.')
else:
    libre = asignar_unidad(fecha, inicio, fin, excluir_cita=self.instance.pk)
    if libre is None:
        cuantas = ('La única unidad dental está ocupada' if len(unidades) == 1
                   else f'Las {len(unidades)} unidades dentales están ocupadas')
        self.add_error('hora_inicio_programada',
                       f'{cuantas} en ese horario. Elija otro espacio o '
                       f'consulte la recomendación del calendario.')
    else:
        limpio['unidad'] = libre
```

La unidad dental no se le pide al usuario. Las unidades son intercambiables entre sí, de modo que preguntar en qué sillón se atenderá añadiría un campo al formulario sin aportar información: el sistema asigna la primera libre al guardar. La asignación automática tiene además la ventaja de que la validación y la asignación usan el mismo cálculo, con lo que no puede ocurrir que el formulario acepte una cita para la que luego no haya sillón.

4.2.3.5 Proceso 5: registro de la atención y realimentación del modelo

El quinto proceso es el que cierra el ciclo y el que hace que el sistema mejore con su propio uso. Al concluir la atención se registran la hora real de inicio y la de finalización; el formulario

calcula la duración efectiva, el sistema deja constancia en el historial de la cita, incorpora el registro al conjunto de entrenamiento del modelo y cierra los renglones del plan de tratamiento que esa cita ejecutó.

app/agenda/views.py — función registrar_tiempos

```
@login_required
@escritura_requerida('agenda')
def registrar_tiempos(request, pk):
    """Registro de los tiempos reales de atención al cerrar la cita."""
    cita = get_object_or_404(Cita, pk=pk, estado=True)
    if request.method == 'POST':
        form = RegistrarTiemposForm(request.POST, instance=cita)
        if form.is_valid():
            HistorialCita.objects.create(
                cita_ref=cita.pk, paciente=str(cita.paciente),
                odontologo=str(cita.odontologo), fecha_cita=cita.fecha,
                estado_cita_anterior=cita.estado_cita, accion='ATENCION',
                realizado_por=request.user.username)
            cita = form.save()
            registrar_dato_entrenamiento(cita)
            messages.success(
                request,
                f'Atención registrada: {cita.duracion_real_min} minutos reales '
                f'frente a {cita.duracion_asignada_min} asignados. '
                f'El dato se incorporó al entrenamiento del modelo.')
            _cerrar_renglones(request, cita)
            # El expediente del paciente queda incompleto sin la nota de la
            # visita, así que se ofrece escribirla en el momento a quien tiene
            # competencia para hacerlo.
            if request.user.puede_escribir('clinico') and not _tiene_nota(cita):
                return redirect(f'{reverse("clinico:nota_nueva", args=[cita.paciente_id])}'
                                f'?cita={cita.pk}')
            return redirect('agenda:lista')
```

La redirección final no es un detalle de comodidad. La duración real sin la nota clínica de la visita es un dato incompleto para el modelo: el número de piezas tratadas, el uso de anestesia y la aparición de complicaciones son tres de sus siete variables y solo se conocen al documentar la atención. Por eso, cuando quien cierra la cita tiene competencia para escribir el expediente, el sistema lo lleva directamente al formulario de la nota con la cita ya vinculada. Cuando no la tiene —el caso de la recepción—, la cita queda cerrada y la nota pendiente para el profesional.

El cierre de los renglones del plan responde al mismo criterio de no pedir al usuario lo que el sistema puede deducir: si la cita que ejecutaba un renglón presupuestado fue atendida, el renglón está ejecutado, y el avance del plan se recalcula sin que nadie lo marque a mano. Cuando el plan queda finalizado con saldo pendiente, el sistema lo advierte, porque es el momento en que la clínica debe gestionar el cobro.

4.2.3.6 Funciones del módulo de Seguridad y Usuarios

Las funciones de este módulo se inventarían en la Tabla 64. La más significativa no es una vista sino los dos decoradores que protegen a todas las demás, porque son el punto único en que se aplica la autorización del sistema.

Archivo	Función o clase	Qué hace
views.py	lista, crear, editar, cambiar_estado	Mantenimiento de usuarios con borrado lógico; cada operación deja constancia en el historial de auditoría.
views.py	historial	Consulta del historial de cambios de los usuarios.
views.py	recuperar_correo, recuperar_pregunta, recuperar_nueva	Recuperación de contraseña en tres pasos mediante la pregunta secreta, con verificación de la etapa anterior en cada paso.
decorators.py	modulo_requerido	Restringe una vista a los usuarios cuyo rol tiene acceso al módulo.
decorators.py	escritura_requerida	Exige acceso al módulo y además que el rol no sea de solo consulta.
decorators.py	puede_gestionar_odontologo	Permite al profesional administrar la disponibilidad de su propia ficha, único caso en que un rol de consulta escribe.
backends.py	EmailBackend.authenticate	Autenticación por correo electrónico con tiempo de respuesta uniforme.
middleware.py	NoCacheMiddleware	Impide el almacenamiento en caché de las páginas con sesión.
context_processors.py	permisos	Expone a todas las plantillas los permisos de escritura del usuario.
models.py	Rol	Declara el acceso por módulo, la condición de solo consulta y el permiso de escritura clínica.
models.py	Usuario.puede, puede_escribir, es_solo_consulta, es_admin	Resuelven el acceso y la escritura por módulo para un usuario.
models.py	Usuario.set_respuesta_secreta, check_respuesta_secreta	Guardan y verifican la respuesta de la pregunta secreta cifrada.
models.py	HistorialUsuario.registrar	Inserta una copia del estado previo del usuario antes de modificarlo.
forms.py	UsuarioCrearForm, UsuarioEditarForm	Validan nombres, correo único y contraseña, y aplican los validadores compartidos del sistema.

Tabla 64: Inventario de funciones del módulo de Seguridad y Usuarios Fuente: elaboración propia.

app/usuarios/decorators.py

```
def modulo_requerido(modulo):
    """Restringe una vista a los usuarios cuyo rol tiene acceso al módulo."""
    def decorador(vista):
        @wraps(vista)
        def _envuelta(request, *args, **kwargs):
            u = request.user
            if not u.is_authenticated:
                return redirect('login')
            if u.puede(modulo):
                return vista(request, *args, **kwargs)
            raise PermissionDenied('No tiene acceso a este módulo.')
        return _envuelta
    return decorador



def escritura_requerida(modulo):
    """Restringe una vista que modifica información.
```

```
    Exige acceso al módulo y, además, que el rol no sea de solo consulta. Se
    aplica a las vistas de alta, edición, cambio de estado y registro de
    tiempos, de modo que la restricción no dependa de ocultar el botón en la
    plantilla: el acceso directo a la dirección también queda denegado.
    """
    def decorador(vista):
        @wraps(vista)
        def _envuelta(request, *args, **kwargs):
            u = request.user
            if not u.puede(modulo):
                raise PermissionDenied('No tiene acceso a este módulo.')
            if u.es_solo_consulta:
                raise PermissionDenied(
                    'Su rol es de solo consulta: no puede modificar información.')
            return vista(request, *args, **kwargs)
        return _envuelta
    return decorador
```

4.2.3.7 Funciones de los módulos de Pacientes y de Catálogo

Los dos módulos comparten una característica: su comportamiento es el mismo para varias entidades. El catálogo administra cuatro —especialidades, procedimientos, odontólogos y unidades dentales— y todas se listan, se crean, se editan y se dan de baja igual. En lugar de escribir cuatro veces el mismo código, las cuatro se resuelven con vistas genéricas parametrizadas por un diccionario de configuración, con lo que agregar una quinta entidad al catálogo requiere declararla y no programarla.

app/catalogo/views.py

```
# Configuración de cada entidad del catálogo: modelo, formulario y etiquetas.
ENTIDADES = {
    'procedimientos': {
        'modelo': Procedimiento, 'form': ProcedimientoForm,
        'singular': 'procedimiento', 'plural': 'Procedimientos',
        'orden': ['nombre'],
        'busqueda': lambda q: Q(codigo__icontains=q) | Q(nombre__icontains=q),
    },
    'unidades': {
        'modelo': UnidadDental, 'form': UnidadDentalForm,
        'singular': 'unidad dental', 'plural': 'Unidades dentales',
        'orden': ['codigo'],
        'busqueda': lambda q: (Q(codigo__icontains=q) | Q(nombre__icontains=q) |
                               Q(ubicacion__icontains=q)),
    },
    ...
}



@login_required
@modulo_requerido('catalogos')
def lista(request, entidad='procedimientos'):
    cfg = _config(entidad)
    ...
```

Los validadores de los datos de entrada están centralizados en un módulo compartido por los dos módulos y por el resto del sistema. El inventario completo de las funciones de ambos se

presenta en la Tabla 65. La expresión regular de los nombres admite tildes, la letra eñe, apóstrofos y guiones, de modo que acepte nombres compuestos y apellidos con guion, y rechace números y caracteres especiales.

app/config/validadores.py

```
# Letras (con tildes y ñ), espacios, apóstrofos y guiones, para admitir nombres
# compuestos como "María José" o apellidos como "López-García".
NOMBRE_REGEX = re.compile(
    r"^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ]+(?:[\s'-][A-Za-zÁÉÍÓÚÜÑáéíóúüñ]+)*$")


# Un código alfanumérico con guiones, por ejemplo PAC-0001 o PROC-014.
CODIGO_REGEX = re.compile(r'^[A-Za-z0-9][A-Za-z0-9\-]*[A-Za-z0-9]$')



def validar_nombre(valor, etiqueta='nombre'):
    """Rechaza números y caracteres especiales en nombres y apellidos."""
    valor = (valor or '').strip()
    if not valor:
        raise ValidationError(f'El campo {etiqueta} es obligatorio.')
    if not NOMBRE_REGEX.match(valor):
        raise ValidationError(
            f'El {etiqueta} solo puede contener letras y espacios '
            '(no se permiten números ni caracteres especiales).')
    return valor
```

Módulo	Función	Qué hace
pacientes	lista	Localiza pacientes por código, nombre o teléfono.
pacientes	crear, editar	Registran y actualizan al paciente con sus datos de identificación y de contacto.
pacientes	ficha	Consolida en una vista los datos personales, los antecedentes médicos y el historial de citas del paciente.
pacientes	cambiar_estado, quitar_antecedente	Aplican el borrado lógico al paciente y a sus antecedentes.
pacientes	_auditar	Inserta en el historial una copia del estado previo del paciente.
pacientes	siguiente_codigo	Genera el siguiente código correlativo de paciente.
catalogo	lista, crear, editar, cambiar_estado	Vistas genéricas que atienden las cuatro entidades del catálogo.
catalogo	_config	Resuelve la configuración de la entidad solicitada y responde con error cuando no existe.
catalogo	ProcedimientoForm	Valida el código, el nombre, el precio y la duración de referencia del procedimiento.
catalogo	OdontologoForm	Valida los datos del profesional, su número de colegiado y su vínculo con un usuario del sistema.

Tabla 65: Inventario de funciones de los módulos de Pacientes y de Catálogo Fuente: elaboración propia.

4.2.3.8 Funciones del módulo de Expediente Clínico

La función más característica del módulo es la que reconstruye el odontograma vigente. La tabla de hallazgos es de solo agregado: cada observación se registra con su fecha y no se modifica, de modo que el expediente conserva la evolución de cada pieza. El estado actual de la boca es, entonces, el último hallazgo activo de cada combinación de pieza y cara, y se obtiene recorriendo

los registros del más antiguo al más reciente: cada escritura sobre la misma clave sustituye a la anterior y, al terminar el recorrido, queda la observación vigente.

app/clinico/models.py

```
def odontograma_vigente(paciente):
    """Último hallazgo activo de cada pieza y cara del paciente.

    Devuelve {numero_de_pieza: {cara: Hallazgo}}. Al recorrer los registros de
    más antiguo a más reciente, cada escritura sobre la misma clave sustituye a
    la anterior, de modo que al terminar queda la observación vigente.
    """
    vigente = {}
    consulta = (Hallazgo.objects
                .filter(paciente=paciente, estado=True)
                .order_by('fecha', 'creado'))
    for h in consulta:
        vigente.setdefault(h.pieza, {})[h.cara] = h
    return vigente
```

La nota clínica de cada visita se vincula con la cita en una relación uno a uno, lo que impide documentar dos veces la misma atención. Tres de sus campos —número de piezas tratadas, uso de anestesia y aparición de complicaciones— cumplen la doble función ya señalada: documentan el acto clínico y constituyen variables del modelo predictivo. El inventario del módulo se presenta en la Tabla 66.

Archivo	Función o clase	Qué hace
views.py	lista, expediente	Localizan al paciente y presentan su expediente con el odontograma vigente, los hallazgos pendientes y la última nota.
views.py	odontograma, quitar_hallazgo	Registran un hallazgo sobre una pieza y una cara, y aplican el borrado lógico al hallazgo registrado por error.
views.py	notas, nota_nueva	Presentan la hoja de evolución y registran la nota de una visita atendida.
views.py	_exigir_escritura_clinica	Verifica el permiso de escritura clínica antes de modificar el expediente.
views.py	_resumen_odontograma	Cuenta las piezas por tipo de hallazgo para el encabezado del expediente.
models.py	Hallazgo	Observación fechada del estado de una pieza; tabla de solo agregado.
models.py	Hallazgo.simbolo, clase_css, es_patologico	Traducen el tipo de hallazgo al símbolo y al color con que la pieza se dibuja, y distinguen los hallazgos por atender.
models.py	odontograma_vigente	Reconstruye el estado actual de la boca a partir del historial de hallazgos.
models.py	NotaClinica	Nota de evolución de una visita, vinculada uno a uno con la cita atendida.
models.py	NotaClinica.lista_piezas	Convierte el campo de piezas tratadas en la lista de números de pieza.
piezas.py	es_valida, nombre, filas_arcada	Validan el número de pieza según la notación FDI, devuelven su nombre anatómico y organizan las piezas en los cuatro cuadrantes.
forms.py	HallazgoForm, NotaClinicaForm	Validan la pieza, la fecha y la coherencia entre el hallazgo y la visita que lo registra.

Tabla 66: Inventario de funciones del módulo de Expediente Clínico Fuente: elaboración propia.

4.2.3.9 Funciones de los módulos de Plan de Tratamiento y de Cobros

El plan de tratamiento es el punto en que el componente predictivo deja de resolver una cita aislada. Cada renglón del plan conserva la duración que el modelo estimó al presupuestarlo, y con esas duraciones el sistema reparte el trabajo pendiente en sesiones de sillón: agrupa los renglones respetando el orden de las fases —una fase posterior no se adelanta— y sin que ninguna sesión exceda las dos horas, que es el tope razonable en jornadas de cuatro. El resultado es una previsión del número de visitas que el tratamiento requerirá, disponible antes de agendar la primera.

app/clinico/models.py — método PlanTratamiento.sesiones_sugeridas

```
def sesiones_sugeridas(self):
    """Reparte los renglones pendientes en sesiones de sillón.

    Los renglones se agrupan respetando su fase (una fase posterior no se
    adelanta) y sin que ninguna sesión pase de `MINUTOS_SESION_MAX`. Es una
    estimación de planificación, no una reserva: agendar cada sesión sigue
    siendo una decisión de la recepción.
    """
    sesiones = []
    actual, minutos, fase_actual = [], 0, None
    for d in self.renglones.order_by('fase', 'orden', 'id'):
        if d.estado_detalle == 'COMPLETADO':
            continue
        cambia_fase = fase_actual is not None and d.fase != fase_actual
        no_cabe = minutos + d.duracion_estimada_min > MINUTOS_SESION_MAX
        if actual and (cambia_fase or no_cabe):
            sesiones.append({'fase': fase_actual, 'renglones': actual,
                             'minutos': minutos})
            actual, minutos = [], 0
        actual.append(d)
        minutos += d.duracion_estimada_min
        fase_actual = d.fase
    if actual:
        sesiones.append({'fase': fase_actual, 'renglones': actual,
                         'minutos': minutos})
    return sesiones
```

El estado financiero del plan no se almacena: se deriva. El subtotal es la suma de los importes de los renglones, el total descuenta la rebaja acordada, lo pagado es la suma de los abonos vigentes y el saldo es la diferencia. Mantener esas cifras como campos obligaría a recalcularlas en cada operación que las afecte, y bastaría que una ruta omitiera el recálculo para que el saldo de un paciente quedara equivocado. Derivarlas en el momento de leerlas hace imposible esa inconsistencia; la anulación de un abono, además, no borra el registro, sino que lo marca como inactivo, con lo que el saldo se corrige y la trazabilidad del cobro se conserva. La Tabla 67 inventaría las funciones de ambos módulos.

app/clinico/models.py — clase PlanTratamiento

```
@property
def subtotal(self):
    return sum((d.importe for d in self.renglones), Decimal('0.00'))

@property
def total(self):
    return max(Decimal('0.00'),
               self.subtotal - (self.descuento or Decimal('0.00')))

@property
def pagado(self):
    return sum((p.monto for p in self.pagos.filter(estado=True)),
               Decimal('0.00'))

@property
def saldo(self):
    return self.total - self.pagado
```

Archivo	Función o clase	Qué hace
views.py	planes, plan_nuevo, plan_detalle	Listan los planes, crean uno nuevo y presentan su detalle con fases, renglones, sesiones sugeridas y estado financiero.
views.py	plan_estado	Mueve el plan entre propuesto, aprobado, en curso, finalizado y anulado, registrando el cambio en el historial.
views.py	plan_quitar_detalle	Aplica el borrado lógico a un renglón del plan.
views.py	agendar_renglon	Lleva al formulario de cita con el paciente, el procedimiento y la duración estimada del renglón ya definidos.
views.py	cobros, anular_pago	Presentan el estado de cobro de los planes aprobados y anulan un abono sin eliminar su registro.
models.py	PlanTratamiento.sesiones_sugeridas	Reparte los renglones pendientes en sesiones que no exceden las dos horas, respetando las fases.
models.py	PlanTratamiento.subtotal, total, pagado, saldo	Derivan el estado financiero del plan a partir de sus renglones y abonos vigentes.
models.py	PlanTratamiento.actualizar_avance	Mueve el plan a en curso o finalizado según sus renglones completados.
models.py	PlanTratamiento.minutos_estimados, minutos_pendientes	Suman los minutos de sillón del plan completo y de lo que resta.
models.py	PlanTratamiento.siguiente_codigo	Genera el código correlativo del plan.
models.py	DetallePlan.importe, etiqueta_origen	Calculan el importe del renglón y declaran de qué fuente proviene su duración estimada.
models.py	Pago, HistorialPlan.registrar	Registran los abonos aplicados al plan y el historial de cambios de estado.
forms.py	DetallePlanForm	Propone la tarifa del catálogo y la duración estimada por el modelo al agregar un renglón.
forms.py	PagoForm	Valida que el abono sea positivo y no exceda el saldo del plan.

Tabla 67: Inventario de funciones de los módulos de Plan de Tratamiento y de Cobros Fuente: elaboración propia.

4.2.3.10 Funciones del módulo de Inteligencia

El módulo de inteligencia se divide en dos partes que no comparten proceso. El servicio de estimación, documentado en la sección 4.2.3.2, opera dentro del proceso web y solo lee el modelo. El entrenamiento opera fuera de él, como un comando administrativo, por la razón de memoria expuesta en la sección 4.1.2.2.

El entrenamiento construye una tubería de procesamiento que serializa en un solo objeto la codificación de las variables categóricas y el estimador. Las tres variables categóricas —procedimiento, odontólogo y día de la semana— se codifican como variables indicadoras, con la instrucción de ignorar las categorías no vistas en el entrenamiento, de modo que un procedimiento nuevo no provoque un error al estimar. Las cuatro numéricas pasan sin transformación.

app/inteligencia/entrenamiento.py

```
CATEGORICAS = ['procedimiento', 'odontologo', 'dia_semana']
NUMERICAS = ['hora_inicio', 'num_piezas', 'anestesia', 'complicacion']



def _construir(algoritmo):
    preparacion = ColumnTransformer([
        ('categoricas', OneHotEncoder(handle_unknown='ignore'), CATEGORICAS),
        ('numericas', 'passthrough', NUMERICAS),
    ])
    if algoritmo == 'RF':
        estimador = RandomForestRegressor(
            n_estimators=200, min_samples_leaf=2, random_state=42, n_jobs=1)
    else:
        estimador = LinearRegression()
    return Pipeline([('preparacion', preparacion), ('estimador', estimador)])
```

El entrenamiento reserva la cuarta parte del conjunto para evaluar y no entrena con ella. Además de las tres métricas habituales, calcula el error de una línea base —estimar siempre el promedio global— porque es contra ella y no contra cero como se juzga si el modelo aporta algo. Un modelo con error de cinco minutos es excelente si la línea base se equivoca en diecisiete y mediocre si se equivoca en seis.

app/inteligencia/entrenamiento.py — función entrenar

```
df = pd.DataFrame(filas)
X = df.drop(columns=['duracion_real_min'])
y = df['duracion_real_min']
X_ent, X_pru, y_ent, y_pru = train_test_split(
    X, y, test_size=PROPORCION_PRUEBA, random_state=42)

pipe = _construir(algoritmo)
inicio = time.perf_counter()
pipe.fit(X_ent, y_ent)
segundos = time.perf_counter() - inicio

prediccion = pipe.predict(X_pru)
# Línea base: estimar siempre el promedio global. Es contra lo que se mide
# si el modelo aporta algo.
base = [y_ent.mean()] * len(y_pru)

resultado = Resultado(
    algoritmo=algoritmo,
    mae=round(float(mean_absolute_error(y_pru, prediccion)), 2),
    rmse=round(float(mean_squared_error(y_pru, prediccion) ** 0.5), 2),
    r2=round(float(r2_score(y_pru, prediccion)), 3),
    registros=len(df), registros_prueba=len(y_pru),
    mae_base=round(float(mean_absolute_error(y_pru, base)), 2),
```

```
    segundos=round(segundos, 3))
```

La Tabla 68 presenta el resultado del entrenamiento de los dos algoritmos implementados sobre el conjunto de 472 atenciones registradas en el sistema, de las cuales 118 se reservaron para evaluar y no intervinieron en el entrenamiento. La tabla reporta dos comparaciones distintas y conviene no confundirlas. La primera es contra la línea base estadística —estimar siempre el promedio global—, que indica si el modelo aprendió algo: ambos algoritmos la reducen en más del setenta por ciento. La segunda es contra la duración de referencia del catálogo, que es el método de bloques fijos que la clínica emplea hoy, y es la que resuelve la hipótesis del proyecto. Ahí los dos algoritmos se separan: la regresión lineal reduce el error un 26.4 %, apenas por encima de la meta del 25 %, mientras que el bosque aleatorio lo reduce un 41.8 %. Esa diferencia, y no el coeficiente de determinación, es la razón por la que el bosque aleatorio es el que opera en producción; su costo —un archivo doscientas setenta veces mayor y un entrenamiento veintisiete veces más largo— no resulta limitante en esta escala.

Métrica	Regresión lineal múltiple	Bosque aleatorio
Registros de entrenamiento / de prueba	354 / 118	354 / 118
Error absoluto medio (MAE)	5.91 minutos	4.67 minutos
Raíz del error cuadrático medio (RMSE)	8.53 minutos	6.47 minutos
Coeficiente de determinación (R^2)	0.894	0.939
MAE de la línea base (promedio global)	19.62 minutos	19.62 minutos
Reducción frente a la línea base	69.9 %	76.2 %
MAE de los bloques del catálogo	8.03 minutos	8.03 minutos
Reducción frente al catálogo (hipótesis)	26.4 %	41.8 %
Meta del proyecto	25 %	25 %
Tiempo de entrenamiento	0.008 segundos	0.221 segundos
Tamaño del archivo serializado	2.0 KB	539.3 KB

Tabla 68: Resultado del entrenamiento de los dos algoritmos implementados sobre 446 atenciones registradas Fuente: elaboración propia.

El bosque aleatorio permite, además, examinar el peso de cada variable en la predicción, que la Tabla 69 presenta. El resultado tiene una consecuencia metodológica directa: las tres variables de mayor peso son el número de piezas tratadas, el uso de anestesia y el tipo de procedimiento, y dos de ellas provienen de la nota clínica de la visita y no de la cita. Es la justificación empírica de haber incorporado el módulo de expediente clínico al sistema: las variables que aporta la nota de evolución reúnen el 66.2 % del peso del modelo, de modo que sin ella la estimación tendría que renunciar a la mayor parte de lo que hoy le permite acertar.

Variable	Peso en el modelo	Procedencia del dato
Número de piezas tratadas	43.6 %	Nota clínica de la visita
Uso de anestesia	19.4 %	Nota clínica de la visita

Procedimiento: tratamiento de conducto	16.5 %	Cita
Procedimiento: colocación de brackets	6.0 %	Cita
Procedimiento: retiro de brackets	5.4 %	Cita
Aparición de complicaciones	3.2 %	Nota clínica de la visita
Hora de inicio de la cita	0.9 %	Cita
Procedimiento: sellante de fosas y fisuras	0.8 %	Cita

Tabla 69: Peso de cada variable en el modelo de bosque aleatorio y procedencia del dato Fuente: elaboración propia.

Las dos comparaciones de la Tabla 68 responden preguntas distintas y la segunda merece detenerse. Medir el modelo contra el promedio global dice si aprendió, pero no si sirve: la clínica no estima con el promedio de todos sus procedimientos, sino reservando el bloque que declara el catálogo. Por eso el entrenamiento arrastra, junto a cada atención, la duración de referencia que el catálogo le asignaba, y calcula con ella el error que la clínica habría cometido sobre exactamente las mismas 118 atenciones del subconjunto de prueba. Esa duración no es una variable del modelo —se separa antes de entrenar, de modo que no pueda influir en la predicción— y solo se emplea para medir.

app/inteligencia/entrenamiento.py — función entrenar

```
df = pd.DataFrame(filas)
# La duración del catálogo no es una variable del modelo: se arrastra para
# medir contra ella en el mismo subconjunto de prueba.
catalogo = df['duracion_catalogo']
X = df.drop(columns=['duracion_real_min', 'duracion_catalogo'])
y = df['duracion_real_min']
X_ent, X_pru, y_ent, y_pru, _, cat_pru = train_test_split(
    X, y, catalogo, test_size=PROPORCION_PRUEBA, random_state=42)
...
resultado = Resultado(
    ...
    mae=round(float(mean_absolute_error(y_pru, prediccion)), 2),
    mae_base=round(float(mean_absolute_error(y_pru, base)), 2),
    mae_catalogo=round(float(mean_absolute_error(y_pru, cat_pru)), 2))
```

El resultado es el que resuelve la hipótesis de trabajo enunciada en la sección 3.4.4. Sobre esas 118 atenciones, reservar el bloque del catálogo produce un error absoluto medio de 8.03 minutos y el modelo uno de 4.67, lo que representa una reducción de 41.8 %. El criterio de decisión declarado era una reducción de al menos 25 %, de modo que la hipótesis nula se rechaza con holgura. Conviene subrayar que la medición se hace sobre registros que el modelo no vio al entrenarse: medirla sobre el conjunto completo arrojaría una cifra mejor y carecería de valor, porque el modelo ya conoce esas atenciones.

La cifra no queda en el informe del entrenamiento: se almacena con la versión del modelo y el panel del módulo la presenta en una franja propia, con el error de cada método, la reducción obtenida, la meta del proyecto y el veredicto. Así, quien administra el sistema puede comprobar

en cualquier momento si la versión en producción sigue cumpliendo el criterio, que es la pregunta que importa cuando el modelo se reentrena con datos nuevos.

Esa medición es retrospectiva: se hace sobre atenciones ya ocurridas. El sistema incorpora además una medición prospectiva, que es la que corresponde a la operación real. Cada vez que se programa una cita con el modelo en producción, la predicción emitida queda registrada con el bloque que se recomendó; cuando esa cita se atiende y se registran sus tiempos reales, la predicción queda contrastada contra lo que efectivamente duró. El panel informa entonces tres cifras: el error del modelo en operación, el del catálogo sobre las mismas citas y la proporción de atenciones que cayeron dentro del bloque recomendado. La última es la que importa en la agenda: una estimación que falla por dos minutos no desordena el día, pero una que se sale del bloque reservado sí. Esta medición se llena conforme la clínica usa el sistema, de modo que al momento de esta entrega está vacía y la pantalla lo declara así en lugar de presentar un resultado que no existe.

app/inteligencia/evaluacion.py

```
def desempeno_en_operacion():
    """Lectura prospectiva: qué tan cerca quedó cada predicción emitida."""
    filas = list(contrastables())
    if not filas:
        return None

    errores_modelo, errores_catalogo, dentro = [], [], 0
    for p in filas:
        real = p.cita.duracion_real_min
        errores_modelo.append(abs(p.duracion_estimada_min - real))
        errores_catalogo.append(
            abs(p.cita.procedimiento.duracion_promedio_min - real))
        if p.bloque_min_recomendado <= real <= p.bloque_max_recomendado:
            dentro += 1
    ...
```

El versionado conserva el histórico completo de los entrenamientos. Cada versión registra su algoritmo, sus métricas y la ruta de su archivo, y una sola está activa a la vez; las demás se conservan para poder comparar y, en caso necesario, volver a una anterior. El comando admite una opción que condiciona la activación a que la versión nueva reduzca el error de la vigente, de modo que un reentrenamiento con datos de mala calidad no degrade la estimación en producción. La Tabla 70 inventaría las funciones del módulo completo.

Comando de entrenamiento

```
python manage.py entrenar --algoritmo RF --activar
python manage.py entrenar --algoritmo RF --solo-si-mejora
python manage.py entrenar --algoritmo LINEAL
```

app/inteligencia/entrenamiento.py — función activar_version

```
def activar_version(version):
    """Pone una versión en producción y retira la anterior.
```

```
    Solo una versión está activa a la vez; las demás se conservan para poder
    comparar y, en caso necesario, volver atrás.
    """
    ModeloVersion.objects.filter(activo=True).exclude(
        pk=version.pk).update(activo=False)
    version.activo = True
    version.save(update_fields=['activo'])
    limpiar_cache()
    return version
```

Archivo	Función	Qué hace
servicio.py	estimar_duracion	Punto único de entrada de la estimación; aplica la cadena de respaldo de tres niveles.
servicio.py	_estimar_con_modelo	Arma el registro de entrada y solicita la predicción al modelo entrenado.
servicio.py	_estimar_con_historico	Promedia la duración real de las atenciones previas, prefiriendo las del mismo profesional cuando hay al menos tres.
servicio.py	_cargar, limpiar_cache	Conservan el modelo en memoria entre peticiones y lo descartan al activarse otra versión.
servicio.py	_anestesia_habitual	Deduce el uso habitual de anestesia del procedimiento, que al agendar todavía no se conoce.
servicio.py	_redondear, _bloque_recomendado	Redondean la estimación a múltiplos de cinco minutos y calculan el rango recomendado del 85 % al 120 %.
servicio.py	registrar_prediccion	Guarda la predicción emitida para contrastarla después con la duración real.
servicio.py	registrar_dato_entrenamiento	Incorpora una cita atendida al conjunto de entrenamiento; actualiza la fila si la atención ya estaba registrada.
entrenamiento.py	_fila_de	Arma, desde la cita y su nota clínica, los valores con que una atención entra al conjunto.
entrenamiento.py	sincronizar	Pone al día el conjunto de entrenamiento: da de alta las atenciones que faltan, corrige las que cambiaron y da de baja las que dejaron de ser válidas.
entrenamiento.py	reunir_datos	Devuelve el conjunto de entrenamiento, sincronizándolo antes de leerlo.
entrenamiento.py	_construir	Arma la tubería de codificación y estimación del algoritmo solicitado.
entrenamiento.py	entrenar	Entrena, evalúa contra la línea base y contra la duración del catálogo, y serializa el modelo; devuelve sus métricas.
entrenamiento.py	_importancias	Calcula el peso relativo de cada variable en el bosque aleatorio.
entrenamiento.py	registrar_version, activar_version	Registran la versión entrenada y la ponen en producción retirando la anterior.
evaluacion.py	resumen_version	Lectura retrospectiva de una versión: su error frente al del catálogo sobre el subconjunto de prueba, con el veredicto respecto de la meta del proyecto.
evaluacion.py	contrastables	Predicciones emitidas cuya cita ya fue atendida y medida.
evaluacion.py	desempeno_en_operacion	Lectura prospectiva: error del modelo y del catálogo sobre las citas realmente agendadas con él, y proporción de atenciones que cayeron dentro del bloque recomendado.
evaluacion.py	desempeno_por_procedimiento	Desglose de esa lectura por procedimiento.
commands/ entrenar.py	Command.handle	Comando administrativo que sincroniza el conjunto y ejecuta el entrenamiento fuera del proceso web, con las opciones de algoritmo, activación y activación condicionada a la mejora.
views.py	panel, activar, _desempeno_vigente	Presentan la validación de la hipótesis, las versiones entrenadas con sus métricas, el desempeño en operación y

		el contraste entre la duración del catálogo y la real; permiten activar una versión.

Tabla 70: Inventario de funciones del módulo de Inteligencia Fuente: elaboración propia.

4.2.3.11 Funciones del módulo de Reportes

El módulo de reportes separa el cálculo de la presentación: un módulo de servicio produce las cifras y la vista únicamente las entrega a la plantilla. Esa separación permite que las mismas cifras se consuman desde la pantalla y desde las verificaciones automatizadas sin duplicar la consulta. El inventario se presenta en la Tabla 71.

Archivo	Función	Qué hace
servicio.py	rango, _dias_habiles	Resuelven el período solicitado y la lista de días hábiles que contiene.
servicio.py	resumen	Calcula las cifras globales del período: citas, atenciones, cancelaciones, horas de sillón, utilización y error medio entre lo asignado y lo real.
servicio.py	atenciones_por_semana	Serie de atenciones y cancelaciones por semana.
servicio.py	carga_por_odontologo	Horas atendidas y utilización por profesional.
servicio.py	procedimientos_frecuentes	Procedimientos más realizados en el período.
servicio.py	precision_por_procedimiento	Contrasta la duración del catálogo, la asignada y la real por procedimiento, y ordena por la magnitud de la desviación.
servicio.py	ocupacion_por_hora	Distribución de las atenciones por hora de la jornada.
views.py	panel	Reúne los siete cálculos, serializa las series para las gráficas y entrega el conjunto a la plantilla.

Tabla 71: Inventario de funciones del módulo de Reportes Fuente: elaboración propia.

La función más importante del módulo es la que contrasta la duración de referencia del catálogo con la que la clínica tarda realmente. Es la cifra que sustenta el componente predictivo, porque identifica los procedimientos en que la duración de referencia se aleja de la realidad, que son aquellos donde la estimación aporta más. La función excluye los procedimientos con menos de tres atenciones medidas, para no reportar como desviación lo que es ruido de una muestra pequeña, y ordena el resultado por la magnitud de la desviación relativa, de modo que lo que más se aparta aparezca primero.

app/reportes/servicio.py — función precision_por_procedimiento

```
def precision_por_procedimiento(desde, hasta):
    """Contraste entre la duración del catálogo, la asignada y la real."""
    filas = []
    for proc in Procedimiento.objects.filter(estado=True).order_by('nombre'):
        base = Cita.objects.filter(
            procedimiento=proc, fecha__range=(desde, hasta), estado=True,
            estado_cita='ATENDIDA', duracion_real_min__isnull=False)
        n = base.count()
        if n < MINIMO_PARA_PROMEDIO:
            continue
        agregado = base.aggregate(real=Avg('duracion_real_min'),
                                  asignado=Avg('duracion_asignada_min'))
        real = round(agregado['real'], 1)
```

```
        catalogo = proc.duracion_promedio_min
        filas.append({
            'nombre': proc.nombre, 'catalogo': catalogo,
            'asignado': round(agregado['asignado'], 1), 'real': real,
            'desvio': round(real - catalogo, 1),
            'desvio_pct': round((real - catalogo) / catalogo * 100, 1)
                          if catalogo else 0,
            'atenciones': n,
        })
    return sorted(filas, key=lambda f: abs(f['desvio_pct']), reverse=True)
```

4.2.4 Ejemplo de gráficas y reportes

El panel de reportes (Figura 52) reúne en una sola pantalla los indicadores del período, tres gráficas y dos tablas. El período es filtrable entre cuatro, ocho, doce y veintiséis semanas, y el conjunto completo se recalcula con la selección.

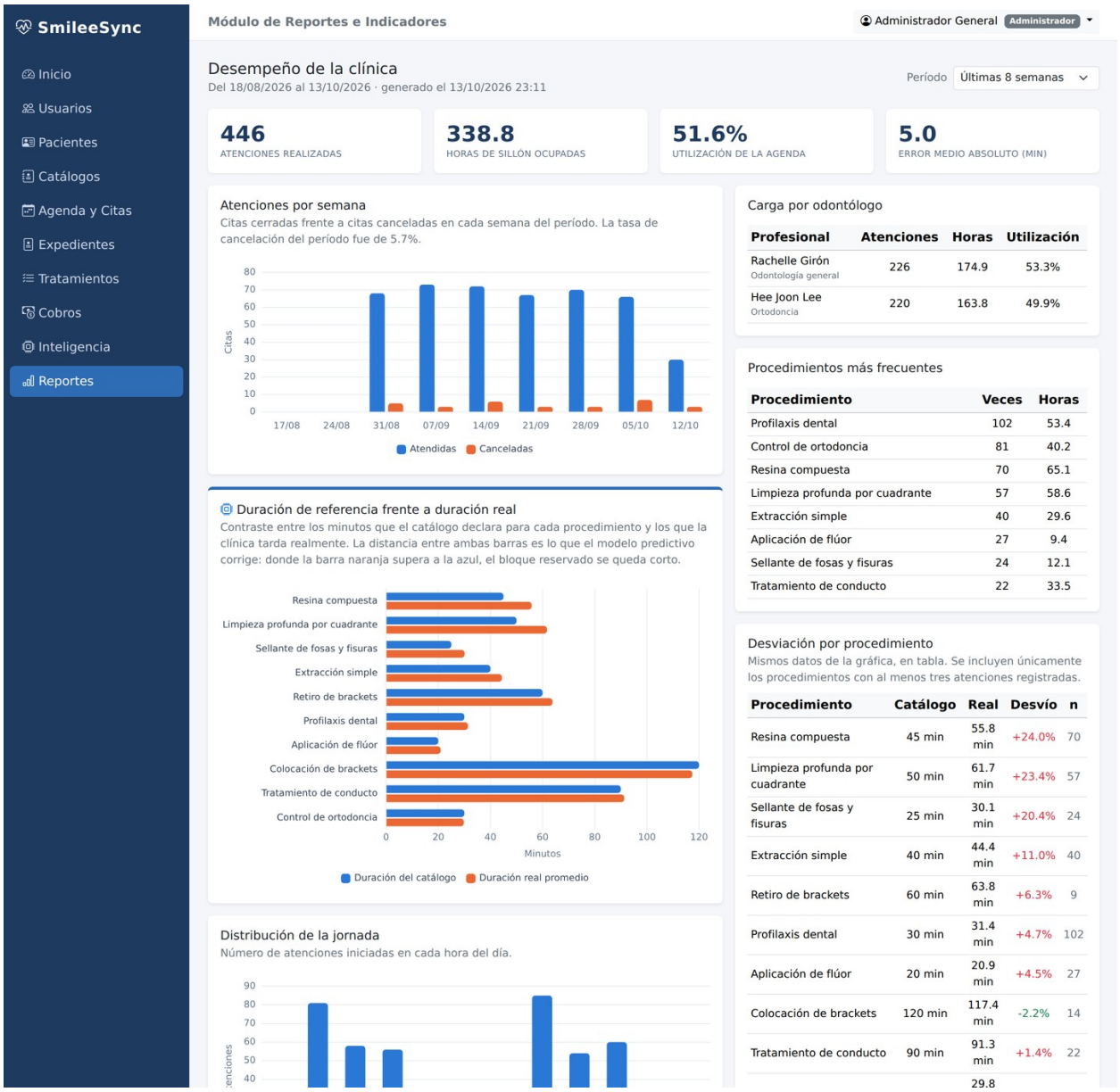


Figura 52: Panel de reportes e indicadores del sistema (datos de demostración) Fuente: elaboración propia.

La gráfica central del panel es la que contrasta la duración de referencia del catálogo con la duración real promedio de cada procedimiento. Su lectura es directa: donde ambas barras coinciden, la duración del catálogo basta; donde se separan, el bloque reservado se queda corto o sobra, y es ahí donde la estimación del modelo aporta. Sobre los datos de demostración, la resina compuesta excede en 24.0 % su duración de referencia, la limpieza profunda por cuadrante en 23.4 % y el sellante en 20.4 %, lo que confirma el problema que el proyecto plantea: la duración declarada en el catálogo subestima sistemáticamente el tiempo real de varios procedimientos.

La implementación de las gráficas sigue un principio: el servidor produce datos y el navegador los dibuja. La vista serializa cada serie a JSON y la plantilla la deposita en la página mediante la etiqueta json_script del framework, que escapa el contenido de modo que un dato con caracteres especiales no pueda cerrar la etiqueta e inyectar código.

app/reportes/views.py — función panel

```
@login_required
@modulo_requerido('reportes')
def panel(request):
    semanas = request.GET.get('semanas', '8')
    semanas = int(semanas) if semanas.isdigit() and int(semanas) in SEMANAS else 8
    desde, hasta = servicio.rango(semanas)

    precision = servicio.precision_por_procedimiento(desde, hasta)
    contexto = {
        'semanas': semanas, 'opciones': SEMANAS,
        'desde': desde, 'hasta': hasta,
        'resumen': servicio.resumen(desde, hasta),
        'carga': servicio.carga_por_odontologo(desde, hasta),
        'frecuentes': servicio.procedimientos_frecuentes(desde, hasta),
        'precision': precision,
        'serie_semanal': json.dumps(servicio.atenciones_por_semana(desde, hasta)),
        'serie_horas': json.dumps(servicio.ocupacion_por_hora(desde, hasta)),
        'serie_precision': json.dumps({
            'etiquetas': [f['nombre'] for f in precision],
            'catalogo': [f['catalogo'] for f in precision],
            'real': [f['real'] for f in precision],
        }),
        'generado': timezone.localtime(),
    }
    return render(request, 'reportes/panel.html', contexto)
```

app/templates/reportes/panel.html

```
{{ serie_semanal|json_script:"d-semanal" }}
{{ serie_precision|json_script:"d-precision" }}
{{ serie_horas|json_script:"d-horas" }}

<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
<script>
(function () {
  if (typeof Chart === 'undefined') return;

  const SERIE_1 = '#2a78d6';
  const SERIE_2 = '#eb6834';
  const leer = (id) => JSON.parse(JSON.parse(
      document.getElementById(id).textContent));

  const precision = leer('d-precision');
  new Chart(document.getElementById('g-precision'), {
    type: 'bar',
    data: {
      labels: precision.etiquetas,
      datasets: [
        { label: 'Duración del catálogo', data: precision.catalogo,
          backgroundColor: SERIE_1, borderRadius: 4, maxBarThickness: 14 },
        { label: 'Duración real promedio', data: precision.real,
          backgroundColor: SERIE_2, borderRadius: 4, maxBarThickness: 14 },
      ],
    },
```

```
    options: Object.assign({}, comunes, { indexAxis: 'y', ... }),
  });
})();
</script>
```

La primera línea del bloque de script comprueba que la biblioteca de gráficas se haya cargado antes de usarla. Sin esa comprobación, un servidor sin salida a internet —o una red de distribución de contenido momentáneamente inaccesible— produciría un error de JavaScript que detendría la ejecución del resto del script de la página. Con ella, la pantalla pierde las gráficas pero conserva los indicadores y las tablas, que es la degradación aceptable.

Esa consideración se extiende al diseño de todo el panel: cada gráfica tiene junto a sí la tabla con los mismos datos. La decisión atiende tres situaciones distintas con una sola medida. La primera es la señalada: si la biblioteca no carga, la información sigue disponible. La segunda es la lectura precisa, porque de una barra se estima una magnitud pero no se lee un valor exacto. La tercera es la accesibilidad: las dos series de las gráficas se distinguen por color, y la tabla garantiza que la información no dependa únicamente de esa distinción. Los dos colores empleados —un azul y un naranja— se seleccionaron verificando que conserven contraste suficiente entre sí bajo las formas más frecuentes de visión cromática deficiente.

El panel del módulo de inteligencia (Figura 53) aplica el mismo patrón a la información del modelo: presenta el error del modelo frente al error del catálogo, el coeficiente de determinación, el número de registros de entrenamiento, el histórico de versiones entrenadas con sus métricas y la documentación de la cadena de estimación. Es la pantalla desde la que se activa una versión del modelo y la única reservada al rol de administrador.

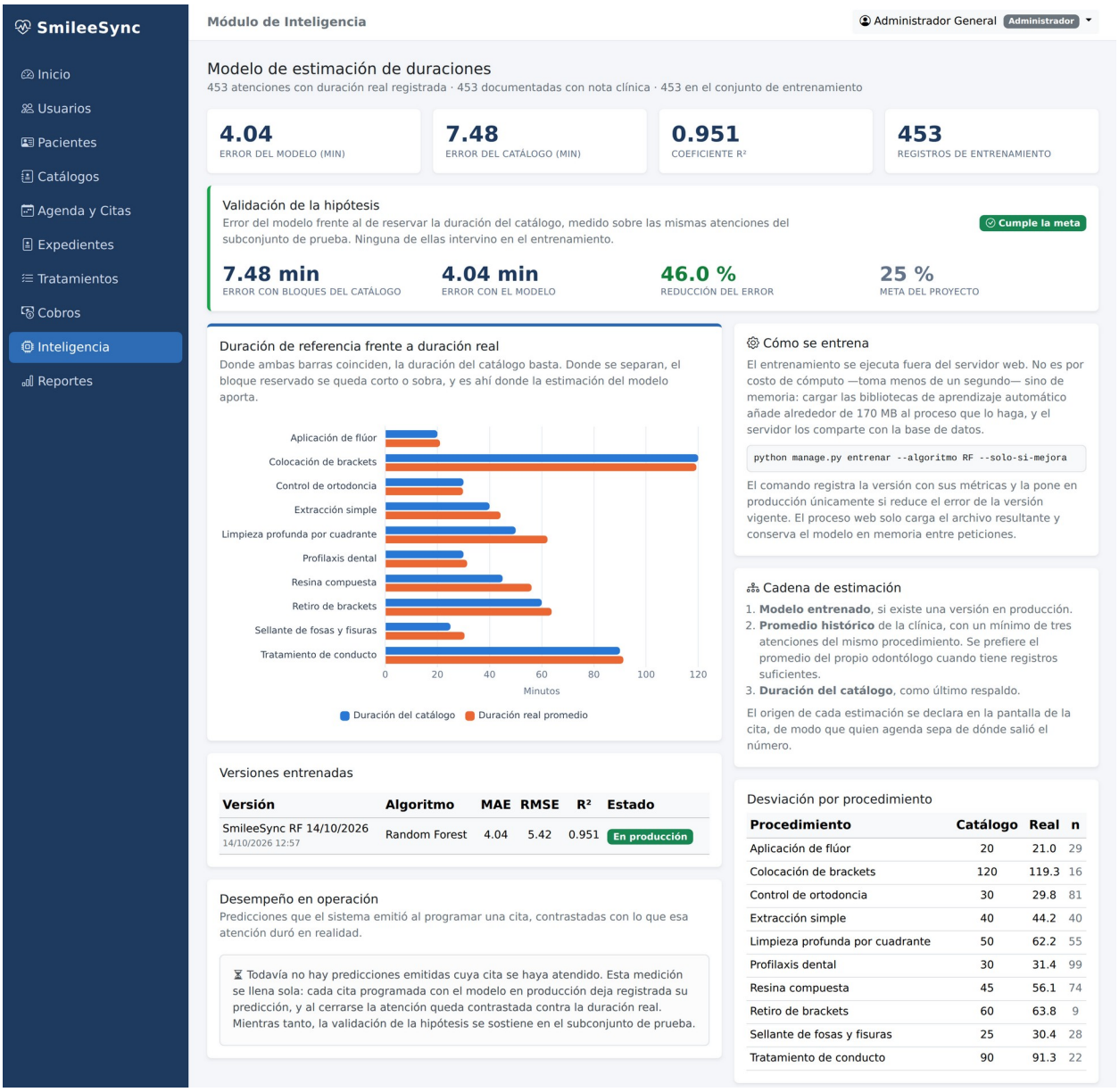


Figura 53: Panel del módulo de Inteligencia con las métricas del modelo y el histórico de versiones (datos de demostración)
Fuente: elaboración propia.

4.3 Pruebas de implementación

Las pruebas de implementación consisten en poner el sistema en manos de sus usuarios reales, en el entorno en que habrán de operarlo, y observar si resuelve el trabajo tal como la clínica lo realiza. Se distinguen de la verificación técnica —que comprueba que el código hace lo que el programador supone— porque lo que ponen a prueba es otra cosa: si el sistema corresponde a la operación real de la clínica, si el personal entiende lo que ve en la pantalla y si las decisiones de diseño que se tomaron en el escritorio sobreviven al uso cotidiano.

La presente sección documenta, en ese orden: el entorno y el alcance de las pruebas; la verificación técnica que se exigió como condición previa a entregar el sistema a los usuarios; los usuarios y los datos con que se preparó el ensayo; el guion de acompañamiento con las preguntas previstas y sus respuestas; el protocolo con que se registra cada sesión; y los hallazgos derivados del uso, con las correcciones que se aplicaron.

4.3.1 Entorno y alcance

El sistema se ejecuta en la instancia de producción descrita en la sección 4.1.1 y se accede a él desde cualquier navegador, sin instalar nada en los equipos de la clínica. Esa condición es parte de lo que se pone a prueba: la clínica no dispone de personal de informática, de modo que un sistema que exigiera instalación o mantenimiento en cada equipo habría fracasado por esa sola razón.

El alcance de las pruebas comprende los tres procesos que constituyen la operación diaria de la clínica y que son también los que el sistema transforma: la programación de una cita con la duración estimada, la atención con el registro de los tiempos reales y la documentación de la visita en el expediente. El resto de los módulos —usuarios, catálogos, pacientes, planes y cobros— se prueba en la medida en que esos tres procesos los requieren, que es también la medida en que la clínica los usa.

4.3.2 Verificación técnica previa a la entrega

Entregar a los usuarios un sistema que falla en lo elemental convierte la prueba de implementación en una sesión de depuración y desperdicia la disposición del personal, que es un recurso escaso. Por esa razón se estableció como condición previa que el sistema superara un conjunto de verificaciones automatizadas, agrupadas en ocho conjuntos según el comportamiento que comprueban. Cada verificación formula una afirmación sobre el sistema y la comprueba contra la base de datos y contra las pantallas; en total son 336 y las 336 se cumplen. La Tabla 72 las relaciona.

Conjunto	Qué comprueba	Verificaciones
Horarios y ausencias	Patrón semanal de atención, registro de ausencias y cálculo de la disponibilidad efectiva como la resta de ambos.	22
Permisos por rol	Acceso de cada rol a cada módulo y a cada vista, incluido el intento de acceso directo por dirección sin pasar por el menú.	33
Calendario	Construcción de la rejilla, cambio de período, filtros por profesional e	31

	indicadores del módulo Principal.	
Recomendación	Estimación de la duración, cálculo de los espacios sugeridos y su presentación sobre el calendario.	20
Flujos completos	Siete recorridos de extremo a extremo, desde el registro del paciente hasta el cobro; incluye la revisión de veintidós pantallas en busca de marcas de plantilla sin procesar.	31
Capacidad y unidades	Cálculo de la capacidad real de la clínica, asignación automática de las unidades dentales y detección de traslapes.	31
Expediente clínico	Odontograma y vigencia de los hallazgos, notas de evolución, planes por fases, reparto en sesiones, presupuesto, abonos y saldos.	82
Modelo predictivo	Sincronización del conjunto de entrenamiento, métricas contra la línea base y contra la duración del catálogo, versionado, caché del modelo, los tres niveles de la cadena de respaldo, el desempeño en operación y los paneles de inteligencia y de reportes.	86
Total		336

Tabla 72: Conjuntos de verificación automatizada exigidos como condición previa a las pruebas de implementación Fuente: elaboración propia.

Dos de esos conjuntos merecen una nota. El de permisos por rol no comprueba que el botón esté oculto, sino que la dirección esté denegada: para cada rol y cada vista que modifica información, solicita la dirección directamente y verifica que el sistema responda con el rechazo correspondiente. El de flujos completos incorpora una verificación que no se anticipó al diseñarlo y que se agregó a raíz de un hallazgo de las pruebas de uso: recorre veintidós pantallas del sistema y falla si encuentra en ellas marcas de plantilla sin procesar, que es la forma en que un comentario mal escrito en el código llega a mostrarse al usuario. Su origen se documenta en la sección 4.3.6.

4.3.3 Usuarios y datos de preparación del ensayo

Cada integrante del personal recibió su propio usuario, con el rol que corresponde a su función. La decisión de no compartir un usuario entre varias personas no es un formalismo: el historial de auditoría registra quién realizó cada cambio, y un usuario compartido convertiría ese registro en una constancia inútil. La Tabla 73 relaciona los usuarios creados para el ensayo.

Usuario	Rol en la clínica	Rol en el sistema	Qué puede hacer
Hee Joon Lee	Odontólogo, especialista en ortodoncia	Odontólogo	Consulta su propia agenda y los pacientes; escribe el expediente clínico, las notas de evolución y los planes de tratamiento; registra sus ausencias.
Rachelle Girón	Odontóloga general	Odontólogo	Las mismas facultades, sobre su propia agenda y sobre los pacientes que atiende.
Ana Recepción	Recepcionista	Recepcionista	Programa, reprograma y cancela citas; registra pacientes y los tiempos reales de atención; consulta el expediente sin escribir en él; registra abonos y consulta los reportes.
Administrador General	Propietario de la clínica	Administrador	Todos los módulos, incluidos el catálogo, los usuarios y el módulo de inteligencia, desde el que se entrena y activa el modelo.

Tabla 73: Usuarios creados para las pruebas de implementación Fuente: elaboración propia.

El sistema no se entregó vacío. Una base de datos sin información no permite probar nada de lo que importa: no hay agenda que consultar, la estimación recae en la duración del catálogo y los reportes no tienen qué reportar. Se preparó, por eso, un conjunto de datos de demostración que reproduce dos meses de operación con el volumen y la variabilidad reales de la clínica. La Tabla 74 lo describe. Los nombres de los pacientes son ficticios; las duraciones reales se generaron a partir de la duración de referencia de cada procedimiento con la variabilidad que la clínica reporta, incluyendo el ritmo distinto de cada profesional y la aparición ocasional de complicaciones.

Elemento	Cantidad	Observación
Período cubierto	31/08/2026 al 30/10/2026	Seis semanas de historial y dos de agenda futura.
Especialidades	5	Catálogo de la clínica.
Procedimientos	10	Con su duración de referencia y su tarifa.
Odontólogos	2	Un ortodoncista y una odontóloga general, con su patrón semanal de veinte franjas de atención.
Unidades dentales	2	Es el recurso que limita la atención simultánea.
Pacientes	24	Con sus antecedentes médicos.
Citas	663	472 atendidas, 155 programadas y 36 canceladas.
Notas de evolución	472	Una por cada atención, con piezas tratadas, anestesia y complicaciones.
Registros de entrenamiento	472	Es el conjunto con que se entrenó el modelo que opera en el sistema; 118 se reservaron para evaluarlo.

Tabla 74: Datos de demostración cargados para las pruebas de implementación Fuente: elaboración propia.

4.3.4 Guion de acompañamiento: preguntas previstas y respuestas

Antes de entregar el sistema se preparó un guion de acompañamiento. Su propósito no es documentar la operación del sistema, que es lo que hace el manual de usuario, sino anticipar las preguntas que cada rol formularía al encontrarse con una forma de trabajar distinta de la que conoce. Las preguntas se derivaron de los puntos en que el sistema se aparta de la agenda en papel: la estimación de la duración, que antes era un criterio del profesional; la imposibilidad de borrar, que antes era tachar; la separación de permisos, que antes no existía porque el cuaderno era de todos.

El guion cumple además una función de diseño. Una pregunta que se anticipa y cuya respuesta resulta difícil de dar señala, casi siempre, una decisión mal resuelta en el sistema antes que una duda del usuario. Dos de los hallazgos de la sección 4.3.6 se identificaron al redactar estas respuestas y no al observar el uso. Las preguntas previstas para la recepción se presentan en la Tabla 75; las de los odontólogos, en la Tabla 76; las del administrador, en la Tabla 77.

Pregunta prevista	Respuesta
¿Cómo sé cuánto tiempo reservarle a un paciente que pide una resina en dos piezas?	En el calendario se elige el procedimiento, el odontólogo que lo atenderá y el número de piezas. El sistema estima la duración, dice de dónde sacó ese número y resalta en verde los espacios de la semana donde esa cita cabe completa. Basta pulsar uno de esos espacios para programarla.
El sistema me dice 75 minutos, pero el doctor dice que él lo hace en 60. ¿A quién le hago caso?	Al doctor: la duración se puede escribir a mano y el sistema la acepta. La estimación es una recomendación, no una restricción. Además, al registrarse los tiempos reales el modelo aprende el ritmo de cada profesional, de modo que si efectivamente lo hace en 60 minutos, en unas semanas el sistema propondrá 60.
¿Por qué no me deja agendar a las 11:30 si el doctor está libre a esa hora?	El sistema exige cuatro condiciones y el mensaje de error indica cuál no se cumple. Las dos más frecuentes son que la cita no termine antes de que cierre la franja de atención —una cita de una hora no cabe a las 11:30 si la jornada termina a las 12:00— y que los dos sillones estén ocupados a esa hora, aunque el odontólogo esté libre.
Un paciente quiere cambiar su cita del martes al jueves. ¿Tengo que cancelarla y crear otra?	No. Se abre la cita y se cambia la fecha: es la misma cita reprogramada. El historial conserva la fecha anterior y quién hizo el cambio. Cancelar y crear de nuevo haría que la clínica apareciera con una cancelación que no ocurrió.
¿Y si el paciente no llega?	Se cancela la cita. El registro no desaparece: queda como cancelada y suma a la tasa de cancelación del reporte, que es información que la clínica necesita. Las citas canceladas no se cuentan para entrenar el modelo, porque no hay duración real que aprender de ellas.
¿Puedo escribir en el expediente del paciente?	Puede consultarlo completo, pero no escribir en él. El odontograma, las notas de evolución y los planes los registra el personal odontológico, porque son actos clínicos. Si abre la pantalla, verá la información sin los botones para modificarla.
¿Dónde registro que el paciente abonó Q300 de su tratamiento?	En el módulo de cobros, como un abono sobre el plan aprobado del paciente. El saldo se recalcula solo: no hay que anotarlo en ninguna parte.
Registré un pago con el monto equivocado. ¿Lo puedo borrar?	Se anula, que no es lo mismo que borrarlo. El abono queda marcado como anulado, el saldo se corrige y queda constancia de que existió un cobro mal registrado. Después se ingresa el abono correcto.
¿Tengo que anotar a qué hora terminó realmente la consulta?	Sí, y es la anotación más importante del sistema. Es lo que permite que la estimación mejore: cada atención con su tiempo real se incorpora al conjunto con que se entrena el modelo. Sin ese registro el sistema seguiría estimando con la duración del catálogo indefinidamente.
Me equivoqué al escribir el apellido de un paciente. ¿Lo puedo corregir?	Sí. Se edita la ficha y el historial conserva el valor anterior, de modo que la corrección quede documentada.
¿Puedo entrar desde mi teléfono?	Sí: es un sitio web y las pantallas se adaptan al tamaño de la ventana. Para el calendario semanal conviene la computadora, porque muestra seis días a la vez.

Tabla 75: Guion de acompañamiento: preguntas previstas de la recepción y sus respuestas Fuente: elaboración propia.

Pregunta prevista	Respuesta
Al entrar, ¿veo la agenda de toda la clínica o solo la mía?	La suya. El sistema reconoce al usuario vinculado a su ficha profesional y preselecciona su agenda al ingresar. Si necesita ver la de la clínica completa, se elige «Todos los odontólogos» en el filtro superior.
¿Puedo agendar una cita yo mismo?	La programación de citas corresponde a la recepción, de modo que su acceso a la agenda es de consulta. Sí puede, en cambio, registrar sus ausencias, porque esa es información suya.
Voy a salir a un congreso el 20 de noviembre. ¿Cómo bloqueo ese día?	En su ficha, en la pantalla de ausencias, se registra la fecha y el rango de horas. Desde ese momento el sistema deja de ofrecer espacios en esa franja y rechaza cualquier intento de agendar ahí, con un mensaje que explica que existe una ausencia registrada. Una ausencia de medio día no cancela el resto de la jornada: el sistema la resta y conserva disponible lo que queda.
¿Tengo que llenar el odontograma completo del paciente la primera vez que lo atiendo?	No. Se registran únicamente los hallazgos que observa; las piezas sin registro se muestran como sanas. El odontograma se va completando con cada visita.
Registré un hallazgo en la pieza equivocada. ¿Se borra el anterior?	El registro se marca como inactivo y deja de contar para el odontograma vigente, pero no desaparece del historial de la pieza. El expediente conserva la secuencia completa de lo que se observó y cuándo, que es lo que permite seguir

	la evolución de un diente.
¿Por qué el sistema me pide el número de piezas tratadas y si usé anestesia?	Porque son dos de las tres variables que más explican cuánto dura una atención. Juntas con la aparición de complicaciones reúnen el 60.7 % del peso del modelo, de modo que son esos tres datos, y no el tipo de procedimiento por sí solo, los que permiten que el sistema estime con precisión. Es también la razón por la que la nota de evolución no es un trámite: es la fuente de la que el sistema aprende.
¿La nota clínica se puede modificar después de guardarla?	No desde la pantalla, y es deliberado: una nota de evolución es la constancia de lo ocurrido en una visita, y un expediente que se puede reescribir a posteriori pierde su valor como documento. Cada visita admite una sola nota; si hay algo que agregar, se consigna en la nota de la visita siguiente.
El plan que presupuesté dice que requiere cinco sesiones. ¿De dónde sale ese número?	De la suma de las duraciones estimadas de cada renglón del plan, repartidas en bloques que no pasan de dos horas de sillón y respetando el orden de las fases, de modo que una fase posterior no se adelante. Es una previsión para conversar con el paciente, no una reserva: agendar cada sesión sigue siendo decisión de la recepción.
¿Puedo ver cuánto me desvío del tiempo que me asignan?	Sí. En la consulta de citas, cada atención muestra el tiempo asignado, el real y la diferencia entre ambos. El reporte que consolida esa desviación por procedimiento corresponde al administrador, porque es información de la clínica y no de un profesional.
Si el sistema no está disponible un día, ¿cómo atiendo?	La agenda del día se puede imprimir desde la vista por día del calendario, y las atenciones de ese día se registran después. El registro posterior no afecta al modelo: lo que importa es la duración real, no el momento en que se capturó.

Tabla 76: Guion de acompañamiento: preguntas previstas de los odontólogos y sus respuestas Fuente: elaboración propia.

Pregunta prevista	Respuesta
¿Cada cuánto hay que volver a entrenar el modelo?	No existe una regla única; el criterio es la cantidad de información nueva. Con el volumen de la clínica —alrededor de setenta atenciones al mes— un reentrenamiento mensual incorpora material suficiente para que las métricas cambien. El comando admite la opción de activar la versión nueva únicamente si reduce el error de la vigente, con lo que reentrenar no implica riesgo.
¿El sistema se detiene mientras se entrena el modelo?	No. El entrenamiento es un comando que se ejecuta aparte del servidor web y tarda menos de un segundo; mientras corre, el sistema sigue estimando con la versión que está en producción. La versión nueva entra en operación solo cuando se la activa.
Si el modelo nuevo resulta peor que el anterior, ¿cómo vuelvo atrás?	El panel de inteligencia conserva el histórico completo de las versiones entrenadas con sus métricas. Se activa la versión anterior desde ese mismo panel y el cambio es inmediato: solo una versión está en producción a la vez.
¿Qué pasa con la estimación cuando agregamos un procedimiento nuevo al catálogo?	El modelo no conoce ese procedimiento, de modo que la estimación recae en la duración de referencia que se declare al darlo de alta. Cuando se acumulan tres atenciones, pasa a usar el promedio real de la clínica; y al siguiente reentrenamiento, el modelo lo incorpora. La pantalla declara en cada caso de dónde proviene el número, de modo que se sabe en qué etapa está.
¿Cómo doy de alta a una asistente nueva?	En el módulo de usuarios, con el rol de asistente, que da acceso de consulta a la agenda y al expediente. El sistema envía al usuario a definir su contraseña y exige que registre una pregunta secreta para poder recuperarla.
¿Puedo eliminar un procedimiento que ya no realizamos?	Se da de baja, lo que lo retira de los formularios y del catálogo activo sin borrarlo. Eliminarlo haría que las citas históricas que lo usaron quedaran sin procedimiento y la clínica perdería su propio historial, que es justamente lo que alimenta la estimación.
Si compramos un tercer sillón, ¿hay que modificar el sistema?	No: se registra en el catálogo de unidades dentales. La capacidad de la clínica, la validación al programar y la recomendación de espacios se calculan a partir de las unidades dadas de alta, de modo que las tres se ajustan solas al nuevo sillón.
¿Quién puede ver el expediente clínico de un paciente?	El personal odontológico lo escribe y lo consulta; la recepción lo consulta sin escribir; el administrador tiene acceso completo. Cada cambio queda registrado con el usuario que lo hizo, de modo que el acceso es trazable.
¿Dónde quedan los datos y qué pasa si se pierde el servidor?	Los datos residen en la base de datos PostgreSQL de la instancia. El respaldo periódico de esa base es la tarea pendiente más importante del plan de puesta en producción, junto con la habilitación de HTTPS; ambas se documentan en la

	sección 4.1.3.5.

Tabla 77: Guion de acompañamiento: preguntas previstas del administrador y sus respuestas Fuente: elaboración propia.

4.3.5 Protocolo y registro de las sesiones

Las sesiones de prueba se organizan en tres etapas sucesivas, de modo que la observación recoja información distinta en cada una. La primera es la capacitación inicial: se recorre el sistema con el usuario, se ejecuta con él el proceso que le corresponde y se resuelven las dudas que surjan, apoyándose en el guion de la sección anterior. La segunda es el uso acompañado: el usuario opera por su cuenta sobre datos de demostración y el observador anota, sin intervenir, en qué puntos se detiene, qué busca donde no está y qué entiende distinto de lo que la pantalla dice. La tercera es el uso autónomo, en el que el personal emplea el sistema sin acompañamiento y comunica lo que encuentra.

La segunda etapa es la que produce los hallazgos más valiosos, y por una razón concreta: en la capacitación el usuario pregunta y en el uso autónomo reporta solo lo que le impide trabajar, mientras que en el uso acompañado se observa aquello que el usuario resuelve por su cuenta sin mencionarlo —volver atrás, repetir un paso, buscar un botón en el lugar equivocado—, que son justamente los defectos de diseño que nadie reporta. Cada sesión se registra con la ficha de la Tabla 78, que fija qué se anota para que las observaciones de sesiones distintas resulten comparables.

Campo de la ficha	Qué se registra
Fecha y duración	Día de la sesión y tiempo efectivo de trabajo.
Usuario y rol	Quién operó el sistema y con qué rol ingresó.
Etapa	Capacitación inicial, uso acompañado o uso autónomo.
Procesos ejercitados	Cuáles de los tres procesos del alcance se ejecutaron y cuántas veces.
Tareas completadas sin ayuda	Tareas que el usuario concluyó por sí solo, con el tiempo que le tomó la primera y las siguientes.
Puntos de detención	Momentos en que el usuario se detuvo, qué buscaba y dónde lo buscó.
Preguntas formuladas	Preguntas del usuario, indicando si estaban previstas en el guion o no. Una pregunta no prevista señala una omisión del guion o del sistema.
Discrepancias con la operación real	Casos en que el sistema exige algo que la clínica no hace, o no admite algo que la clínica sí hace.
Defectos observados	Comportamientos incorrectos, con los pasos para reproducirlos.
Comentarios del usuario	Observaciones espontáneas, incluidas las favorables, que indican qué conviene conservar.
Acciones derivadas	Corrección, ajuste de la capacitación o decisión de no cambiar nada, con la justificación.

Tabla 78: Ficha de registro de las sesiones de prueba de implementación Fuente: elaboración propia.

Con esa ficha se planificaron tres sesiones, una por cada perfil de usuario del sistema y en el orden en que la operación las requiere: primero el profesional que escribe el expediente, después

la profesional que lo consulta y lo continúa, y por último la recepción, que es quien convierte todo eso en citas. La Tabla 79 las describe con su objetivo, las tareas que se ejercitan en cada una y lo que se observa. Al momento de esta entrega las sesiones están planificadas y no ejecutadas, de modo que lo que se reporta es el diseño del ensayo y no sus resultados; estos se incorporarán conforme se realicen, con la ficha de la Tabla 78 como instrumento de registro.

Sesión	Participante y etapa	Objetivo y tareas	Qué se observa
1	Dr. Hee Joon Lee, ortodoncista. Capacitación inicial. Duración prevista: 45 minutos.	Validar el módulo de expediente clínico con el profesional que lo escribirá. Tareas: recorrer el expediente de un paciente, registrar hallazgos sobre el odontograma en al menos tres piezas, redactar la nota de evolución de una visita atendida y elaborar un plan de tratamiento por fases con su presupuesto.	Si la secuencia de la pantalla corresponde al orden en que el profesional trabaja; si la notación del odontograma le resulta familiar sin explicación previa; si entiende por qué el sistema le pide el número de piezas y el uso de anestesia; y qué información del expediente en papel no encuentra en la pantalla.
2	Dra. Rachelle Girón, odontóloga general. Uso acompañado. Duración prevista: 60 minutos.	Ejercitar el expediente sin acompañamiento en las tareas y con observación silenciosa. Tareas: consultar el odontograma vigente de un paciente que la Dra. no atendió, continuar un tratamiento registrado por otro profesional, documentar una visita con complicación y consultar su propia agenda.	En qué puntos se detiene y qué busca donde no está; si distingue el hallazgo vigente del histórico de la pieza; si advierte que la nota no se puede modificar después de guardarla y cómo reacciona a ello; y qué preguntas formula que no estén previstas en el guion de la Tabla 76, pues cada una de ellas señala una omisión del guion o del sistema.
3	Ana Recepción, recepcionista. Uso acompañado y autónomo. Duración prevista: 60 minutos.	Ejercitar el ciclo completo de la agenda con datos de prueba. Tareas: registrar pacientes ficticios, consultar la recomendación de espacios para tres procedimientos de distinta duración, programar citas a ambos profesionales hasta ocupar las dos unidades dentales, reprogramar una cita, cancelar otra y registrar los tiempos reales de una atención concluida.	Si la recomendación de espacios resulta comprensible sin explicación; si los mensajes de validación indican qué hacer cuando un espacio se rechaza; cuánto tarda en programar la primera cita y cuánto la quinta; y si registra los tiempos reales sin que se le recuerde, que es la tarea de la que depende el aprendizaje del modelo.

Tabla 79: Sesiones de prueba de implementación previstas, con su objetivo y los aspectos por observar Fuente: elaboración propia.

El orden de las sesiones no es arbitrario. La primera y la segunda recaen sobre el expediente clínico porque es el módulo que incorpora el acto profesional y, al mismo tiempo, el que alimenta al modelo predictivo: si el profesional no documenta la visita, el sistema pierde las tres variables que explican dos terceras partes de la duración, conforme a la Tabla 69. La tercera cierra el ciclo con el perfil que opera la agenda a diario y que es, en la práctica, el usuario principal del sistema.

4.3.6 Hallazgos derivados del uso y correcciones aplicadas

Los hallazgos que se presentan a continuación provienen del uso del sistema sobre las pantallas construidas y no de la verificación automatizada, que los daba todos por correctos. Esa

es precisamente la razón de documentarlos: ninguno de los siete habría aparecido en una prueba de código, porque en seis de ellos el sistema hacía exactamente lo que el programador le había indicado y el problema estaba en lo que el programador había indicado. Cada corrección se acompañó de una verificación automatizada nueva, de modo que el defecto no pueda reaparecer sin que el conjunto de pruebas lo señale. La Tabla 80 los relaciona.

Hallazgo	Efecto para el usuario	Corrección aplicada	Verificación agregada
Comentarios del código visibles en la pantalla del calendario.	Sobre el encabezado del calendario aparecía un párrafo con texto destinado al programador, que el usuario leía como si fuera parte del sistema.	La sintaxis de comentario empleada solo es válida en una línea; al ocupar varias, el motor de plantillas la trataba como texto. Se sustituyó por la etiqueta de comentario de bloque en las tres pantallas afectadas.	Un recorrido por veintidós pantallas que falla si encuentra marcas de plantilla sin procesar en la respuesta.
El resultado de la estimación no se leía en el calendario.	El usuario veía los espacios recomendados resaltados en la rejilla, pero no la duración estimada ni su procedencia, que era la información que explicaba por qué esos espacios y no otros.	La fila que contenía el resultado sumaba trece columnas de las doce disponibles, de modo que el resultado se comprimía en un tercio del ancho. Se rediseñó como una franja propia a todo lo ancho, con la cifra destacada.	Comprobación de que la pantalla presenta la duración estimada, el bloque recomendado y la etiqueta de origen.
Elegir primero el procedimiento y después el odontólogo descartaba el procedimiento.	La estimación no aparecía nunca si el usuario llenaba los campos en ese orden, que es el orden natural: el paciente pide un procedimiento y luego se decide quién lo atiende.	El selector de odontólogo pertenecía al formulario de filtro y el de procedimiento al de consulta, de modo que enviar uno perdía el otro. Se agregaron los campos ocultos que conservan la consulta en curso y el selector de odontólogo se trasladó al panel de recomendación.	Comprobación de que la consulta responde con ambos campos enviados en cualquier orden.
Mensaje de error con redacción incorrecta cuando hay una sola unidad dental activa.	El sistema respondía «Las 1 unidades dentales están ocupadas», lo que resta credibilidad a un mensaje que el usuario debe tomar en serio.	El mensaje distingue el caso de una unidad del de varias.	Comprobación del texto del mensaje con una sola unidad activa en el catálogo.
Un renglón del plan podía guardarse con precio cero.	El formulario mostraba 0.00 como valor inicial del precio, de modo que un renglón guardado sin tocar ese campo quedaba presupuestado en cero y el total del plan resultaba inferior al real.	El campo se presenta vacío y el formulario propone la tarifa del catálogo al elegir el procedimiento, con lo que el valor que se guarda es deliberado y no un valor predeterminado del modelo.	Comprobación de que el renglón exige un precio y de que la tarifa propuesta corresponde a la del catálogo.
El archivo del modelo se leía del disco en cada estimación.	No era visible como defecto, pero cada consulta de recomendación deserializaba medio megabyte, de modo que el calendario respondía más lento conforme el modelo crecía.	Se agregó una caché en memoria del módulo, que lee el archivo una vez por versión y lo descarta solo al activarse otra.	Medición de que la primera estimación tras limpiar la caché es la más costosa y de que las siguientes se mantienen por debajo de cien milisegundos.
El odontólogo encontraba al entrar la agenda de toda la clínica.	El profesional veía las citas de ambos consultorios y tenía que filtrar su propia agenda en cada ingreso, que es lo único que necesita ver.	El sistema reconoce al usuario vinculado a la ficha profesional y preselecciona su agenda, distinguiendo el primer ingreso de la elección explícita de ver la	Comprobación de que el odontólogo recibe su agenda preseleccionada y de que puede elegir ver la de todos.

		clínica completa.	

Tabla 80: Hallazgos derivados del uso del sistema, correcciones aplicadas y verificaciones agregadas Fuente: elaboración propia.

Los tres primeros hallazgos comparten un rasgo que conviene señalar. En los tres el sistema calculaba correctamente y el defecto estaba en la presentación: la estimación existía, los espacios recomendados se calculaban bien y la consulta funcionaba, pero el usuario no veía el resultado o no lograba llegar a él. Es la clase de defecto que la verificación automatizada no detecta, porque comprueba que el dato esté en la respuesta y no que esté legible en la pantalla, y es también la que justifica que las pruebas de implementación no puedan sustituirse por pruebas de código.

El sexto hallazgo es de naturaleza distinta y vale la pena distinguirlo. No lo reportó ningún usuario ni lo habría reportado, porque su efecto —unas décimas de segundo en cada consulta— se percibe como lentitud general y no como un defecto concreto. Apareció al medir el tiempo de respuesta de la recomendación, lo que confirma que la observación del uso debe complementarse con la medición: hay defectos que el usuario sufre sin poder nombrarlos.

4.3.7 Resultado y tareas pendientes

El sistema completa los tres procesos del alcance sobre el entorno de producción, con los usuarios de cada rol y con dos meses de datos de operación cargados: la programación de una cita con la duración estimada, el registro de la atención con los tiempos reales y la documentación de la visita en el expediente. Las 336 verificaciones automatizadas se cumplen, el modelo en producción reduce en 41.8 % el error del método de bloques fijos —frente a la meta de 25 % del proyecto— y los siete defectos derivados del uso fueron corregidos, cada uno con su verificación añadida al conjunto.

Quedan pendientes tres tareas antes de considerar concluida la implantación, que se enuncian para que el alcance de lo verificado no se confunda con el de lo definitivo. La primera es la habilitación de HTTPS con un certificado y la activación de las marcas de seguridad correspondientes, condición para operar con datos de pacientes reales. La segunda es la configuración del respaldo periódico de la base de datos. La tercera es la sustitución de los datos de demostración por el historial real de la clínica, momento en que el modelo deberá reentrenarse sobre ese historial y en que las métricas reportadas en la Tabla 68 habrán de recalcularse: las cifras

de este capítulo corresponden al conjunto de 472 atenciones con que se construyó y validó el sistema.

Referencias

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cayirli, T., & Veral, E. (2003). Outpatient scheduling in health care: A review of literature. Production and Operations Management, 12(4), 519–549. https://doi.org/10.1111/j.1937-5956.2003.tb00218.x

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Date, C. J. (2004). An introduction to database systems (8.ª ed.). Addison-Wesley.

Django Software Foundation. (2024). Django documentation. https://docs.djangoproject.com/

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Erdogan, S. A., Denton, B. T., & Otten, J. (2015). Surgery planning and scheduling. En M. Brandeau, F. Sainfort, & W. Pierskalla (Eds.), Handbook of Healthcare Operations Management (pp. 577–609). Springer. https://doi.org/10.1007/978-1-4614-5885-2_20

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2.ª ed.). O'Reilly Media.

Gupta, D., & Denton, B. (2008). Appointment scheduling in health care: Challenges and opportunities. IIE Transactions, 40(9), 800–819. https://doi.org/10.1080/07408170802165880

Han, J., Kamber, M., & Pei, J. (2011). Data mining: Concepts and techniques (3.ª ed.). Morgan Kaufmann.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2.ª ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hung, M., Voss, M. W., Rosales, M. N., Li, W., Su, W., Xu, J., & Licari, F. W. (2020). Application of machine learning for diagnostic prediction of root caries. Gerodontology, 36(4), 395–404. https://doi.org/10.1111/ger.12432

Klassen, K. J., & Rohleder, T. R. (1996). Scheduling outpatient appointments in a dynamic environment. Journal of Operations Management, 14(2), 83–101. https://doi.org/10.1016/0272-6963(95)00044-5

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. Proceedings of the 14th International Joint Conference on Artificial Intelligence, 1137–1143.

Laudon, K. C., & Laudon, J. P. (2016). Management information systems: Managing the digital firm (14.ª ed.). Pearson.

Mell, P., & Grance, T. (2011). The NIST definition of cloud computing (NIST Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Mitchell, T. M. (1997). Machine learning. McGraw-Hill.

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). Introduction to linear regression analysis (5.ª ed.). Wiley.

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. https://www.postgresql.org/docs/

Pressman, R. S., & Maxim, B. R. (2015). Software engineering: A practitioner's approach (8.ª ed.). McGraw-Hill.

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2011). Recommender systems handbook. Springer. https://doi.org/10.1007/978-0-387-85820-3

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum: La guía definitiva de Scrum, las reglas del juego. https://scrumguides.org/

Sebesta, R. W. (2016). Concepts of programming languages (11.ª ed.). Pearson.

Sommerville, I. (2011). Software engineering (9.ª ed.). Pearson.

Srinivas, S., & Ravindran, A. R. (2018). Optimizing outpatient appointment system using machine learning algorithms and scheduling rules: A prescriptive analytics framework. Expert Systems with Applications, 102, 245–261. https://doi.org/10.1016/j.eswa.2018.02.022

Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. Nature Medicine, 25(1), 44–56. https://doi.org/10.1038/s41591-018-0300-7

Tsai, C. F., Hu, Y. H., & Chen, Z. Y. (2016). Improving patient adherence to clinical practice guidelines through an automated appointment scheduling and reminder system. Computers in Human Behavior, 55, 506–519. https://doi.org/10.1016/j.chb.2015.09.033

Van Rossum, G., & Drake, F. L. (2009). Python 3 reference manual. CreateSpace.

Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. Climate Research, 30(1), 79–82. https://doi.org/10.3354/cr030079

Glosario

Agenda clínica: Instrumento de planificación que registra las citas, los procedimientos y los tiempos asignados a los pacientes en un establecimiento de salud, y que permite coordinar los recursos disponibles con la demanda de atención.

Aprendizaje automático: Rama de la inteligencia artificial que desarrolla algoritmos capaces de aprender patrones a partir de datos históricos y de generar predicciones sin haber sido programados explícitamente para cada caso.

Aprendizaje supervisado: Paradigma del aprendizaje automático en el que el algoritmo aprende a partir de datos etiquetados, es decir, registros para los que se conoce de antemano el valor que se desea predecir.

Borrado lógico: Técnica que marca un registro como inactivo en lugar de eliminarlo físicamente de la base de datos, de modo que deja de ofrecerse en las operaciones corrientes pero conserva su vínculo con la información histórica.

Bosques aleatorios: Algoritmo de ensamble que construye múltiples árboles de decisión y combina sus predicciones mediante promedio, lo que reduce el sobreajuste frente a un árbol individual.

Coeficiente de determinación (R^2): Métrica que mide la proporción de la varianza de la variable dependiente explicada por el modelo. Su valor máximo es 1; los valores cercanos a 1 indican mayor capacidad explicativa, y un valor negativo sobre datos no vistos indica que el modelo predice peor que el promedio de los datos.

Conjunto de datos: Colección organizada de registros representada como una tabla, donde cada fila corresponde a una observación y cada columna a una variable. Se divide habitualmente en subconjuntos de entrenamiento y prueba.

Correlación de Pearson: Medida estadística que cuantifica la relación lineal entre dos variables continuas. Su coeficiente varía entre -1 y 1.

DERCAS: Documento de Especificación de Requerimientos y Criterios de Aceptación de Software. Delimita qué debe hacer el sistema y bajo qué condiciones se considera que un requerimiento fue satisfecho.

Diagrama entidad-relación: Representación gráfica de la estructura lógica de una base de datos, que describe las entidades, sus atributos y las relaciones que existen entre ellas.

Error absoluto medio (MAE): Promedio de las diferencias absolutas entre los valores predichos y los valores reales. Expresa el error en las mismas unidades que la variable estimada, lo que facilita su interpretación.

Error cuadrático medio (MSE): Promedio de los cuadrados de las diferencias entre los valores predichos y los reales. Penaliza con mayor severidad los errores de gran magnitud.

Gradient boosting: Técnica de ensamble en la que los modelos se construyen de forma secuencial y cada uno corrige los errores del conjunto precedente.

Hiperparámetro: Parámetro de configuración de un algoritmo cuyo valor se establece antes del entrenamiento y no se aprende de los datos, como la profundidad máxima de un árbol o la tasa de aprendizaje.

Infraestructura como servicio (IaaS): Modelo de computación en la nube en el que el proveedor ofrece recursos de cómputo, almacenamiento y red bajo demanda, y el cliente administra el sistema operativo y las aplicaciones.

Interfaz de programación de aplicaciones (API): Conjunto de definiciones que permite que dos componentes de software se comuniquen entre sí mediante un contrato de peticiones y respuestas.

Mapeo objeto-relacional (ORM): Capa de software que traduce las operaciones sobre objetos del lenguaje de programación en sentencias del lenguaje de consulta de la base de datos.

Modelo predictivo: Representación matemática o computacional que, a partir del análisis de datos históricos, genera estimaciones sobre el valor futuro o desconocido de una variable de interés.

Modelo vista plantilla (MVT): Patrón arquitectónico empleado por Django que separa el acceso a los datos, la lógica de control y la presentación, de manera análoga al patrón modelo vista controlador.

Muestreo estratificado: Técnica de muestreo que divide la población en grupos homogéneos y extrae unidades de cada uno, de modo que todos los grupos queden representados en la muestra.

Optimización de recursos: Distribución eficiente de los recursos disponibles —tiempo, personal y espacio— con el fin de maximizar la productividad o minimizar los costos operativos de un sistema.

Raíz del error cuadrático medio (RMSE): Raíz cuadrada del promedio de los cuadrados de los errores de predicción. Expresa el error en las unidades de la variable estimada y es sensible a los errores de gran magnitud.

Regresión: Técnica estadística y de aprendizaje automático cuyo objetivo es modelar la relación entre una o más variables independientes y una variable dependiente de naturaleza continua.

Regresión lineal múltiple: Modelo que asume una relación lineal entre la variable dependiente y varias variables independientes, y estima los coeficientes que minimizan la suma de los cuadrados de los residuos.

Sistema de información: Conjunto integrado de componentes que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, la coordinación y el control dentro de una organización.

Sistema de recomendación: Sistema computacional que sugiere elementos o decisiones relevantes a partir de datos históricos y de la identificación de patrones. En este proyecto sugiere el bloque horario adecuado para una cita.

Sobreajuste: Situación en la que un modelo se ajusta en exceso a los datos de entrenamiento y capta ruido que no se generaliza, de modo que su precisión cae al aplicarse a datos nuevos.

Subajuste: Situación en la que un modelo resulta demasiado simple para captar los patrones subyacentes, lo que produce un rendimiento deficiente tanto en entrenamiento como en datos nuevos.

Tiempo de espera: Intervalo transcurrido entre la llegada del paciente a la clínica y el inicio efectivo de la atención. Su reducción es uno de los principales indicadores de calidad en la gestión de agendas.

Tiempo muerto: Período de la jornada en el que el profesional se encuentra disponible pero sin pacientes asignados, como consecuencia de cancelaciones, ausencias o errores de programación.

Validación cruzada: Técnica de evaluación que divide el conjunto de datos en k subconjuntos y realiza k iteraciones de entrenamiento y evaluación, de manera que cada subconjunto se utilice una vez como conjunto de prueba.

Variable dependiente: Variable cuyo valor constituye el objetivo de la predicción. En este proyecto, la duración real del procedimiento y el desajuste en la asignación del tiempo.

Variable independiente: Variable que se utiliza como entrada del modelo para estimar el comportamiento de la variable dependiente, como el tipo de procedimiento o el odontólogo responsable.

XGBoost: Implementación optimizada del gradient boosting, diseñada para el procesamiento eficiente de grandes conjuntos de datos tabulares y de amplio uso en problemas de predicción.

Anexos

Anexo A. Modelo de la base de datos

La base de datos del sistema fue diseñada bajo el modelo relacional e implementada en PostgreSQL. Se compone de veinticuatro tablas distribuidas en los ocho módulos del sistema, aplicando el borrado lógico mediante el campo estado y tablas de histórico para la auditoría de los cambios. La Figura 54 presenta nuevamente el diagrama entidad-relación completo, a mayor tamaño, en el que se observan las entidades, sus atributos y las relaciones entre ellas.

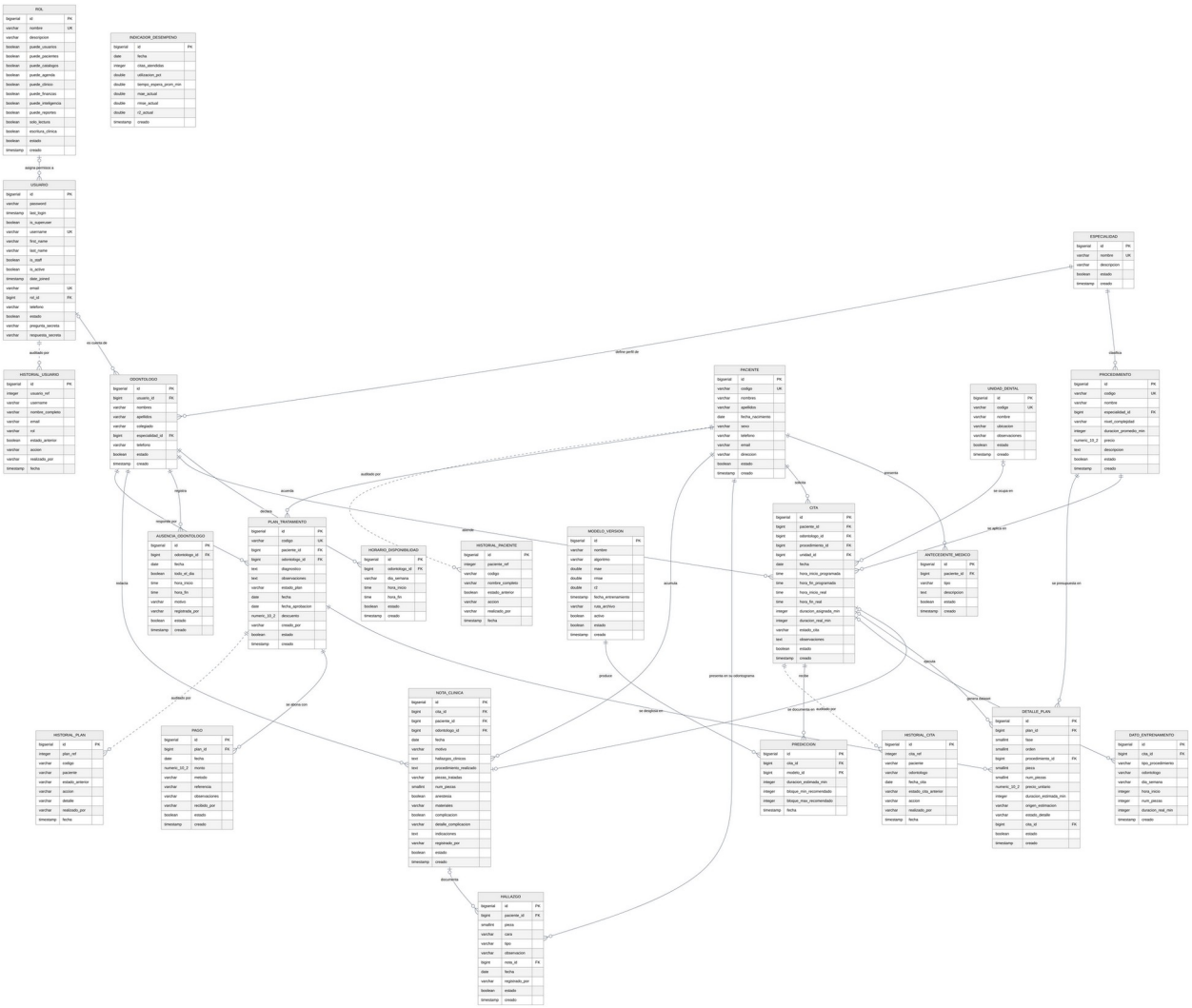


Figura 54: Diagrama entidad-relación completo del sistema SmileeSync (vista ampliada) Fuente: elaboración propia.

Anexo B. Diccionario de datos

La Tabla 81 presenta el diccionario de datos del sistema. Describe cada tabla del modelo y sus campos, indicando el tipo de dato, si admite valores nulos y una breve descripción. Las claves primarias corresponden al campo id y las claves foráneas se señalan con la referencia a la tabla relacionada.

Tabla 81: Diccionario de datos del sistema **Fuente:** elaboración propia.

Entidad: rol (Seguridad y Usuarios)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
nombre	varchar(50)	No	Nombre. El valor no puede repetirse en la tabla.
descripcion	varchar(200)	No	Descripción.
puede_usuarios	boolean	No	Módulo usuarios.
puede_pacientes	boolean	No	Módulo pacientes.
puede_catalogos	boolean	No	Módulo catálogos.
puede_agenda	boolean	No	Módulo agenda.
puede_clinico	boolean	No	Módulo expediente clínico.
puede_finanzas	boolean	No	Módulo cobros.
puede_inteligencia	boolean	No	Módulo inteligencia.
puede_reportes	boolean	No	Módulo reportes.
solo_lectura	boolean	No	Solo consulta. Permite ver los módulos asignados, sin crear ni modificar registros.
escritura_clinica	boolean	No	Puede registrar en el expediente. Permite registrar odontograma, notas clínicas y planes de tratamiento. Corresponde al personal odontológico.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: usuario (Seguridad y Usuarios)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
password	varchar(128)	No	Contraseña.
last_login	timestamp	Sí	Último inicio de sesión.
is_superuser	boolean	No	Estado de superusuario. Indica que este usuario tiene todos los permisos sin asignárselos explícitamente.
username	varchar(150)	No	Nombre de usuario. El valor no puede repetirse en la tabla. Requerido. 150 carácteres como máximo. Únicamente letras, dígitos y @/./+/-/_
first_name	varchar(150)	No	Nombre.
last_name	varchar(150)	No	Apellidos.
is_staff	boolean	No	Es staff. Indica si el usuario puede entrar en este sitio de administración.
is_active	boolean	No	Activo. Indica si el usuario debe ser tratado como activo. Desmarque esta opción en lugar de borrar la cuenta.
date_joined	timestamp	No	Fecha de alta.
email	varchar(254)	No	Correo electrónico. El valor no puede repetirse en la tabla.
rol_id	bigint	Sí	Rol. Llave foránea hacia la tabla «rol».
telefono	varchar(20)	No	Teléfono.

estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
pregunta_secreta	varchar(10)	No	Pregunta secreta. Valores admitidos: MASCOTA, CIUDAD, ESCUELA, MADRE, APODO.
respuesta_secreta	varchar(128)	No	Respuesta secreta (cifrada).

Entidad: historial_usuario (Seguridad y Usuarios)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
usuario_ref	integer	No	Id de usuario.
username	varchar(150)	No	Username.
nombre_completo	varchar(200)	No	Nombre completo.
email	varchar(254)	No	Email.
rol	varchar(50)	No	Rol.
estado_anterior	boolean	No	Estado anterior.
accion	varchar(20)	No	Accion. Valores admitidos: CREACION, ACTUALIZACION, DESACTIVACION, ACTIVACION.
realizado_por	varchar(150)	No	Realizado por.
fecha	timestamp	No	Fecha.

Entidad: especialidad (Catálogo Odontológico)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
nombre	varchar(80)	No	Nombre. El valor no puede repetirse en la tabla.
descripcion	varchar(200)	No	Descripción.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: procedimiento (Catálogo Odontológico)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
codigo	varchar(20)	No	Código. El valor no puede repetirse en la tabla.
nombre	varchar(120)	No	Nombre.
especialidad_id	bigint	No	Especialidad. Llave foránea hacia la tabla «especialidad».
nivel_complejidad	varchar(5)	No	Nivel de complejidad. Valores admitidos: BAJA, MEDIA, ALTA.
duracion_promedio_min	integer	No	Duración promedio (min).
precio	numeric(10,2)	No	Precio de lista (Q).
descripcion	text	No	Descripción.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: odontologo (Catálogo Odontológico)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
usuario_id	bigint	Sí	Usuario del sistema. Llave foránea hacia la tabla «usuario».
nombres	varchar(80)	No	Nombres.
apellidos	varchar(80)	No	Apellidos.

colegiado	varchar(30)	No	Número de colegiado.
especialidad_id	bigint	No	Especialidad. Llave foránea hacia la tabla «especialidad».
telefono	varchar(20)	No	Teléfono.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: unidad_dental (Catálogo Odontológico)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
codigo	varchar(20)	No	Código. El valor no puede repetirse en la tabla.
nombre	varchar(80)	No	Nombre.
ubicacion	varchar(120)	No	Ubicación.
observaciones	varchar(200)	No	Observaciones.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: paciente (Pacientes)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
codigo	varchar(20)	No	Código. El valor no puede repetirse en la tabla.
nombres	varchar(80)	No	Nombres.
apellidos	varchar(80)	No	Apellidos.
fecha_nacimiento	date	Sí	Fecha de nacimiento.
sexo	varchar(1)	No	Sexo. Valores admitidos: M, F.
telefono	varchar(20)	No	Teléfono.
email	varchar(254)	No	Correo.
direccion	varchar(200)	No	Dirección.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: antecedente_medico (Pacientes)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
paciente_id	bigint	No	Paciente. Llave foránea hacia la tabla «paciente».
tipo	varchar(60)	No	Tipo.
descripcion	text	No	Descripción.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: historial_paciente (Pacientes)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
paciente_ref	integer	No	Id de paciente.
codigo	varchar(20)	No	Codigo.
nombre_completo	varchar(200)	No	Nombre completo.
estado_anterior	boolean	No	Estado anterior.
accion	varchar(20)	No	Accion. Valores admitidos: CREACION, ACTUALIZACION, DESACTIVACION, ACTIVACION.

realizado_por	varchar(150)	No	Realizado por.
fecha	timestamp	No	Fecha.

Entidad: horario_disponibilidad (Agenda y Citas)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
odontologo_id	bigint	No	Odontólogo. Llave foránea hacia la tabla «odontologo».
dia_semana	varchar(3)	No	Día de la semana. Valores admitidos: LUN, MAR, MIE, JUE, VIE, SAB.
hora_inicio	time	No	Hora de inicio.
hora_fin	time	No	Hora de fin.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: cita (Agenda y Citas)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
paciente_id	bigint	No	Paciente. Llave foránea hacia la tabla «paciente».
odontologo_id	bigint	No	Odontólogo. Llave foránea hacia la tabla «odontologo».
procedimiento_id	bigint	No	Procedimiento. Llave foránea hacia la tabla «procedimiento».
unidad_id	bigint	Sí	Unidad dental. Llave foránea hacia la tabla «unidad_dental».
fecha	date	No	Fecha.
hora_inicio_programada	time	No	Hora inicio programada.
hora_fin_programada	time	No	Hora fin programada.
hora_inicio_real	time	Sí	Hora inicio real.
hora_fin_real	time	Sí	Hora fin real.
duracion_asignada_min	integer	No	Duración asignada (min).
duracion_real_min	integer	Sí	Duración real (min).
estado_cita	varchar(12)	No	Estado de la cita. Valores admitidos: PROGRAMADA, ATENDIDA, CANCELADA, REPROGRAMADA.
observaciones	text	No	Observaciones.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: historial_cita (Agenda y Citas)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
cita_ref	integer	No	Id de cita.
paciente	varchar(200)	No	Paciente.
odontologo	varchar(200)	No	Odontologo.
fecha_cita	date	No	Fecha cita.
estado_cita_anterior	varchar(12)	No	Estado cita anterior.
accion	varchar(20)	No	Accion.
realizado_por	varchar(150)	No	Realizado por.
fecha	timestamp	No	Fecha.

Entidad: ausencia_odontologo (Agenda y Citas)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
odontologo_id	bigint	No	Odontólogo. Llave foránea hacia la tabla «odontologo».
fecha	date	No	Fecha.
todo_el_dia	boolean	No	Toda la jornada.
hora_inicio	time	No	Hora de inicio.
hora_fin	time	No	Hora de finalización.
motivo	varchar(200)	No	Motivo.
registrada_por	varchar(150)	No	Registrada por.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: hallazgo (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
paciente_id	bigint	No	Paciente. Llave foránea hacia la tabla «paciente».
pieza	smallint	No	Pieza dental (FDI). Valores admitidos: 18, 17, 16, 15, 14, 13, 12, 11, 21, 22, 23, 24, 25, 26, 27, 28, 48, 47, 46, 45, 44, 43, 42, 41, 31, 32, 33, 34, 35, 36, 37, 38.
cara	varchar(4)	No	Cara. Valores admitidos: TODA, O, M, D, V, L.
tipo	varchar(12)	No	Hallazgo. Valores admitidos: SANA, CARIES, OBTURADA, SELLANTE, ENDODONCIA, CORONA, PROTESIS, IMPLANTE, FRACTURA, EXTRACCION, AUSENTE.
observacion	varchar(200)	No	Observación.
nota_id	bigint	Sí	Visita en que se registró. Llave foránea hacia la tabla «nota_clinica».
fecha	date	No	Fecha del hallazgo.
registrado_por	varchar(150)	No	Registrado por.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: nota_clinica (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
cita_id	bigint	Sí	Cita atendida. Llave foránea hacia la tabla «cita».
paciente_id	bigint	No	Paciente. Llave foránea hacia la tabla «paciente».
odontologo_id	bigint	No	Odontólogo. Llave foránea hacia la tabla «odontologo».
fecha	date	No	Fecha de la visita.
motivo	varchar(200)	No	Motivo de la consulta.
hallazgos_clinicos	text	No	Hallazgos clínicos.
procedimiento_realizado	text	No	Procedimiento realizado.
piezas_tratadas	varchar(120)	No	Piezas tratadas. Números FDI separados por coma, p. ej. 16, 26.
num_piezas	smallint	No	Número de piezas tratadas.
anestesia	boolean	No	Se aplicó anestesia.
materiales	varchar(200)	No	Materiales utilizados.
complicacion	boolean	No	Hubo complicación.
detalle_complicacion	varchar(200)	No	Detalle de la complicación.
indicaciones	text	No	Indicaciones al paciente.
registrado_por	varchar(150)	No	Registrado por.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.

creado	timestamp	No	Creado.

Entidad: plan_tratamiento (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
codigo	varchar(20)	No	Código. El valor no puede repetirse en la tabla.
paciente_id	bigint	No	Paciente. Llave foránea hacia la tabla «paciente».
odontologo_id	bigint	No	Odontólogo responsable. Llave foránea hacia la tabla «odontologo».
diagnostico	text	No	Diagnóstico.
observaciones	text	No	Observaciones.
estado_plan	varchar(12)	No	Estado del plan. Valores admitidos: BORRADOR, APROBADO, EN_CURSO, FINALIZADO, RECHAZADO.
fecha	date	No	Fecha de elaboración.
fecha_aprobacion	date	Sí	Fecha de aprobación.
descuento	numeric(10,2)	No	Descuento (Q).
creado_por	varchar(150)	No	Elaborado por.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: detalle_plan (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
plan_id	bigint	No	Plan. Llave foránea hacia la tabla «plan_tratamiento».
fase	smallint	No	Fase. Las fases se ejecutan en orden.
orden	smallint	No	Orden dentro de la fase.
procedimiento_id	bigint	No	Procedimiento. Llave foránea hacia la tabla «procedimiento».
pieza	smallint	Sí	Pieza dental. Valores admitidos: 18, 17, 16, 15, 14, 13, 12, 11, 21, 22, 23, 24, 25, 26, 27, 28, 48, 47, 46, 45, 44, 43, 42, 41, 31, 32, 33, 34, 35, 36, 37, 38.
num_piezas	smallint	No	Número de piezas.
precio_unitario	numeric(10,2)	No	Precio unitario (Q).
duracion_estimada_min	integer	No	Duración estimada (min).
origen_estimacion	varchar(12)	No	Origen de la estimación.
estado_detalle	varchar(10)	No	Estado. Valores admitidos: PENDIENTE, AGENDADO, COMPLETADO, ANULADO.
cita_id	bigint	Sí	Cita que lo atiende. Llave foránea hacia la tabla «cita».
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: pago (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
plan_id	bigint	No	Plan. Llave foránea hacia la tabla «plan_tratamiento».
fecha	date	No	Fecha del abono.
monto	numeric(10,2)	No	Monto (Q).
metodo	varchar(14)	No	Forma de pago. Valores admitidos: EFECTIVO, TARJETA, TRANSFERENCIA, CHEQUE.
referencia	varchar(60)	No	Referencia o número de recibo.
observaciones	varchar(200)	No	Observaciones.

recibido_por	varchar(150)	No	Recibido por.
estado	boolean	No	Vigente. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: historial_plan (Expediente Clínico y Cobros)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
plan_ref	integer	No	Id del plan.
codigo	varchar(20)	No	Codigo.
paciente	varchar(200)	No	Paciente.
estado_anterior	varchar(12)	No	Estado anterior.
accion	varchar(20)	No	Accion. Valores admitidos: CREACION, ACTUALIZACION, APROBACION, RECHAZO, FINALIZACION, ANULACION, PAGO.
detalle	varchar(200)	No	Detalle.
realizado_por	varchar(150)	No	Realizado por.
fecha	timestamp	No	Fecha.

Entidad: modelo_version (Inteligencia)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
nombre	varchar(80)	No	Nombre.
algoritmo	varchar(10)	No	Algoritmo. Valores admitidos: LINEAL, RF.
mae	double precision	Sí	MAE.
rmse	double precision	Sí	RMSE.
r2	double precision	Sí	R^2.
mae_catalogo	double precision	Sí	MAE del catálogo.
fecha_entrenamiento	timestamp	Sí	Fecha de entrenamiento.
ruta_archivo	varchar(255)	No	Ruta del modelo (.joblib).
activo	boolean	No	En producción.
estado	boolean	No	Activo. Soporta el borrado lógico: el registro inactivo se conserva.
creado	timestamp	No	Creado.

Entidad: dato_entrenamiento (Inteligencia)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
cita_id	bigint	Sí	Cita origen. Llave foránea hacia la tabla «cita».
tipo_procedimiento	varchar(120)	No	Tipo de procedimiento.
odontologo	varchar(200)	No	Odontólogo.
dia_semana	varchar(3)	No	Día de la semana.
hora_inicio	integer	No	Hora de inicio (0-23).
num_piezas	integer	No	Número de piezas.
anestesia	boolean	No	Se aplicó anestesia.
complicacion	boolean	No	Hubo complicación.
duracion_catalogo_min	integer	No	Duración de referencia del catálogo (min).
duracion_real_min	integer	No	Duración real (min).
creado	timestamp	No	Creado.

Entidad: prediccion (Inteligencia)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
cita_id	bigint	No	Cita. Llave foránea hacia la tabla «cita».
modelo_id	bigint	No	Modelo. Llave foránea hacia la tabla «modelo_version».
duracion_estimada_min	integer	No	Duración estimada (min).
bloque_min_recomendado	integer	No	Bloque mínimo (min).
bloque_max_recomendado	integer	No	Bloque máximo (min).
fecha	timestamp	No	Fecha de la predicción.

Entidad: indicador_desempeno (Reportes e Indicadores)

Campo	Tipo	Nulo	Descripción
id	bigserial	No	Identificador único de la tabla (llave primaria).
fecha	date	No	Fecha.
citas_atendidas	integer	No	Citas atendidas.
utilizacion_pct	double precision	Sí	Utilización de agenda (%).
tiempo_espera_prom_min	double precision	Sí	Tiempo de espera promedio (min).
mae_actual	double precision	Sí	MAE del modelo.
rmse_actual	double precision	Sí	RMSE del modelo.
r2_actual	double precision	Sí	R^2 del modelo.
creado	timestamp	No	Creado.